

Wireless Sensor Network

无线传感器网络

第一讲

WSN 的基础概念

WSN 的定义：

一种由大量在微小的体积内集成传感器、嵌入式微处理器和无线收发器等器件的微型传感器节点组成的面向任务的无线自组织网络系统。

这些传感器节点被密集部署在某个指定的地理区域内，通过无线通信和自组织方式形成多跳的无线网络（通过无线通信技术组成的分布式网络系统），用来感知、采集和处理所监测区域内的各种环境数据或目标信息（如温度、湿度、光照、声音、振动等），并将所感知和采集到的数据和信息传送给监控中心或终端用户，协作完成指定的任务。

WSN 的特点：

- ①具有自组织性：节点可自动组网，适应环境变化（如节点故障或移动）。
- ②多跳传输：数据通过多个节点接力传输，解决单个节点通信距离短的问题，扩展网络覆盖范围。
- ③微型化与低功耗：节点可适应复杂环境，如需长时间工作，有可依赖的节能设计（如休眠机制）；
- ④以感知和采集环境数据为目标：数据传输具有实时性和可靠性要求。
- ⑤动态拓扑：网络结构可能因节点移动、能量耗尽或环境干扰而变化。

大规模网络特点

是指其通常由大量传感器节点组成，这些节点协同工作以实现特定的监测或控制任务。

节点数量庞大，分布范围广，部署密集，可增大网络覆盖监测区域，通过不同视角获取信息，减少盲区。由此带来的庞大的数据量需要高效的数据管理策略，常要对数据进行聚合和压缩。通过大量采集信息，可降低对单个节点传感器精度要求，即使部分节点数据不准确，仍可通过数据融合算法得到准确结果。为提高网络的可靠性和鲁棒性，通常会在同一区域部署多个冗余节点，使得系统具有更强的容错能力，即使部分节点失效，其他节点仍可正常工作，确保网络整体功能不受影响，提升了系统的可靠性。另外，网络中可能包含不同类型的节点（如地面、空中、水下节点），进行多层次、多维度的监测，实现多层次覆盖。

自组织（Self-Organizing Network, SON）网络特点

是指网络中的节点能够自主地、动态地组织和管理自身，而无需依赖预先部署的基础设施或集中控制。该特点是无线传感器网络的核心优势之一，使其能够适应复杂和动态的环境。

通常传感器节点位置无法预先精确确定，使用过程中可能因能量耗尽、环境因素或人为干扰随时加入或退出网络。节点的自组织能力使其能自动进行配置和管理，通过拓扑控制机制（包括节点发现、链路建立、路由选择等，例如，采用分布式拓扑控制算法，节点可根据邻居节点信息动态调整通信半径，减少能耗，提高网络覆盖效率）和网络协议自动形成多跳无线网络系统，适应使用过程中网络拓扑结构动态变化。例如，在节点失效时，其他节点可通过重新路由建立通信链路，确保网络连通性；在新节点加入时，可自动融入网络并参与数据采集和传输。

自组织能力主要体现在以下几个方面：

（1）自动组网

节点能够自动发现邻居节点并建立通信链路。节点启动后，能够通过广播或监听机制自动发现周围的邻居节点；节点之间根据通信需求和链路质量，自动建立通信链路；在网络部署初期，节点能够自主完成网络初始化，形成基本的通信结构。

（2）动态拓扑管理

网络能够自动根据节点加入（即能自动融入）、退出或失效的情况，动态调整拓扑结构，

保持连通性。

（3）自适应路由

数据通过多跳方式从源节点传输到汇聚节点，节点能够自动根据网络状态（如能量、链路质量）选择最优的通信路径。路由协议既能考虑节点的剩余能量，避免能量耗尽的节点成为瓶颈，又能根据链路质量（如信号强度、误码率）选择稳定的通信路径。

（4）分布式控制

网络中没有严格的控制中心（“去中心化”），节点之间通过分布式算法协同工作。每个节点根据本地信息（如邻居状态、能量水平）做出决策，而不需要全局信息。

自组织网络的优势：

（1）灵活性与适应性

动态环境适应：自组织网络能够适应节点移动、链路变化等动态环境。

快速部署：无需预先部署基础设施，网络能够快速建立并投入使用。

（2）可扩展性

节点动态加入：新节点能够自动加入网络，扩展网络规模。

网络分区与合并：网络能够根据节点分布自动分区或合并，保持连通性。

（3）鲁棒性与可靠性

故障容忍：当部分节点或链路失效时，网络能够自动调整拓扑，保持通信功能。

负载均衡：通过分布式算法均衡网络负载，避免单点故障。

（4）低成本

无需基础设施：自组织网络无需依赖预先部署的基础设施，降低了部署成本。

分布式管理：无需集中控制，减少了管理和维护成本。

动态性网络特点

是指网络的拓扑结构、节点状态和通信环境可能随时间发生变化。这种动态性是由多种因素引起的，包括节点能量耗尽、节点移动、环境变化以及外部干扰等。

WSN 的拓扑结构可能因环境原因或电能耗尽造成节点失效或故障，环境条件变化造成无线通信链路带宽变化，节点、监测对象、观察者具有移动性，新节点加入等原因发生改变。当节点失效时，可通过重新配置路由或启动备用节点恢复网络功能，以适应拓扑结构变化。同时，网络协议和算法也需具备动态适应性，能根据网络状态动态调整参数，确保网络性能优化。对于具有移动性的节点、监测对象和观察者，网络需支持动态位置更新和数据传输。

动态性主要体现在以下几个方面：

（1）拓扑结构变化

节点加入、退出或移动导致网络拓扑结构发生变化。

（2）节点状态变化

节点能量耗尽、故障或恢复、根据任务需求或能量管理策略在休眠状态和唤醒状态之间切换都会导致节点状态发生变化。

（3）通信环境变化

环境因素（如温度、湿度、障碍物）或外部干扰（如电磁干扰）导致通信链路质量（如信号强度、误码率）发生变化。无线信道的多径效应、衰落和干扰可能导致信道条件随时间变化。

（4）任务需求变化

网络中的任务需求可能随时间变化（如事件检测、数据采集），动态的任务分配会导致节点的工作状态和通信模式发生变化。另外，某些任务可能具有更高的优先级，需要动态调整网络资源分配。

可靠性网络要求

是指网络在各种环境条件下能够稳定、持续地完成其预期功能的能力。可靠性是 WSN 设计和部署中的关键目标之一，尤其是在恶劣或不可预测的环境中。

Wireless Sensor Network

一方面，传感器网络的维护十分困难甚至不可维护，特别是在野外或危险环境中部署的节点，传感器节点应坚固、不易损坏，能在高温、低温、潮湿、强电磁干扰等各种恶劣的环境条件下长期稳定运行。另一方面，防止监测数据被盗取或被伪造十分必要，网络需采用加密技术、身份认证技术和数据完整性校验技术以确保数据安全。

可靠性主要体现在以下几方面：

（1）数据可靠性

通过数据复制或多路径传输，确保数据能够准确、完整地从源节点传输到目标节点。

（2）网络连通性

在同一区域部署多个冗余节点，数据通过多跳方式从源节点传输到目标节点，确保即使部分节点或链路失效，网络也能够保持通信连接。同时，可通过拓扑管理来提高可靠性。例如，当检测到节点或链路失效时，网络可以进行动态拓扑调整，重新选择通信路径；也可以采用分簇算法，如 LEACH，将网络划分为多个簇，由簇头负责数据处理和转发，从而提高网络可靠性和能量效率。

（3）任务完成率

确保网络能够高效地完成监测、控制或其他任务。通过优化路由和任务分配，均衡网络中的能量消耗，避免部分节点过早失效。节点在不工作时进入休眠状态，减少能量消耗，延长网络寿命。

（4）容错能力

在网络中出现节点故障、链路失效或其他异常情况时，仍能保持正常运行。假设真的出现错误，可通过校验码（如 CRC）检测数据传输中的错误，还可以通过前向纠错（FEC）或重传机制纠正数据传输中的错误。

应用相关性

是指网络的设计、部署和优化需要根据具体的应用需求进行定制。

不同的传感器网络应用关心不同的物理量，例如，在农业监测中，主要监测土壤酸碱度、湿度、温度和光照强度；在工业监测中，重点监测设备温度、压力和振动频率。不同的应用背景的要求也不同，从而造成硬件平台、软件系统和网络协议的差别。例如，在医疗监测中，对数据实时性和准确性要求高，需采用高精度传感器和低延迟通信协议。同时，不同应用对节点的部署方式、网络拓扑结构、数据处理方式和能量管理策略也有不同要求，需根据具体应用需求进行定制化设计。

（1）网络拓扑

部署方式：不同的应用场景可能需要不同的节点部署方式。例如，环境监测可能需要随机部署节点，而工业控制可能需要精确部署节点。

拓扑结构：某些应用可能需要分簇结构（如 LEACH 协议），而其他应用可能需要扁平结构（如 AODV 协议）。

（2）节点硬件

传感器类型：不同的应用场景需要不同类型的传感器。例如，温度监测需要温度传感器，而水质监测需要 pH 传感器。

处理器和存储器：某些应用可能需要高性能的处理器和存储器（如视频监控），而其他应用可能只需要低功耗的硬件（如温度监测）。

（3）通信协议

MAC 协议：不同的应用场景可能需要不同的 MAC 协议。例如，低功耗应用可能需要 S-MAC 或 T-MAC 协议，而高实时性应用可能需要 TDMA 协议。

路由协议：某些应用可能需要能量高效的路由协议（如 LEACH），而其他应用可能需要高可靠性的路由协议（如 AODV）。

（4）数据处理

数据聚合：某些应用可能需要数据聚合以减少数据传输量（如环境监测），而其他应用可能需

要保留原始数据（如医疗监测）。

数据压缩：某些应用可能需要数据压缩以节省能量（如图像传输），而其他应用可能不需要压缩（如温度监测）。

（5）能量管理

能量采集：某些应用可能需要能量采集技术（如太阳能、振动能）以延长网络寿命（如环境监测），而其他应用可能依赖电池供电（如医疗监测）。

休眠调度：某些应用可能需要频繁的休眠调度以节省能量（如低功耗监测），而其他应用可能需要持续工作（如实时监控）。

以数据为中心的网络

是指网络的设计和操作主要围绕数据的采集、传输、处理和存储展开，而不是围绕特定的节点或设备。这一特点使得无线传感器网络能够高效地管理和利用数据资源，满足各种应用需求。

WSN 是任务型的网络，其主要任务是完成数据采集、处理和传输，以满足用户对特定事件的监测需求。用户使用 WSN 查询事件时，直接通告给网络其关心的事件，而不是通告给某个确定编号的节点，网络会自动将查询请求分发到该区域的相关节点，节点采集数据后返回给用户。同时要完成数据驱动的网络优化，根据数据流量和数据类型动态调整网络拓扑结构和通信协议，提高数据传输效率，并且采用数据融合技术对采集的数据进行处理，减少数据冗余，降低能耗，提高网络整体性能。数据可以在节点本地进行分布式存储，减少汇聚节点的存储压力，也可以在汇聚节点进行集中式存储，便于后续分析和使用。

WSN 的应用场景：

- ①环境监测：森林火灾预警、气象监测、水质检测等。
- ②智能家居/建筑：温湿度控制、安防系统、家电设备控制等。
- ③医疗健康：远程监护患者心率、体温等。
- ④工业自动化：设备状态监控、工厂机械振动检测、生产流程优化等。
- ⑤农业物联网：土壤湿度监测、自动灌溉等。
- ⑥智能交通：车流量感知、交通信号灯动态配时、桥梁/隧道结构健康监测等。

WSN 的核心组成：

①传感器节点

每个节点包含传感器（感知环境）、处理器（处理数据）、无线通信模块（传输数据）和电源（常为电池）。

特点是体积小、成本低、功耗低，但计算和存储能力有限。

②网络结构

节点通过自组织方式形成多跳网络（无需预先部署基础设施）。

通常包含汇聚节点（Sink Node），负责将数据汇总到互联网或服务器。

WSN 的技术挑战：

- ①能源限制：电池寿命直接影响网络寿命。
- ②数据传输可靠性：无线信号易受天气、障碍物等的干扰。
- ③安全性：如何防止数据被窃听或篡改。
- ④大规模部署：如何高效管理成千上万个节点。

WSN 与其他网络的区别：

- ①对比 Wi-Fi、蓝牙等传统无线网络，WSN 更注重低功耗和自组织能力，而非高速率传输。
- ②WSN 是 IoT（物联网）的底层感知层技术之一，专注于数据采集与传输。

WSN 与 IoT 的关系

- ①二者的目标、范围和应用层次不同，是从属与包含的关系。

WSN 作为 IoT 的“感官神经末梢”，负责通过分布式传感器节点感知和采集环境中的各种原始数据，通过无线多跳网络将数据传输到汇聚节点，再接入 IoT 平台，实现物理世界与数

字世界的连接。

IoT 作为 WSN 的“大脑与骨架”，是一个更为广泛的概念，指通过互联网将传感器、摄像头、智能终端等各种物理设备**互联**，实现数据采集、传输、分析和智能决策的生态系统。IoT 平台对 WSN 采集的**数据**进行**处理**和**分析**，提取有价值的信息并基于此实现**智能决策**和自动化控制。

二者协同的优势：实现了从数据感知到应用服务的完整闭环。例如：在智慧工厂中，WSN 节点监测机器振动和温度，数据通过 IoT 平台分析后，预测设备故障并自动触发维护工单，同时通知管理人员；农业 IoT 系统中，WSN 负责监测土壤湿度，IoT 平台则根据数据控制灌溉系统；WSN 监测城市空气质量，IoT 平台结合交通数据优化尾气排放管控。

②二者功能互补

特性	WSN	IoT
核心目标	环境感知与数据采集	设备互联与智能服务
网络规模	局部区域（农田、工厂等）	广域覆盖（城市、全球互联）
数据流向	单向为主（传感器→汇聚节点）	双向交互（设备↔云平台↔用户）
节点能力	低功耗、低成本、计算能力弱	设备类型多样（低功耗传感器->高性能终端）
典型应用	环境监测、工业传感	智能家居、智慧城市、车联网

值得一提的是，WSN 也为 IoT 提供边缘计算支持，部分 WSN 节点具备初步数据处理能力（如过滤噪声数据），减轻 IoT 云端负担。并且 WSN 的节能设计（如 ZigBee、LoRa 协议）为 IoT 在偏远或无电源场景（如野外监测）提供了可行的组网方案。

另外，IoT 对 WSN 也进行扩展，IoT 将 WSN 采集的原始数据与其他数据（如用户行为、社交媒体）结合，通过 AI 和大数据分析实现智能决策。并且 IoT 可实现跨网络集成，通过网关将 WSN 与其他网络（如 4G/5G、Wi-Fi）互联，实现数据向云端的无缝传输。同时也整合了执行器（如智能开关）、用户终端（如手机 APP）等，形成闭环控制系统，使服务多样化。

③技术挑战的交叉

能源效率：WSN 的低功耗设计与 IoT 设备的长期运行需求高度关联。

安全性：两者均需应对数据加密、身份认证等安全威胁。

标准化：WSN 的通信协议（如 ZigBee、6LoWPAN）需与 IoT 标准（如 MQTT、CoAP）兼容。

WSN 的发展历程

早期传感器系统通常为有线连接，布线复杂，扩展性差。功能单一，仅能采集少量数据，无网络协作能力，数据传输距离有限。后来无线传感器网络的出现解决了布线问题，提高了系统的灵活性和可扩展性。另外，微型化技术使传感器节点体积更小，功耗更低；多跳自组网技术也提高了网络的覆盖范围和稳定性。

传统传感器系统阶段

特点

独立工作：传感器为单一功能设备，无网络协作能力。

有线连接：通过电缆直接连接到中央控制器或数据采集设备。

集中式架构：数据直接传输至中心节点，无分布式处理能力。

高功耗与大体积：传感器依赖外部电源，体积较大，部署灵活性差。

例如

工业温度监测系统：多个温度传感器通过电缆连接到中央控制室的 PLC（可编程逻辑控制器）。每个传感器独立采集数据，无本地处理能力，所有数据需通过电缆传输到中心设备进行分析。

局限性：布线成本高，无法适应复杂或移动场景，扩展性差。

气象站单点监测：使用独立的风速计、湿度计和气压计，数据通过有线方式传输到记录仪，人工定期读取数据。

局限性：无法实时传输，且设备部署受限于电源和布线。

传感器网络节点集成化阶段

特点

无线化与模块集成：传感器节点开始集成无线通信模块（如 RF 收发器）和微处理器，**无需布线，部署灵活。**

初步组网能力：支持短距离无线传输，形成星型或简单树状网络。

低功耗设计：采用电池供电，但节点寿命仍受限于通信能耗。

单向数据流：数据从传感器节点单向传输至汇聚节点，无多跳协作。

例如

ZigBee 无线温湿度监测系统：多个温湿度传感器节点通过 ZigBee 协议与中心网关连接。每个节点集成传感器、微控制器和无线模块，数据直接发送至网关，再由网关上传至服务器。

局限性：网络覆盖范围受限于单跳通信距离。

家庭安防系统：无线门磁传感器和红外探测器与中央报警主机通信，触发报警后主机通过电话线通知用户。

局限性：网络拓扑固定，节点故障可能导致局部失效。

多跳自组网阶段

特点

自组织与多跳路由：节点自动组网，数据通过多跳中继传输，网络拓扑动态适应环境变化。

分布式协同：节点具备本地数据处理能力（边缘计算），协作完成感知任务。

超低功耗设计：采用休眠调度、能量采集（如太阳能）等技术延长网络寿命。

大规模部署：支持成千上万个节点协同工作，覆盖广域复杂环境。

例如

森林火灾监测系统：许多传感器节点部署在森林中，每个节点集成温度、烟雾和 GPS 模块，通过 LoRa 或 6LoWPAN 协议自组织成多跳网络。数据通过相邻节点接力传输至卫星或基站，即使部分节点被火灾破坏，网络仍能自动重构路径。

关键技术：AODV（自组织按需距离向量）路由协议、动态休眠机制。

智能城市交通监测：道路两侧的振动传感器和摄像头通过多跳 Wi-Fi Mesh 网络互联，实时监测车流量和路况。数据经边缘节点预处理后传输至云端，优化信号灯控制。

优势：支持高动态环境，网络鲁棒性强。

对比维度	传统传感器系统	传感器网络节点集成化	多跳自组网
网络结构	有线集中式（点对点）	无线星型/树状（单跳）	无线多跳网状（动态自组织）
通信方式	电缆直连	短距离无线	多跳中继
节点功能	感知+传输	感知+简单处理+无线传输	感知+处理+协作路由+自适应
能耗与寿命	依赖外部电源，无节能设计	电池供电，寿命数月至1年	超低功耗+能量采集，寿命 5-10 年
扩展性	低（受限于布线）	中等（覆盖范围有限）	高（支持大规模动态部署）
典型应用	工业控制、气象站	智能家居、农业监测	环境监测、智慧城市、军事侦察
代表技术	4-20mA 电流环、RS-485 总线	ZigBee、蓝牙	AODV 路由、RPL 路由、TSCH 调度

WSN 节点的组成

硬件组成：

包括传感器单元（感知环境数据）、微处理器（处理数据）、无线通信模块（传输数据）和电源模块（提供能量）。

软件组成：

包括操作系统（管理硬件资源）、通信协议栈（实现节点间通信）和应用层软件（实现特定功能）。

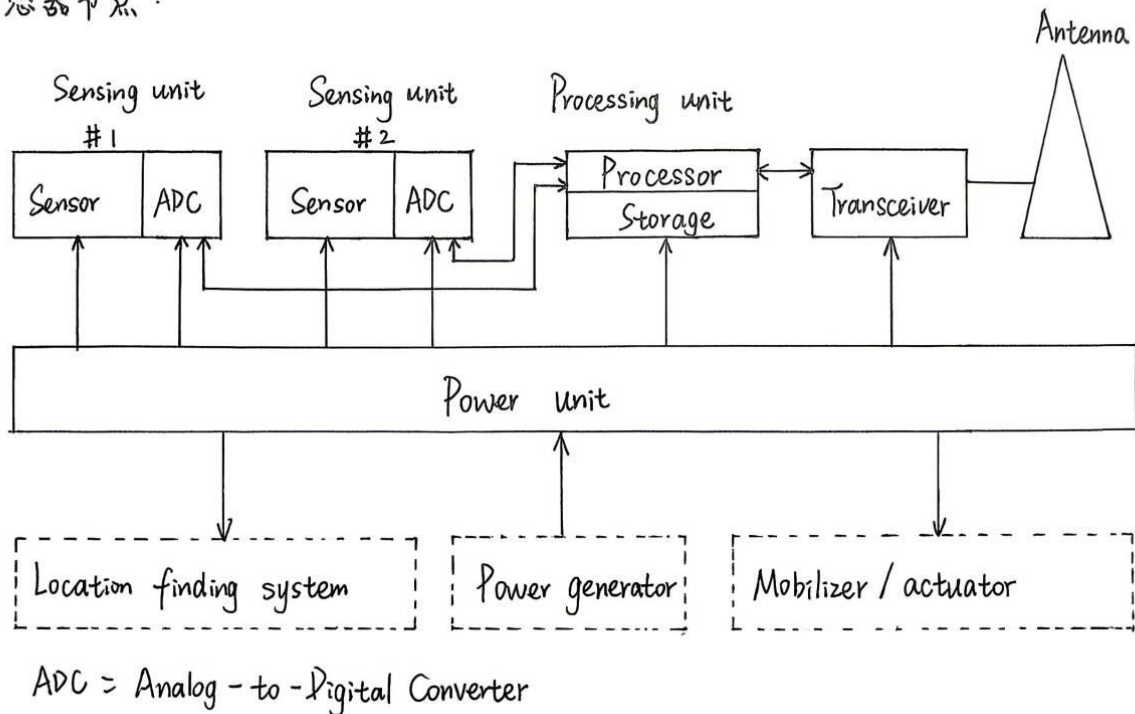
Wireless Sensor Network

部署方式：

随机部署或按特定模式部署，如网格状、随机散布等。

部署方式影响网络的覆盖范围和数据采集效率。

传感器节点：



传感器模块

功能：感知物理或化学参数（如温度、湿度、光照、振动等）。

例如：

环境监测节点：集成温湿度和光照传感器。

工业监测节点：使用振动传感器（压电式）检测设备异常。

微处理器（MCU）

功能：控制传感器采集、处理数据（如滤波、压缩）、管理通信协议。

例如：

低功耗 MCU：Texas Instruments MSP430、Nordic nRF52 系列。

高性能 MCU：ARM Cortex-M4（支持复杂算法）。

无线通信模块

功能：与其他节点或汇聚节点通信，支持无线协议。

例如：

短距离：ZigBee（2.4GHz）、Bluetooth Low Energy（BLE）。

长距离：LoRa、NB-IoT（需蜂窝网络支持）。

电源模块

功能：为节点供电，需优化能量使用以延长寿命。

例如：

电池：锂电池、纽扣电池。

能量管理：动态电压调节（DVS）、休眠模式（仅唤醒时工作）。

存储模块

功能：缓存传感器数据（尤其在通信中断时）。

例如：

闪存：存储历史数据。

EEPROM：保存节点配置参数。

可选扩展模块

例如：

定位模块（如 GPS）

用途：记录节点位置（适用于移动或广域部署）。

代价：增加功耗，需权衡精度与能耗。

能量采集模块

技术：太阳能板、振动能量采集器（压电材料）、射频能量收集。

应用场景：野外监测、无电池更换需求的长期部署。

数据流向过程：

- ①感知阶段：传感器模块采集环境原始数据，并通过 ADC 转换为数字信号。
- ②处理阶段：MCU 对数据进行滤波、压缩（舍去异常值、冗余数据等）。
- ③存储/传输阶段：数据暂存于闪存，随后通过无线模块发送至相邻节点或汇聚节点。
- ④能量管理：若通信失败，MCU 会让设备进入低功耗模式，等条件合适再醒来工作以节省电量。

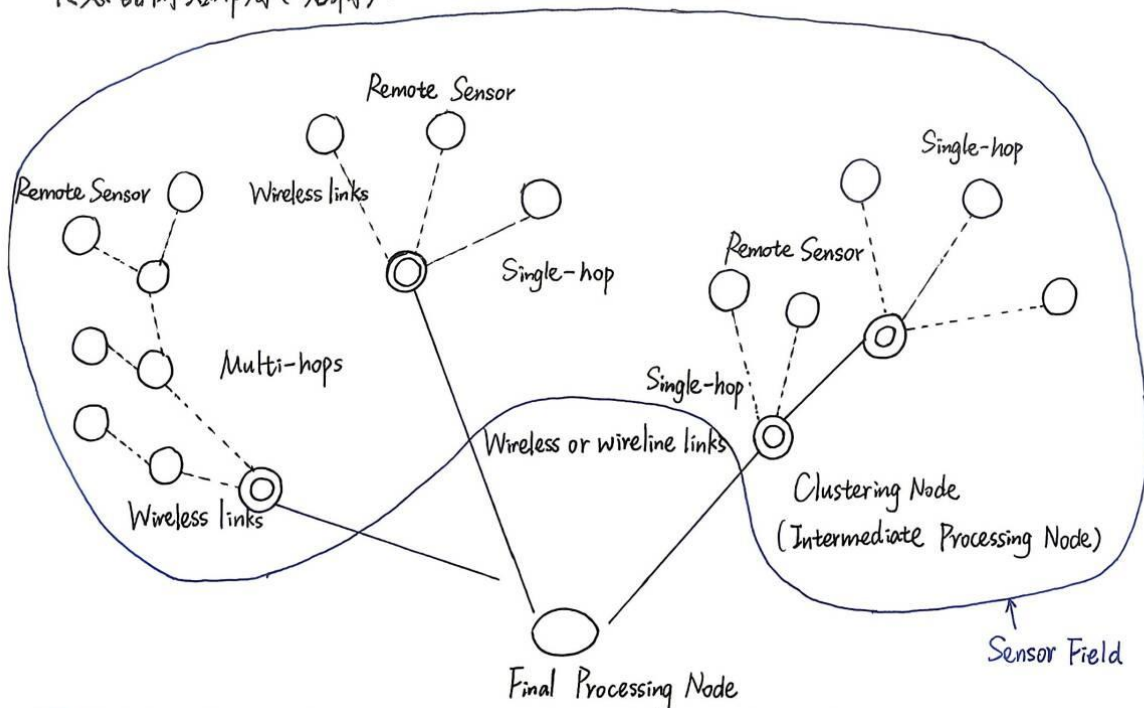
对比不同模块的能耗占比：

模块	能耗占比	节能设计
传感器模块	10%-30%	间歇性采样（如每分钟唤醒一次）
处理器（MCU）	20%-40%	动态降频（根据负载调整时钟）
无线通信模块	50%-70%	短时高速传输、长休眠周期
其他（存储 / 扩展）	<5%	低功耗模式

第二讲

WSN 的组成

传感器网络布局（结构）：



Wireless Sensor Network

终端节点是数据采集的源头, 配备传感器, 感知和采集环境数据发送至汇聚节点

汇聚节点负责收集终端节点数据并上传至外部系统, 是网络的数据汇聚点

簇首节点负责管理簇内节点, 协调通信, 优化数据传输路径, 降低能耗

中继节点用于多跳网络, 接力传输数据, 扩大网络覆盖范围, 增强鲁棒性

单跳数据传输指终端节点直接与汇聚节点通信, 传输路径短但能耗高, 适用于近距离简单场景

多跳数据传输通过多个节点接力传输数据, 可降低单节点能耗以提高网络寿命, 适用于长距离、复杂环境

终端节点

是部署在监测区域内的底层设备, 通常由低功耗硬件构成, **直接与环境交互**, 完成数据感知和传输任务。

主要功能:

数据采集——通过传感器模块获取物理环境信息。

数据预处理——对原始数据进行滤波、压缩或简单计算(如平均值、阈值判断), 以减少传输数据量。

数据传输——将处理后的数据通过无线通信模块发送至其他节点(如中继节点或簇头)或直接传至汇聚节点。

接收指令——响应来自汇聚节点或其他管理节点的控制命令(如调整采样频率、进入休眠模式)。

角色职责:

在分层网络(如分簇结构)中, 终端节点通常作为簇成员, 仅与簇头通信, 不参与路由决策。

在平面网络中, 终端节点可能需承担简单的中继任务, 但以数据采集为核心职能。

关键特点:

低功耗——通过休眠-唤醒机制、动态功率调整等技术最大限度节能。

大规模部署——数量庞大, 分布广泛, 支持高密度监测。

资源受限——计算能力、存储空间和能量有限, 需轻量化算法设计。

自组织性——支持自动加入网络、自适应拓扑变化(**如节点失效**)。

汇聚节点

是 WSN 中**唯一与外部网络**(如互联网、云平台或用户终端) **直接通信**的节点。

主要功能:

数据收集——接收来自所有传感器节点的监测数据(如温度、湿度、位置等)。

数据聚合——对原始数据进行融合、压缩或去冗余处理, 降低传输负载。

协议转换——将 WSN 内部通信协议(如 ZigBee、LoRa)转换为外部网络协议(如 TCP/IP、HTTP)。

角色职责:

通信枢纽——所有传感器节点的数据最终通过单跳或多跳路由传输至汇聚节点。

能量中心——通常汇聚节点能量供应更充足(如外接电源), 而普通传感器节点依赖有限电池。

用户接口——用户通过汇聚节点访问网络数据或下发指令(如调整采样频率)。

部署方式:

①固定式 Sink: 位置固定(如基站、塔台), 适合静态网络。

优点: 稳定性高, 易于维护。

缺点: 若节点远离 Sink, 多跳通信可能导致“能量空洞”。

Wireless Sensor Network

能量空洞：在固定式 Sink 的 WSN 中，靠近 Sink 的一圈节点因承担大量转发任务（不只要传自己的数据，还要转发外围所有节点的数据）而过早耗尽能量，导致外围节点无法向 Sink 传输数据，从而在 Sink 周围形成的失效区域。

②**移动式 Sink：**通过无人机、机器人或车载设备动态移动收集数据。

优点：均衡网络能耗，避免热点问题；可覆盖更大区域。

缺点：需复杂路由协议支持（如预测移动路径——节点要知道 Sink 会往哪走，提前做好传输路径，不然数据还没发完，Sink 就跑了、时间同步——Sink 飞到节点附近时，节点要能及时和它建立连接，不然错过窗口就收不到数据了）。

终端节点与汇聚节点的对比

特性	终端节点	汇聚节点
功能	数据采集与初步处理	数据聚合、协议转换、网络管理
能量供应	依赖电池，能量受限	外接电源，能量充足
通信范围	短距离（如 ZigBee 覆盖 10-100 米）	长距离（如通过 4G/5G 连接互联网）
部署数量	大规模（成百上千）	少量（1 个或多个）
网络层级	底层（数据源）	顶层（数据出口）

簇首节点（Cluster Head, CH）

是 WSN 分层结构中的关键角色，负责管理一个簇内的多个终端节点，并协调簇内外的数据通信。

主要功能：

数据聚合——接收簇内终端节点的原始数据，进行去冗余、压缩或融合处理（如计算平均值、提取特征值），减少传输数据量。

数据转发——将聚合后的数据通过多跳通信发送至汇聚节点或其他簇首节点。

资源管理——分配簇内节点的通信时隙（如 TDMA 调度）、监控节点状态（如能量水平）、协调休眠机制以优化能耗。

角色职责：

在分簇网络架构（如 LEACH、PEGASIS 协议）中，簇首节点扮演以下核心角色：

本地控制中心——管理簇内终端节点的数据采集与传输任务；维护簇内拓扑，处理节点加入/退出事件。

能耗均衡枢纽——通过轮换机制（如周期性选举新簇首）避免单个节点因频繁转发而过快耗尽能量；通常由能量较高的节点担任，以延长网络生命周期。

簇首节点的选择与轮换机制：

①选择标准：

剩余能量——优先选择电池电量高的节点。

地理位置——选择靠近簇中心或汇聚节点的位置，减少传输距离。

计算能力——需支持数据聚合算法（如压缩、加密）。

②轮换策略：

随机轮换（如 LEACH 协议）——每轮随机选举新簇首，均衡负载。

能量阈值触发——当簇首能量低于阈值时，重新选举。

固定周期轮换——按预设时间间隔切换簇首。

优点：减少终端节点的直接通信距离，降低能耗；通过数据聚合大幅减少网络流量；提升网络可扩展性，适合大规模部署。

缺点：簇首节点因频繁通信可能成为能耗“热点”；轮换机制可能引入额外控制开销；簇首选举算法复杂度影响实时性。

Wireless Sensor Network

簇首节点与终端节点、汇聚节点的对比

特性	簇首节点	终端节点	汇聚节点
功能	数据聚合、簇内管理、多跳转发	数据采集、简单中继	全网数据收集、协议转换、外部通信
能量消耗	较高（需处理数据并转发）	低（以采集为主）	最高（外接电源支持）
部署密度	少量（占节点总数 5%-20%）	大规模（80%-95%）	极少量（1 个或多个）
网络层级	中间层（簇内与簇间协调）	底层（数据源）	顶层（数据出口）

中继节点

是网络中专门负责转发数据的节点，通常不具备传感器节点的数据感知功能。其核心任务是在源节点（如传感器节点）和目标节点（如汇聚节点）之间建立通信链路，帮助完成数据的多跳传输。

中继节点与普通节点的对比

特性	中继节点	普通传感器节点
主要功能	数据转发、路由优化	数据感知、采集与初步处理
能耗	以通信能耗为主	感知、计算、通信综合能耗
部署密度	可能低于普通节点	通常密集部署
硬件能力	可能具备更强的通信模块或能量储备	通常资源受限（能量、计算能力）

WSN 的网络结构

单跳与多跳网络结构

单跳网络结构简单，节点直接与汇聚节点通信（所有节点均位于汇聚节点的直接通信范围内），无需通过任何中继节点转发数据。适合网络规模较小、节点分布密集且距离汇聚节点较近的场景，但扩展性差。

多跳网络结构复杂，数据经多跳传输至汇聚节点，可覆盖大范围，适应复杂环境，但需复杂路由算法。

①单跳数据传输（Single-hop Data Transmission）是指数据从源节点（如传感器节点）直接发送到目标节点（如汇聚节点），仅通过一次无线通信完成从源到目的地的传输，中间不经过任何中继节点转发的通信方式。

适用场景：

源节点与目标节点之间距离较近（在有效通信范围内），且信道质量良好（无遮挡、干扰小）。

②多跳数据传输（Multi-hop Data Transmission）是指数据从源节点（如传感器节点）到目标节点（如汇聚节点）的传输过程中，通过多个中继节点逐跳转发的通信方式。

适用场景：

大规模、长距离的复杂环境。

特性	单跳网络结构	多跳网络结构
通信路径	直接通信，路径单一	通过多个中继节点逐跳转发，路径复杂
能耗	远距离时能耗高 （能耗与距离的平方或更高次方成正比）	每跳距离短，总体能耗可能更低 （但中继节点自身需额外能耗）
延迟	低延迟（仅一次传输）	较高延迟（逐跳处理与转发）
可靠性	依赖信道质量，远距离易丢包	可通过冗余路径或中继节点规避信道问题
网络拓扑	结构简单，无需路由协议	需复杂路由协议支持（如 AODV、LEACH）
覆盖范围	较小（受限于汇聚节点的通信范围）	较大（通过中继节点扩展覆盖范围）

平面与分层网络结构

平面结构中所有节点地位平等，直接参与通信，易于部署，但易出现通信冲突，管理复杂。

分层结构将节点分层管理，簇首节点协调簇内通信，简化管理，提高效率，但簇首节点能耗高。

Wireless Sensor Network

①平面结构（Flat Architecture）是一种去中心化的网络拓扑组织形式，其核心特征是网络中所有节点的功能与地位平等，逻辑上处于同一层级，无固定角色分工，每个节点既可以是数据生产者，也可以是数据转发者，共同参与路由发现和数据传输（数据通过节点之间的协作和多跳通信完成传输）。

关键特点：

对等性——节点功能相同，无主从关系。

自组织性——网络通过分布式协议动态构建路由路径。

多跳传输——数据通过相邻节点逐跳转发至汇聚节点。

②分层结构（Hierarchical Architecture）是一种层次化的网络拓扑组织形式，其核心特征是将网络中的节点划分为不同的层级，每个层级承担特定的功能。分层结构通过引入簇首节点或中继节点，优化网络资源的使用，提高能量效率和可扩展性。

关键特点：

层次化——节点按功能划分为不同层级。

分工明确——网络被划分为多个簇，每个簇由一个簇首节点和若干普通节点组成。普通节点负责采集环境数据并发送给簇首节点，簇首节点接收普通节点的数据并进行聚合处理后转发至汇聚节点，汇聚节点最终接收所有簇首节点的数据。

能量高效——通过数据聚合减少冗余传输，延长网络寿命。

平面结构与分层结构的对比

特性	平面结构	分层结构（如分簇结构）
节点角色	所有节点功能平等	分为簇首节点和普通节点，簇首负责数据聚合与转发
路由复杂度	高（需动态路由协议）	低（路由路径由簇首管理）
能量效率	低（所有节点均参与转发）	高（簇首集中处理数据，普通节点能耗低）
扩展性	差（适合小规模网络）	好（适合大规模网络）

网络分类细化

静止网络与移动网络：确定性与非确定性网络；单汇聚与多汇聚节点网络；同构与异构网络...

①静止网络&移动网络

静止网络指节点位置固定的网络。

关键特点：

传感器节点部署后位置不再移动，网络拓扑结构保持稳定而无需频繁调整。

移动网络指节点可以移动的网络。

关键特点：

传感器节点可以在网络中移动，网络拓扑结构随节点移动而变化（动态拓扑）。

②确定性网络&非确定性网络

确定性网络指节点位置和通信路径预先规划的网络。

关键特点：

节点位置和通信路径预先经过精心规划和设计，部署后位置与路径通常保持固定，路由稳定且协议简单。

非确定性网络指节点位置和通信路径不预先规划的网络。

关键特点：

节点位置随机部署且会动态变化，节点之间的通信路径根据实时情况动态确定，路由协议需具备自适应性。

③静止汇聚节点网络&移动汇聚节点网络

静止汇聚节点网络指汇聚节点位置固定的网络。

关键特点：

汇聚节点部署后位置不再移动，传感器节点将数据发送到固定的汇聚节点，网络拓扑结构保持稳定而无需频繁调整。（传感器节点可以是静止的或移动的）

移动汇聚节点网络指汇聚节点可以移动的网络。

关键特点：

汇聚节点可以在网络中移动，网络拓扑结构随汇聚节点移动而变化（动态拓扑）。

④**单汇聚节点网络&多汇聚节点网络**

单汇聚节点网络指网络中只有一个汇聚节点负责收集所有传感器节点数据的网络。

关键特点：

网络中只有唯一一个汇聚节点，所有传感器节点将数据发送到该汇聚节点。

多汇聚节点网络指网络中存在多个汇聚节点共同负责收集传感器节点数据的网络。

关键特点：

网络中有多个汇聚节点，数据收集任务由它们共同承担。

⑤**同构网络&异构网络**

同构网络指所有传感器节点具有相同硬件和软件配置的网络。

关键特点：

所有传感器节点的软、硬件配置都相同，节点之间具有一致的功能和性能，网络管理相对简单。

异构网络指网络中存在不同配置、不同功能、不同性能的传感器节点的网络。

关键特点：

节点的硬件或软件配置不完全相同，不同节点可能具有不同的计算、存储、通信、能量供应性能，节点之间具有明显分工，网络中可能存在普通传感器节点、簇首节点或汇聚节点等不同角色，网络结构更灵活。

节点部署（Node Deployment）

是指将传感器节点按照特定策略和目标布置在监测区域内的过程，是 WSN 设计和应用的基础环节，直接影响网络的可靠性和寿命。

其目的是通过合理规划传感器节点的位置、密度和拓扑结构，实现网络覆盖（确保目标区域内所需位置被传感器覆盖）、连通可靠（节点之间形成稳定的通信链路，数据可传输至汇聚节点）、资源优化（平衡能量消耗、硬件成本和网络性能）。

节点部署的常见策略对比

部署策略	描述	适用场景	优缺点
确定性部署	节点位置由人工精确规划（如网格、环形等规则布局）。	环境可控的小范围监测（如工厂、实验室）	优点：覆盖均匀，连通性可控。 缺点：成本高，不适合复杂地形。
随机部署	节点通过随机抛洒（如飞机空投）分散在监测区域。	大规模或危险环境（如战场、灾区）	优点：快速部署，成本低。 缺点：可能产生覆盖盲区，需冗余节点补充。
混合部署	结合确定性与随机部署（如关键区域人工布设，其他区域随机抛洒）。	复杂地形或重点监测区域	优点：平衡成本与性能。 缺点：管理复杂度高。
移动节点辅助部署	部署少量可移动节点（如无人机、机器人），动态填补覆盖漏洞或优化网络拓扑。	动态环境或需后期调整的场景	优点：灵活修复网络。 缺点：移动节点成本高，能耗大。

节点部署的考虑因素

①**覆盖要求：**

Wireless Sensor Network

区域覆盖：节点感知范围需覆盖整个目标区域（如温度监测需全覆盖）。

点覆盖：仅需覆盖特定关键点（如火灾监测重点保护区域）。

②连通性保障：

节点通信半径需重叠，形成多跳路径，避免网络分裂。

③能量效率：

部署密度需平衡：过密会增加能耗和干扰，过疏会导致覆盖不足。

④环境约束：

地形障碍（如山体、建筑）、气候条件（如风雨干扰）影响节点布置。

⑤成本控制：

在满足性能前提下，尽量减少节点数量和硬件成本。

节点部署的挑战

①覆盖与连通的权衡：

高密度部署可提高覆盖，但会增加能耗和信号干扰。

②动态环境适应：

节点失效或环境变化（如植被生长）需动态调整部署。

③能量空洞问题：

靠近汇聚节点的中继节点因频繁转发数据而能量耗尽，形成“能量空洞”。

④三维空间部署：

水下、空中等三维场景的部署复杂度远高于二维平面。

典型应用场景的部署策略

智能农业：

网格化部署——在农田中按规则网格布置节点，均匀监测温湿度。

城市交通监测：

沿道路部署——节点沿街道布局，优化车流数据采集。

环境监测（如森林火灾）：

随机部署——通过无人机抛洒节点，快速覆盖大面积区域。

冗余设计——增加节点密度以应对部分节点失效。

军事监控：

混合部署——重点区域人工布设高精度节点，其他区域随机抛洒低成本节点。

例子.

大鸭岛（Great Duck Island）的 WSN 部署是一个经典的案例，展示了其在环境监测中的应用。该项目旨在监测海燕的栖息地环境。

为了研究海燕的栖息环境，研究人员需要长期监测岛上的温度、湿度、光照等环境参数。传统的人工监测方法成本高、效率低，且可能干扰海燕的生活。因此，研究团队决定部署 WSN，实现自动化、连续的环境监测。网络部署如下：

（1）节点部署

部署了传感器节点和网关节点这两种类型的节点，传感器节点被部署在海燕的巢穴附近，负责采集温度、湿度、光照等环境数据，网关节点（**汇聚节点**）被部署在岛上的高点以确保通信覆盖，负责收集传感器节点的数据，并通过卫星或无线电将数据传输到研究中心。

（2）网络拓扑

其一，网络采用分簇结构，每个簇选择一个簇头负责数据聚合和转发。其二，传感器节点通过多跳方式将数据传输到网关节点，减少单跳通信的能量消耗。

部署完成后，网络拓扑可能会发生变化，主要原因包括：

①节点位置变动（Position）

②可达性变化（Reachability）

主要由信号干扰（jamming）、环境噪声（noise）、移动障碍物（moving obstacles）等因素导致

③节点剩余能量变化（Available energy）

④节点故障（Malfunctioning）

网络拓扑变动

是指网络中节点之间的连接关系或通信路径发生变化。以下是导致网络拓扑变动的主要原因：

①节点移动

若网络中存在移动节点，则节点的位置变化会导致通信链路的中断或重建。

例如：

在目标跟踪或车辆网络中，节点的移动性会频繁改变网络拓扑。

②节点故障

能量耗尽：传感器节点能量耗尽后停止工作，导致其与其他节点的连接中断。

硬件故障：节点硬件损坏或软件崩溃，导致其退出网络。

环境干扰：恶劣环境可能导致节点失效。

③通信链路变化

信号衰减：由于距离增加或障碍物阻挡，节点之间的通信链路可能中断。

无线干扰：其他无线设备或环境噪声可能导致通信链路质量下降或中断。

多径效应：无线信号反射、折射等可能导致链路不稳定。

④能量管理策略

睡眠调度：为节省能量，节点可能进入睡眠模式，导致其暂时退出网络拓扑。

动态功率调整：节点根据通信需求调整发射功率，可能影响其通信范围。

⑤网络扩展或缩减

新增节点：新节点加入网络会改变现有拓扑结构。

节点移除：节点被移除或主动退出网络会导致拓扑变化。

⑥环境变化

物理环境变化：如建筑物倒塌、树木生长等可能阻断节点之间的通信。

天气条件：恶劣天气可能影响无线信号传播。

⑦路由协议动态调整

自适应路由：某些路由协议会根据网络状态动态调整通信路径，导致拓扑变化。

负载均衡：为均衡网络负载，节点可能选择不同的父节点或汇聚节点。

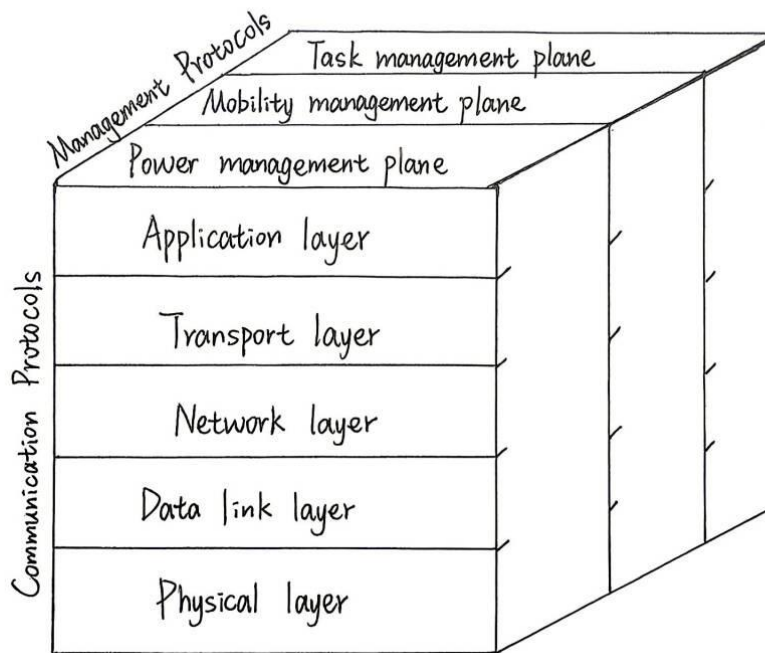
⑧攻击或安全事件

恶意攻击：如拒绝服务攻击（DoS）可能导致节点无法正常工作。

节点被捕获：攻击者捕获节点并篡改其行为，导致拓扑异常。

WSN 协议栈架构与功能

传感器网络通用协议栈:



应用层: 提供面向用户的各种传感器网络应用,如环境监测、智能家居等,满足用户多样化需求

传输层: 提供可靠的数据传输服务,确保数据完整准确到达目的地,是网络传输保障

网络层: 为数据提供路由,依据网络拓扑和节点状态选择最优路径,确保数据高效传输,是网络核心

数据链路层: 实现数据流复用、帧发送与接收、媒体接入与差错控制,保障数据可靠传输,提高网络效率

物理层: 负责信号在物理媒体上的传输,包括频率选择、调制解调等,确保信号稳定传输,是网络基础

WSN 与现有无线网络 (如 Wi-Fi、蜂窝网络等) 的区别

与无线自组网相比, WSN 节点数量更多、分布更密集,且节点能量、处理能力等有限,需优化设计。例如,无线自组网节点通常有持续能量供给,而 WSN 节点需考虑能量优化,以延长网络寿命。另外,传感器节点能量、处理能力、存储能力和通信能力有限,需设计低能耗、高效率协议。例如,采用节能通信协议和数据压缩算法,可降低节点能耗,提高节点性能。WSN 网络拓扑易受环境影响,节点易出现故障,需动态调整拓扑,以保持网络稳定运行。例如,在野外监测场景中,受天气等环境因素影响,节点可能出现故障,需动态调整网络结构。

①设计目标

WSN:

数据为中心: 专注于数据采集和环境监测。

低功耗: 节点通常由电池供电,需长时间运行。

自组织: 节点自组织成网络,无需基础设施支持。

现有无线网络:

Wireless Sensor Network

用户为中心：专注于为用户提供高速数据传输和通信服务。

高带宽：强调高数据传输速率和低延迟。

基础设施依赖：依赖基站、路由器等基础设施。

②网络规模与密度

WSN：

大规模部署：通常包含成百上千个节点。

高密度：节点密集部署以覆盖监测区域。

现有无线网络：

规模较小：通常覆盖有限区域（如家庭、办公室）。

密度较低：设备数量相对较少。

③节点能力

WSN：

资源受限：节点计算能力、存储能力和能量有限。

专用性强：节点通常针对特定任务设计。

现有无线网络：

资源丰富：设备（如手机、笔记本电脑）具有强大的计算和存储能力。

通用性强：支持多种应用和服务。

④通信模式

WSN：

多跳通信：数据通过多个节点中继传输到汇聚节点。

低数据率：通常传输少量数据（如传感器读数）。

现有无线网络：

单跳通信：设备直接与基站或路由器通信。

高数据率：支持高速数据传输（如视频流、文件下载）。

第三讲

传感器节点的限制

主要来自于**电源能量有限、通信能力有限、计算和存储能力有限**三方面。

能量限制与管理

传感器节点能量采集来源多样，但采集效率受限。能量消耗主要集中在通信、计算和感知模块，其中通信模块能耗占比最高，可达 70%以上。节点在 Idle（空闲）状态下能耗较高，主要原因是部分模块未关闭，仍处于低功耗监听状态，以等待数据传输指令，导致持续消耗能量。另外，节点在休眠状态下，由于部分硬件电路无法完全关闭（如时钟电路需持续运行以维持休眠计时，确保节点能按时唤醒），所以仍会产生能耗。通过优化硬件设计和采用更先进的低功耗芯片，可降低休眠能耗，延长节点寿命。

传感器节点的能量供应有限是一个核心限制因素，直接影响网络的性能和寿命。

①能量来源有限

依赖电池供电：大多数传感器节点依靠电池供电，电池容量有限，且能量密度较低。

难以补充能量：节点常部署在偏远、危险或难以接近的环境中，更换电池或充电几乎不可行。

能量收集技术受限：虽然太阳能、风能、振动能等能量收集技术可用于补充能量，但受环境条件限制且不稳定。

②能量消耗的主要来源

数据采集：传感器模块在采集环境数据时会消耗能量。而且采样频率越高，数据采集的能耗越大，通常需要根据应用需求优化采样频率。

数据处理：节点需要对采集的数据进行初步处理（如滤波、压缩、融合），这会消耗计算能量。而且数据融合、加密等复杂算法会显著增加能量消耗。另外，节点可能需要将数据暂存在本

地，存储操作也耗能。

无线通信：通信模块（如射频模块）是能量消耗的主要来源，尤其是在长距离传输或高频率通信时。发送数据时，射频模块需要将电信号转换成电磁波并发射出去，这一过程消耗大量能量。发射功率与通信距离的平方或更高次方成正比，长距离通信会显著增加能耗。而且帧头、校验码、同步信号等协议开销的添加所造成的额外数据的发送也会增加能耗。接收数据时，射频模块需要捕获电磁波并将其转换为电信号，这一过程的能耗低于发送数据的，但仍不可忽视。另外，节点在未发送或接收数据时，可能处于空闲侦听状态以等待可能的通信请求，这会持续消耗能量。数据包可能受到冲突或干扰，需要重传，也会增加额外的能量消耗。发送数据的能耗远高于接收数据：

节点发送数据的能量消耗：

$$E_{tx} = E_{elec} \times k + E_{amp} \times k \times d^2$$

其中， E_{elec} 为电子设备能耗， E_{amp} 为放大器能耗， k 为数据量， d 为传输距离。

发送数据的能量消耗 E_{tx} 可以通过以下公式计算：

$$E_{tx} = P_{tx} \times T_{tx}$$

其中 P_{tx} 是发送功率（单位为瓦特，W）， T_{tx} 是发送数据所需的时间（单位为秒，s）。

(1) 发送功率 P_{tx}

发送功率取决于通信距离、硬件效率和环境条件，通常可以通过以下公式估算：

$$P_{tx} = P_{elec} + P_{amp} \times d^n$$

其中 P_{elec} 是射频模块的电路功耗（通常为常数，例如 50 mW）， P_{amp} 是功率放大器的功耗系数

（通常为常数，例如 100 pJ/bit/m²）， d 是通信距离（单位为米，m）， n 是路径损耗指数（通常为 2 到 4，取决于环境）。

(2) 发送时间 T_{tx}

发送时间取决于数据量 D （单位为比特，bit）和传输速率 R （单位为比特/秒，bps）：

$$T_{tx} = \frac{D}{R}$$

(3) 发送能量消耗的完整公式

$$E_{tx} = (P_{elec} + P_{amp} \times d^n) \times \frac{D}{R}$$

节点接收数据的能量消耗：

$$E_{rx} = E_{elec} \times k$$

接收能耗主要由电子设备决定，与数据量成正比，与传输距离无关。

接收数据的能量消耗 E_{rx} 可以通过以下公式计算：

$$E_{rx} = P_{rx} \times T_{rx}$$

其中 P_{rx} 是接收功率（单位为瓦特，W）， T_{rx} 是接收数据所需的时间（单位为秒，s）。

(1) 接收功率 P_{rx}

接收功率通常是一个常数，取决于射频模块的电路功耗：

$$P_{rx} = P_{elec}$$

其中 P_{elec} 是射频模块的电路功耗（通常与发送功率中的 P_{elec} 相同）。

(2) 接收时间 T_{rx}

接收时间与发送时间相同，取决于数据量 D 和传输速率 R ：

$$T_{rx} = \frac{D}{R}$$

(3) 接收能量消耗的完整公式

$$E_{rx} = P_{elec} \times \frac{D}{R}$$

休眠与唤醒：节点在休眠和唤醒状态之间切换时也会消耗能量，频繁切换会加剧能量消耗。

③能量消耗的不均衡性

多跳通信中的能量消耗：在基于多跳通信的 WSN 中，靠近汇聚节点的节点需要转发更多数据，能量消耗更快，可能导致网络分区或失效。

热点问题：某些节点因承担更多任务（如数据融合、路由）而成为能量消耗的热点，加速能量耗尽。

能量的限制对网络性能的影响：

①网络寿命缩短

能量耗尽会导致节点失效，进而影响网络的覆盖范围和数据传输能力。

②通信质量下降

能量不足可能导致节点无法完成数据发送或接收，增加丢包率和通信延迟。

③网络拓扑变化

节点失效会改变网络拓扑，增加路由维护和数据传输的复杂性。

能量管理的挑战与应对策略：

①能量高效利用

需要在数据采集、处理和通信之间找到平衡，避免不必要的能量消耗。在硬件层面采用低功耗的处理器、传感器和通信模块，从源头降低能耗；在节点本地对数据进行融合与压缩，减少需要传输的数据量；减少空闲侦听。

②动态能量管理

根据网络负载和环境条件动态调整节点的工作模式，优化能量使用效率。让节点在不工作时进入休眠状态，减少能量消耗；利用核反应能、太阳能、风能、振动能、热能（热电效应）、声能等为节点动态补充能量（核反应能是不可再生能源，其他都是可再生能源，功率密度越高，说明这种能量源越适合给 WSN 节点供电，但是能量采集技术受环境影响很大，比如声能的功率密度会随噪音分贝数提升而显著增加，这也会带来**不稳定**）。

能量采集技术：

通过从周围环境中捕获能量并将其转换为电能，为节点提供持续的能源支持，延长节点的使用寿命并提高网络的可持续性。

太阳能采集——利用光伏效应，将太阳光能转换为电能，适用于田野、沙漠、建筑物外部等光照充足的户外环境，能量密度较高，但受天气和昼夜影响，也会增加节点体积。

振动能量采集——利用压电效应或电磁感应，将机械振动转换为电能，适用于工业设备、桥梁、交通工具等振动源丰富的环境，设备体积小，易于集成，但能量输出较低，适合低功耗节点，且对振动频率和幅度敏感。

热能采集——利用热电效应（如塞贝克效应），将温度梯度转换为电能，适用于工厂设备、人体表面、地热区域等存在温差的环境，无需运动部件，可靠性高，但能量转换效率较低，且需要较大的温差才能产生足够的电能。

射频能量采集——通过天线捕获环境中的 Wi-Fi、蓝牙、蜂窝信号等射频信号，并将其转换为电能，适用于射频信号密集的城市环境或室内场景，利用现有无线信号而无需额外能源，设备体积小，易于集成，但能量密度极低，通常只能为低功耗设备供电，且受信号强度和距离限制。

风能采集——利用小型风力发电机，将风能转换为电能，适用于山顶、海岸、草原等风力资源丰富的户外环境，可为较高功耗的节点供电，但设备体积较大，可能影响节点部署。

水流能量采集——利用微型水力发电机，将水流动能转换为电能，适用于河流、管道等水流稳定的环境，能量输出较高，但设备体积较大，部署复杂，且受水流速度和流量限制。

生物能量采集——利用生物燃料电池，将葡萄糖、微生物等生物质的化学能转换为电能，适

Wireless Sensor Network

用于土壤、植物、人体等生物环境，可应用于医疗或生物监测，但能量输出较低，且成本较高。

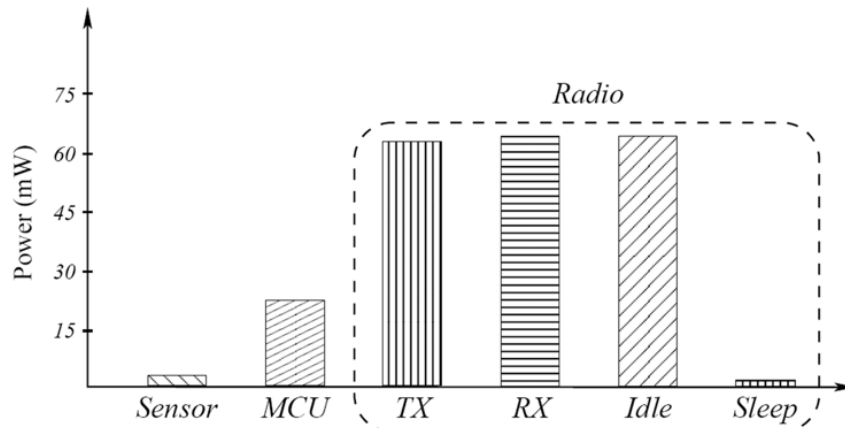
混合能量采集——结合多种能量采集技术，以提高能量供应的稳定性和可靠性，适用于能量来源多样化的复杂环境，但是设计复杂，成本较高。

③能量均衡

根据能量供应情况，通过优化路由协议和动态调整任务分配，避免部分节点过早耗尽能量。（**热点与空洞？**）采用分簇与层次化网络结构，将网络划分为多个区域，减少长距离通信的能量消耗。

④能量存储

使用超级电容器或可充电电池存储采集的能量，以应对能量供应的波动。



上图是 WSN 节点各模块的功耗对比柱状图，直观展示了不同模块的功耗水平，可以看出无线通信（Radio）是节点功耗的最大来源，远高于传感和计算模块。这也是为什么 WSN 里常说“通信功耗占总功耗的 70%以上”。值得注意的是，由于空闲侦听时节点即使没有收发数据，射频模块也一直开着，所以功耗几乎和收发数据一样高，而且休眠时功耗很少，所以 WSN 节能的核心思路之一是让节点尽可能多休眠，少处于 Idle 状态。

Idle（空闲）状态

是指节点处于开启但未进行任何有效通信或数据处理的状态。

节点在 **Idle 状态**下的能耗虽然低于发送或接收数据的状态，但仍然较高，常介于发送状态和休眠状态之间。主要原因如下：

①射频模块的静态功耗

节点的射频模块（如射频收发器）仍处于开启状态以保持对信道的监听，射频模块的电路（如振荡器、放大器、滤波器等）需要持续消耗能量来维持运行。另外，射频模块需要保持与网络中其他节点的时钟同步，这需要持续的能量供应。

②信道监听

节点需要持续监听信道，一方面为检测是否有数据包需要接收和转发；另一方面在某些基于竞争的 MAC 协议（CSMA/CA）中，通过信道监听进行冲突检测，避免多个节点同时发送产生数据碰撞，持续监听使射频模块长期处于活跃状态，造成较大能量消耗。

③协议开销

在某些协议（如 IEEE 802.15.4 的信标模式）中，节点需要定期（周期性地）监听信标帧、控制消息、路由更新来完成时间同步与网络配置。另外，在多跳转场景下，节点也要频繁监听信道，等待接收和转发数据包。长期保持信道监听会让射频模块处于活跃状态，大幅增加能耗。

④时钟运行

节点需要保持内部时钟的运行，以支持时间同步、状态切换和任务调度。另外，节点可能使用定时器来触发数据采集、状态切换等周期性任务，这些都会增加能耗。

⑤环境因素

在复杂的无线环境中，节点可能需要提高监听灵敏度以克服干扰和噪声，这会增加射频模块的能耗。另外，极端的环境条件（如高温、高湿度）可能导致硬件效率下降，增加能耗。

休眠（Sleep）状态

是一种低功耗模式，旨在最大限度地减少能量消耗。然而，即使在休眠状态下，节点仍然会产生一定的能耗，主要原因包括以下几个方面：

①静态功耗（漏电流）

在休眠状态下，节点的处理器、存储器和射频模块等半导体器件并未完全断电，而是处于低功耗模式，仍然存在漏电流。另外，部分电路（如时钟电路、定时器）需要保持运行以支持唤醒功能，这也会产生一定的能耗。

②时钟与定时器

节点在休眠状态下通常需要保持内部时钟的运行，以支持时间同步和定时唤醒功能。节点还可能使用定时器来触发周期性唤醒（如数据采集、状态切换）。

③唤醒机制

在某些低功耗 MAC 协议中，节点在休眠状态下需要定期唤醒并监听信道（低功耗监听-LPL），以检测是否有数据包需要接收。某些节点可能配备唤醒无线电这种专用低功耗监听模块。平时主射频深度休眠，仅由唤醒无线电持续监听唤醒信号；检测到信号后再唤醒主射频进行数据收发。其功耗虽低，但长期持续工作仍会产生一定能耗。

④存储器保持

节点需要持续的供电去保持存储器（如 RAM、Flash）中的数据不丢失，这需要持续供电以维持存储器的状态。另外，在进入休眠状态之前，节点可能需要保存当前的运行状态（如寄存器值、配置参数），这些操作也会消耗能量。

计算与存储能力限制

WSN 节点多采用微型嵌入式设备，其特点是价格低、功耗小，适合大规模部署。但受限于体积和成本，计算和存储能力有限，难以满足复杂算法和大量数据存储需求，难以处理复杂数据，需要优化算法和数据管理策略，采用高效数据压缩和存储策略，同时合理分配存储空间，优先存储关键数据，定期清理冗余数据，提高存储利用率，确保节点正常运行。

节点计算与存储能力受限的原因：

①能量供应的限制

大多数传感器节点依赖电池供电，能量有限且难以补充。虽然能量采集技术可以补充能量，但其供应不稳定且能量密度不高。而高性能的计算和存储单元通常能耗较高，会显著缩短节点的使用寿命。

②硬件尺寸的限制

传感器节点通常需要小型化以便于部署，采用高度集成的硬件设计以降低成本和功耗，这就限制了处理器和存储器的尺寸与扩展，从而导致其计算和存储能力受限。

③成本限制

WSN 通常需要部署大量节点，因此每个节点的成本必须尽可能低。设计节点时需要在性能和成本之间进行权衡，通常优先选择低功耗、低成本的硬件。而高性能的处理器和存储器会增加成本，不符合大规模部署的需求。

④应用需求的限制

多数传感器节点主要承担数据采集、简单处理和转发任务，对计算与存储性能要求不高。因此设计时通常优先考虑低功耗、低成本和长寿命，而不是高性能。

第四讲

物理层基础概念

物理层的定义

Wireless Sensor Network

物理层负责在无线介质上传输原始比特流，将数据转换为适合无线传输的信号，并处理传输过程中的信号调制与解调、编码与解码、频率选择等。

物理层为建立、维护和释放数据链路实体之间的二进制比特传输的物理连接，提供机械的、电气的、功能的和规程性的特性。它是网络协议栈的最底层，负责物理介质上的信号传输。

物理层的主要功能

其主要职责是确保数据能够在物理介质上可靠地传输，为上层协议提供稳定的传输服务，是整个网络通信的基础。

①为数据终端设备提供传送数据的通路

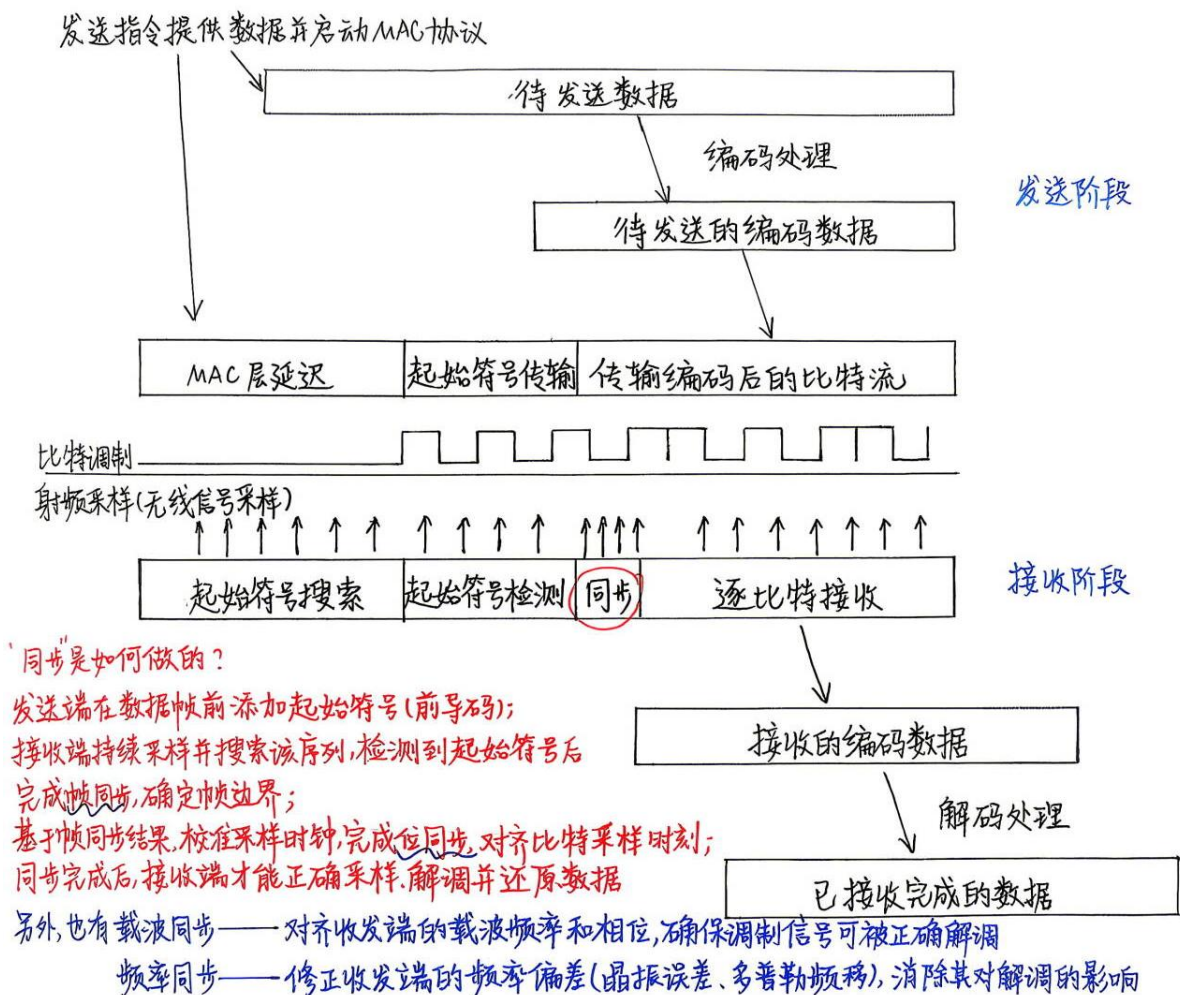
一次完整的数据传输包括激活物理连接、传送数据和终止物理连接，确保数据传输的完整性和可靠性。在此过程中，物理层通过**信号调制与解调**实现数字信号与传输所需模拟信号的转换与还原；通过**信道编码与解码**增加冗余位，保障数据传输的可靠性；同时借助**信号放大与滤波**增强信号覆盖能力、过滤噪声干扰，为数据通路提供稳定的物理支撑。

②传输数据

物理层要形成适合传输需要的实体为数据传输服务，并提供足够的带宽，以满足不同应用对数据传输速率的要求。为了适配复杂的传输环境，物理层会**合理选择频段、规避干扰并高效利用频谱资源**；同时通过**控制数据传输速率**，根据实时信道条件动态调整传输速率，平衡传输速率与可靠性。

③其它管理工作

如信道状态评估、能量检测等，这些功能有助于优化物理层的性能，提高数据传输效率和可靠性。此外，物理层还承担能量管理的职责，通过**调整发射功率和休眠模式**等方式**优化节点能量使用，延长节点寿命**，进一步提升网络整体的稳定性与续航能力。



无线通信

无线通信概述

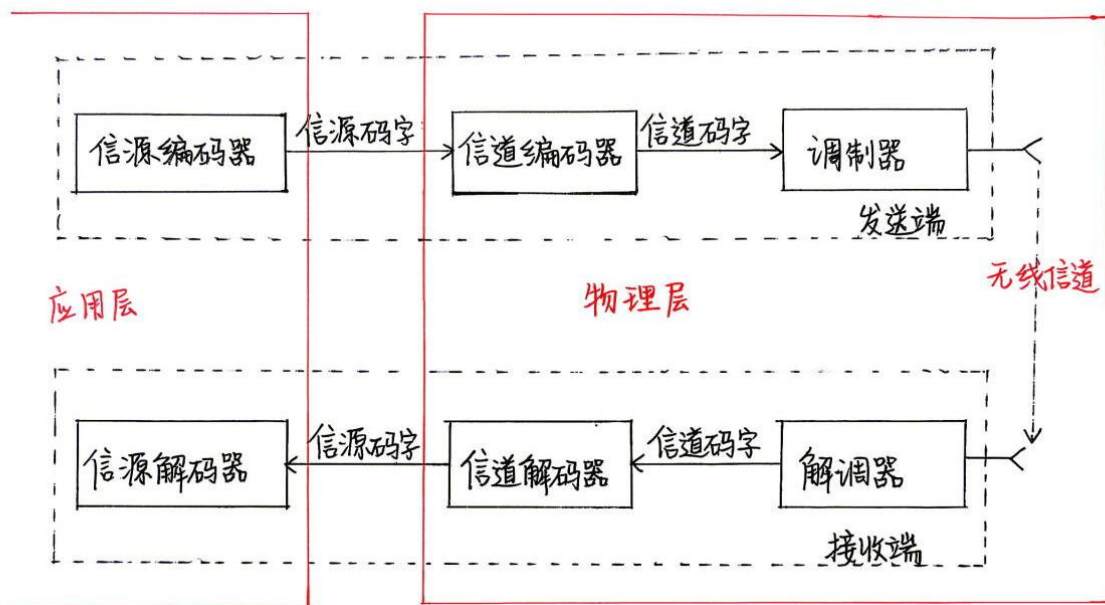
无线通信是一种通过电磁波在空气中传输信息的技术，无需电缆或光纤等物理连接，广泛应用于移动通信、卫星通信、无线网络等领域。

关键组成部分：

- ①发射器：将信息转换为电磁波信号并发送。
- ②接收器：接收电磁波信号并还原为原始信息。
- ③信道：电磁波传播的介质，通常是空气或真空。

主要技术：

- ①调制与解调：调制可以将信息加载到载波信号上，解调可以从载波信号中提取信息。
- ②多址技术：允许多个用户共享同一信道，常见方法包括 FDMA、TDMA、CDMA 和 OFDMA。
- ③编码与解码：编码可以增加冗余以提高抗干扰能力，解码可以还原原始信息并纠正错误。
- ④天线技术：用于发射和接收电磁波，其设计影响通信质量。



信源编码（数据压缩）

是通信系统中对原始信息进行编码的过程，旨在尽量保证信息完整性的同时，减少数据量（压缩、去冗余）以提高传输效率。在发送端，信源编码由应用层实现，它通过研究信源的统计特性，用一些信息位表示信源数据，生成信源码字。

要根据信源特性和应用需求选择合适的编码技术，常见技术如下：

①无损编码

压缩后可以完全恢复原始数据，适用于不允许信息丢失的场景。例如霍夫曼编码、算术编码、LZ77、LZ78 等。

②有损编码

压缩后会丢失部分信息，但通常是人眼或耳不易察觉的部分，适用于多媒体数据。例如 JPEG（图像）、MP3（音频）、MPEG（视频）等。

优势：提高传输效率，减少数据量，提升传输速度；节省存储空间，降低存储需求；提高频谱利用率，优化带宽使用。

挑战：某些编码算法计算复杂，增加系统负担；有损编码可能导致信息丢失，需权衡压缩率与质量；实时通信系统需快速编解码，对算法效率要求高。

信道编码（差错控制编码）

对信源码字进行再编码而生成信道码字，是通信系统中通过在数据中添加冗余信息来检测和纠正传输中的错误以提高传输可靠性的技术，旨在增强数据在传输过程中的抗干扰和纠错能力，减少无线信道差错对信源产生的影响。

优势：能有效应对噪声和干扰，提高可靠性；增强纠错能力，减少数据传输错误；在恶劣信道条件下保持稳定传输，能适应复杂环境。

挑战：引入冗余信息会占用额外带宽；某些编码算法计算复杂，会增加系统负担；编解码过程可能引入额外延迟，影响实时性。

交织

是一种按特定规则重排数据顺序以分散突发错误的技术，对经过信道编码的码字进行交织，以此抑制突发错误；信道编码与交织共同作用，能够帮助接收机识别比特位错误并启动重传，或直接纠正有限数量的错误比特位，提升传输可靠性与系统的抗干扰能力。

类型：

①块交织

将数据按矩阵排列，按行写入、按列读出，或反之。简单易实现，适用于固定长度数据。

②卷积交织

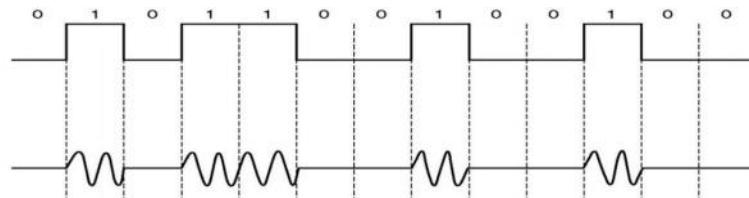
数据通过多个延迟线交织，延迟长度逐渐增加。适用于连续数据流，延迟较小。

③随机交织

按伪随机序列重新排列数据。性能较好，但复杂度较高。

调制

是通过改变载波信号的某些特性（如幅度、频率或相位），将基带信号的信息加载到载波上，生成适合信道传输的高频载波信号的过程，实现了比特流的波形变换，确保信号能够高效、可靠地通过传输介质。它可将基带信号搬移到更高频段，便于无线传输；可通过不同载波频率实现多路信号同时传输；高频信号抗干扰能力更强，适合长距离传输。



类型：

$m(t)$ 表示基带信号， $I(t)$ 表示同相分量， $Q(t)$ 表示正交分量。

①幅度调制（AM）

$$s(t) = [A + m(t)] \cos(2\pi f_c t)$$

通过改变载波信号的幅度来传递信息。简单但抗噪声能力较弱。

②频率调制（FM）

$$s(t) = A \cos(2\pi f_c t + 2\pi k_f \int m(t) dt)$$

通过改变载波信号的频率来传递信息。抗噪声能力较强，常用于广播。

③相位调制（PM）

$$s(t) = A \cos(2\pi f_c t + k_p m(t))$$

通过改变载波信号的相位来传递信息。抗噪声能力强，常用于数字通信。

④正交幅度调制（QAM）

在 4-QAM 图中，每个点的横坐标就是 $I(t)$ ，纵坐标就是 $Q(t)$ 。

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t)$$

Wireless Sensor Network

同时改变载波信号的幅度和相位，频谱效率高，适用于高速数据传输，广泛应用于现代通信系统。

⑤幅移键控（ASK）

通过改变载波信号的振幅来表示数字信号。

⑥频移键控（FSK）

通过改变载波信号的频率来表示数字信号。

⑦相移键控（PSK）

通过改变载波信号的相位来表示数字信号。

基带信号

是指未经调制的原始信号，通常包含需传输的信息，频率范围集中在零频附近。它是通信系统中最基本的信号形式，包含需传输的原始信息，频率较低，不适合直接通过无线信道传输。

类型：

①模拟基带信号

幅度和频率随时间连续变化的信号，如音频信号。

②数字基带信号

幅度和频率随时间离散变化（通常表示为高低电平）的信号，如二进制数据流。

处理：

①调制

将基带信号加载到高频载波上，生成适合无线传输的调制信号。例如 AM、FM、PM、QAM。

②编码

对基带信号进行编码，增加冗余信息以提高传输可靠性。例如 PCM、DPCM、DM。

③滤波

对基带信号进行滤波，去除噪声和干扰，提高信号质量。例如低通滤波器、带通滤波器。

载波信号

是一种高频电磁波（通常为正弦波或余弦波），用于在通信系统中承载信息信号。通过调制将低频信息信号加载到高频载波上，便于传输；高频信号抗干扰能力强，适合长距离传输；不同载波频率可实现多路信号同时传输。

载波信号可表示为：

$$C(t) = A \cos(2\pi f_c t + \phi)$$

其中 A 是振幅， f_c 是载波频率， ϕ 是相位。

交织与调制的关系：

两者结合可以显著提高系统的抗干扰能力和传输可靠性。

在调制前进行交织，通过重新排列数据顺序，分散突发错误，可以确保调制后的信号在传输中遇到突发错误时，错误不会集中在某一段数据中。解调后，通过解交织恢复原始顺序，使纠错码能有效纠正分散的错误。

交织后，将数字信息调制到模拟信号上，产生适合信道传输的波形，天线就可以通过电磁波的辐射将无线电波发送出去。

天线发送无线电波

基本原理：

天线将电信号转换为电磁波并辐射到空间中。这一过程基于麦克斯韦方程组，变化的电场和磁场相互激发，形成电磁波。

发送过程：

传感器节点生成需发送的电信号，通常为低频基带信号。基带信号通过调制加载到高频

载波上，生成适合无线传输的高频信号。高频信号经过功率放大器放大，以具备足够的发射功率。放大后的高频信号通过馈线送入天线。天线将高频电信号转换为电磁波并辐射到空间中（高频信号在天线中产生变化的电场和磁场，二者相互激发形成电磁波从天线向空间辐射）。

介质

是指电磁波传播的媒介或通道。无线信道使用的介质主要是自由空间（空气或真空），但电磁波在传播过程中会受到多种介质的影响。

①自由空间

是指没有障碍物和干扰的理想传播环境，如空气或真空。电磁波以直线传播，遵循自由空间路径损耗公式。信号强度随距离增加而衰减。应用于卫星通信、深空通信、无线局域网（Wi-Fi）等。

②大气层

是地球表面的气体层，电磁波在传播过程中会穿过不同密度和成分的大气层。具有折射、吸收、散射的特点——电磁波在大气层中传播时发生折射会导致传播路径弯曲；某些频率的电磁波会被大气层中的气体（如水蒸气、氧气）吸收，导致信号衰减；大气层中的微粒（如雨滴、雾、灰尘）会导致电磁波散射。应用于地面无线通信、卫星通信、气象雷达等。

③地面和建筑物

是电磁波传播过程中遇到的主要障碍物。具有反射、衍射、穿透的特点——电磁波遇到地面和建筑物表面时会发生反射，产生多径传播；电磁波可以发生衍射，绕过建筑物和地形的边缘，传播到障碍物后方；某些频率的电磁波能够穿透建筑物，但信号强度会减弱。应用于城市无线通信、室内无线通信、移动通信等。

④水体

包括海洋、湖泊、河流等，电磁波在水体中的传播特性与在空气中不同。具有高衰减、低频传播的特点——水对电磁波的吸收较强，尤其是高频信号，导致信号快速衰减；低频电磁波（如 ELF、VLF）在水体中传播较远。应用于水下通信、海洋探测、潜艇通信等。

⑤植被

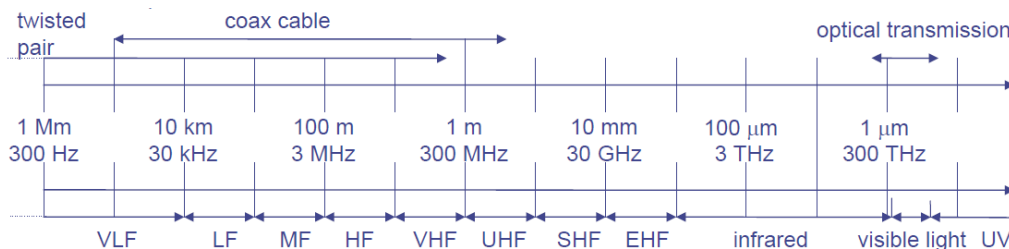
包括森林、草地、农作物等，电磁波在植被中的传播会受到叶片、枝干等的影响。植被会导致电磁波的散射和吸收，增加信号衰减。也存在多径效应，植被中的多径传播会导致信号衰落和时延扩展。应用于农业无线传感器网络、森林监测、环境监测等。

⑥电离层

是地球大气层的一部分，含有大量自由电子和离子，对电磁波的传播有显著影响。低频电磁波（如短波）可以被电离层反射，实现远距离传播；高频电磁波（如微波）会穿透电离层，但会受到一定程度的衰减。应用于短波通信、卫星通信、无线电广播等。

频段

是指用于传输无线信号的特定频率范围。不同的频段具有不同的传播特性和应用场景。



- VLF = Very Low Frequency
- LF = Low Frequency
- MF = Medium Frequency
- HF = High Frequency
- VHF = Very High Frequency

- UHF = Ultra High Frequency
- SHF = Super High Frequency
- EHF = Extra High Frequency
- UV = Ultraviolet Light

①低频段 (LF)

30kHz 到 300kHz。波长较长，穿透力强，能够绕过障碍物。传播距离远，适合地面波传播。应用于长波广播（如 AM 广播）、导航系统（如 LORAN）等。

②中频段 (MF)

300kHz 到 3MHz。地面波传播，白天主要依靠地面波，夜间可以利用天波（电离层反射）。传播距离中等。应用于中波广播（如 AM 广播）、航空导航等。

③高频段 (HF)

3MHz 到 30MHz。主要依靠天波传播，通过电离层反射实现远距离通信。受电离层变化影响较大，传播不稳定。应用于短波广播、远距离通信（如海事通信、军事通信）等。

④甚高频段 (VHF)

30MHz 到 300MHz，主要以直线传播（视距传播），受地形和障碍物影响较大。传播距离较短，但信号质量较好。应用于 FM 广播、电视广播、航空通信等。

⑤超高频段 (UHF)

300MHz 到 3GHz。主要以直线传播，穿透力较强，适合城市环境。传播距离较短，但带宽较大，适合高速数据传输。应用于电视广播、移动通信（如 4G LTE）、Wi-Fi 等。

⑥微波频段 (Microwave)

3GHz 到 30GHz。直线传播，受大气吸收和雨衰影响较大。高带宽，适合高速数据传输。应用于卫星通信、雷达系统、点对点微波通信等。

⑦毫米波频段 (Millimeter Wave)

30GHz 到 300GHz。波长极短，直线传播，穿透力差，易受障碍物阻挡。带宽极高，适合超高速数据传输。应用于 5G 移动通信、高频雷达、短距离高速无线通信等。

⑧红外和可见光频段

300GHz 到 790THz（红外）和 790THz 到 430THz（可见光）。直线传播，易受障碍物阻挡。高带宽，适合短距离高速通信。应用于红外通信（如遥控器）、可见光通信（如 Li-Fi）等。

解调

是从已调制的载波信号中恢复原始信息信号的过程。就是利用调制阶段建立的数据位与波形之间的一对一映射关系，将接收到的波形与预设波形进行匹配，从而还原出原始数据位的过程。通过天线接收已调制的射频信号；通过低噪声放大器（LNA）放大微弱信号，通过带通滤波器滤除带外噪声和干扰；使用本地振荡器和混频器将高频信号转换为中频或基带信号，便于处理；根据调制方式选择解调方法（AM 解调——包络检波或同步检波、FM 解调——鉴频器或相位锁定环（PLL）、PM 解调——相位检测器），提取原始信息信号；对解调后的信号进行滤波、放大与数字化处理（如 ADC 转换）以便进一步分析。

挑战：

①信号在传输中会受到噪声和干扰，需要设计高性能的滤波器和抗干扰算法。

②解调时需要与发射端的载波频率和相位同步（即载波同步），否则会导致解调失败或信号失真。

③信号经不同路径到达接收端，会产生时延和相位偏移（即多径效应），需要均衡技术和多径抑制算法。

④非线性失真：放大器、混频器等器件的非线性特性可能引入失真，需优化硬件设计和信号处理算法。

⑤复杂调制方式的解调：高阶调制（如 QAM、OFDM）的解调复杂度高，需高性能算法和硬件支持。

⑥无线环境的动态变化（如多普勒效应）要求解调系统能够快速适应，需实时调整参数以保持性能。

扩频 (SS)

Wireless Sensor Network

通过将信号的频谱扩展到一个更宽的频带上进行传输，以提高抗干扰、抗截获和多用户共存能力。扩频的基本原理是通过特定技术将原始信号的带宽扩展到远大于其原始带宽，使扩频后的信号在更宽的频带上传输。通常，发送端会使用伪随机序列（PN 码）对原始信号进行调制，从而实现频谱扩展；接收端则使用相同的伪随机序列进行解扩，恢复出原始信号。由于扩频信号占用的带宽更大，在接收端解扩后可以获得处理增益，从而提高系统的信噪比（SNR）和抗干扰能力。处理增益通常表示为：

$$G_p = \frac{B_{ss}}{B_{info}}$$

其中 B_{ss} 表示扩频后的带宽， B_{info} 表示原始信息信号的带宽。

分类：

①直接序列扩频（DSSS）

发送端将原始信号与伪随机序列相乘，扩展频谱；接收端用相同的伪随机序列解扩，恢复信号。抗干扰能力强，适合短距离通信（如 Wi-Fi、蓝牙）。应用于 GPS、无线局域网（WLAN）、军事通信。

②跳频扩频（FHSS）

载波频率按伪随机序列快速跳变；接收端同步跳频模式，解调信号。抗窄带干扰能力强，适合多用户环境。应用于蓝牙、军事通信、无线个域网（WPAN）。

图 1：扩频的优点与应用

优势：

- ①抗干扰能力强：扩频信号在更宽的频带上传输，窄带干扰影响较小。
- ②抗截获性高：信号功率谱密度低，难以被检测和截获。
- ③多用户共存：不同用户使用不同的伪随机序列，实现多用户通信。
- ④抗多径效应：扩频信号对多径干扰有较强抵抗力。
- ⑤高安全性：伪随机序列增加了信号破解难度。

应用：

①民用领域：

Wi-Fi——IEEE 802.11 标准使用 DSSS 和 OFDM。

蓝牙——采用 FHSS 技术。

GPS——使用 DSSS 技术实现精确定位。

②军事领域：

抗干扰和抗截获的通信系统。

③卫星通信：

用于卫星导航和通信系统。

无线信道传播特性

传输波形在信道中传播时，会受到多种因素的干扰，导致衰减或失真。无线信道的特性对信号传输质量有重要影响，无线信道的传播特性包括路径损耗、反射、折射、衍射和散射等。这些特性使得信号在传播过程中发生复杂的传播现象，影响信号的传输距离和质量。

由于反射、散射等，无线电通信不仅限于直接的视距传输。多径效应是指信号在传播过程中，经过不同的路径到达接收端，形成多个信号副本。这些信号副本在时间和相位上存在差异，会导致信号的叠加和干扰。多径效应会使信号的幅度和相位发生变化，产生信号的衰落现象。这种衰落现象对信号传输质量产生严重影响，可能导致信号失真和误码率增加。

信号的自干扰，与来自多路径的多个备份作用。在无线通信中，信号不仅会受到自身多径效应的干扰，还会受到其他因素的干扰，如噪声和第三方干扰。噪声与接收机灵敏度有关，取决于温度、环境和接收机结构等。噪声是无线信道中不可避免的干扰因素，它会降低信号的信噪比，影响信号的检测和解调。来自第三方的干扰包括带内信道干扰和相邻信道干

扰，这些干扰会增加信号的误码率，降低通信质量。

理想特例——自由空间传播

在理想的无障碍物环境中，电磁波以直线传播。信号强度随距离增加而衰减，遵循自由空间路径损耗公式。适用于卫星通信和深空通信。

四大基础现象

① 衰减 (Attenuation)

随着信号波在空中传播，信号强度随之减弱。衰减与在空中传播的距离成正比，会导致空中无线电波的路径损耗。

② 反射与折射 (Reflection and refraction)

当信号从一种介质进入到另一种介质，一部分波从交界面弹回叫反射，还可能有一部分波穿过交界面继续传播，叫折射。

电磁波遇到障碍物（如建筑物、地面）时会发生反射，反射波与直射波叠加，可能导致信号增强或减弱（多径效应）。常见于城市环境和室内通信。

电磁波从一种介质进入另一种介质时会发生折射，传播方向发生改变，可能导致信号传播路径偏移，影响接收信号强度。常见于在大气层中的传播，如无线电波通过不同密度的大气层。

③ 衍射 (Diffraction)

电磁波遇到障碍物边缘时，会绕过障碍物继续传播。衍射使信号能够传播到障碍物后方，但信号强度会减弱。当信号波在陡峭的边缘传播，如山顶或建筑物顶端，陡峭边缘形成一个波源，产生新波束，信号强度在新波束中分散开来。常见于山区和城市环境中，信号能够绕过建筑物和地形障碍。

④ 散射 (Scattering)

现实中无理想光滑的表面，电磁波遇到粗糙表面或小物体时，会被散射到各个方向。散射导致信号能量分散，接收信号强度减弱。常见于复杂环境中，如城市和森林地区。

四大衍生效应

① 多径传播

电磁波通过直射、反射、折射和衍射的多条路径到达接收端，导致信号时延扩展和频率选择性衰落，可能引起码间干扰，影响通信质量。常见于移动通信和室内无线通信。

② 衰减

电磁波在传播过程中能量逐渐减弱，信号强度随距离增加而降低，影响通信质量。包括自由空间路径损耗、吸收、散射等，所有无线通信系统都需要考虑衰减的影响。

③ 衰落

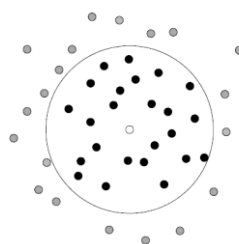
接收端信号强度随时间、频率和空间变化，忽强忽弱，影响通信可靠性。大尺度衰落由距离和障碍物引起，变化较慢；小尺度衰落由多径传播引起，变化较快。常见于移动通信和无线传感器网络。

④ 干扰

来自其他信号源或环境的电磁波干扰，降低信噪比，影响通信质量。常见于密集的无线通信环境，如城市和室内。

无线信道模型

圆盘图表单元模型 (DGUM)



Wireless Sensor Network

假设每个传感器节点的通信范围是一个圆形区域，节点只能与位于其通信范围内的其他节点直接通信。如果两个节点的距离小于或等于通信半径 R ，则它们之间存在一条“边”，即可以直接通信，反之，则无法直接通信。

数学表示：

设节点 i 和节点 j 的坐标分别为 (x_i, y_i) 和 (x_j, y_j) ，节点 i 和节点 j 之间的距离为：

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

若 $d(i, j) \leq R$ ，则节点 i 和节点 j 之间存在一条边。

特点：

① 对称性

如果节点 i 能与节点 j 通信，则节点 j 也能与节点 i 通信。

② 确定性

通信范围是固定的，不考虑信号强度、障碍物等因素。

③ 理想化假设

假设通信范围内的信号质量是均匀的，忽略了实际环境中的多径效应、干扰等。

优点：

简单易用——模型易于理解和实现，适合理论分析和仿真。

计算高效——基于几何距离的判断计算复杂度低。

缺点：

过于理想化——未考虑实际环境中的信号衰减、障碍物、干扰等因素。

缺乏动态性——假设通信范围固定，无法反映节点移动或环境变化。

应用：

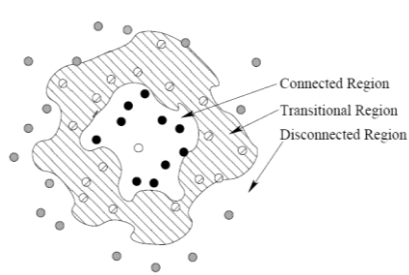
① 网络拓扑分析——用于研究网络的连通性、覆盖范围等。

② 路由算法设计——基于该模型设计高效的路由协议，如最短路径算法。

③ 覆盖优化——通过调整节点位置或通信半径，优化网络覆盖。

④ 仿真与实验——在仿真工具中用于模拟无线传感器网络的行为。

统计信道 (SCM)



与确定性模型不同，统计信道模型考虑了无线信道中的随机性和不确定性，能够更真实地反映实际环境中的信号传播行为。无线信号在传播过程中会受到多径效应、阴影衰落、路径损耗、干扰等因素的影响，导致信号强度随机变化。统计信道模型预测信号强度、误码率、信道容量等关键参数，通过概率分布来描述随机现象，为网络设计、性能分析和优化提供理论依据。

特点：

① 随机性

考虑了信号传播中的随机因素，更贴近实际环境。

② 动态性

能够反映信道随时间、位置和环境变化的特性。

③复杂性

模型参数较多，需要大量实验数据支持。

优点：

真实——能够反映实际信道中的随机性和不确定性。

适用范围广——可用于不同环境和场景的信道建模。

缺点：

复杂度高——模型参数多，计算和分析较复杂。

数据依赖性强——需要大量实测数据来校准模型参数。

物理层设计要素

①传输介质选择

传输介质的选择要考虑信号传输距离、传输速率、抗干扰能力等因素，要根据具体的应用场景和性能要求进行权衡，以满足不同应用的需求。常见的无线传输介质包括无线电波、微波等，无线电波适用于长距离传输，但容易受到环境干扰；微波则具有较高的传输速率和抗干扰能力，但传输距离相对较短。

②帧结构设计

帧结构的设计要考虑到数据传输的效率和可靠性。帧结构通常包括帧头、数据载荷和帧尾等部分。帧头用于携带控制信息，如同步信号、地址信息等；数据载荷用于传输用户数据；帧尾用于携带校验信息，用于检测和纠正传输错误。

合理的帧结构设计可以提高数据传输的效率，减少传输延迟和误码率。例如，通过在帧头中添加同步信号，可以实现接收端的快速同步，提高信号的解调性能；通过在帧尾中添加校验信息，可以检测和纠正传输错误，提高数据传输的可靠性。

③编码调制技术选择

编码调制技术包括信源编码、信道编码和调制技术等。信源编码用于减少数据冗余，提高数据传输效率；信道编码用于增加冗余信息，提高数据传输的可靠性；调制技术用于将数字信号转换为适合无线信道传输的模拟信号。

不同的编码调制技术具有不同的性能特点和适用场景。例如，振幅-幅移键控（ASK）调制技术简单易实现，但抗干扰能力较差；频率-频移键控（FSK）调制技术抗干扰能力较强，但传输速率较低。在选择编码调制技术时，需要根据具体的应用需求和信道特性进行综合考虑。

802.15.4 PHY 标准（一种低速率无线个人区域网络（LR-WPAN）标准）

802.15.4 PHY 是 WSN 中常用的物理层标准，它定义了无线传感器网络的物理层规范，包括频段、调制方式、传输速率等技术参数。该标准通过合理的频段选择（2.4 GHz、868 MHz 和 915 MHz）、调制方式（O-QPSK、BPSK）和传输速率设计，为 WSN 提供一种低功耗、低速率、低成本、高灵活性（支持多频段和多信道）、高可靠性（通过调制和信道选择抗干扰）的物理层解决方案。（低功耗设计使得设备能够在有限的电池容量下长时间工作，降低了设备的维护成本；低速率设计保证了信号的传输质量和可靠性，减少了误码率；高可靠性设计提高了网络的稳定性和可用性）

该标准适用于短距离无线通信场景，如智能家居、工业自动化等。它通过优化物理层设计，降低了设备的功耗与成本，提高了网络的可靠性和稳定性。不同的频段适用于不同的应用场景，不同的调制方式具有不同的性能特点。

调制方式：

①O-QPSK（偏移正交相移键控）

用于 2.4 GHz 频段。具有较高的频谱效率和抗干扰能力。

②BPSK（二进制相移键控）

用于 868 MHz 和 915MHz 频段。实现简单，抗噪声能力强。

信道划分：

Wireless Sensor Network

2.4 GHz 频段——16 个信道，信道宽度为 5 MHz。

868 MHz 频段——1 个信道，信道宽度为 600 kHz。

915 MHz 频段——10 个信道，信道宽度为 2 MHz。

帧结构：

①前导码

用于同步接收机，通常为 4 字节。

②帧起始定界符

标识帧的开始，通常为 1 字节。

③帧长度

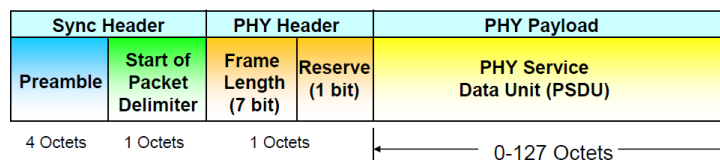
指示帧的长度，1 字节。

④物理层负载

包含 MAC 层数据，最大长度是 127 字节。

■ PHY packet fields

- **Preamble** (32 bits) – synchronization
- **Start of packet delimiter** (8 bits) – shall be formatted as “11100101”
- **PHY header** (8 bits) –PSDU length
- **PSDU** (0 to 127 bytes) – data field



物理层的功能：

①数据发送与接收

将 MAC 层数据转换为无线信号并发送，接收无线信号并解调为 MAC 层数据。

②信道能量检测（ED）

测量信道能量，用于信道选择。

③链路质量指示（LQI）

评估接收信号的质量。

④空闲信道评估（CCA）

检测信道是否空闲，避免冲突。

第五讲

WSN 中介质访问控制（MAC）协议

MAC 协议的定义：

MAC 协议是网络通信的关键组成部分。它决定了无线信道的使用方式，负责在传感器节点之间分配有限的无线通信资源。MAC 协议构建了传感器网络系统的底层基础结构，为数据传输提供了基本的通信机制，是网络正常运行的基石。

MAC 协议的目的：

减少冲突、节省能量、提高网络吞吐量、降低延迟。

- ①冲突避免：MAC 协议的重要目标之一。在高密度的传感器网络中，节点之间的通信冲突会导致数据丢失和传输延迟。
- ②节省能量：MAC 协议的关键设计目标。传感器节点通常由电池供电，能量有限，因此 MAC 协议需要通过优化通信过程，减少能量消耗，延长节点的使用寿命。
- ③可扩展性：MAC 协议能够适应不同规模的传感器网络。随着网络节点数量的增加，协议仍能保持良好的性能，不会出现性能瓶颈。
- ④降低时延：衡量 MAC 协议性能的重要指标之一。在某些应用场景中，如实时监测和控制，对数据传输的时延要求较高，MAC 协议需要尽量减少数据传输的延迟。
- ⑤公平性：确保所有传感器节点都能公平地获得通信资源，不会出现某些节点长时间占用信道而其他节点无法访问信道的情况。
- ⑥提高吞吐量：反映网络在单位时间内能够传输的数据量。MAC 协议通过优化信道分配和通信调度，提高网络的吞吐量，满足不同应用场景对数据传输量的需求。

MAC 协议的作用：

- ①MAC 协议能够管理节点对共享无线信道的访问，有效避免节点之间的通信冲突，确保高效、准确、可靠的数据传输。
- ②MAC 协议还注重能量的有效利用，通过合理的资源分配和通信调度，延长传感器节点的使用寿命，提高整个网络的性能和效率。

MAC 协议的技术挑战：

①高节点密度

节点密度较高时，多个节点可能同时尝试访问有限的无线信道资源，通信冲突的概率会显著增加。另外，高节点密度还会导致控制帧开销增大，过多的控制帧会占用宝贵的信道资源，降低数据传输的有效带宽，影响网络的性能和效率。

②冲突避免

冲突避免协议可以在节点之间进行有效的协调和调度，确保数据传输的顺利进行。例如，采用随机退避机制和优先级调度策略，可以减少节点之间的冲突概率，提高网络的通信性能和可靠性。

③低功耗需求

传感器节点的能量有限，节点失效会影响整个网络的连通性和性能。因此，MAC 协议需要在保证通信质量的前提下，尽量减少能量消耗。节点的能量消耗主要来自于发送、接收和空闲侦听等过程。其中，空闲侦听是能量浪费的主要原因之一，因为节点在侦听信道时并未进行实际的数据传输。低功耗的 MAC 协议通过优化通信过程，减少控制帧开销，避免空闲侦听和频繁的无线电状态切换。例如，采用节能的睡眠模式和动态功率控制技术，可以在不降低通信性能的情况下，显著降低能量消耗，延长网络的使用寿命。

通信过程能量浪费的主要来源：

- 1) 空闲侦听：节点对信道进行侦测但是无数据传输。无线收发开关保持开启，可能导致节点串音（节点接收到发往其它节点的分组或消息）。
- 2) 收发功耗比：信号收发的能量消耗。

3) 冲突: 两个或两个以上相邻节点同时向同一个接收机发送数据包时可能导致接收机发生数据接收错误, 就要重新发送。

4) 协议开销: 通信协议控制帧能量消耗。

④有限的处理与存储能力

传感器节点通常具有有限的处理和存储能力, 这使得复杂的算法难以在节点上实现。因此, MAC 协议需要设计简单而高效的调度算法, 这些算法能够在节点的有限处理能力下, 有效地进行通信调度和资源分配, 通过减少计算复杂度, 提高协议的运行效率。另外, 传统的分层结构虽然简化了协议栈, 但可能不适合传感器网络, 需要采用跨层优化和自配置、分布式协议, 通过增强层间协作, 以充分利用节点的有限资源, 提高网络的性能和可扩展性, 更好地适应动态环境。跨层优化通过在不同协议层之间进行交互、信息共享和协同工作, 进一步提高网络的整体性能。例如, 在 MAC 层和网络层之间进行联合优化, 可以更好地利用有限的资源, 提高网络的吞吐量和可靠性。

跨层优化:

跨层优化的主要目标

- ①能量效率: 优化能量使用, 延长网络寿命。
- ②提高吞吐量: 通过层间协作提升数据传输效率。
- ③降低延迟: 减少数据传输和处理时间。
- ④增强可靠性: 通过层间信息共享提高数据传输的可靠性。

常见的跨层优化方法

①物理层与 MAC 层的协作

动态功率控制——根据信道状态调整发射功率, 节省能量并减少干扰。

自适应调制和编码——根据信道条件调整调制和编码方案, 提高传输效率。

②MAC 层与网络层的协作

路由与调度结合——MAC 层调度与网络层路由协同, 优化路径选择和时隙分配。

拥塞控制——MAC 层检测拥塞并通知网络层调整路由。

③网络层与应用层的协作

数据聚合——在网络层进行数据聚合, 减少传输量, 节省能量。

QoS 保障——根据应用层需求调整网络层路由和 MAC 层调度。

④跨层信息共享

全局信息共享——各层共享信道状态、能量水平等信息, 优化决策。

事件驱动优化——根据网络事件动态调整各层参数。

跨层优化的优势

- ①提高资源利用率: 通过层间协作更高效地利用资源。
- ②增强适应性: 更好地适应网络环境变化。
- ③简化协议设计: 减少冗余操作, 简化协议栈。

跨层优化的挑战

- ①复杂性增加: 设计和管理跨层优化方案更为复杂。
- ②兼容性问题: 与传统分层协议的兼容性可能存在问题。
- ③调试和测试难度: 跨层优化的调试和测试更具挑战性。

用好理解的话来说呢, 就是

我们先从“传统分层架构”说起:

传统的网络协议的各层是“各司其职、互不干扰”的, 这种设计的好处是结构清晰、好维护, 但在无线传感器网络 (WSN) 里, 就有个大问题:

传感器节点的能量、带宽、算力都极其有限, 每层自己做决策, 不跟别的层通气, 很容易做出“局部最优但全局吃亏”的决定。

比如: MAC 层为了不冲突, 一直让节点休眠, 结果网络层想发数据时, 节点全睡死了, 导

致数据延迟飙升、能耗反而变高。

跨层优化就是打破这种“各层互不搭理”的壁垒：

允许不同协议层之间直接交换信息、协同做决策，比如物理层把信道质量告诉 MAC 层，MAC 层再把节点剩余电量告诉网络层，大家一起做最省电、最高效的决定，让整个网络性能达到全局最优。

4 个核心目标（为什么要做？）

1. 能量效率：WSN 节点大多靠电池供电，换电池几乎不可能。跨层优化的首要目标就是少耗电、多干活，比如根据信道质量动态调发射功率，不用一直满功率发射，延长网络寿命。
2. 提高吞吐量：让数据传得更快、更多。比如 MAC 层和路由层配合，选信道质量好的路径 + 高效的时隙分配，减少重传和冲突，提升传输效率。
3. 降低延迟：减少数据从采集到送达的时间。比如路由层不用等 MAC 层按部就班调度，直接根据 MAC 层的实时状态选低延迟路径。
4. 增强可靠性：减少数据丢包。比如物理层把信道干扰情况告诉上层，上层就可以提前调整传输策略，避免数据传丢。

常见的跨层优化方法（具体怎么做？）

就是不同协议层之间两两协作，典型的几类：

1. 物理层 ↔ MAC 层 协作

- 动态功率控制：物理层实时检测信道质量，如果信道干净，就调低发射功率，既省电又不干扰邻居节点；如果信道差，再适当提高功率，避免重传浪费电。
- 自适应调制编码（AMC）：信道好的时候，用高阶调制（比如 64QAM），一次传更多数据；信道差的时候，用低阶调制 + 强纠错编码，保证数据不丢，而不是死用一套方案。

2. MAC 层 ↔ 网络层 协作

- 路由与调度结合：MAC 层知道哪个节点现在是空闲的、哪个信道没被占用，就把这些信息告诉网络层，网络层选路由时，优先选这些节点和信道，避免选了路径但节点正在休眠、信道被占着的尴尬情况。
- 拥塞控制：MAC 层发现某个节点周围信道冲突严重、排队数据太多，就通知网络层：“别再往这儿发数据了，换条路吧！”，避免节点被数据“撑爆”、能耗飙升。

3. 网络层 ↔ 应用层 协作

- 数据聚合：应用层知道现在要传的是温度数据，网络层就可以把多个节点的温度数据合并成一条发出去，不用每个节点单独发，大幅减少传输次数，省电又省带宽。
- QoS 保障：应用层告诉网络层：“这条数据是紧急报警数据，要优先传！”，网络层就会给它分配高优先级路径，MAC 层也会优先给它分配时隙，保证低延迟送达。

4. 全局跨层信息共享

- 所有层共享关键信息：比如节点剩余电量、信道质量、网络负载、拓扑变化，所有层都能看到这些数据，做决策时就不会“盲人摸象”。
- 事件驱动优化：比如某个节点突然电量不足了，这个信息会同步给所有层：物理层调低功率、MAC 层减少它的发送次数、网络层绕开它，动态调整整个网络的策略。

跨层优化的优势与挑战（必讲的“正反面”）

✓ 优势

1. 资源利用率更高：所有层协同起来，避免了“各做各的”导致的资源浪费，比如不用为了保证不冲突，一直让节点休眠，也不用为了保证传输，一直满功率发射。
2. 适应性更强：WSN 的环境很容易变（比如节点没电了、信道被干扰了），跨层优化能根据实时信息动态调整策略，比固定的分层协议更能应对复杂情况。

3. 简化冗余设计：有些分层协议里，每层都要做一次“纠错、重传、调度”，跨层优化可以把这些功能整合，减少重复操作，简化协议栈。

🔥 挑战

1. 复杂度飙升：本来每层的逻辑是独立的，现在要让不同层的逻辑耦合在一起，设计、实现、维护都比传统分层协议难很多，容易出现“牵一发而动全身”的问题。
2. 兼容性问题：传统分层协议是标准化的，跨层优化打破了这种标准，和其他厂商的设备、旧版本协议可能不兼容，没法直接对接。
3. 调试和测试更难：出了问题，不知道是哪一层的锅，也没法像分层协议一样逐层排查，调试成本很高。

一句话总结

跨层优化就是 WSN 里的“全局协同作战”：打破各层的壁垒，让物理层、MAC 层、网络层、应用层互通信息、协同决策，在资源有限的情况下，用最少的电、最高的效率，实现更长的网络寿命和更好的性能，同时也要面对复杂度、兼容性和调试的挑战。

MAC 协议的分类：

① 竞争型 MAC 协议

竞争型 MAC 协议采用按需使用信道的方式，当节点需要发送数据时，通过竞争方式使用信道。如果发生冲突，节点按照事先设定的某种策略重传数据，直到数据发送成功或放弃。这种协议的优点是简单灵活，能够根据网络的实际需求动态分配信道资源。然而，它也存在一些缺点，如在高负载情况下，冲突概率会增加，导致数据传输延迟和能量浪费。

在传统无线网络中，ALOHA 和 CSMA（载波侦听多路访问）是常见的竞争型 MAC 协议。ALOHA 协议（不侦听信道，想发就发或者是分时隙错开发送，冲突了就靠随机等待重发来解决）简单易实现，但冲突概率较高，适用于低负载的网络环境。CSMA 协议通过侦听信道的忙闲状态来决定是否发送数据，能够有效减少冲突概率，提高信道利用率。然而，它仍然存在隐藏终端和暴露终端等问题，需要进一步优化。

基础前提

无线信号有覆盖范围，A 能收到 B 的信号，不代表 A 能收到 C 的信号，这是产生两类问题的根源。

约定：节点 A、B、C 在一条直线上，B 在中间， $A \leftrightarrow B$ 、 $B \leftrightarrow C$ 彼此在信号覆盖内，A 和 C 互相听不到对方。

隐藏终端 (Hidden Terminal)

定义

两个节点互相检测不到对方，但它们的信号会在共同的接收节点处发生冲突，这两个节点互为隐藏终端。

场景演示

1. A 要发数据给 B，先监听信道，听不到 C 的信号，判定信道空闲，开始发送；
2. 与此同时，C 也要发数据给 B，同样监听不到 A，也判定信道空闲，开始发送；
3. A、C 的信号在 B 处叠加冲突，B 收不到任何有效数据。

核心问题

信道监听失效，引发数据碰撞，是无线网络里最典型的问题，CSMA/CA 也没法完全根治。

常用解决办法

RTS/CTS 握手（请求发送 / 允许发送）：

A 先发短报文 RTS 给 B，B 回复 CTS；周边节点收到 CTS 就知道信道被占用，主动静默，避免争抢。

暴露终端 (Exposed Terminal)

定义

Wireless Sensor Network

节点听到邻居正在通信，误以为信道被占用，明明自己可以正常发包，却被迫等待，这个节点就是暴露终端。

场景演示

1. B 正在发数据给 A, C 监听信道，听到了 B 的信号；
2. C 想发数据给旁边其他节点（不是 B），本身不会和 B 的传输产生冲突；
3. 但 C 误以为信道忙，停止发送、原地等待，白白浪费信道资源。

核心问题

过度避让，降低信道利用率、造成带宽浪费，不会产生数据冲突。

对比总结

类型	现象	后果
隐藏终端	互相听不到，同时发包	数据冲突、丢包
暴露终端	听到邻节点通信，不敢发包	信道空闲、资源浪费

在 WSN 中，通过竞争方式使用信道是指节点通过竞争机制来获取共享无线信道的访问权限。这种方式通常用于分布式网络环境中，节点在发送数据前需要先检测信道状态，确保信道空闲后再进行传输，以避免冲突。

竞争方式的基本原理：

载波侦听多路访问（CSMA）：

载波侦听——节点在发送数据前先监听信道，检测是否有其他节点正在传输。

多路访问——如果信道空闲，节点可以开始传输；如果信道忙，节点需要等待一段随机时间后再尝试。

冲突避免（CA）：

随机退避——当信道忙时，节点等待一个随机时间后再尝试，以减少冲突概率。

冲突检测——虽然 WSN 中通常不进行冲突检测（由于硬件限制），但可以通过确认机制（ACK）来确保数据传输成功。

竞争方式的优缺点：

优点：

简单易实现——竞争协议通常较为简单，易于在资源受限的 WSN 中实现。

分布式控制——无需中央协调器，适合分布式网络环境。

适应性强——能够适应节点数量和网络拓扑的变化。

缺点：

冲突概率——在高负载情况下，冲突概率增加，导致性能下降。

能量消耗——由于需要频繁监听信道，可能导致较高的能量消耗。

隐藏终端问题——节点可能无法检测到所有正在传输的节点，导致冲突。

ALOHA 协议：

一种早期的随机接入协议，最初用于无线数据通信。它允许多个用户共享同一信道，适用于 WSN 等分布式系统。ALOHA 协议及其变体通过不同的机制来提高信道利用率和减少冲突。

基本 ALOHA 协议

工作原理：

随机发送——节点有数据时立即发送，不进行信道监听。

冲突处理——如果多个节点同时发送，数据包会冲突，发送节点在随机退避后重发。

优点：简单易实现，无需复杂控制机制；低延迟，数据准备好后立即发送。

缺点：低信道利用率；在负载较高时具有高冲突概率。

时隙 ALOHA（Slotted ALOHA）

改进：

时隙划分——时间被划分为固定时隙，节点只能在时隙开始时发送。

减少冲突窗口——冲突仅发生在相同时隙内。

优点：提高信道利用率；冲突窗口缩小一半，减少了冲突。

缺点：需要时间同步，增加了复杂性；尤其在负载较高时仍然存在冲突。

帧时隙 ALOHA (Framed Slotted ALOHA)

改进：

帧结构——时间划分为多个时隙组成的帧，节点每帧选择一个时隙发送。

减少冲突——通过帧结构进一步降低冲突概率。

优点：信道利用率更高，适用于大规模网络；通过帧和时隙结构优化进一步减少了冲突。

缺点：需要更复杂的控制机制，复杂性增加了；节点需严格同步。

②非竞争型 MAC 协议

非竞争型 MAC 协议将共享的信道根据时间、频率或伪噪声码划分成一组子信道，并将子信道分配给各个节点。每个节点拥有一个专用的子信道进行数据发送，从而避免了节点之间的通信冲突。

这种协议的优点是能够有效避免冲突，提高数据传输的可靠性和效率。然而，它也存在一些缺点，如信道分配的灵活性较差，难以适应网络动态变化的需求。

在传统无线网络中，TDMA（时分多址）、FDMA（频分多址）和 CDMA（码分多址）是非竞争型 MAC 协议的典型代表。TDMA 协议通过时间分割将信道分配给不同节点，每个节点在分配的时间段内独占信道进行数据发送。FDMA 协议则通过频率分割将信道分配给不同节点，每个节点在分配的频段内进行数据发送。CDMA 协议利用伪噪声码对数据进行编码，通过码分多址的方式实现多个节点在同一信道上的同时通信。

TDMA（时分多址，Time Division Multiple Access）

一种信道接入技术，通过将时间划分为多个时隙，使多个用户共享同一频率信道。每个用户在指定的时隙内传输数据，从而避免冲突并提高信道利用率。

基本原理：

时间划分——时间被划分为多个帧，每个帧进一步划分为多个时隙；每个用户分配一个或多个时隙，用于数据传输。

同步机制——所有用户必须严格同步，以确保在正确时隙发送和接收数据（时间同步）；通常通过基站或参考节点提供同步信号（时钟同步）。

数据传输——用户在分配时隙内发送数据，其他时隙保持静默；帧结构周期性重复，确保用户持续传输数据。

优点：通过时间划分，允许多个用户共享同一频率，提高信道利用率；时隙分配避免用户同时发送数据，减少冲突；适合传输突发性数据，如语音和视频；时隙分配固定，数据传输延迟可预测。

缺点：所有用户需严格同步，增加系统复杂性；用户无数据传输时，时隙可能浪费；用户需等待分配时隙，可能增加初始延迟。

FDMA（频分多址，Frequency Division Multiple Access）

一种信道接入技术，通过将频带划分为多个子信道，使多个用户能够同时使用不同的频率进行通信。每个用户分配一个唯一的频率子信道，从而避免冲突并提高信道利用率。

基本原理：

频率划分——将总频带划分为多个子信道，每个子信道占用不同的频率范围；每个用户分配一个或多个子信道，用于数据传输。

并行传输——通过滤波器隔离不同频率信号，确保信号不互相干扰；多个用户可以在不同频率上同时传输数据，互不干扰。

信道管理——合理规划频率分配，避免频率冲突和干扰；根据网络需求动态调整频率分配，提高资源利用率。

优点：频率划分和分配相对简单，易于实现和管理；多个用户可以同时通信，提高信道利用

率（并行通信）；适合需要连续传输的应用，如语音通信；用户无需等待时隙，数据传输延迟较低。

缺点：总频带有限，子信道数量受限；用户无数据传输时，分配的子信道可能浪费；相邻频率可能产生干扰，影响通信质量；需要复杂的滤波器和频率合成器，增加硬件成本。

CDMA（码分多址，Code Division Multiple Access）

一种信道接入技术，通过为每个用户分配唯一的编码序列，使多个用户能够在同一频率和时间上同时传输数据。CDMA 利用编码序列的正交性或准正交性来区分不同用户的信号，从而实现多用户共享同一信道。

基本原理：

编码序列——每个用户分配一个唯一的编码序列（扩频码），用于调制数据；编码序列之间具有正交性或准正交性，确保接收端能够分离不同用户的信号。

扩频技术——用户数据通过扩频码进行调制，将窄带信号扩展为宽带信号；提高了信号的抗干扰能力和隐蔽性。

信号分离——接收端使用与发送端相同的扩频码进行相关检测，分离出目标用户的信号，抑制其他用户的信号干扰。

优点：通过编码序列区分用户，允许多用户共享同一频率和时间，提高信道容量；扩频技术提高了信号的抗干扰能力和隐蔽性；用户数量增加时，系统性能逐渐下降，但不会突然中断；同一频率可以在不同小区复用，提高频谱利用率。

缺点：编码序列的生成和相关检测需要复杂的硬件和算法；所有用户必须严格同步，以确保编码序列的正交性；近处用户的信号可能淹没远处用户的信号，影响通信质量（“远近效应”）。

假设用户 A 的编码为向量 $\mathbf{C}_A$ ，用户 B 的编码为向量 $\mathbf{C}_B$ 。它们满足正交特性：

$$\mathbf{C}_A \cdot \mathbf{C}_B = 0$$

同时，编码自身的内积为：

$$\mathbf{C}_A \cdot \mathbf{C}_A = 1$$

当 A 发送数据 d_A ， B 发送数据 d_B 时，空中混合后的总信号为：

$$\mathbf{S} = d_A \mathbf{C}_A + d_B \mathbf{C}_B$$

当 A 的接收端收到总信号 $\mathbf{S}$ 后，只需要乘以 A 的本征码 $\mathbf{C}_A$ ：

$$\mathbf{S} \cdot \mathbf{C}_A = (d_A \mathbf{C}_A + d_B \mathbf{C}_B) \cdot \mathbf{C}_A = d_A (\mathbf{C}_A \cdot \mathbf{C}_A) + d_B (\mathbf{C}_B \cdot \mathbf{C}_A)$$

由于正交性，后半部分变为 0：

$$\mathbf{S} \cdot \mathbf{C}_A = d_A \cdot 1 + d_B \cdot 0 = d_A$$

用户 B 的信号直接就被完美过滤掉了！

③混合型 MAC 协议

混合型 MAC 协议有效结合了竞争型和非竞争型 MAC 协议的优点，既能够根据网络的实际需求动态分配信道资源，又能够有效减少节点之间的通信冲突，改善网络的传输性能。这种协议通过灵活的信道分配和调度策略，能够在不同的网络负载和应用场景下，实现较好的性能平衡，提高网络的可靠性和效率。

IEEE 802.11 协议是一种典型的混合型 MAC 协议，它包括分布式协调（DCF）和点协调（PCF）两种工作方式。DCF 是 802.11 的基本访问控制方式，采用 CSMA/CA 机制和随机退避时间，实现无线信道的共享。PCF 是基于优先级的无竞争访问方式，通过访问接入点（AP）协调节点的数据收发，采用轮询机制查询当前哪些节点有数据发送请求，并在必要时给予数据发送权。这种混合型机制能够根据不同的应用场景和网络负载，灵活地选择合适的信道分配和调度策略，提高网络的性能和可靠性。

分布式协调功能（DCF）

DCF 是 IEEE 802.11 的默认访问机制，基于载波侦听多路访问 / 冲突避免（CSMA/CA, Carrier Sense Multiple Access with Collision Avoidance）。它是一种分布式、基于

Wireless Sensor Network

竞争的协议，适合异步数据传输（如普通数据包传输）。

工作机制：

- 载波侦听：节点在发送数据前先监听信道，如果信道空闲，则等待一个短的时间间隔（DIFS, Distributed Inter-Frame Space）。
- 随机退避：如果信道忙，节点会启动一个随机退避计时器，等待一段时间后再尝试发送。
- 冲突避免：通过 RTS（Request to Send）和 CTS（Clear to Send）握手机制来避免隐藏终端问题。
- 确认机制：接收方成功接收数据后，会发送一个 ACK（Acknowledgment）帧，以确保数据传输的可靠性。

特点：

分布式控制——无需中央协调器，所有节点平等竞争信道。

适合突发性数据——适合传输非实时数据（如网页浏览、文件下载）。

简单易实现——适用于大多数 Wi-Fi 场景。

点协调功能（PCF）

PCF 是一种可选的访问机制，基于集中式调度，由接入点（AP, Access Point）协调节点的数据传输。它适合实时性要求较高的应用（如语音、视频流）。

工作机制：

- 轮询机制：AP 作为中央控制器，轮流询问每个节点是否有数据需要发送。
- 无竞争期（CFP, Contention-Free Period）：在 PCF 模式下，时间被划分为竞争期（CP）和无竞争期（CFP）。在 CFP 期间，AP 控制信道访问，节点只有在被轮询时才能发送数据。在 CP 期间，退回到 DCF 模式，大家还是按 CSMA/CA 抢信道。PCF 并非全程轮询，而是 CFP 与 CP 交替出现。
- 优先级访问：PCF 可以为实时数据提供优先级，确保低延迟和高可靠性。

特点：

集中式控制——AP 负责调度所有节点的数据传输。

适合实时数据——适合传输语音、视频等对延迟敏感的数据。

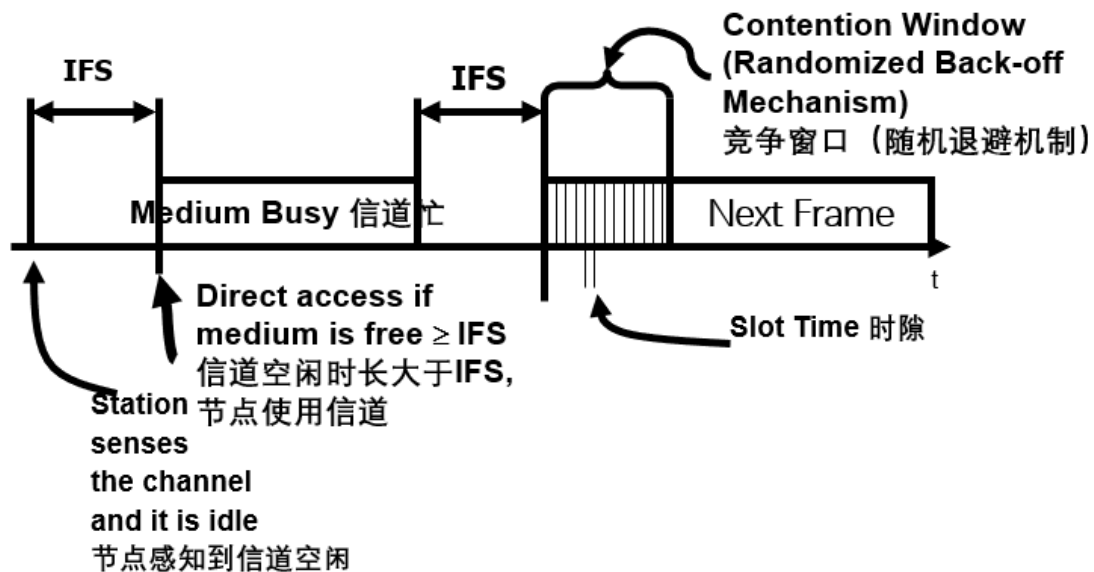
较少使用——由于实现复杂且效率较低，PCF 在实际应用中较少使用。

MAC 协议的关键技术：

①CSMA/CA

CSMA/CA（载波侦听多路访问/冲突避免）机制是无线局域网中常用的 MAC（介质访问控制）协议机制之一。它通过节点对信道的侦听来判断信道的忙闲状态，从而决定是否发送数据。当节点有数据需要传输时，它首先感知信道。如果信道空闲，节点等待一个特定的时间间隔（如 DIFS），然后立即传输数据。如果信道忙，节点则推迟数据传输，继续监测信道，直到当前传输结束。

在 CSMA/CA 机制中，当节点感知到信道忙时，它会采用二进制指数退避方案来决定何时再次尝试发送数据。节点首先选取一个随机时隙数作为计数器的初始值，然后递减退避计数器。当计数器递减至 0 时，节点开始传输数据。如果在传输过程中仍然发生冲突，竞争窗口会加倍，退避时间呈指数增加。这种退避方案能够有效减少冲突概率，提高信道利用率。



基本的 CSMA/CA 工作流程：

当站点有数据待发时，首先监听信道。若信道空闲，站点先等待一个帧间间隙（IFS）。若为高优先级帧（如 ACK），可直接发送；普通数据帧则进入冲突避免的随机退避阶段。若检测到信道忙碌，站点将持续监听，直到当前传输结束、信道重新空闲，再等待一个 IFS，之后进入随机退避阶段。在退避阶段，各站点在竞争窗口内随机抽取一个由若干时隙组成的退避时间并开始倒计时：每经历一个空闲时隙，计数器减 1；若信道变忙，计数器暂停；信道恢复空闲并等待 IFS 后，继续倒计时。计数器率先归零的站点抢占信道发送下一帧；其余站点的计数器保持冻结，等待下一次信道空闲时继续倒数，以此错开发送，避免冲突。

（一）不带 ACK 和 RTS/CTS 的基本 CSMA/CA 机制

如果不使用 ACK（确认机制）和 RTS/CTS（握手机制），CSMA/CA（载波侦听多路访问 / 冲突避免）的机制会简化，但仍然保留了核心的冲突避免功能。以下是不带 ACK 和 RTS/CTS 的基本 CSMA/CA 机制：

（1）载波侦听（Carrier Sense）

节点在发送数据之前，会先监听信道，检测是否有其他节点正在传输数据。

- 物理载波侦听：通过射频模块检测信道中的信号强度。
- 虚拟载波侦听（可选）。

如果信道空闲，节点可以继续下一步；如果信道忙，节点会推迟发送。

（2）帧间间隔（Inter-Frame Spaces）

为了减少冲突并提高网络效率，CSMA/CA 使用了多种帧间间隔（Interframe Space, IFS）来控制节点访问信道的时机，协调设备之间的数据传输。帧间间隔是指在不同类型的帧之间插入的固定时间间隔，用于确保设备能够有序地访问介质。CSMA/CA 定义了多种帧间间隔，每种间隔的长度不同，优先级也不同。设备必须等待相应的帧间间隔后才能尝试访问介质。以下是常见的帧间间隔类型：

SIFS（Short Interframe Space，短帧间间隔）

- 作用：SIFS 是最短的帧间间隔，用于高优先级的帧传输，如确认帧（ACK）、清除发送帧（CTS）和数据帧的后续部分。
- 特点：由于 SIFS 时间最短，使用 SIFS 的设备可以优先访问介质，确保关键帧能够快速传输。
- 典型应用：ACK 帧、CTS 帧、数据帧的分片等。

PIFS (PCF Interframe Space, 点协调功能帧间间隔)

- 作用：PIFS 用于点协调功能（PCF）模式下的帧传输，优先级高于普通数据帧。
- 特点：PIFS 的长度介于 SIFS 和 DIFS 之间，允许点协调器（如接入点）优先访问介质。
- 典型应用：PCF 模式下的轮询帧。

DIFS (DCF Interframe Space, 分布式协调功能帧间间隔)

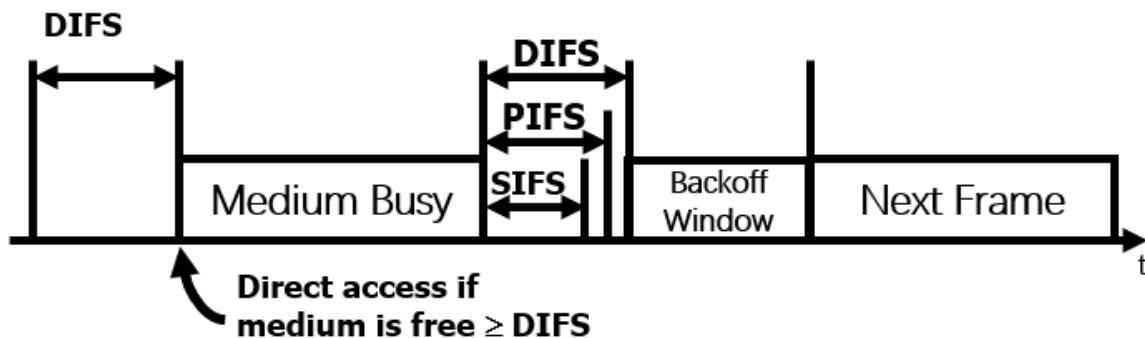
- 作用：DIFS 是分布式协调功能（DCF）模式下设备在发送数据帧前必须等待的固定时间间隔，节点在信道空闲后，需要等待 DIFS 时间。
- 特点：DIFS 的长度比 SIFS 和 PIFS 长，用于普通数据帧的传输。
- 典型应用：普通数据帧的发送。

帧间间隔的作用：

优先级控制——通过设置不同长度的帧间间隔，CSMA/CA 可以确保高优先级的帧（如 ACK、CTS）能够优先访问介质，减少冲突并提高网络效率。

冲突避免——设备在发送数据前必须等待相应的帧间间隔，这有助于分散设备的发送时间，减少多个设备同时发送数据的可能性。

有序传输——帧间间隔确保了设备之间的传输顺序，避免了介质访问的混乱。



帧间间隔的工作流程：

设置不同长度的帧间间隔（IFS）实现优先级控制与冲突避免。SIFS 最短、优先级最高，用于 ACK、CTS 等关键控制帧；PIFS 次之，用于 PCF 模式下的轮询帧；DIFS 最长，用于普通数据帧。当节点检测到信道空闲时（信道空闲时长 $\geq$ DIFS），需根据帧类型等待相应的 IFS，普通数据帧在等待 DIFS 后，还需进入退避窗口执行随机退避，倒计时归零后才能发送数据，以此确保控制帧优先传输，同时通过分级等待和随机退避分散节点发送时机，避免多节点同时访问信道引发冲突，保障网络有序高效传输。

在进行载波侦听的过程中，设备通过侦听通信介质（如电缆或无线频谱）来判断是否有其他设备在传输数据。这一过程的核心原理是通过检测介质上的信号能量或特定的信号特征来确定介质是否正在被使用。以下是具体的判断原理和过程：

信号能量检测

- 原理：设备通过物理层的硬件（如无线网卡或以太网卡）检测介质上的信号能量。如果检测到的信号能量超过某个阈值，设备会认为介质正在被使用。
- 过程：设备持续监测介质的信号强度。如果信号强度较低或没有信号，设备认为介质空闲；如果信号强度较高，设备认为介质繁忙。

信号特征识别

- 原理：除了检测信号能量，设备还可以通过识别特定的信号特征（如帧前导码、同步信号等）来判断是否有其他设备在传输数据。
- 过程：设备会解析接收到的信号，检查是否符合已知的数据帧格式。如果检测到有效的数据帧，设备会认为介质正在被使用。

(3) 随机退避 (Random Backoff)

是 CSMA/CA (载波侦听多路访问 / 冲突避免) 协议中的核心部分, 用于减少多个节点同时访问信道时发生冲突的概率。它的基本思想是: 当节点检测到信道空闲时, 如果信道在 DIFS 时间内保持空闲, 节点不会立即发送数据, 而是等待一段随机的时间 (退避时间), 从而分散节点的发送时机, 降低冲突的可能性。

退避计时器:

节点从 $[0, CW-1]$ 的范围内随机选择一个退避时隙 (slot), 其中 CW (Contention Window) 是竞争窗口的大小, 每个时隙的长度是固定的。

退避机制的工作流程 (不带 ACK 和 RTS/CTS 的 CSMA/CA 工作流程):

节点在发送数据前, 先监听信道。如果信道空闲, 节点等待一个固定的帧间间隔 (DIFS); 如果信道忙, 节点推迟发送, 并启动退避过程。节点从 $[0, CW-1]$ 的范围内随机选择一个整数作为退避时隙数 (退避时间为退避时隙数 $\times$ 退避时隙长度)。节点每经过一个退避时隙, 退避计时器减 1。如果信道在退避期间变为忙状态, 退避计时器会暂停, 直到信道再次空闲并保持空闲状态超过 DIFS 时间。当退避计时器减到 0 时, 节点可以开始发送数据。

竞争窗口 (CW) 的动态调整:

竞争窗口的大小直接影响退避时间的范围和冲突概率。为了适应网络负载的变化, CW 的大小会根据冲突情况动态调整。CW 的初始值通常较小 (例如 $CW = CW_{min} = 15$), 如果发送的数据帧发生冲突 (例如未收到 ACK), CW 的值会加倍, 直到达到最大值 CW_{max} (例如 $CW_{max} = 1023$)。这种指数增长的机制称为二进制指数退避 (Binary Exponential Backoff)。如果数据帧成功发送 (例如收到 ACK), CW 会重置为最小值 CW_{min} 。

示例

假设:

- 退避时隙长度 = $9 \mu s$
- $CW_{min} = 15, CW_{max} = 1023$

示例 1: 第一次发送

1. 节点随机选择一个退避时隙数, 例如 5。
2. 退避时间 = $5 \times 9 \mu s = 45 \mu s$ 。
3. 节点等待 $45 \mu s$ 后发送数据。

示例 2: 冲突后的退避

1. 如果数据帧发生冲突, CW 的值加倍, 例如从 15 变为 31。
2. 节点从 $[0, 30]$ 中随机选择一个退避时隙数, 例如 12。
3. 退避时间 = $12 \times 9 \mu s = 108 \mu s$ 。
4. 节点等待 $108 \mu s$ 后重传数据。

优点:

减少冲突——通过随机退避时间, 分散节点的发送时机, 降低多个节点同时发送数据的概率。

适应网络负载——动态调整竞争窗口大小, 适应网络负载的变化。在高负载情况下, CW 增大, 退避时间变长, 进一步减少冲突。

公平性——所有节点使用相同的退避机制, 确保每个节点有公平的机会访问信道。

缺点:

延迟增加——退避时间可能导致数据传输的延迟增加, 尤其是在高负载情况下。

效率受限——在高负载情况下, 退避时间可能较长, 降低信道利用率。

复杂性——动态调整竞争窗口和退避计时器增加了协议的复杂性。

(4) 发送数据

当退避计时器为 0 时, 节点开始发送数据帧。由于没有 ACK 机制, 发送方不会等待

接收方的确认。

(5) 冲突处理

由于没有 ACK 机制，发送方无法知道数据是否成功到达接收方。如果发生冲突：数据帧可能会丢失，而且发送方不会进行重传（因为没有 ACK 机制）。

不带 ACK 和 RTS/CTS 的 CSMA/CA 的特点：

优点：

简单高效——去掉了 ACK 和 RTS/CTS 机制，减少了控制帧的开销。适用于对可靠性要求不高的场景。

低延迟——由于不需要等待 ACK 或 RTS/CTS，数据传输的延迟较低。

缺点：

可靠性低——没有 ACK 机制，发送方无法确认数据是否成功到达接收方。数据帧可能因冲突或噪声丢失。

冲突概率较高——没有 RTS/CTS 机制，隐藏终端问题可能导致冲突。冲突发生后，数据帧会被丢弃，发送方不会重传。

不适合高负载网络——在高负载情况下，冲突概率增加，信道利用率下降。

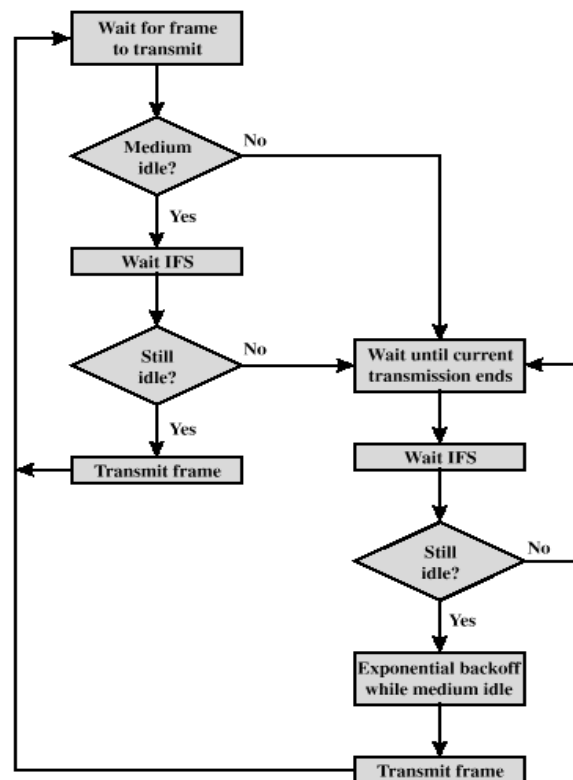


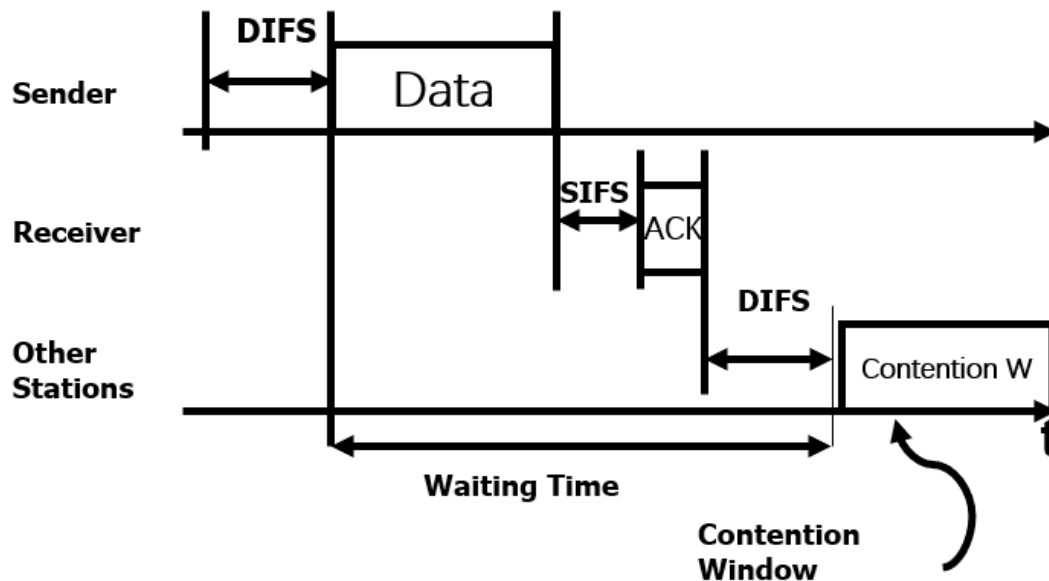
Figure 14.6 IEEE 802.11 Medium Access Control Logic

(二) 带 ACK 的分布式无线介质访问控制协议

当节点发送数据时，接收方在正确接收数据后会立即发送一个 ACK（确认帧）。ACK 机制用于确认数据的正确接收，从而提高数据传输的可靠性。如果发送方在规定的时间内没有收到 ACK，它会认为数据传输失败，从而执行重传操作，能有效减少数据丢失和传输错误，提高网络的通信质量。

在实现 ACK 机制时，发送方在发送数据前会等待一个 DIFS 时长。如果在 DIFS 时长内信道空闲，发送方开始传输数据。接收方在正确接收数据后，立即发送 ACK。接收方在传输 ACK 前，不对信道进行感知，以减少信道感知的开销。如果 ACK 丢失，发送方会执行重传操作。通过这种机制，可以确保数据的可靠传输，提高网络的性能和可靠性。

Wireless Sensor Network



发送方在检测到信道空闲并等待一个分布式帧间间隙（DIFS）后，进入竞争窗口进行随机退避，退避计时器归零后开始发送数据帧（Data）；接收方成功接收后，为了确保回复的最高优先级，仅需等待一个更短的短帧间隙（SIFS）便可向发送方回传确认帧（ACK）；在此期间，其余站点在整个传输时间内保持挂起和等待（Waiting Time）；当 ACK 发送完毕、信道重新空闲后，其余站点必须再次强制等待一个 DIFS，随后正式进入竞争窗口（Contention Window），各自抽取随机时隙数进行退避倒计时，率先归零的站点将夺取下一轮的数据发送权。

（三）CSMA/CA 中存在的问题

隐藏终端问题（Hidden Terminal Problem）

这个问题会导致数据冲突，降低网络效率。

定义：

在网络中，某些设备（节点）由于地理位置或信号传播的限制，无法直接检测到彼此的存在，从而导致它们在不知情的情况下同时向同一个接收设备发送数据，引发数据冲突。

场景：

假设有三个设备：A、B 和 C。设备 A 和设备 C 都在设备 B 的通信范围内，但设备 A 和设备 C 彼此不在对方的通信范围内。这种情况下：

- 设备 A 可以向设备 B 发送数据。
- 设备 C 也可以向设备 B 发送数据。
- 但设备 A 和设备 C 无法直接检测到彼此的信号。

原因：

信号传播范围限制——无线信号的传播范围有限，设备之间的距离可能导致某些设备无法直接通信。

障碍物阻挡——物理障碍物（如墙壁、建筑物）可能阻挡信号，使得设备无法检测到彼此。

影响：

数据冲突——当设备 A 和设备 C 同时向设备 B 发送数据时，设备 B 会接收到冲突的信号，导致数据无法正确解码。

降低网络效率——由于数据冲突，设备需要重传数据，增加了网络延迟并降低了吞吐量。

资源浪费——冲突导致信道资源的浪费，减少了有效的数据传输时间。

解决办法：

（1）RTS/CTS 机制：

Wireless Sensor Network

请求发送 (RTS) —— 发送设备在发送数据前, 先向接收设备发送一个 RTS 帧, 请求发送数据。

清除发送 (CTS) —— 接收设备在接收到 RTS 帧后, 回复一个 CTS 帧, 表示可以发送数据。

作用: RTS/CTS 机制通过广播 RTS 和 CTS 帧, 使得隐藏终端能够感知到数据传输, 从而避免冲突。

(2) 功率控制:

调整发射功率 —— 设备可以动态调整发射功率, 确保信号能够覆盖到所有相关设备。

作用: 通过增加发射功率, 设备可以扩大通信范围, 减少隐藏终端问题。

(3) 定向天线:

使用定向天线 —— 定向天线可以集中信号在特定方向, 减少信号干扰和隐藏终端问题。

作用: 定向天线可以提高信号的指向性, 使得设备能够更有效地检测到彼此。

(4) 时间同步:

时间分片 —— 通过时间同步机制, 设备在特定的时间片内发送数据, 避免同时发送。

作用: 时间同步可以减少设备之间的竞争, 降低冲突概率。

暴露终端问题 (Exposed Terminal Problem)

这个问题会导致信道资源的浪费, 降低网络效率。

定义:

在网络中, 某些设备 (节点) 由于能够检测到其他设备的信号, 误以为介质正在被使用, 从而不必要地延迟自己的数据传输, 即使这些传输不会导致冲突。

场景:

假设有四个设备: A、B、C 和 D。

- 设备 A 和设备 B 在彼此的通信范围内;
- 设备 C 和设备 D 在彼此的通信范围内;
- 设备 A 和设备 C 在彼此的通信范围内 (A 能听到 C 的信号), 但设备 A 和设备 D 不在彼此的通信范围内 (A 给 B 发数据, 不会影响 D); 同理, 设备 C 和设备 B 也不在彼此的通信范围内 (C 给 D 发数据, 不会影响 B)。

这种情况下:

- 设备 A 正在向设备 B 发送数据;
- 设备 C 能听到 A 的信号, 误以为信道被占用, 不敢向设备 D 发送数据;
- 但实际上, C 给 D 发数据和 A 给 B 发数据, 完全不会互相干扰。

原因:

信号传播范围限制 —— 无线信号的传播范围有限, 设备之间的距离可能导致某些设备能够检测到彼此的信号, 即使它们的传输不会相互干扰。

过度谨慎的载波侦听 —— 设备在检测到任何信号后都会延迟自己的传输, 即使这些信号不会导致冲突。

影响:

信道资源浪费 —— 设备 A 检测到设备 C 的信号后, 会误以为介质正在被使用, 从而延迟自己的数据传输, 即使设备 A 和设备 C 的数据传输不会冲突。

降低网络效率 —— 不必要的延迟导致信道利用率下降, 减少了有效的数据传输时间。

增加延迟 —— 设备需要等待更长的时间才能发送数据, 增加了数据传输的延迟。

解决办法:

(1) RTS/CTS 机制:

请求发送 (RTS) —— 发送设备在发送数据前, 先向接收设备发送一个 RTS 帧, 请求发送数据。

清除发送 (CTS) —— 接收设备在接收到 RTS 帧后, 回复一个 CTS 帧, 表示可以发送数

据。

作用：RTS/CTS 机制通过广播 RTS 和 CTS 帧，使得暴露终端能够更准确地判断介质的使用情况，从而减少不必要的延迟。

（2）功率控制：

调整发射功率——设备可以动态调整发射功率，确保信号只覆盖到必要的设备。

作用：通过减少发射功率，设备可以缩小通信范围，减少暴露终端问题。

（3）定向天线：

使用定向天线——定向天线可以集中信号在特定方向，减少信号干扰和暴露终端问题。

作用：定向天线可以提高信号的指向性，使得设备能够更有效地判断介质的使用情况。

（4）时间同步：

时间分片——通过时间同步机制，设备在特定的时间片内发送数据，避免不必要的延迟。

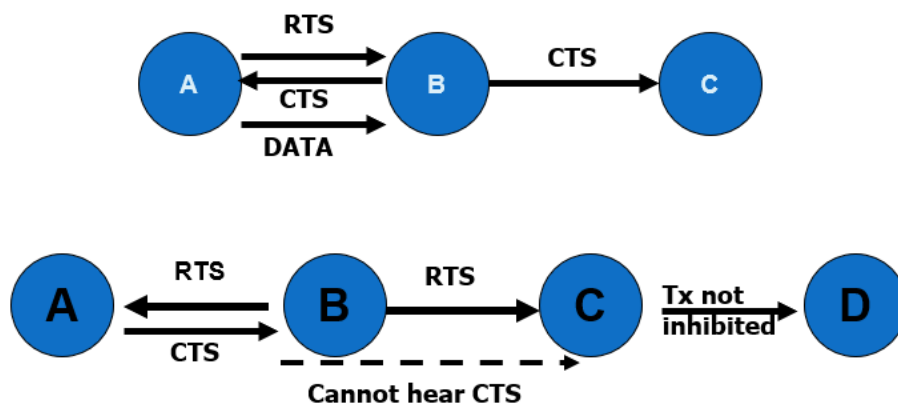
作用：时间同步可以减少设备之间的竞争，降低不必要的延迟。

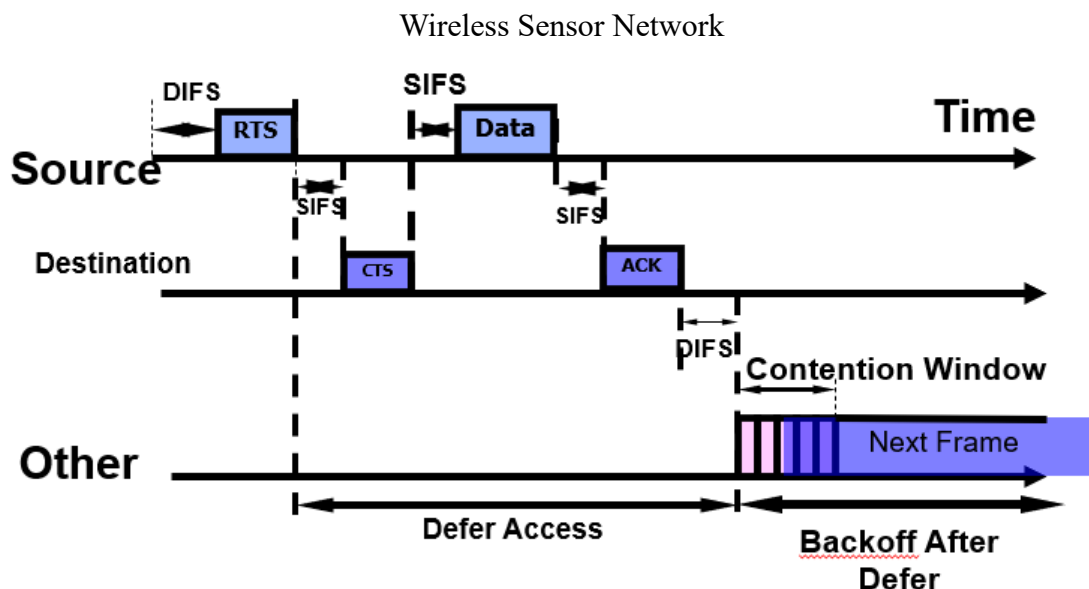
（四）带 RTS/CTS 的分布式无线介质访问控制协议

RTS/CTS 机制的作用：

带 RTS/CTS 的分布式无线介质访问控制协议通过引入 RTS（请求发送）和 CTS（清除发送）控制帧，解决了隐藏终端和暴露终端问题，进一步提高了信道利用率和数据传输的可靠性。在这种协议中，发送方在发送数据前首先发送一个 RTS 帧，请求接收方的发送权限。接收方在收到 RTS 帧后，回应一个 CTS 帧，表示同意接收数据。通过这种方式，可以有效地避免数据传输过程中的冲突。

带RTS/CTS的分布式无线介质访问控制协议





RTS/CTS 机制的工作流程:

发送方检测到信道空闲时间大于 DIFS 时, 发送一个 RTS 帧, 通知接收设备和周围的设备, 它准备发送数据。接收方在接收到 RTS 帧后, 如果检测到信道空闲时间大于 SIFS 时, 回应一个 CTS 帧, 通知发送设备和周围的设备, 它准备接收数据。RTS 帧和 CTS 帧都包含发送方地址、接收方地址和包大小 (预计传输的数据长度) 等信息, 周围节点收到后会设置 NAV (网络分配向量), 在数据传输期间保持静默。发送方在收到 CTS 帧后, 开始发送数据帧 (实际要传输的数据)。其他节点通过监听 RTS 和 CTS 帧, 了解信道的使用情况, 从而避免在数据传输过程中发生冲突。接收设备在成功接收到数据帧后, 同样等待 SIFS 间隔, 随后发送一个 ACK 帧 (包含接收设备的地址), 通知发送设备数据已成功接收。当 ACK 帧发送完毕, 整个传输周期结束, 信道重新空闲。此时, 之前处于静默状态的其他节点, 会先等待一个 DIFS 间隔, 然后进入 Contention Window (竞争窗口), 通过随机退避机制抢占信道, 准备发送自己的下一帧数据。

优点:

减少冲突——通过预先的握手过程, 确保发送设备和接收设备之间的通信是独占的, 减少数据冲突。

解决隐藏终端问题——通过 RTS 和 CTS 帧的广播, 使得隐藏终端能够感知到数据传输, 从而避免冲突。

提高信道利用率——通过协调设备之间的传输, 减少不必要的重传和延迟, 提高信道利用率。

缺点:

增加开销——RTS 和 CTS 帧的传输增加了额外的开销, 可能会降低网络的整体效率。

增加延迟——握手过程增加了数据传输的延迟, 特别是在低负载情况下。

注意:

RTS/CTS 机制并不能完全消除冲突, RTS 帧在发送时仍然可能发生碰撞; 当节点经历 RTS 冲突后, 会采用二进制指数退避方案重新竞争信道。不过, 由于 RTS 帧本身长度远小于 DATA 帧 (RTS 包比 DATA 短), RTS 冲突所造成的信道资源浪费和传输开销, 都比直接发生 DATA 冲突要小得多, 这也是 RTS/CTS 机制的核心优势之一。

RTS (Request To Send) 包 (20 字节): 在发送数据前, 发送方用一个短的 RTS 包向接收方请求发送权限。

CTS (Clear To Send) 包 (16 字节): 一旦就绪, 接收方给予发送方发送数据的权限。

包含: (发送方地址; 接收方地址; 包大小)

网络分配向量（NAV）

作用：

NAV 是 802.11 协议中用于实现虚拟载波侦听的重要机制。它通过在数据帧中设置时间持续字段，将介质预留一段固定时间，从而实现对信道的虚拟忙状态管理。NAV 机制能够有效减少信道感知的开销，提高信道利用率。通过这种方式，可以更好地协调节点之间的通信，避免冲突，提高网络的性能和可靠性。

NAV 的工作流程：

发送方在发送数据帧时，将 NAV 设置为期望使用信道的时间。当其他节点收到 RTS/CTS 帧时，会从中读出一个叫“Duration（持续时间）”的字段，这个字段里写着“本次从 CTS 到 ACK 结束，一共要占用信道多久”，只要 NAV 大于 0，其他节点就会认为信道是忙的，了解到这个信道预留情况之后它们就会把这个时间写入自己的 NAV 计时器，然后就进入“静默模式”——在 NAV 倒计时归零前，无论物理上有没有听到信号，都认为信道是“忙”的，绝不发送任何数据，从而推迟自己的传输。这样一来，发送方和接收方之间的数据传输就不会被打扰，完美解决了隐藏终端问题。当 NAV 为 0 时，其他节点可以开始竞争信道。通过这种方式，NAV 机制能够有效管理信道的使用，避免冲突，提高网络的性能和可靠性。

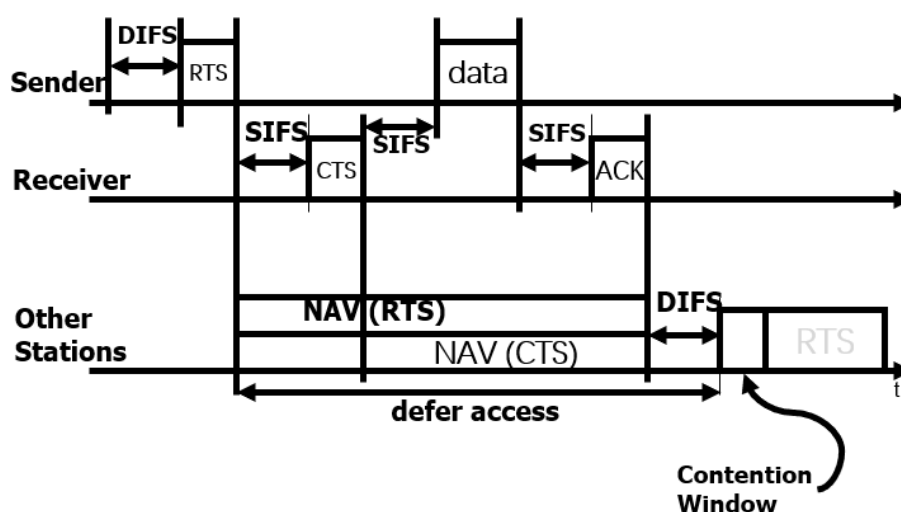
虚拟载波侦听（Virtual Carrier Sense）

是无线局域网（如 Wi-Fi）中的一种机制，用于通过协议层面的信息来预测介质的繁忙状态，而不仅仅依赖于物理层的信号检测。虚拟载波侦听通过使用 NAV 来实现，帮助设备更有效地管理介质访问，减少冲突和提高网络效率。

工作原理：

虚拟载波侦听依靠网络分配向量 NAV 实现，NAV 是用于指示介质未来占用时长的计时器，可帮助设备预判信道忙闲状态，避免在信道被占用时发送数据。无线帧头部设有持续时间字段（Duration Field），指示本次传输将占用信道的总时长；其他设备收到帧后，会读取该字段并据此更新自身 NAV，以此完成虚拟载波侦听。

带RTS/CTS的分布式无线介质访问控制协议(网络分配向量)



虚拟载波侦听的工作流程：

发送方检测到信道空闲时间大于 DIFS 后发送 RTS 帧，接收方收到后等待 SIFS 回复 CTS 帧，两者均携带本次传输的持续时间信息；其他节点监听到 RTS 或 CTS 帧后读取帧头中的持续时间字段以了解当前传输将占用介质的时间，更新自身 NAV 计时器，在 NAV

归零前推迟访问信道并保持静默，避免在介质被占用时尝试发送数据；发送方收到 CTS 后等待 SIFS 发送数据帧，接收方成功接收后再等待 SIFS 回复 ACK 帧；当 NAV 归零，且物理层载波侦听确认信道空闲后，其他节点等待 DIFS 间隔，再进入竞争窗口执行随机退避，重新参与信道竞争，该机制有效解决了隐藏终端问题，降低了数据冲突概率，提升了无线信道利用率与传输可靠性。

第六讲（数据链路层（MAC 子层）协议）

S-MAC 协议原理与机制

基本概念：

①周期休眠与侦听机制

S-MAC 协议中，节点进入周期休眠模式，关闭射频模块并设置唤醒计时器。当计时器到期，节点醒来侦听是否有数据要发送给自己。这种机制有效减少了节点的能耗，解决了“空闲侦听”消耗大量能量的问题。

休眠时间与侦听周期是应用相关的，所有节点都设置为同一值，以确保节点间的协调工作。通过这种方式，S-MAC 协议在保证通信的同时，最大限度地降低了节点的能耗。

②同步机制与调度维护

周期同步包的交换维持同步调度，同步期是节点发送 SYNC 包的时间段。接收节点在收到 SYNC 包后，立即调整计时器，确保节点间的同步性。这种同步机制保证了节点在休眠和侦听状态的协调一致。每个节点维护一个包含所有已知邻居调度信息的调度表。节点通过侦听信道一段时间来获取邻居的调度信息，并根据这些信息选择自己的调度。若节点在选择自己的调度前收到邻居节点的 SYNC 包，则跟随邻居节点的调度。

③冲突避免与数据传输

S-MAC 基于竞争，当多个邻居同时想向同一邻居发送数据时，节点开始侦听并尝试传输数据。与 IEEE802.11 一样，S-MAC 使用 RTS/CTS 机制解决隐藏终端问题，启动传输前进行载波侦听。每个数据包中的持续字段表明余下的传输将持续多长时间。若一个节点接收到自己非目标节点的包，它将知道休眠多长时间。节点记录 NAV 中的值，并为之设置一个计数器。当节点有数据需发送时，它首先查看 NAV，若该值不为 0，则介质忙。

文献：Medium Access Control With Coordinated Adaptive Sleeping for Wireless Sensor Networks

S-MAC (Sensor-MAC) 是一种专为无线传感器网络设计的介质访问控制协议。其核心目标是通过协调自适应睡眠机制显著降低能耗，延长网络寿命。S-MAC 通过以下关键技术实现低占空比操作：

①周期性睡眠与监听：节点周期性关闭射频模块以减少空闲监听能耗。

②虚拟集群同步：相邻节点通过同步睡眠计划减少控制开销。

③自适应监听：通过监听 RTS/CTS 信号预测数据传输时间，减少多跳延迟。

④消息传递与冗余接收避免：通过长消息分片传输减少控制包开销，并避免非目标节点的冗余接收。

关于 WSN 中的能源效率问题，具体挑战包括：

①空闲监听 (Idle Listening)：节点长期处于接收状态导致能耗浪费（占接收能耗的 50%-100%）。

②数据碰撞 (Collision)：多节点竞争信道导致重传与额外能耗。

③冗余接收 (Overhearing)：节点接收非目标数据包。

④控制包开销：RTS/CTS 等控制包占用带宽与能量。

⑤多跳网络中的延迟累积：传统协议在多跳场景下因节点睡眠导致延迟线性增长。

问题提出的基础：

传感器网络具有以下特性，使得传统无线 MAC 协议（如 802.11）不适用：

①能源受限：节点依赖电池供电，需长期运行（如环境监测）。传感器节点靠电池供电，野

外、地下等场景无法频繁充电 / 换电池，要求网络能长期工作。

而 IEEE 802.11 (WiFi) 设计优先保证传输性能，节点会持续监听信道，空闲状态也不断耗电，电池很快耗尽，完全不适合传感器网络。

②低占空比需求：多数时间无数据，突发流量需快速响应。802.11 节点全程保持活跃、随时收发，长期空闲监听造成大量电量浪费。

③多跳协作：数据需通过多跳传输，但传统协议（如 TDMA）难以动态调整。TDMA 协议靠固定时隙分配通信，节点增减、拓扑变化时，时隙很难动态调整，扩展性差。

④应用容忍延迟：监测类应用可接受秒级延迟，但需平衡延迟与能耗。传统无线协议（802.11）把低时延、高吞吐量作为核心目标，为了极致性能不惜消耗大量电能；传感器网络则可以主动牺牲一部分时延，换取更低功耗、更长的电池寿命。

问题的解决方法：

S-MAC 通过以下机制解决上述问题：

①周期性睡眠与虚拟集群同步：节点默认低占空比操作，通过同步睡眠计划形成虚拟集群，减少同步开销；周期性广播 SYNC 包维护时钟同步，容忍时钟漂移。

②自适应监听 (Adaptive Listen)：邻居节点通过监听 RTS/CTS 预测数据传输结束时间，提前唤醒以减少多跳延迟。

③避免冗余接收：邻居节点在收到 RTS/CTS 后进入睡眠，避免监听长数据包。

④消息传递 (Message Passing)：将长消息分片为突发传输，仅需一次 RTS/CTS 交互，减少控制开销。

解决方法和现有方法的区别与联系：

对比维度	传统方法（如 IEEE 802.11）	S-MAC
能耗优化目标	次要目标，主要关注带宽利用与公平性	核心目标，通过低占空比、冗余接收避免、消息传递降低能耗。
空闲监听处理	节点持续监听信道，仅在空闲超时后进入节能模式（可选）。	默认低占空比操作，周期性睡眠为基本模式。
多跳同步	假设单跳网络，难以适应多跳拓扑（如 802.11 PS 模式）。	虚拟集群同步，允许不同步的邻居共存，通过自适应监听减少延迟。
冗余接收处理	无专门机制，邻居节点持续监听所有传输。	基于 NAV 的睡眠机制，邻居节点在数据传输期间休眠。
消息传输	分片传输需多次 RTS/CTS 交互，公平性优先。	消息分片突发传输，牺牲公平性以降低控制开销与延迟。

一、什么是「仅需一次 RTS/CTS 交互」

先对比 IEEE 802.11 和 S-MAC 的差异：

1. 传统 802.11 做法

如果要发送一条长消息，会把它拆成多个独立小数据分片。

每发送一个分片，都要完整走一遍：RTS → CTS → 分片数据 → ACK。

也就是每一个分片都要单独竞争信道、单独握手，RTS/CTS 反复执行，控制帧开销大、耗电、延迟高。

2. S-MAC 消息传递机制（一次 RTS/CTS 交互）

把整条长消息的所有分片当成一整批连续数据：

- 只在整批分片开始前，做一次 RTS/CTS 握手，一次性预约整条消息的信道占用时间；
- 之后所有分片连续突发发送，分片之间不再重复发起 RTS/CTS；
- 每个分片发完仍回复 ACK 做校验，出错只重传当前分片，不释放信道。

简单总结：

一次 RTS/CTS 交互 = 一次握手预约信道，连续传完所有数据分片，不再重复握手。

作用：大幅减少控制帧数量，降低能耗与传输延迟。

二、S-MAC 为什么「牺牲公平性」

1. 先理解 MAC 层公平性

信道公平：多个节点同时有数据要发时，每个节点获得平等的信道使用机会，不会出现一个节点长期霸占信道、其他节点一直等待。

IEEE 802.11 严格保证公平：

哪怕一个节点有大量分片，发完一个分片后也要释放信道，重新参与竞争，把机会让给其他节点。

2. S-MAC 如何牺牲公平性

S-MAC 为了节能、降低延迟，采用整批分片连续突发传输：

- 某个节点一旦通过一次 RTS/CTS 拿到信道，就会连续把所有分片全部发完，全程不释放信道；
- 这段时间里，其他有短数据包的邻居节点，哪怕优先级更高、等待更久，也必须一直等到整批消息传输结束，才能竞争信道。

举例

节点 A 有 10 个分片（长消息），节点 B 只有 1 个小包：

- 802.11：A 发 1 个分片 → 让信道 → B 可以发 → A 再竞争信道发下一个分片，两者机会均等，保证公平。
- S-MAC：A 一次握手后连续发完 10 个分片，全程占用信道，B 只能一直等待。

从传统 MAC 标准来看：单个节点长期占用信道，其他节点被插队、等待，就是牺牲了节点间的信道公平性。

3. 为什么 S-MAC 主动放弃公平性？

这是场景取舍：

无线传感器网络的目标不是“人人平等上网”，而是协同完成感知、数据采集、网内数据聚合。

1. 传感器节点为同一个任务工作，节点间不需要严格公平；
2. 频繁切换信道、反复 RTS/CTS 会增加能耗和延迟，违背传感器网络优先节能的核心需求；
3. 长消息（如一组采集数据）需要完整接收才能做聚合、计算，拆分传输反而会拉高整体时延。

因此 S-MAC 主动牺牲单节点信道公平性，换取更低能耗、更低传输时延。

三、精简版（直接写笔记 / 答题）

1. 仅需一次 RTS/CTS 交互

S-MAC 将长消息拆分为多个数据分片，仅在整批分片传输前进行一次 RTS/CTS 握手预约信道，所有分片连续突发传输，分片之间不再重复执行 RTS/CTS 交互，以此减少控制帧开销与能耗。

2. S-MAC 牺牲公平性的原因

传统 802.11 要求每个数据分片传输后释放信道，保证所有节点平等竞争信道（信道公平）。而 S-MAC 节点一旦获取信道，会连续传完所有分片、长期占用信道，其他节点只能等待，破坏了节点间的信道公平性。

由于传感器网络以协同工作、节能为首要目标，可容忍该问题，因此协议主动牺牲公平性，换取更低的能耗与传输延迟。

解决方法存在的问题：

①动态拓扑适应性：S-MAC 依赖“虚拟集群同步”，大家一起睡一起醒。但如果有新节点加入、旧节点没电关机，整个同步节奏就容易乱套。为了维持同步，节点必须每 2 分钟就广播一次“邻居发现包”，频繁收发这些控制包会额外消耗电量，和“节能”的初衷有点矛盾。

Wireless Sensor Network

②延迟与吞吐量折中：低占空比操作导致吞吐量下降。为了省电，节点大部分时间都在睡觉，只在短时间内醒来传数据。这就导致两个问题：延迟变大（数据要等到下一个“醒来窗口”才能发出去）和吞吐量下降。

③复杂环境下的可靠性：S-MAC 里的“自适应监听”机制，需要邻居节点成功收到 RTS/CTS 信号，才能提前醒来准备接收数据。但如果环境复杂、信道有干扰，RTS/CTS 包可能丢包，邻居节点就不知道要提前醒，数据传输就会失败，导致可靠性下降。

④实验场景局限性：测试基于理想环境，而实际部署中多路径衰落与移动性可能影响性能。
改进方法建议：

①动态占空比调整：根据网络负载自适应调整睡眠周期（如流量高时增加监听时间），平衡延迟与能耗。（流量低时，多睡会儿，省点电；流量高时，多醒一会儿，提高吞吐量、降低延迟。）

②混合协议设计：结合 TDMA（时分复用，无冲突）与 CSMA（竞争访问，灵活）优势，静态时段分配关键控制包（如 SYNC），动态时段使用竞争机制传输数据。

③增强同步鲁棒性：引入基于贝叶斯估计的时钟漂移补偿算法，不用那么频繁发 SYNC 包也能维持同步，减少控制包开销，更省电。

④机器学习驱动的睡眠策略：用 LSTM 这种机器学习模型，预测网络什么时候会有突发流量，提前把节点唤醒，不用一直等固定的唤醒窗口，既能降低延迟，又不用一直保持高占空比耗电。

⑤跨层优化：联合 MAC 层与路由层设计，路由选路时优先选择能量充足的中继节点，电量少的节点就少干点活，以此延长网络整体寿命。

S-MAC 协议性能分析：

①能耗特性与节能优势

S-MAC 协议消耗的能量比无休眠机制的类 802.1 协议少。S-MAC 通过减少空闲侦听与优化长消息传输而节省能量。在重负载下，空闲侦听很少发生，休眠节省的能量有限。在轻负载下，周期休眠起关键作用。通过合理设置休眠时间和侦听周期，S-MAC 协议能够有效降低节点的能耗，延长网络的使用寿命。

②时延与冲突率问题

S-MAC 依靠周期性休眠调度降低节点能耗，却也带来了明显的时延问题：节点多数时间处于休眠状态，数据需等待节点唤醒后才能转发，多跳传输场景下时延会不断累积，因此该协议不适用于对实时性要求高的应用。为改善这一缺陷，S-MAC 设计了自适应侦听机制，为节点提供多跳感知功能，节点可通过监听到的 RTS、CTS 帧预判传输结束时间并提前唤醒，有效降低周期休眠造成的多跳延迟。但由于节点仍受休眠周期约束，且信道干扰可能导致唤醒失效，所以自适应侦听只能减小时延，不能将其完全消除。

③适用场景与局限性

S-MAC 协议在处理长数据时，把长数据拆成多个分片，只做一次 RTS/CTS 握手预约信道，然后连续发完所有分片。这就意味着，发送长数据的节点会长时间霸占信道，其他有短数据的节点只能一直等待，无法获得信道使用权，这会导致公平性问题。这些问题限制了 S-MAC 协议在某些特定场景下的应用。

TRAMA 协议原理与机制

基本概念：

①基于时隙的结构

TRAMA 协议是一种基于时隙的结构，时间分为随机访问时期和调度访问时期。随机访问时期用于发送控制信息，如同步与更新两跳邻居信息；调度访问时期用于节点间无竞争的数据交换。这种基于时隙的结构使得 TRAMA 协议能够有效地避免冲突，提高信道利用率。通过合理分配时隙，节点可以在无竞争的环境中进行数据传输，从而提高通信效率。

②TRAMA 组件与功能

TRAMA 协议包含三个主要组件：邻居协议（NP）、调度交换协议（SEP）和自适应选择算法（AEA）。邻居协议用于收集 2 跳邻居信息，调度交换协议用于收集 1 跳的通信调度信息，自适应选择算法用于选择发送方与接收方。邻居协议在随机访问期使用控制信息收集 2 跳邻居信息，若无更新，控制信息包作为“keep alive”信标。调度交换协议基于数据包产生速率，每个节点计算调度间隔，并将时间调度表通报给邻居节点。自适应选择算法使用节点标号和全局哈希函数计算优先级，为节点分配时隙。

③时隙分配与调度传播

TRAMA 协议通过信号时隙（对应随机访问时期）和传输时隙（对应调度访问时期）实现时隙分配与调度传播。信号时隙由邻居协议使用，在随机访问期传播一跳邻居信息，获取一致的两跳拓扑信息。传输时隙用于无冲突的数据交换和调度传播。节点使用调度时间表交换协议与邻居节点交换基于通信情况的信息，通过自适应选择算法选择发送方与接收方，让其他节点自由选择低功率休眠模式。

文献：Energy-Efficient, Collision-Free Medium Access Control for Wireless Sensor Networks

总体描述文献内容：

TRAMA (Traffic-Adaptive Medium Access) 是一种新型的无线传感器网络介质访问控制协议，旨在通过动态调度和流量适配机制实现无冲突通信与能量高效。TRAMA 的核心设计包括 3 个部分：

①邻居协议（NP）：交换两跳邻居信息以维护拓扑一致性。

不仅要知道谁能直接听到我说话（1 跳），还要知道听我说话的人还能听到谁说话（2 跳）。为什么要管这么宽？因为要彻底解决无线通信里的“隐藏终端问题”，必须保证在我的两跳范围内，同时只有一个节点在发言。另外，有一个保活机制：如果没有更新，节点就会定期发一个简短的“Keep Alive”信号。

②调度交换协议（SEP）：广播流量调度信息，指示节点在特定时隙的接收目标。

节点根据自己手里有多少数据包要发，算出一个“调度窗口”，然后用一个位图（Bitmap）告诉邻居：“在接下来的某某时隙，我要发给谁，其他人可以去睡觉了。”另外，有一个动态重用机制：如果某节点本来占了位置，但临时没数据发了，它就会释放这个时隙，让别的邻居顶上。

③自适应选举算法（AEA）：基于优先级和流量需求动态选举发送节点，非发送/接收节点进入低功耗睡眠模式。

到了每一个传输时隙，大家都用“节点 ID + 全局哈希函数”算一个优先级分数。得分最高且确实有数据要发的节点，赢得该时隙的发送权。被该发送者指定的目标接收节点，强制保持清醒准备接收。其他既不用发、也不用收的节点，立刻进入低功耗睡眠模式。

实验表明，TRAMA 在能量节省和吞吐量上明显优于竞争型协议（如 CSMA、802.11、S-MAC）和调度型协议（如 NAMA）。

研究聚焦于解决无线传感器网络中以下关键问题：

①能量浪费：传统竞争型协议（如 CSMA、802.11）因空闲监听、数据冲突和控制包开销导致能耗过高。

②冲突与低吞吐量：隐藏终端问题和随机竞争机制降低信道利用率。

Wireless Sensor Network

③静态调度的局限性：现有调度协议（如 NAMA）未考虑能量优化，且无法动态适应流量变化。

④广播/组播支持不足：多数协议仅优化单播场景，忽略传感器网络中常见的多播需求。

问题提出的基础：

①能源受限：节点依赖电池供电，需最大化寿命（如环境监测场景需运行数月）。

②流量突发性与稀疏性：事件触发型应用（如火灾监测）长期空闲，但突发时需快速响应。

③多跳动态拓扑：节点可能因能量耗尽失效或新节点加入，需要有自组织能力。

④应用多样性：需同时支持单播、广播和组播（如区域温度采集需多播至多个汇聚节点）。

问题的解决方法：

TRAMA 的核心机制如下：

①时间分片与调度：时间划分为随机访问时隙（信令交互）和调度访问时隙（数据传输）。

节点通过 NP 协议维护两跳邻居信息，SEP 协议广播调度窗口（含目标接收节点的位图）。

②自适应选举算法（AEA）：每个时隙基于哈希函数计算节点优先级，优先级最高且需发送数据的节点获得信道。非目标接收节点进入睡眠状态，减少空闲监听。

③动态调度重用：节点若在调度窗口内无数据发送，则释放时隙供其他节点重用，提升信道利用率。

解决方法和现有方法的区别与联系：

对比维度	现有方法	TRAMA
能量优化机制	竞争型协议（如 S-MAC）依赖固定占空比睡眠，调度型协议（如 NAMA）无睡眠优化。	动态睡眠：非发送 / 接收节点强制睡眠，能量节省与流量动态适配。
冲突避免	竞争型协议通过 RTS/CTS 减少冲突，但仍存在隐藏终端问题。	无冲突调度：基于优先级选举确保两跳范围内仅一个节点发送。
多播支持	多数协议仅优化单播，广播依赖简单竞争（易冲突）。	位图指示接收节点：支持单播、广播、组播，且无冲突。
拓扑适应性	静态调度协议（如 TDMA）难以适应节点动态加入 / 退出。	NP 协议维护两跳邻居信息，支持动态拓扑更新。

解决方法存在的问题：

①同步开销大：NP 和 SEP 协议需频繁交换邻居与调度信息，控制包占用带宽。

②高延迟：调度机制导致数据排队延迟，尤其在均匀流量场景下（实验显示 TRAMA 延迟高于 NAMA）。

③计算复杂度：AEA 需维护两跳邻居信息并计算优先级，对资源受限节点负担较重。

④动态场景适应性：节点快速加入/退出时，调度更新延迟可能导致临时信道浪费。

改进方法建议：

①轻量级信息交换：采用增量更新代替全量广播，仅传输拓扑变化部分（如新加入节点信息）。

②混合调度机制：事件触发时隙分配。在低负载时使用竞争机制（如 CSMA），高负载时切换为调度模式，平衡延迟与能耗。

③预测性优先级计算：利用流量历史数据训练轻量级模型（如线性回归），预测节点优先级，减少实时计算开销。

④分簇调度优化：将网络划分为簇，簇内使用 TRAMA 调度，簇间通过低功耗路由协议（如 LEACH）通信，降低全局同步成本。

⑤自适应时隙长度：根据流量负载动态调整时隙长度（如突发流量时扩展时隙），减少空闲时间占比。

TRAMA 协议性能分析：

①能效与冲突避免

Wireless Sensor Network

TRAMA 协议通过基于时隙的结构和自适应选择算法，实现了能源有效的冲突避免。节点在无竞争的时隙中进行数据传输，减少了冲突的发生，提高了信道利用率。通过合理分配时隙，TRAMA 协议能够在高负载下有效避免冲突，保证通信的稳定性。

②时延与调度开销

TRAMA 协议的时延主要由时隙分配和调度传播引起。由于需要进行复杂的时隙分配和调度传播，TRAMA 协议的时延相对较高。此外，显式调度传播的开销也较大，增加了协议的复杂性和通信成本。这些因素在一定程度上限制了 TRAMA 协议的性能，尤其是在对时延要求较高的应用场景中。

③适用场景与局限性

TRAMA 协议适用于对能效要求较高、通信冲突较多的无线传感器网络。通过基于时隙的结构和自适应选择算法，TRAMA 协议能够有效避免冲突，提高信道利用率。然而，TRAMA 协议的复杂选择算法和数据结构增加了协议的实现难度和通信开销。此外，更高的排队时延也限制了 TRAMA 协议在某些实时性要求较高的场景中的应用。

Z-MAC 协议原理与机制

基本概念:

Z-MAC 协议是一种混合 MAC 协议，结合了 CSMA 和 TDMA 的优点。

CSMA 属于竞争型协议，其核心特点是“先听后发、按需接入”，节点仅在发送需求时去随机竞争抢占信道，这使得它在低负载下无需时间同步、随来随发且延迟极低，但在高负载下会因多节点无序抢占导致冲突剧烈、重传激增乃至网络瘫痪；反之，TDMA 属于调度型协议，它通过“严格时分、独占时隙”的方式将时间切块并固定分配给每个节点，虽需要高精度的全网时间同步，且在低负载无数据发送时会因时隙空置而造成资源浪费与不必要的排队延迟，但它在重负载下能够实现全网完美无冲突，提供稳定、可预测的吞吐量与服务质量（QoS）保证。

它使用由 DRAND（分布式时隙分配，每个节点只跟两跳内节点协商，以保证任意两个两跳内节点时隙不同，而两跳外可以复用同一个时隙，空间复用，省时隙）创建的 TDMA 作为传输调度的参考，在访问介质时，时隙拥有者具有比非拥有者更高的优先级。与 TDMA 不同，若时隙拥有者无数据传输，非拥有者可以使用该时隙。这种混合机制使得 Z-MAC 协议能够在不同竞争级别下灵活切换，兼顾了信道利用率和公平性。

Z-MAC 协议根据竞争级别在 CSMA 和 TDMA 之间切换。在低竞争级别时，Z-MAC 的性能像 CSMA，具有高的信道利用率和低时延；在高竞争级别时，Z-MAC 的性能像 TDMA，具有高的信道利用率、公平性和低的竞争开销。这种基于竞争级别的切换机制使得 Z-MAC 协议能够根据网络的实际需求动态调整协议的行为，从而在不同的网络环境下都能保持较好的性能。

Z-MAC 协议通过时隙分配和优先级机制实现高效的信道访问。每个节点根据自身的数据传输需求和时隙分配情况，计算优先级并选择合适的时隙进行数据传输。在时隙分配过程中，Z-MAC 协议考虑了节点的竞争级别和数据传输需求，确保时隙的合理分配和高效利用。通过这种方式，Z-MAC 协议能够在不同竞争级别下实现良好的性能平衡。

文献：Z-MAC: A Hybrid MAC for Wireless Sensor Networks

总体描述文献内容:

本文提出了一种新型混合 MAC 协议 Z-MAC (Zebra MAC)，结合了 TDMA（时分多址）和 CSMA（载波侦听多址）的优势，以解决无线传感器网络（WSN）中的能量效率和信道利用率问题。Z-MAC 的核心设计包括以下关键特性：

- ①动态模式切换：低竞争时采用类似 CSMA 的随机接入机制，高竞争时切换为类似 TDMA 的调度机制。
- ②优先级机制：通过调整竞争窗口大小，赋予时隙“所有者”更高的访问优先级，减少冲突。
- ③显式竞争通知（ECN）：节点通过 ECN 消息通知两跳邻居（避免隐藏终端问题）进入高竞争模式（HCL），优化调度。
- ④本地时间同步：仅需局部时钟同步，降低全局同步开销。
- ⑤鲁棒性设计：容忍时隙分配错误、时钟漂移和动态拓扑变化，极端情况下退化为 CSMA。

实验基于 TinyOS 平台和 Mica2 节点，结果表明 Z-MAC 在高竞争下吞吐量较 B-MAC 提升 40%，能量效率显著优于传统协议，并支持单播、组播和广播流量。

文献研究的问题:

- ①CSMA 的局限性：高竞争场景下隐藏终端问题导致频繁冲突，RTS/CTS 控制开销大（占带宽 40% 以上）。
- ②TDMA 的不足：静态调度难以适应动态拓扑和流量变化，时钟同步开销高，信道利用率低。
- ③混合协议的需求：现有混合协议（如 S-MAC、T-MAC）无法平衡低延迟与高吞吐量，且

对同步和拓扑变化敏感。

④能量效率与实时性矛盾：传统协议难以在低功耗和高性能间取得平衡。

问题提出的基础：

①能源受限：节点依赖电池供电，需最大化寿命（如环境监测需运行数月）。

②动态拓扑：节点可能因能量耗尽失效或新节点加入，需自组织能力。

③流量多样性：从低速率事件驱动到高速率实时监控，需灵活适应流量变化。

④干扰不规则性（Interference Irregularity）：通信范围与干扰范围不一致，静态调度难以避免冲突。

问题的解决方法：

①混合接入机制：

低竞争模式（LCL）——非所有者节点可“窃取”空闲时隙，类似 CSMA。

高竞争模式（HCL）——仅所有者及其一跳邻居可竞争当前时隙，类似 TDMA。

②动态时隙分配：使用分布式算法 DRAND 分配时隙，确保两跳邻居无冲突；本地帧规则（TF Rule），节点根据局部拓扑动态调整帧长，提升信道复用率。

③显式竞争通知（ECN）：节点通过噪声退避次数检测高竞争，广播 ECN 触发邻居进入 HCL 模式；抑制机制避免 ECN 泛洪，仅关键节点转发通知。

当节点收到两跳 ECN 时，表明其所在两跳范围内竞争剧烈、隐藏终端引发冲突严重，LCL 的宽松竞争机制已无法有效抑制冲突；因此必须切换至 HCL，通过限制两跳外节点接入、提升时隙所有者优先级，将网络收敛为类 TDMA 调度状态，从而降低冲突率、保障重负载下的吞吐量。

④优先级控制：所有者使用更小的竞争窗口（如 8 时隙），非所有者使用较大窗口（如 32 时隙），减少冲突。

⑤轻量级同步：基于 RTP/RTCP 的局部同步协议，仅发送节点需周期性同步，接收节点被动同步。

解决方法和现有方法的区别与联系：

对比维度	现有方法（如 S-MAC、B-MAC、PTDMA）	Z-MAC
竞争处理	S-MAC 依赖固定占空比，B-MAC 无冲突避免机制，PTDMA 需全局同步。	动态模式切换：LCL 类似 CSMA，HCL 类似 TDMA，适应流量变化。
时隙分配	TDMA 需集中式调度（如 NAMA），难以适应动态拓扑。	分布式 DRAND 算法：局部时隙分配，支持动态节点加入 / 退出。
同步开销	PTDMA 需严格全局同步，能耗高。	局部同步：仅发送节点同步，接收节点被动调整，降低开销。
鲁棒性	传统 TDMA 对时隙分配错误和时钟漂移敏感。	退化为 CSMA：时隙冲突或同步失败时，性能不劣于 CSMA。
多跳支持	S-MAC 依赖虚拟集群同步，拓扑变化时延迟高。	ECN 机制：通过两跳通知优化冲突避免，提升多跳吞吐量。

解决方法存在的问题：

①初始化开销：DRAND 时隙分配和全局同步（如 TPSN）在部署初期消耗较多能量。

②复杂参数调优：竞争窗口大小、帧长等参数需根据场景手动调整。

③局部同步局限性：低流量节点并不能频繁收发数据包，没有机会及时校准修正，时钟漂移的累积会导致局部不同步，影响 HCL 模式性能。

④ECN 延迟：ECN 检测依赖噪声退避统计，高动态场景下响应速度不足。

⑤实验场景限制：测试基于静态拓扑和固定流量模式，移动和突发流量下的性能未充分验证。

改进方法建议：

①轻量化初始化流程

按需时隙分配——仅在高竞争区域触发 DRAND，减少全局开销。

增量同步——新节点加入时复用现有时隙分配，避免全网重调度。

②自适应参数调整

动态竞争窗口——根据实时冲突率调整竞争窗口大小，平衡延迟与吞吐量。

机器学习预测——利用历史流量数据训练模型，动态优化帧长和 ECN 触发阈值。

③增强同步鲁棒性

混合时钟源——结合硬件时钟（如晶振）与软件同步，降低漂移影响。

冗余同步消息——关键路径节点增加同步频率，提升局部稳定性。

④跨层优化

路由-MAC 协同——根据路由负载动态调整 MAC 模式（如高负载路径强制 HCL）。

能量感知调度——优先分配时隙给剩余能量高的节点，延长网络寿命。

⑤移动性支持

时隙预留机制——为移动节点预留弹性时隙，减少拓扑变化引发的冲突。

快速邻居发现——优化 ping 消息间隔，加速新节点融入调度体系。

S-MAC、TRAMA、Z-MAC 三种协议对比分析

①S-MAC 协议能效特点

S-MAC 协议通过周期休眠和侦听机制有效降低了节点的能耗。在轻负载下，周期休眠起关键作用，能够显著减少节点的空闲侦听时间，从而降低能耗。然而，在重负载下，S-MAC 协议的节能效果有限，因为此时空闲侦听很少发生。此外，S-MAC 协议在处理长数据时，将数据分割成小的数据包，增加休眠时间，但可能导致公平性问题，从而影响整体能效。

②TRAMA 协议能效特点

TRAMA 协议通过基于时隙的结构和自适应选择算法实现了能源有效的冲突避免。节点在无竞争的时隙中进行数据传输，减少了冲突的发生，提高了信道利用率，从而降低了能耗。然而，TRAMA 协议的复杂选择算法和数据结构增加了协议的实现难度和通信开销，显式调度传播的开销也较大，这些因素在一定程度上影响了协议的能效。

③Z-MAC 协议能效特点

Z-MAC 协议通过混合机制和基于竞争级别的切换实现了较高的信道利用率和公平性。在低竞争级别时，Z-MAC 协议像 CSMA 一样，能够快速响应数据传输需求，提高信道利用率；在高竞争级别时，Z-MAC 协议像 TDMA 一样，能够有效避免冲突，保证公平性。

与 S-MAC 和 TRAMA 协议相比，Z-MAC 协议在能效方面具有明显的优势。通过合理的时隙分配和优先级机制，Z-MAC 协议能够在不同竞争级别下实现良好的能效平衡，从而满足不同应用场景的需求。

第七讲

WSN 网络层基础与特点

作用：

WSN 是由大量传感器节点组成的分布式自组织网络。为完成特定任务，节点需要将采集或检测数据传送给汇聚节点做进一步处理，供终端用户使用。由于节点能力有限，WSN 采用多跳方式进行数据传输，大多需要中间节点进行转发。

路由协议将数据分组从源节点通过网络转发至目的节点，主要包括两方面功能：寻找源节点和目的节点间的优化路径，将数据分组沿着优化路径正确转发。

WSN 中的网络层协议主要负责数据的路由和传输，确保数据能够从源节点有效地传递到目标节点。网络层协议负责选择从源节点到目标节点的最佳路径，即进行路由选择。由于无线传感器网络中的节点通常能量有限且分布密集，路由选择需要考虑多种因素，如能量消

耗、路径长度、网络拓扑等。常见的路由协议包括基于地理位置的路由、基于能量的路由和多路径路由等。

网络层作用与传输模式

多跳数据传输模式：

WSN 由大量传感器节点组成，节点能力有限，采用多跳方式传输数据。大多需中间节点转发，以完成数据从采集节点到汇聚节点的传递，供终端用户进一步处理使用。例如，一个环境监测场景中，分布在不同位置的传感器节点采集温度、湿度等数据，通过多跳方式将数据传至汇聚节点，再由汇聚节点汇总分析，为用户提供整体环境信息。

路由协议核心功能：

路由协议负责将数据分组从源节点通过网络转发至目的节点，主要功能包括寻找源节点和目的节点间的优化路径，以及将数据分组沿着优化路径正确转发。以智能家居系统为例，传感器节点检测到异常情况，如烟雾报警，路由协议需快速找到一条最优路径，将报警信息及时传至控制中心节点，确保信息准确传达，保障家庭安全。

WSN 路由协议

特点：

①能量效率优先考量

WSN 节点常由电池供电，能量有限，尽量减少能耗、延长网络生存期是路由协议设计重要目标。选择数据传输路径时，需考虑节点能量消耗及网络能量均衡，如果节点很快耗尽电量，就会导致网络瘫痪。可以通过选择能量高效的路径、减少数据传输量、动态调整传输功率和休眠机制等方法来提升能量效率。采用能量优先的路由协议，可合理分配能量，延长网络运行时间。

②自组织性

WSN 通常是自组织的，节点可以动态加入或离开网络。路由协议需要能够适应网络拓扑的变化，自动调整路由。可以通过分布式算法和自适应机制来实现自组织性。

③基于局部拓扑信息

WSN 采用多跳通信模式，节点存储和计算资源有限，难以存储大量路由信息和进行复杂路由计算。在只能获取局部拓扑信息和资源有限条件下，实现简单高效路由机制是基本问题。以农田灌溉监测网络为例，每个传感器节点仅了解其周围少数节点信息，通过基于局部拓扑信息的路由协议，节点可快速找到合适转发路径，将土壤湿度等数据传至汇聚节点，实现精准灌溉控制。

④以数据为中心

传统路由协议以地址标识节点确定传输路径，WSN 大量节点随机部署，关注的是感兴趣区域（ROI）的感知数据，是一种以数据为中心的网络。用户关心指定区域数据，而非特定节点数据。在城市交通流量监测场景中，分布在各路口的传感器节点采集车流量数据，以数据为中心的路由协议会根据数据需求，将各路口数据汇聚至中心节点，为交通管理部门提供整体交通流量信息，辅助交通疏导决策。

⑤应用相关性

WSN 针对具体应用设计部署，不同应用对 WSN 要求不同，应用环境区别造成网络通信模式不同，无通用路由机制适合所有应用。需针对具体用户需求实现适应路由协议。比如，工业生产中的设备故障监测网络与环境监测网络在数据采集频率、传输实时性要求等方面差异很大，需分别设计适应其应用特点的路由协议，以满足不同场景下的监测需求。

挑战：

①邻居节点发现

路由协议要求相邻节点交换信息，不同算法产生不同结果。地理路由需知邻居节点位置信息，以数据为中心路由需节点观测值。在不影响路由选择精度条件下，需减少局部信息交换。例如，在大型仓库的货物库存监测网络中，传感器节点频繁发现邻居节点位置信息以建

立路由，会消耗大量能量。采用优化算法减少不必要的邻居节点信息交换，可降低能耗，延长网络寿命。

②通信与计算能耗

来自不同节点的数据需在信息内容不被损坏前提下融合至同一数据包，减少数据流量。在每个中继节点进行数据处理，避免路由选择信息冗余。以海洋水质监测网络为例，多个传感器节点采集的水质数据在中继节点融合处理，减少传输至汇聚节点的数据量，降低通信能耗，同时提高数据传输效率。

③网络全局信息获取

WSN 由大量节点高密度部署组成，大量节点阻碍网络全局信息获取，分布式协议缺乏网络全局拓扑认识，需协议具有可扩展性。在智能城市的大规模传感器网络中，节点数量众多，难以获取完整全局拓扑信息。具有良好可扩展性的路由协议可适应节点数量增加，确保网络稳定运行，实现城市多维度数据监测。

④局部信息交换限制

同样由于节点密度高，需限制局部信息交换以降低网络功耗。在不增加功耗前提下，协议应支持网络内部来自节点的信息融合。以大型农场的土壤养分监测网络为例，高密度部署的传感器节点若频繁进行局部信息交换，会增加功耗。限制局部信息交换并支持信息融合的路由协议，可在保证数据传输效率的同时，降低功耗。

⑤唯一固定地址分配困难

WSN 中节点数量大，难以为每一个节点分配唯一固定地址。用户兴趣为来自多个节点的关于同一现象的信息，而非来自单个传感器的信息。在大规模气象监测网络中，节点数量庞大，为每个节点分配唯一固定地址不现实。采用无需每个节点都有唯一地址的寻址机制，可满足用户对特定区域气象数据的需求，简化网络管理。

⑥鲁棒性要求

WSN 多跳通信模式，在中间节点上运行路由协议，节点为低成本组件易故障，无线信道通信易丢包。路由协议应提供单节点鲁棒性，防止单节点失效，且有效性不依赖于单个数据包。例如，在工业生产环境中的设备状态监测网络中，节点可能因恶劣环境或设备故障失效。具有良好鲁棒性的路由协议可在节点失效时，快速重新建立路由，确保数据传输不间断，保障生产过程正常运行。

分类：

在 WSN 中，路由协议可以分为平面路由协议和分层路由协议两大类。

①平面路由协议

平面结构网络特点：

用于平面结构的网络，所有节点平等，每个节点在路由功能上的地位相同。不需要设置特殊功能的节点，易于实现。例如，在小型智能家居系统中，所有传感器节点地位平等，采用平面路由协议，每个节点都参与路由决策，可快速实现数据传输，满足家庭环境监测等简单应用场景需求。

平面路由协议优势：

实现简单，易于部署和维护。由于所有节点功能相同，无需复杂节点角色划分和管理，降低了网络部署和维护成本。

平面路由协议（Flat Routing Protocol）是一种简单的路由结构，其中所有节点在网络中具有相同的角色和功能，没有层次结构或分簇机制。以下是平面路由协议的详细解释：

（1）平等性：在平面路由协议中，所有节点在网络中具有相同的地位和功能，没有专门的簇头节点或汇聚节点。优点是简化了网络结构，减少了管理和维护的复杂性。

（2）分布式控制：路由决策是分布式的，每个节点根据局部信息独立做出路由决策。优点是提高了网络的鲁棒性和自适应性。

（3）简单性：平面路由协议通常具有简单的路由算法和协议机制。优点是易于实现和部

署，适合小规模网络。

②分层路由协议

分层结构网络组成：

用于分层结构的网络，节点被分成多个簇，每个簇由簇头和若干个簇成员组成。簇成员节点将数据传送给簇头，簇头负责收集和处理簇内所有成员节点采集的数据，并将收集和处理后的数据经过其他簇头传送给汇聚节点。例如，在大型工业园区的环境监测网络中，采用分层路由协议，将园区划分为多个区域，每个区域形成一个簇，簇头节点负责汇总本区域数据，再传至汇聚节点，可有效管理大量节点，提高数据传输效率。

分层路由协议优势：

通过簇头节点集中管理和数据处理，可有效减少网络中的数据传输量，降低能耗。同时，簇头节点可对簇内数据进行预处理，提高数据质量。

分层路由协议（Hierarchical Routing Protocol）是一种将网络节点组织成层次结构的路由方法。与平面路由协议不同，分层路由协议通过将网络划分为不同的层次或簇（Cluster），来优化能量消耗、提高网络的可扩展性和管理效率。以下是分层路由协议的详细解释：

（1）层次结构：网络节点被组织成多个层次或簇，通常包括簇头节点（Cluster Head）和普通节点（Cluster Member）。优点是层次结构简化了网络管理，提高了路由效率。

（2）能量效率：簇头节点负责数据聚合和转发，减少了普通节点的能量消耗。优点是集中管理和数据聚合，延长了网络的生命周期。

（3）可扩展性：分层结构使得网络能够更好地适应大规模部署，减少了路由开销。优点是提高了网络的可扩展性，适合大规模 WSN。

③基于能量的路由协议

能量感知路由策略：

在确定数据传输路径时将节点的剩余能量和链路传输功率作为主要考虑因素，以获得最优能量效率的传输路径。可采用不同能量感知策略，如节点最大剩余能量路由、最小功率路由、节点最大化最小剩余能量路由等。例如，在偏远山区的野生动物监测网络中，传感器节点依靠电池供电，采用基于能量的路由协议，根据节点剩余能量选择传输路径，可延长网络运行时间，确保持续监测野生动物活动。

基于能量路由优势：

有效延长网络生存期，提高网络整体性能。通过合理分配能量，避免部分节点过早耗尽电量导致网络瘫痪，确保网络长期稳定运行。

基于能量的路由协议（Energy-Based Routing Protocol）是一类专门设计来优化能量消耗和延长网络生命周期的路由协议。这些协议在选择路由路径时，优先考虑节点的剩余能量，以确保能量消耗均衡，避免某些节点过早耗尽能量而失效。以下是基于能量的路由协议的详细解释：

（1）能量感知：路由决策基于节点的剩余能量，优先选择能量较高的节点进行数据传输。优点是均衡能量消耗，延长网络的生命周期。

（2）能量高效：通过优化路由路径，减少高能量消耗的传输，提高整体能量效率。优点是减少能量消耗，提高网络的可持续性。

（3）动态调整：根据节点的能量状态动态调整路由路径，适应网络的变化。优点是提高网络的适应性和鲁棒性。

④基于多路径的路由协议

多路径传输原理：

路由协议根据可选传输路径数量分为单路径和多路径路由协议。单路径路由传送到同一目的节点的数据总是采用同一条传输路径，可能造成网络负载和能耗不均衡。多路径路由传送到同一目的节点的数据可选择多条不同路径，可根据平均分配、按比例分配、随机分配等规则分配。例如，在工业生产中的设备故障监测网络中，采用多路径路由协议，当数据传输

出现拥塞或节点故障时，可自动切换至其他路径，确保数据及时传输，提高网络可靠性。

多路径路由优势：

均衡网络中的流量分布、带宽资源和能量消耗，提高网络性能，延长网络工作寿命。对丢失率高的网络，多路径可实现数据冗余传输，提高数据传送成功率。

基于多路径的路由协议（**Multipath Routing Protocol**）是一种通过建立和维护多条路径来传输数据的路由方法。与单一路径路由协议不同，多路径路由协议利用多条路径来提高网络的可靠性、容错性和负载均衡能力。以下是基于多路径的路由协议的详细解释：

（1）多条路径：同时建立和维护多条从源节点到目标节点的路径。优点是提高数据传输的可靠性和容错性。

（2）负载均衡：通过在多条路径上分配数据流量，均衡网络负载。优点是减少单一路径的拥塞，提高网络效率。

（3）容错性：当某条路径失效时，可以快速切换到其他路径，确保数据传输的连续性。优点是提高网络的鲁棒性和可靠性。

⑤基于移动性的路由协议

移动汇聚节点应用：

所有节点静止不动时，汇聚节点周围区域易产生热点效应。使用移动汇聚节点，可均衡网络负载和能耗。在传感器节点和移动汇聚节点间建立传输路径。例如，在大型农场的农作物生长监测网络中，采用移动汇聚节点，可定期在不同区域收集数据，避免固定汇聚节点周围区域节点能耗过快耗尽，延长网络寿命。

移动性路由优势：

有效解决汇聚节点周围热点效应问题，均衡网络负载和能耗，提高网络整体性能和寿命。同时，移动汇聚节点可根据需要灵活调整位置，适应不同监测场景需求。

基于移动性的路由协议（**Mobility-Based Routing Protocol**）是一类专门设计来处理网络中节点移动性的路由协议。这些协议考虑了节点的移动性，能够适应网络拓扑的动态变化，确保数据在节点移动的情况下仍然能够可靠传输。以下是基于移动性的路由协议的详细解释：

（1）动态拓扑适应：能够适应网络中节点的移动性，动态调整路由路径。优点是提高网络的适应性和鲁棒性。

（2）路径维护：定期检测和更新路由路径，确保路径的有效性。优点是减少因节点移动导致的数据丢失和延迟。

（3）低延迟：通过快速适应拓扑变化，减少数据传输的延迟。优点是提高实时性要求高的应用的性能。

⑥基于位置的路由协议

位置信息利用：

协议根据传感器节点的位置、邻居节点位置、目的节点位置等信息进行多跳分组转发，直至分组到达目的节点。节点位置信息可通过 GPS 或基于网络的定位技术获取。例如，在智能交通系统中的车辆定位监测网络中，传感器节点通过 GPS 获取自身位置信息，根据车辆位置和目的地位置，采用基于位置的路由协议，将车辆行驶数据实时传输至交通管理中心，实现交通流量优化。

基于位置路由优势：

可根据节点位置信息快速确定数据传输路径，提高数据传输效率。同时，位置信息可用于优化网络拓扑结构，提高网络性能。

基于位置的路由协议（**Location-Based Routing Protocol**）是一类利用节点的地理位置信息来进行路由决策的协议。这些协议通过使用节点的位置信息来选择最优的路径，从而提高路由效率、减少能量消耗并增强网络的可靠性。以下是基于位置的路由协议的详细解释：

（1）地理位置信息：路由决策基于节点的地理位置信息，如 GPS 坐标或通过其他定位技术

获得的位置数据。优点是利用位置信息可以优化路径选择，减少不必要的跳数和能量消耗。

(2) 能量效率：通过选择最短或最优的地理路径，减少数据传输的能量消耗。优点是延长网络的生命周期，提高能量效率。

(3) 可扩展性：基于位置的路由协议通常具有良好的可扩展性，适合大规模网络。优点是能够适应大规模部署，减少路由开销。

⑦基于机会的路由协议

机会路由原理：

机会路由或机会转发，在确定传输路径和转发数据过程中，充分利用无线媒体的广播特性（单次发送，可能被多个相邻节点收到）提供的各种转发机会，提高丢失率高的 WSN 的路由性能。例如，在恶劣环境下的地质灾害监测网络中，传感器节点通信易受干扰，采用基于机会的路由协议，可充分利用无线广播特性，提高数据传输成功率，确保灾害预警信息及时传达。

机会路由优势：

提高单跳传输可靠性、降低端到端延迟、提高网络吞吐量。在高丢包率网络环境下，机会路由可有效提高数据传输效率，确保网络稳定运行。

基于机会的路由协议（Opportunistic Routing Protocol）是一种利用网络中随机出现的传输机会来进行数据传输的路由方法。与传统的确定性路由协议不同，基于机会的路由协议不预先确定固定的路径，而是根据当前的网络状况动态选择下一跳节点。以下是基于机会的路由协议的详细解释：

(1) 动态路径选择：根据当前的网络状况动态选择下一跳节点，不预先确定固定的路径。

优点是提高网络的适应性和鲁棒性。

(2) 利用传输机会：利用网络中随机出现的传输机会，如节点的移动、信道质量的变化等。优点是提高数据传输的成功率和效率。

(3) 容错性：当某条路径失效时，可以快速切换到其他路径，确保数据传输的连续性。优点是提高网络的可靠性和容错性。

⑧以数据为中心的路由协议

数据驱动传输机制：

汇聚节点向指定区域发送查询消息，并接收和处理指定区域内传感器节点发送来的数据，感知到特定现象的节点将感知数据向汇聚节点汇报。例如，在城市空气质量监测网络中，汇聚节点可向指定区域发送查询指令，区域内传感器节点将空气质量数据传输至汇聚节点，实现对特定区域空气质量的实时监测。

数据中心路由优势：

以数据为中心的传输机制符合 WSN 应用需求，用户可直接获取感兴趣区域的数据，无需关注具体节点信息。同时，可根据数据需求动态调整数据传输路径，提高网络灵活性和适应性。

以数据为中心路由协议（Data-Centric Routing Protocol）是一种将数据的特性和内容作为路由决策核心的路由方法。与传统的以地址为中心的路由协议不同，以数据为中心的路由协议关注的是数据本身，而不是数据的来源或目的地。以下是这种路由协议的详细解释：

(1) 数据特性：路由决策基于数据的特性和内容，如数据类型、数据重要性等。优点是提高数据传输的效率和针对性。

(2) 数据聚合：在传输过程中对数据进行聚合，减少冗余数据的传输。优点是减少能量消耗和网络拥塞。

(3) 查询驱动：路由协议通常由查询驱动，即根据用户或应用的查询需求进行数据传输。优点是提高数据传输的灵活性和响应性。

第八讲（网络层（路由）协议）

数据汇报方式概述

①周期性数据汇报

节点在网络部署完成后，按固定时间间隔向汇聚节点汇报感知数据，无需实时触发，适用于稳定监测场景，如环境温度监测，可定时获取数据变化趋势。此方式便于数据同步与分析，但可能产生大量冗余数据，尤其在数据变化不明显时，增加网络传输负担。

适用于数据变化缓慢且规律性强的场景，如土壤湿度监测，可设定每小时汇报一次，便于长期观察土壤水分变化，为农业灌溉提供精准数据支持；在工业生产中，对设备运行参数的周期性监测，如电机转速、温度等，可及时发现设备异常，预防故障发生。

②事件驱动数据汇报

当特定事件发生时，节点才向汇聚节点汇报，如火灾传感器检测到烟雾，立即上报，具有高实时性，能快速响应紧急情况。节点大部分时间处于低功耗状态，只有在事件触发时才工作，有效节省能源，延长网络寿命。

能有效减少网络中无用数据传输，提高数据传输效率，避免因大量无效数据导致的网络拥塞。在安防监控领域，如入侵检测系统，一旦检测到异常入侵行为，立即触发报警并上报，保障安全。

③基于查询数据汇报

需建立汇聚节点同被查询节点间的路径，由汇聚节点向传感器节点扩散查询消息，节点按需响应，实现数据按需获取。汇聚节点可灵活查询不同节点数据，提高数据获取的针对性，减少无效数据传输。

在智能交通系统中，交通管理中心可按需查询各路段传感器的交通流量数据，实时掌握路况，优化交通信号控制。在环境监测中，研究人员可随时查询特定区域的传感器数据，如空气质量、水质等，便于开展研究工作。

Flooding 协议（网络层的泛洪路由协议）

核心思想：

当一个节点收到数据包时，它会将数据包广播给所有邻居节点，邻居节点继续重复这一过程，直到数据包到达目标节点或达到最大跳数限制。

工作原理：

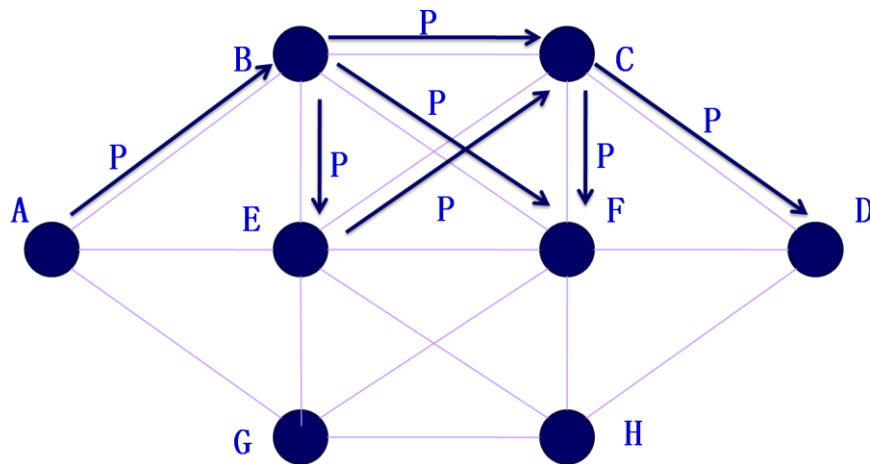
源节点广播数据包给所有邻居节点，每个收到数据包的节点再次广播给其邻居节点。

注意！终止条件：数据包到达目标节点，或达到预设的最大跳数（TTL，Time to Live）。

特点：

- （1）无状态：节点不需要维护路由表或拓扑信息。
- （2）简单易实现：无需复杂的路由算法或网络发现机制。
- （3）高可靠性：由于数据包被广泛传播，泛洪协议在理论上能够确保数据包到达目标节点。

Wireless Sensor Network



一个节点产生或接收数据后，以广播方式向所有相邻节点发送或转发，直至数据到达目的节点或 TTL 到期，过程重复执行。该机制简单直接，无需复杂路由计算，能快速将数据扩散至整个网络，但易引发信息内爆、重叠及资源浪费等问题。每个节点只需将接收到的数据包进行广播，无需查找路由表、选择下一跳节点，也无需特殊算法保持网络拓扑信息更新。简化了节点的路由计算与维护工作，降低了节点的硬件与软件要求，但牺牲了网络资源利用效率。

优点：

(1) 路径容错性好（可靠性高）

数据通过多条路径传输，即使部分路径出现故障，数据仍可通过其他路径到达目的节点，提高数据传输可靠性。适用于对数据可靠性要求高、网络拓扑动态变化的应用场景，如军事侦察、灾害预警等，确保关键数据能成功送达。

(2) 传输延时短

数据无需复杂路由计算与选择，直接广播扩散，能快速到达目的节点，降低传输延时。在实时性要求高的应用中，如远程医疗设备监测，可及时获取患者生命体征数据。

(3) 简单性

无需复杂的路由计算或拓扑维护，适合资源受限的传感器节点。部署成本低，适用于临时性或小规模网络。

(4) 鲁棒性

对节点故障和链路中断具有较强的容忍能力。在网络初始化或拓扑未知的情况下，泛洪协议是一种有效的通信方式。

缺点：

(1) 信息内爆现象（数据冗余）

网络中目标节点收到一个数据的多个副本，导致大量重复数据传输，增加网络负载，降低传输效率，甚至可能导致网络拥塞和冲突，也增加了数据处理和存储的开销。例如在密集部署的传感器网络中，一个数据包可能被多个节点重复转发，使汇聚节点收到大量重复数据。

(2) 部分重叠现象

由于节点密集部署，若干节点对同一事件反应相同，所感知信息在数据性质、数值上相似，邻居节点收到的数据副本相关性大。如在火灾监测场景中，多个相邻传感器可能同时检测到烟雾，向汇聚节点发送大量相似数据，造成数据冗余。

(3) 高能耗

数据包被广播到所有节点，导致大量冗余传输，增加了能量消耗。在密集网络中，能量浪费尤为严重。

(4) 资源利用不合理

Wireless Sensor Network

每个节点单纯广播数据，不考虑节点能量消耗、下一跳节点可行性，缺乏自适应性，导致网络资源浪费。在大规模网络中，这种无差别的广播方式会使节点能量快速耗尽，降低网络生命周期。

（5）可扩展性差

随着网络规模增大，泛洪协议的能量消耗和网络负载呈指数增长。不适合大规模或高密度部署的传感器网络。

1. 信息内爆：同一个包，被重复发 N 遍

比如你发的那个 Flooding 示意图里：

- 源节点 A 发出了数据包 P ，
- 路径 1: $A \rightarrow B \rightarrow C \rightarrow D$
- 路径 2: $A \rightarrow B \rightarrow E \rightarrow C \rightarrow D$
- 路径 3: $A \rightarrow B \rightarrow F \rightarrow D$
- 路径 4: $A \rightarrow B \rightarrow E \rightarrow F \rightarrow D$

目标节点 D 收到的，全都是 A 发出的同一个数据包 P 的多个副本。

- 这些包的内容、ID 完全一样，只是走了不同的路径，在网络里被重复传输了很多次。
- 这就是“信息内爆”：同一个数据在传输中被无限复制，导致大量重复传输。

2. 部分重叠：不同节点，发了内容很像的包

比如火灾监测场景：

- 节点 1、节点 2、节点 3，三个相邻的传感器，同时检测到了烟雾浓度超标。
- 它们各自生成了数据包：
 - 节点 1：“烟雾浓度 = 80%”
 - 节点 2：“烟雾浓度 = 78%”
 - 节点 3：“烟雾浓度 = 82%”
- 这三个包，是不同节点生成的不同数据包，但内容高度相似，都在说“这里有火灾”。
- 这就是“部分重叠”：感知到的数据本身就高度相关，导致汇聚节点收到了大量内容重复的“信息”，但它们不是同一个包的副本。

改进：

（1）选择性泛洪（Selective Flooding）

核心思想：节点根据一定的规则选择性地转发数据包，而不是盲目广播。

实现方式：节点以一定概率转发数据包，减少冗余传输。节点根据与目标节点的距离决定是否转发数据包。

优点：减少能量消耗和网络拥塞。

缺点：可能降低数据包的到达率。

（2）基于 TTL 的泛洪（TTL-Based Flooding）

核心思想：通过设置数据包的最大跳数（TTL）限制泛洪范围。

实现方式：每经过一个节点，TTL 值减 1，当 TTL 为 0 时停止转发。

优点：控制泛洪范围，减少能量消耗。

缺点：需要合理设置 TTL 值，否则可能导致数据包无法到达目标节点。

（3）基于邻居信息的泛洪（Neighbor-Aware Flooding）

核心思想：节点根据邻居节点的信息优化转发策略。

实现方式：节点记录已转发数据包的邻居节点，避免重复广播。使用邻居表或广播历史表（Broadcast History Table）减少冗余。

优点：显著降低冗余传输和能量消耗。

缺点：需要额外的存储和计算资源。

Wireless Sensor Network

（4）基于能量感知的泛洪（Energy-Aware Flooding）

核心思想：优先选择能量充足的节点进行数据转发。

实现方式：节点根据剩余能量决定是否转发数据包。高能量节点承担更多转发任务，低能量节点减少转发。

优点：延长网络寿命，均衡能量消耗。

缺点：需要实时监测节点能量状态，增加复杂度。

应用场景：

（1）网络初始化

在网络部署初期，节点之间缺乏拓扑信息，泛洪协议可用于快速建立通信链路。

（2）紧急事件通知

在火灾监测、地震预警等紧急场景中，泛洪协议可确保关键信息快速传播。

（3）小规模网络

在节点数量较少、能量资源充足的网络中，泛洪协议是一种简单有效的通信方式。

（4）动态拓扑网络

在节点移动频繁或拓扑变化快的网络中，泛洪协议具有较强的适应性。

Gossiping 路由协议

核心思想：

Gossiping 路由协议是一种基于随机转发的路由机制。通过随机选择邻居节点，进行局部信息交换来实现数据的传播，适用于资源受限的无线传感器网络。

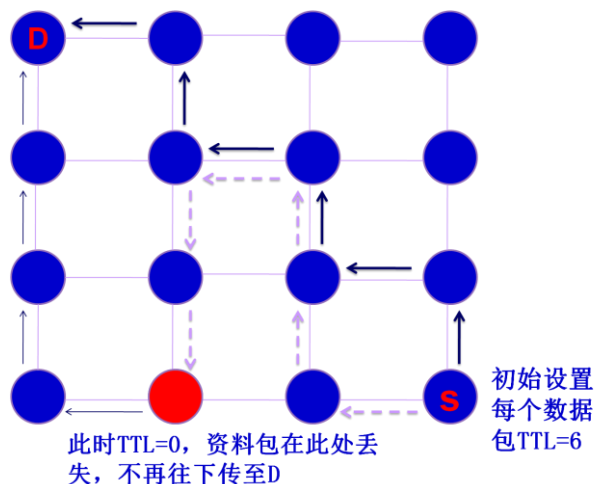
工作原理：

源节点生成数据包，并将其发送给随机选择的一个邻居节点。每个收到数据包的节点以一定的概率（通常小于 1）随机选择一个邻居节点转发数据包。

注意！终止条件：数据包在达到目标节点或经过一定跳数后停止转发。

特点：

- （1）随机性：每个节点独立决定是否转发数据包，转发概率可以固定或动态调整。
 - （2）局部信息：节点仅依赖本地信息（如邻居列表）进行决策，无需全局网络拓扑信息。
 - （3）避免泛洪：通过随机转发，减少了数据包的冗余传输。
-



节点接收到数据后，按一定概率随机选择一个邻居节点进行转发，直至数据到达汇聚节点。该机制在一定程度上减少了数据的广播量，降低了信息内爆与重叠现象。而且考虑了节点能量消耗，每次只选择一个节点进行转发，避免了节点过度广播导致的高能耗与冗余。有助于延长网络中节点的使用寿命，提高网络的整体性能。

优点：

- （1）节能

相比泛洪协议，Gossiping 显著减少了数据包的重复传输，降低了能耗。适用于能量受限的无线传感器网络。

- （2）简单性

协议逻辑简单，易于实现，适合计算能力和存储资源有限的传感器节点。

- （3）减少冗余：

通过随机转发，避免了泛洪协议中大量冗余数据包的问题，减少了网络拥塞。

缺点：

- （1）延迟较高

由于数据包通过随机路径传输，可能导致较长的传输延迟。不适用于对实时性要求较高的应用场景。而且路由选择不理想，未采用路径优化算法，所选路由可能增加数据包端到端延时，或在到达目的节点前生命周期结束。在大规模网络中，可能导致数据传输效率低下，影响应用性能。

- （2）可靠性低

Wireless Sensor Network

随机转发可能导致数据包无法到达目标节点，尤其是在稀疏网络中。数据包的传递成功率依赖于转发概率和网络密度。在对数据可靠性要求较高的场景中，如金融数据传输，可能无法满足需求。

（3）网络拓扑依赖

协议性能受网络拓扑结构影响较大，节点分布不均匀可能导致性能下降。

（4）能量不均衡

某些节点可能被频繁选中转发数据包，导致能量消耗不均衡，缩短网络生命周期。

改进：

（1）动态转发概率

根据网络状态（如节点能量、网络密度，高则提高）动态调整转发概率，以提高可靠性和降低延迟。

（2）混合路由协议

将 Gossiping 与其他路由协议（如 AODV、DSDV，相当于指定了路径，增加了可靠性）结合，形成混合路由协议，以兼顾节能和可靠性。

（3）能量感知机制

在转发数据包时优先选择能量较高的节点，以平衡网络能量消耗。

（4）地理信息辅助

结合地理位置信息，优先选择靠近目标节点的邻居进行转发，以减少延迟。

（5）多路径转发

通过多条路径同时转发数据包，提高数据传递的可靠性。

SPIN 路由协议

SPIN (Sensor Protocols for Information via Negotiation) 是一种基于协商的路由协议。SPIN 协议通过元数据 (metadata) 描述传感器节点收集的数据, 并在数据传输前进行协商, 以减少冗余数据的传输, 从而降低能耗并提高网络效率。SPIN 协议的核心思想是通过“协商”机制优化数据传播, 避免泛洪协议中的冗余问题。

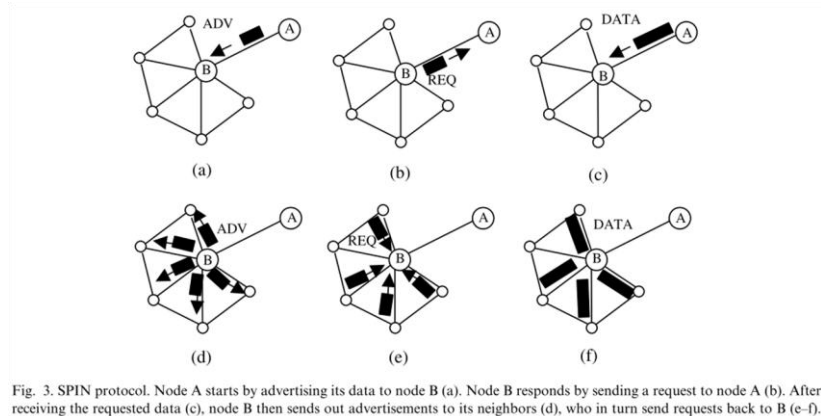


Fig. 3. SPIN protocol. Node A starts by advertising its data to node B (a). Node B responds by sending a request to node A (b). After receiving the requested data (c), node B then sends out advertisements to its neighbors (d), who in turn send requests back to B (e-f).

图中 (a)-(f) 的分步解析

1. 节点 A → 节点 B: ADV 广播 (图 a)

- 节点 A 生成了新数据, 它先向邻居节点 B 发送 ADV 包 (Advertisement, 数据广告), 只包含数据的元信息 (比如数据类型、ID、大小), 而不是完整数据。
- 目的: 告诉邻居“我有新数据, 你们要不要?”, 避免一上来就发大包浪费能量。

2. 节点 B → 节点 A: REQ 请求 (图 b)

- 节点 B 收到 ADV 后, 如果它没有这份数据, 并且对数据感兴趣, 就会向节点 A 回复一个 REQ 包 (Request, 数据请求), 表示“我想要这份数据”。
- 如果节点 B 已经有了这份数据, 或者不感兴趣, 就不会回复 REQ, 传输直接终止, 节省开销。

3. 节点 A → 节点 B: DATA 发送 (图 c)

- 节点 A 收到节点 B 的 REQ 请求后, 确认对方确实需要数据, 就把完整的 DATA 包 (实际数据) 发送给节点 B。
- 到这里, A 和 B 之间的一次完整 SPIN 传输就完成了。

4. 节点 B 向所有邻居广播 ADV (图 d)

- 节点 B 收到完整数据后, 会像节点 A 一样, 向自己的所有邻居节点 (除了 A) 广播 ADV 包, 把“有新数据”的消息扩散出去。
- 这一步就像接力一样, 让数据的广告在网络里传播, 但依然只传元信息, 不发完整数据。

5. 邻居节点向节点 B 回复 REQ (图 e)

- 收到 ADV 的邻居节点, 如果对数据感兴趣, 就会向节点 B 发送 REQ 请求, 和之前 B 向 A 请求的逻辑一样。

6. 节点 B 向请求的邻居发送 DATA (图 f)

- 节点 B 收到 REQ 后, 向请求的邻居节点发送完整的 DATA 包, 完成数据转发。
- 这个过程会在网络中不断接力, 直到所有需要数据的节点都收到数据, 同时避免了大量无效传输。

SPIN 协议的核心优势 (对比泛洪)

- 避免数据冗余: 只有感兴趣的节点才会请求并接收数据, 不会像泛洪那样把数据发给所有节点, 大幅减少了无效传输。

2. 减少内爆问题：通过 ADV/REQ 协商机制，避免了数据包在网络中无限复制，降低了信道冲突和能量消耗。
 3. 适配 WSN 场景：节点资源有限，先协商再传输的方式，能显著延长网络生命周期。
-

一句话总结流程

SPIN 协议通过 ADV（广告）→ REQ（请求）→ DATA（数据）的 3 步协商，实现了“按需传输”：先告诉邻居我有什么，等对方明确要了，再发完整数据，以解决泛洪的冗余问题。

工作流程：

当一个源节点有新数据时，它会广播一个 ADV 消息，其中包含数据的元数据（描述数据的摘要信息）。如果某个邻居节点对 ADV 消息中的数据感兴趣，它会发送一个 REQ 消息请求完整数据。发送 ADV 消息的源节点在收到 REQ 消息后，向请求节点发送 DATA 消息，即把完整的数据发送给请求节点。收到 DATA 消息的节点重复上述过程，将数据传播到整个网络。

目标：

（1）解决传统协议问题

解决 Flooding 与 Gossiping 协议的内爆、重叠及资源利用不合理现象，提高数据传输效率与网络性能。通过节点间协商建立传输路径，优化数据传输过程。

（2）提高数据传输效率

通过协商机制，确保只有有用信息被传输，减少冗余数据，提高网络资源利用率。

数据包类型：

（1）ADV 数据包

路由请求发起数据包，节点接收到数据包后向邻居节点广播 ADV 数据包，通告是否需要接收数据。由于 ADV 数据包体积小，消耗能量资源少，可有效降低网络能耗。

（2）REQ 数据包

请求响应数据包，邻居节点接收 ADV 数据包后，若需要接收数据，则向请求发起节点发送 REQ 数据包。实现了节点间的双向通信与协商，提高了数据传输的针对性。

（3）DATA 数据包

包含传感采集的数据内容，是实际传输的数据包，带有元数据头。元数据头用于描述数据的空间、时间等属性，便于节点进行数据比较与处理。

协商机制：

（1）协商过程

节点在传输数据前进行协商，通过 ADV、REQ 与 DATA 三种数据包的交互，建立传输路径。协商过程有效避免了信息内爆与重叠现象，提高了数据传输效率。

（2）元数据作用

元数据用于完全描述数据，必须小于实际数据，便于节点快速比较与处理。不同应用的元数据具有特定格式，如传感器可使用地理位置或节点 ID 作为元数据。

（3）资源自适应算法

能量充足时，使用三阶段握手协议通信；能量接近低阈值时，节点接收 ADV 后不发送 REQ，保留能量用于感知事件。该算法提高了网络的资源自适应性，延长了网络寿命。

优点：

（1）节能

通过元数据协商机制，减少了冗余数据的传输，降低了能耗。

（2）减少冗余

只有对数据感兴趣的节点才会请求数据，避免了泛洪协议中的冗余问题。通过协商机制与元数据管理，有效解决了传统协议的问题，提高了数据传输效率与网络性能。

（3）自适应性强

SPIN 协议能够适应网络拓扑的变化，适合动态网络环境。

(4) 简单高效

协议逻辑简单，易于实现，适合资源受限的传感器节点。每个节点接收新数据时决策少，无需转发表。

缺点：

(1) 元数据开销

元数据的传输增加了额外的通信开销，尤其是在数据量较小时，可能得不偿失。

(2) 可靠性问题

如果中间节点对数据不感兴趣，可能导致数据无法传播到目标节点。在网络分区或稀疏网络中，数据传播可能失败。

(3) 能量不均衡

靠近源节点的节点可能因频繁参与数据转发而能量消耗过快。

(4) 不适用于高密度网络

在高密度网络中，ADV 和 REQ 消息的广播可能导致信道拥塞。协商过程增加了数据传输的复杂性，可能导致传输延迟增加。

应用场景：

(1) 数据为中心的网络

如环境监测、农业监测等，节点对特定类型的数据感兴趣。

(2) 动态网络

如移动传感器网络，网络拓扑可能频繁变化。

(3) 中等密度网络

节点密度适中，避免因高密度导致的信道拥塞。

性能分析：

(1) 能耗

SPIN 协议通过减少冗余数据传输降低了能耗，但元数据的传输增加了额外开销。

(2) 延迟

由于协商机制，SPIN 协议的延迟较高，尤其是在大规模网络中。

(3) 可靠性

在网络分区或稀疏网络中，数据传播可能失败，可靠性较低。

(4) 可扩展性

适用于中等规模的网络，但在高密度网络中性能可能下降。

Flooding、Gossiping、SPIN 三种协议对比分析

①Flooding 与 Gossiping 对比：

Flooding 协议简单直接，路径容错性好，传输延时短，但易引发信息内爆、重叠及资源浪费问题。Gossiping 协议是 Flooding 的改进，考虑了节点能量消耗，减少了数据广播量，但路由选择不理想，可能导致数据传输效率低下。

②SPIN 与传统协议对比：

SPIN 协议通过协商机制与元数据管理，有效解决了传统协议的问题，提高了数据传输效率与网络性能。但 SPIN 协议数据广播开销较大，协商过程增加了传输延迟，适用于对数据传输效率与可靠性要求较高的场景。

不同场景下的协议选择：

在对数据可靠性要求高且网络规模较小的场景中，可选择 Flooding 协议，如军事侦察、灾害预警等。

在考虑节点能量消耗且网络规模较大的场景中，可选择 Gossiping 协议，如环境监测、智能家居等。

在对数据传输效率与可靠性要求高且网络规模较大的场景中，可选择 SPIN 协议，如智

能交通、工业自动化等。

协议选择需综合考虑网络规模、节点能量、数据传输要求等因素，以达到最佳的网络性能与应用效果。在实际应用中，可根据具体需求对协议进行优化与改进，提高网络的适应性与性能。

第九讲

定向扩散路由协议（Directed Diffusion）

概述：

Directed Diffusion（定向扩散）是一种以数据为中心的无线传感器网络路由协议，专为高效数据收集和能量优化设计。其核心思想是通过“兴趣（Interest）传播”和“梯度（Gradient）建立”机制，动态构建从数据源到汇聚节点（Sink）的最优传输路径。与传统的基于地址的路由协议不同，**Directed Diffusion** 通过内容命名数据，并利用本地化操作减少全局通信开销，适用于大规模、动态变化的传感器网络。

核心机制：

（1）兴趣传播（Interest Propagation）

汇聚节点（Sink）向全网广播“兴趣”消息（Interest Message），描述需要收集的数据类型。中间节点收到兴趣后，缓存兴趣信息并继续广播，最终覆盖整个网络。数据以属性-值对（如{type=temperature, location=area_A}）命名，而非基于节点地址。

（2）梯度建立（Gradient Setup）

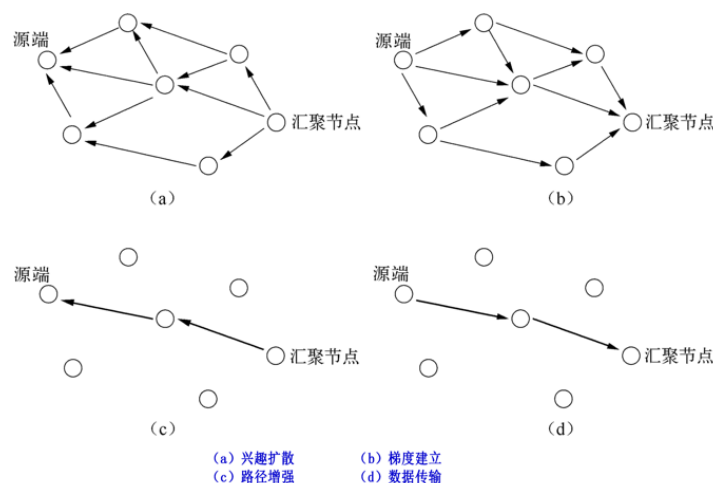
每个节点根据接收到的兴趣消息，为相邻节点建立“梯度”，表示数据传输的方向和优先级（例如，根据链路质量或能量水平）。梯度形成从数据源到汇聚节点的反向路径，为后续数据传输做准备。

（3）数据传播（Data Propagation）

当传感器节点检测到与兴趣匹配的数据时，沿梯度方向（反向路径）向汇聚节点发送数据。中间节点可对多路数据进行聚合（如取平均值），减少冗余传输。

（4）路径加强（Reinforcement）

汇聚节点根据接收到的数据质量（如延迟、丢包率），选择性能最佳的路径，并发送“加强消息”以提升该路径的梯度优先级。网络拓扑变化时（如节点失效），通过重新广播兴趣或调整梯度实现路径更新。



这张图把定向扩散分成了 4 个关键阶段，对应 (a) (b) (c) (d) 四个子图：

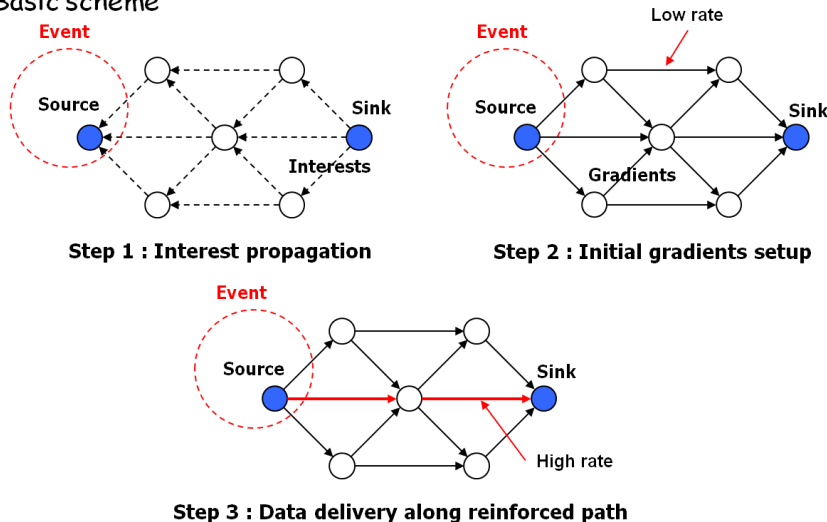
(a) 兴趣扩散（Interest Propagation）

- 动作：汇聚节点（Sink）广播“兴趣消息（Interest）”，也就是它需要什么类型的数据（比如“温度 > 30°C”）。

Wireless Sensor Network

- 过程：兴趣消息像泛洪一样扩散到全网，沿途每个节点都记录下“收到兴趣的邻居”，建立反向的“兴趣路径”。
 - 结果：源端（Source，产生数据的节点）收到了兴趣，知道网络里有人要自己的数据。
- (b) 梯度建立（Gradient Setup）
- 动作：收到兴趣的节点，会向发送兴趣的邻居建立一个“梯度（Gradient）”，也就是一条指向汇聚节点的“数据转发路径”，并标记初始数据传输速率（通常是低速率）。
 - 过程：梯度从源端到汇聚节点逐级建立，形成多条潜在的路径。
 - 结果：全网形成了从源端指向汇聚节点的“梯度场”，数据可以沿着这些梯度传输。
- (c) 路径增强（Path Reinforcement）
- 动作：源端先按低速率沿梯度发送数据，汇聚节点收到后，会对“质量最好（比如延迟最低、丢包最少）”的路径发送“增强消息”，提升这条路径的数据传输速率。
 - 过程：增强消息逐级传回源端，对应的节点把这条路径标记为“主路径”，其他路径的速率保持不变或降低。
 - 结果：网络中出现了一条“高优先级”的主路径。
- (d) 数据传输（Data Transmission）
- 动作：源端沿被增强的主路径，以高速率持续向汇聚节点发送数据。
 - 过程：数据只在最优路径上传输，其他路径不再参与或仅做备份。
 - 结果：数据高效、稳定地到达汇聚节点，同时减少了冗余传输，节省了能耗。

Basic scheme



这张图把定向扩散简化为 3 个核心步骤，和第一张图是一一对应的：

Step 1: Interest Propagation（兴趣扩散）

- 对应第一张图的（a）阶段。
- 汇聚节点（Sink）广播兴趣消息，虚线表示兴趣的传播路径，源端（Source）收到后，知道了汇聚节点的需求。

Step 2: Initial gradients setup（初始梯度建立）

- 对应第一张图的（b）阶段。
- 节点根据收到的兴趣，建立指向汇聚节点的初始梯度（箭头），并设置低速率（Low rate），形成多条潜在传输路径。

Step 3: Data delivery along reinforced path（沿增强路径传输数据）

- 对应第一张图的（c）+（d）阶段。

Wireless Sensor Network

- 汇聚节点收到初始数据后，对最优路径发送增强消息，将其升级为高速率（High rate）的主路径。源端之后的数据都沿这条红色的增强路径传输。

工作原理与机制：

（1）兴趣消息扩散

汇聚节点根据应用需求定义兴趣请求消息，通过洪泛方式发送至网络中的传感器节点。这种洪泛策略确保了消息的广泛传播，使网络中的节点都能接收到兴趣请求，为后续的数据收集和传输做好准备。另外，在兴趣消息洪泛过程中，每个节点会将收到的兴趣请求缓存起来，以便后续与接收到的数据进行匹配。这种缓存机制提高了数据传输的效率，减少了不必要的数据传输，节省了网络资源和节点能量。

（2）数据传输梯度建立

当兴趣消息到达源节点后，源节点会沿着之前建立好的传输梯度进行正向传输，将数据发送回汇聚节点。数据传输梯度的建立是定向扩散协议的关键环节，它为数据的高效传输提供了路径。传输梯度的建立基于节点之间的通信关系和数据传输方向，通过这种方式，数据能够沿着最优路径传输，减少了传输延迟，提高了数据传输的可靠性。

（3）路径加强与优化

在数据传输过程中，汇聚节点会对接收到的数据进行评估，并根据数据质量对路径进行加强。路径加强的目的是提高数据传输的效率和质量，确保数据能够稳定、可靠地传输到汇聚节点。通过路径加强机制，定向扩散协议能够适应网络拓扑的动态变化，优化数据传输路径，提高网络的性能和稳定性。

优点：

（1）能量高效

数据聚合减少冗余传输，梯度机制优化路径选择，显著降低能耗。

（2）动态适应性

支持网络拓扑动态变化（如节点移动或失效），通过重新扩散兴趣实现路径更新。

（3）以数据为中心

基于内容命名数据，无需维护全局节点地址，适合大规模网络。所有通信都是基于邻居节点之间的直接通信，无需节点寻址机制。这种通信方式简化了通信过程，降低了通信开销，提高了通信效率。每个节点都可以进行数据聚合和缓存，进一步减少了数据传输量，节省了网络带宽和节点能量。数据聚合和缓存机制在无线传感器网络中具有重要意义，能够有效提高网络的性能和寿命。

（4）负载均衡

多路径梯度机制避免单点瓶颈，延长网络生命周期。

缺点：

（1）初始化延迟高

兴趣扩散和梯度建立需要时间，初次数据传输延迟较高。

（2）控制开销较大

兴趣广播和梯度维护需频繁通信，可能增加网络负载。

（3）路径维护复杂

动态网络环境中，频繁的路径加强和兴趣重传可能消耗额外能量。

（4）依赖汇聚节点

汇聚节点故障可能导致全网路由失效。

适用场景：

定向扩散协议适用于基于查询的应用场景，如事件驱动的监测任务。在这种场景下，用户可以通过定义兴趣请求来获取所需的数据，网络能够快速响应并传输相关数据。

该协议并不适合需要连续数据传输的应用，如环境监测等。因为定向扩散是基于查询的按需路由，依靠兴趣消息触发数据传输。而环境监测等场景需要节点连续、不间断地上报数

Wireless Sensor Network

据，该协议频繁的兴趣扩散、梯度建立流程会增大传输延迟，且查询机制无法匹配持续数据流，容易造成数据丢失，因此并不适用。

性能分析：

指标	Directed Diffusion 表现
能耗	低（数据聚合 + 梯度优化），但兴趣广播和控制消息可能增加开销。
延迟	初次传输延迟高，路径加强后延迟降低。
可靠性	多路径机制提高容错性，但依赖汇聚节点的稳定性。
可扩展性	适用于大规模网络，但兴趣广播的范围需合理控制。

对比其他协议：

协议	Directed Diffusion	SPIN	Flooding
核心思想	兴趣传播 + 梯度优化	元数据协商	全网广播
能耗	低（数据聚合）	中（协商开销）	高（冗余传输）
适用场景	大规模、动态网络	中等规模、数据为中心	小规模、紧急事件
可靠性	高（多路径）	低（依赖节点兴趣）	高（但冗余代价大）

文献：Directed Diffusion: A Scalable and Robust Communication Paradigm for Sensor Networks

总体描述文献内容：

Directed Diffusion（定向扩散）这一无线传感器网络通信协议以数据为中心，通过兴趣传播、梯度建立和多路径强化机制，实现高效的数据传输与能源管理。其核心思想是通过本地化交互减少冗余通信，适应动态网络环境。

文献研究的问题：

论文旨在解决无线传感器网络中的以下关键问题：

- （1）能源效率：传统路由协议（如泛洪）的高冗余通信导致能耗过高。
- （2）动态适应性：网络拓扑频繁变化（如节点失效、移动）时如何维持可靠通信。
- （3）可扩展性：如何支持大规模节点部署，避免全局路由表维护的开销。
- （4）数据为中心的需求：传感器网络需按内容查询数据，而非基于节点地址通信。

问题提出的基础：

传统方法（如 IP 路由、泛洪）的局限性：

- IP 路由依赖全局地址和端到端通信，不适合存算以及能量资源受限的传感器节点。
- 泛洪导致高冗余和高能耗，无法适应动态拓扑。
- 集中式路由（如全知多播）假设全局拓扑信息，难以在大规模网络中实现。

问题的解决方法：

- （1）数据命名：使用属性-值对（如 type=animal, location=[x,y]）命名数据，而非节点地址。
- （2）兴趣传播：Sink 节点广播兴趣（Interest）请求特定数据，中间节点缓存兴趣并建立梯度（Gradient）。
- （3）梯度建立：节点根据兴趣建立数据传输方向，形成反向路径。
- （4）数据聚合：中间节点对多路数据进行聚合（如去冗余、取平均值），减少传输量。
- （5）路径强化：根据数据质量动态选择最优路径，通过强化（提高数据速率）和负强化（降低低效路径优先级）优化路径。

解决方法和现有方法的区别与联系：

对比维度	Directed Diffusion	现有方法（泛洪 / IP 路由）
通信模式	数据为中心，按内容命名与传播	基于节点地址或全网广播
路径建立	本地梯度交互，动态多路径优化	依赖全局路由表或固定路径
能源效率	通过聚合和路径强化显著降低能耗	高冗余传输导致能耗高
动态适应性	支持快速路径修复与调整	拓扑变化时需全局路由更新，开销大

Wireless Sensor Network

关联技术	借鉴多播路由的加入机制，但没有用复杂的全局多播协议，改用本地梯度实现	类似泛洪但引入智能抑制机制
------	------------------------------------	---------------

解决方法存在的问题：

- （1）初始化延迟高：兴趣扩散和梯度建立需要时间，初次数据传输延迟较高。
- （2）控制开销大：兴趣的周期性广播和梯度维护增加了网络负载。
- （3）路径维护复杂：动态路径调整可能导致额外能耗，尤其在频繁拓扑变化时。
- （4）依赖汇聚节点：若 Sink 节点故障，可能影响全网通信可靠性。
- （5）假设限制：默认节点能量均匀分布，实际场景中可能存在能量不均衡问题。

改进方法建议：

- （1）混合地理路由：结合 GEAR 协议，利用地理位置信息限制兴趣扩散范围，减少广播开销。
- （2）分簇架构：引入分簇机制（如 LEACH），由簇头负责兴趣传播与数据聚合，降低控制流量。
- （3）智能强化策略：利用机器学习预测网络状态，动态调整路径强化阈值，减少路径切换频率。
- （4）多 Sink 容错：部署多个 Sink 节点，分散通信负载，避免单点故障。
- （5）跨层优化：结合 TDMA MAC 层协议，减少空闲监听能耗，提升能效（已指出 802.11 MAC 的高能耗问题）。
- （6）能量感知梯度：在梯度建立时优先选择剩余能量高的节点，延长网络生命周期。

LEACH 路由协议

LEACH (Low-Energy Adaptive Clustering Hierarchy) 是一种经典的分簇式路由协议，专为无线传感器网络 (WSN) 设计。其核心目标是通过动态分簇和簇头轮换机制，均衡网络能量消耗，延长网络生命周期。LEACH 协议通过将网络划分为多个簇 (Cluster)，由簇头 (Cluster Head, CH) 负责数据聚合和转发，避免所有节点直接与基站 (Base Station) 通信，从而减少能量消耗。

工作原理与机制：

①簇头选举机制

LEACH 是一种低功耗自适应聚类路由协议，其核心思想是将节点组织成簇结构，每个簇有一个簇头节点。簇头节点负责管理簇内节点的通信和数据传输，通过轮转的方式选举簇头节点，使网络中的节点能量消耗更加均匀。

在簇头选举过程中，每个节点随机选择一个介于 0 和 1 之间的数字，如果该数字小于某个阈值，则该节点成为簇头节点。这种随机选举机制能够有效避免节点长期担任簇头而过早耗尽能量，提高了网络的稳定性和寿命。

②簇内通信与数据传输

在 LEACH 协议中，簇内通信采用 TDMA (时分多址) 方式，每个节点被分配一个时间槽用于数据传输。簇头节点负责为簇内节点分配时间槽，并协调节点之间的通信，避免了节点之间的通信冲突。

簇头节点接收簇内节点传输的数据后，进行数据聚合和融合，然后将聚合后的数据传输到基站。数据聚合和融合机制能够减少数据传输量，降低通信开销，提高网络性能与效率。

③簇间通信与干扰控制

簇间通信采用 CDMA (码分多址) 方式，每个簇使用不同的扩频码进行通信，以减少簇间干扰。扩频码的随机分配能够有效降低簇间通信的干扰，提高通信的可靠性。

总之，通过 TDMA 和 CDMA 的结合使用，LEACH 协议能够实现簇内和簇间的高效通信，提高网络的整体性能和稳定性。

工作流程：

LEACH 协议将网络运行划分为多个轮次 (Round)，每轮包括两个阶段：簇建立阶段和数据传输阶段。

①簇建立阶段

(1) 簇头选举：每个节点根据预设概率 p 随机决定是否成为簇头。若节点在最近的 $1/p$ 轮内未担任过簇头，则以概率 p 成为候选簇头。最终簇头通过广播宣布自身身份，其他节点选择信号最强的簇头加入其簇。

(2) 簇形成：非簇头节点加入最近的簇头，形成以簇头为中心的星型拓扑结构。簇头为成员节点分配通信时隙 (TDMA 调度)，避免数据冲突。

②数据传输阶段

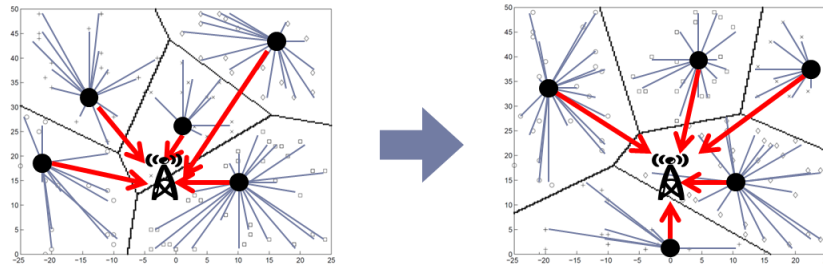
(1) 数据收集：成员节点在分配的时隙内将数据发送给簇头。簇头对接收到的数据进行聚合 (如压缩、去冗余)，减少传输量。

(2) 数据转发：簇头将聚合后的数据通过单跳或多跳方式发送至基站 (Sink)。

③簇头轮换机制

每轮结束后重新选举簇头，避免固定簇头能量过快耗尽。通过动态轮换实现全网能量负载均衡。

Wireless Sensor Network



All nodes marked with a given symbol belong to the same cluster, and the cluster head nodes are marked with a ●.

1. 图中元素说明

- 黑色圆点 (●): 簇头节点 (Cluster Head), 每个簇的管理者。
- 不同符号的空心节点: 普通传感器节点, 同一符号的节点属于同一个簇。
- 蓝色细线: 簇内普通节点 → 簇头的通信链路。
- 红色粗线: 簇头 → 基站 (Sink, 图中的塔形符号) 的通信链路。
- 黑色分割线: 簇的边界, 把整个网络划分成多个独立的区域 (簇)。

2. 左图: 上一轮的分簇状态

- 网络被划分为多个簇, 每个簇由一个簇头和若干普通节点组成。
- 普通节点把采集的数据发给本簇的簇头 (蓝色线), 簇头汇总数据后, 统一发送给基站 (红色线)。
- 此时的簇头分布不均: 靠近基站的节点承担了大量长距离通信任务, 能耗消耗快; 部分远离基站的节点也被选为簇头, 长距离传输能耗更高。

3. 右图: 下一轮的分簇状态 (簇头轮换后)

- 协议通过周期性重新选举簇头, 更新了所有簇的管理者 (新的黑色圆点)。
- 新的簇头重新收集本簇数据, 再统一发送给基站。
- 核心变化: 能耗从固定节点转移到了新的节点上, 避免了部分节点因长期担任簇头而过早耗尽能量, 实现了全网负载均衡。

4. 背后的核心原理 (以 LEACH 为例)

1. 分簇的作用:

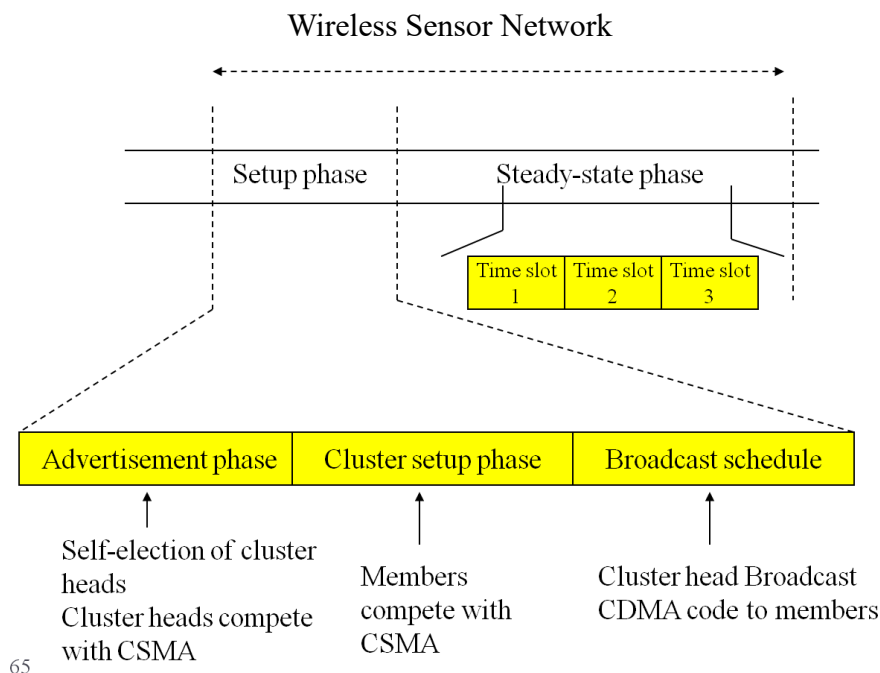
- 普通节点只需要短距离把数据发给簇头, 大幅降低了单跳传输的能耗。
- 簇头可以对数据进行聚合 (如取平均、去冗余), 减少发往基站的数据量, 进一步省电。

2. 簇头轮换的意义:

- 簇头需要承担数据接收、聚合、长距离发送的任务, 能耗远高于普通节点。
- 周期性轮换, 让所有节点轮流担任簇头, 避免 “能量空洞” 问题, 延长整个网络的生命周期。

5. 考试 / 笔记版总结

这两张图直观展示了分簇路由协议的簇头轮转机制: 网络被划分为多个簇, 簇内节点将数据发送给簇头, 再由簇头转发至基站; 通过周期性重新选举簇头, 让能耗在所有节点间均匀分布, 从而解决了传统路由中节点能耗不均、网络寿命短的问题。



一、整体结构：一个轮次分为两大阶段

LEACH 协议的每一轮 (Round) 都分为两个主要阶段：

1. *Setup phase* (簇建立阶段)：负责选簇头、建簇、分配资源
2. *Steady-state phase* (稳定传输阶段)：簇内节点按调度表把数据发给簇头

二、*Setup phase* (簇建立阶段) 的三步拆解

Setup phase 又细分为三个子阶段，图里的黄色框就是这三步：

1. *Advertisement phase* (宣告阶段)

- 核心动作：簇头自选举 + 广播宣告
- 过程：

1. 节点按预设概率随机竞选簇头，当上簇头的节点用 CSMA (载波监听多路访问) 广播自己的簇头宣告消息 (Advertisement)。
2. 普通节点收到多个簇头的消息后，根据信号强度选择加入哪个簇。

2. *Cluster setup phase* (簇建立阶段)

- 核心动作：普通节点申请加入簇
- 过程：

1. 普通节点向自己选定的簇头发送“加入请求” (Join Request)。
2. 多个节点同时请求时，同样用 CSMA 竞争信道，避免冲突。

3. *Broadcast schedule* (广播调度阶段)

- 核心动作：簇头分配时隙和 CDMA 码
- 过程：

1. 簇头收到所有成员的加入请求后，为每个节点分配专属的 TDMA 时隙 (Time slot)，避免簇内节点同时发送数据造成冲突。
2. 同时给每个节点分配 CDMA 码，减少不同簇之间的干扰。
3. 簇头把这个调度表广播给所有成员节点。

三、*Steady-state phase* (稳定传输阶段)

- 簇建立完成后，网络进入稳定数据传输阶段：

1. 普通节点按簇头分配的 TDMA 时隙，在自己的专属时间片里把采集的数据发给簇头 (蓝色细线的通信)。

2. 簇头收到数据后，先做数据聚合（比如去冗余、取平均），再把汇总后的数据包统一发送给基站（Sink）。
3. 图里的 Time slot 1 / 2 / 3 就是给不同节点分配的专属传输时间片，保证无冲突。

四、LEACH 这样设计的目的

1. 降低能耗：
 - 普通节点只需要短距离把数据发给簇头，不用直接长距离传给基站，大幅省电。
 - TDMA 时隙调度让节点只在自己的时间片里唤醒，其他时间休眠，减少空闲监听的能耗。
2. 减少冲突：
 - 簇内用 TDMA，簇间用 CDMA，双重机制避免了不同节点、不同簇之间的信号干扰。
3. 负载均衡：
 - 每一轮都重新选簇头，让所有节点轮流承担簇头的高能耗任务，避免单个节点过早耗尽能量，延长网络整体寿命。

考试 / 笔记版总结

这张图展示了 LEACH 协议一轮的完整流程，分为 Setup 阶段 和 Steady-state 阶段：

- Setup 阶段通过“簇头宣告 → 节点申请 → 调度表广播”三步完成分簇，用 CSMA 解决信道竞争，用 TDMA/CDMA 分配资源。
- Steady-state 阶段节点按 TDMA 时隙向簇头发送数据，簇头聚合后再转发给基站。
- 整个设计通过分簇、时隙调度和簇头轮换，实现了能耗均衡和高效传输，延长了无线传感器网络的生命周期。

优点：

（1）能量高效

簇内成员仅需短距离通信，簇头负责长距离传输，减少节点能耗。簇头聚合数据，降低传输总量。数据聚合和融合机制进一步减少了数据传输量，降低了通信开销，节省了节点能量。在无线传感器网络中，能量消耗是一个关键问题，LEACH 协议通过多种节能机制，有效解决了这一问题。

（2）负载均衡

通过簇头轮转机制，使网络中的节点能量消耗更加均匀，有效延长了网络的寿命。这种能量均衡策略能够避免节点过早耗尽能量，提高了网络的稳定性和可靠性。动态簇头轮换避免单点能量耗尽，延长网络生命周期。

（3）可扩展性

适用于大规模网络，分簇机制支持节点动态加入或退出。

（4）自适应性

这是一种分布式协议，无需全局网络信息，仅依赖本地通信完成分簇。LEACH 节点依靠预设概率自主竞选簇头，仅向周边邻居广播身份；普通节点根据本地信号强度就近选择簇头加入，簇头也只对簇内节点做资源调度。整个分簇流程仅在相邻节点间完成，不需要获取和维护全网拓扑、节点分布等全局信息，也降低了复杂性。能够快速适应网络拓扑的变化，提高了网络的灵活性和可扩展性。

缺点：

（1）簇头分布不均

随机选举可能导致簇头位置集中或边缘化，影响分簇效率。

(2) 单跳通信限制

簇头直接与基站通信，若基站距离较远，簇头能量消耗急剧增加。在大规模网络或节点分布不均匀的情况下，部分节点可能无法直接传输到基站，导致数据丢失或传输失败。

(3) 动态分簇开销

每轮重新分簇需广播控制消息，增加通信开销。

(4) 假设限制

假设所有节点初始能量相同且基站位置固定，实际场景中可能不成立。

(5) 节点复杂度与成本上升

协议要求每个节点支持 TDMA 和 CDMA，增加了节点的复杂性和成本。

(6) 时间同步开销大

需要进行时间同步，增加了协议的实现难度，以及额外的通信与能量开销。

性能分析：

指标	LEACH 表现
能耗	低（分簇 + 数据聚合），但簇头单跳通信能耗较高。
延迟	中等（分簇阶段引入延迟，数据聚合减少传输次数）。
可靠性	依赖簇头稳定性，若簇头失效可能导致簇内数据丢失。
可扩展性	适用于大规模网络，但簇头选举机制需优化。

对比其他协议：

协议	LEACH	Directed Diffusion	SPIN
核心思想	动态分簇 + 簇头轮换	兴趣传播 + 梯度优化	元数据协商
能耗	低（分簇优化）	低（数据聚合）	中（协商开销）
适用场景	周期性数据收集	动态数据查询	事件驱动型应用
可靠性	中等（依赖簇头）	高（多路径）	低（依赖节点兴趣）

文献：Energy-Efficient Communication Protocol for Wireless Microsensor Networks

总体描述文献内容：

LEACH（Low-Energy Adaptive Clustering Hierarchy）协议是一种用于无线传感器网络的能源高效分簇路由协议。其核心思想是通过动态分簇和簇头轮换机制，将能量负载均匀分布到所有节点，显著降低全局能耗并延长网络生命周期。论文通过理论分析和仿真实验验证了 LEACH 的优势：与传统协议（直接传输、MTE 路由、静态分簇）相比，LEACH 可实现高达 8 倍的能耗降低，并将系统寿命延长至传统方法的 3 倍以上。此外，LEACH 通过数据聚合减少传输量，并通过本地化协调实现动态网络的扩展性和鲁棒性。

文献研究的问题：

1. 能源效率低：传统协议（如直接传输、MTE 路由）导致节点能量快速耗尽。
2. 能量消耗不均衡：静态分簇协议中固定簇头节点能量耗尽过快，导致网络寿命缩短。
3. 动态适应性差：网络拓扑变化时传统协议无法有效调整路径，导致通信中断。
4. 数据传输冗余：大量原始数据直接传输至基站，未利用本地计算降低通信开销。

问题提出的基础：

传统方法的局限性：

- 直接传输：节点直接与远距离基站通信，能耗极高（公式显示能耗与距离平方成正比）。
- MTE 路由：多跳传输虽减少单跳距离，但中间节点频繁转发导致局部能量耗尽（近基站节点快速失效）。
- 静态分簇：固定簇头导致其能量快速耗尽，簇内节点无法继续通信（静态分簇寿命显著低于 LEACH）。

传感器网络的特殊性（节点能量受限、大规模部署、动态拓扑）催生了 LEACH 的需求。

问题的解决方法：

LEACH 的核心机制包括：

(1) 动态分簇：每轮随机选举簇头，避免固定节点负载过重。选举公式为：

$$T(n) = \begin{cases} \frac{P}{1 - P * (r \bmod \frac{1}{P})} & \text{if } n \in G \\ 0 & \text{otherwise} \end{cases}$$

其中，P 为簇头比例，r 为当前轮次，G 为未担任簇头的节点集合。

(2) 数据聚合：簇头对簇内数据进行压缩（如波束成形），减少传输量。

(3) TDMA 调度：簇头为成员分配通信时隙，非簇头节点仅在发送时唤醒，降低空闲能耗。

(4) 簇头轮换：通过概率阈值确保所有节点轮流担任簇头，均衡能量消耗。

解决方法和现有方法的区别与联系：

对比维度	LEACH	现有方法
簇头选择	动态随机轮换	固定簇头（静态分簇）或无需分簇（直接传输）
能耗分布	全局能量均衡，节点随机失效	局部能量耗尽（如 MTE 路由中近基站节点）
数据传输	簇内短距离通信 + 簇头远距离聚合传输	直接远距离传输或多跳转发
控制开销	分簇阶段需广播和调度（短期开销）	无分簇开销（直接传输）或路由维护开销（MTE）
关联技术	结合分簇与数据聚合，类似静态分簇但动态	基于传统路由与分簇思想

解决方法存在的问题：

(1) 分簇开销：每轮分簇需广播簇头广告和成员加入消息，增加控制能耗（论文提到未来研究需优化分簇频率）。

(2) 簇头分布不均：随机选举可能导致簇头位置集中或边缘化，影响分簇效率。

(3) 假设限制：默认节点初始能量相同，实际场景中可能存在能量异构性。

(4) 大规模扩展性：分簇管理复杂度随节点数量增加而上升，未支持多级分簇。

改进方法建议：

(1) 能量感知簇头选举：在选举阈值中引入剩余能量参数，优先高能量节点担任簇头。

(2) 地理分簇优化：结合节点位置信息优化簇头分布，减少簇内通信距离（类似 GEAR 协议思想）。

(3) 混合通信模式：簇头间采用多跳转发至基站，避免单跳远距离传输。（类似 LEACH-M 协议：①簇间多跳转发。簇头不直接单跳去基站，而是找更近、能量更高的其他簇头做中继，多跳传到基站。相当于：簇头 → 邻居簇头 → ... → 基站，每跳距离变短，大幅降能耗。②能量感知簇头选举。选簇头时看剩余能量：能量越高，当选概率越大；能量太低直接不让参选。避免“快没电的节点当簇头”，延长簇头存活时间。）

(4) 多级分簇架构：引入超级簇头（Super Cluster Head）分层管理，支持大规模网络（论文提及未来研究方向）。

(5) 动态分簇频率调整：根据网络负载动态调整分簇周期，降低控制开销。

(6) 机器学习辅助分簇：利用聚类算法预测最优簇头分布，减少随机性影响。

第十讲（传输层）

传输层基础概述

无线信道特性及感知数据汇聚导致网络拥塞，感知数据丢失严重。传输层提供端到端可

靠、透明的数据传输服务，支持拥塞控制和差错控制，保障数据传输质量。传统网络以 TCP 为基础，其端到端差错控制和拥塞控制机制消耗资源大，可扩展性和自适应能力差。而 WSN 具有很强的应用相关性，不同应用对传输可靠性要求不同。传输层通过提供可靠的数据传输服务，确保感知数据能够准确地从传感器节点传输到汇聚节点，从而支持各种应用的正常运行。

特点：

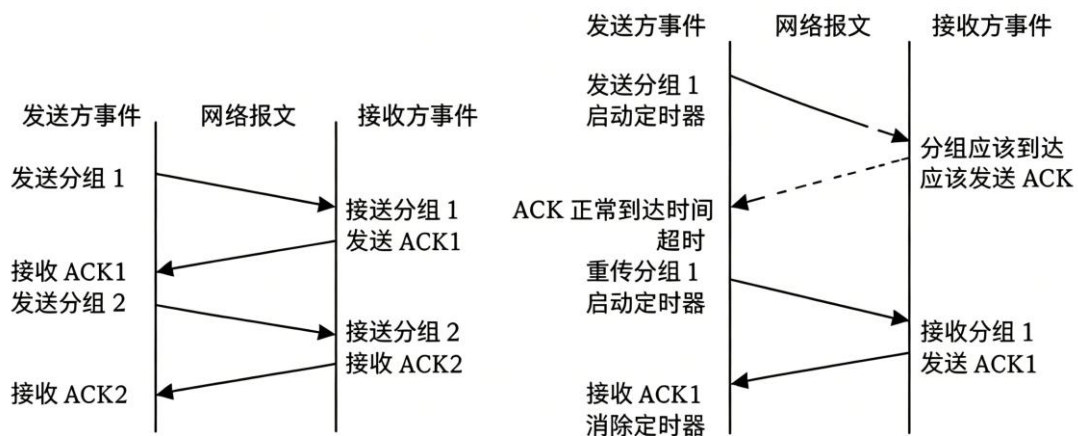
传统传输协议不适用。TCP 关注端到端功能，中间节点仅转发数据，而 WSN 中间节点需对数据编码融合。TCP 假设链路可靠，WSN 中包丢失原因复杂且随机。TCP 要求节点有唯一网络地址，WSN 不一定需要。TCP 基于数据包可靠传输，WSN 面向应用，可能只需传输足够数据即可完成任务。TCP 面向连接，WSN 数据分组短，连接建立开销大，且为多对一通信模式。

WSN 传输协议节能优先，因传感器节点计算和能量存储能力有限。多对一传输模式，大量数据在汇聚节点附近聚集，易造成拥塞。以数据为中心，获得感知数据为主要任务。应用相关性强，不同应用对传输可靠性要求不同。

WSN 传输层的特点使其能够更好地适应 WSN 的特殊需求，如节能、多对一传输、应用相关性等，从而提高网络的整体性能和应用效果。

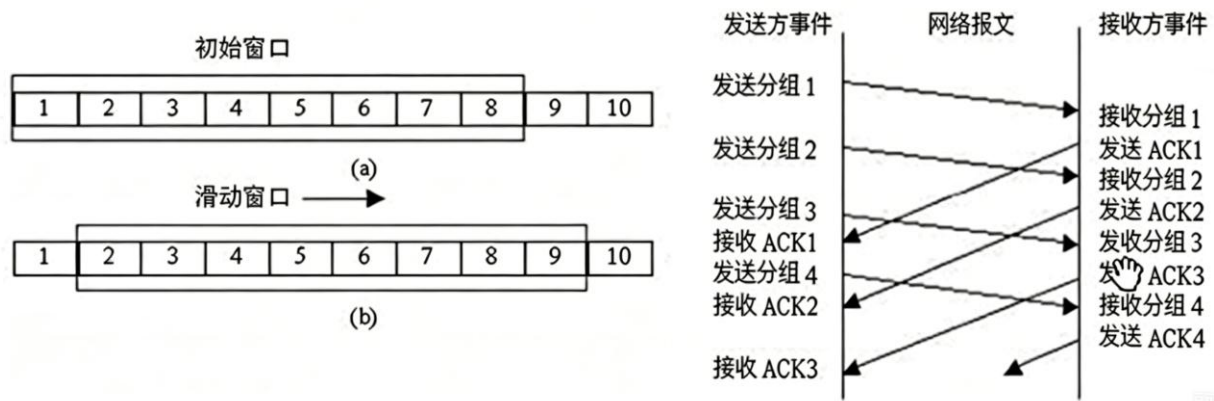
（一）TCP 重传机制

- 带重传功能的肯定确认（Positive acknowledge with retransmission）。
- 接收方正确收到数据后发 ACK。
- 发送方备份每个发出分组，在送出分组的同时启动一个定时器，如果超时且没有收到 ACK，重发分组；在发送下一分组前要收到 ACK。

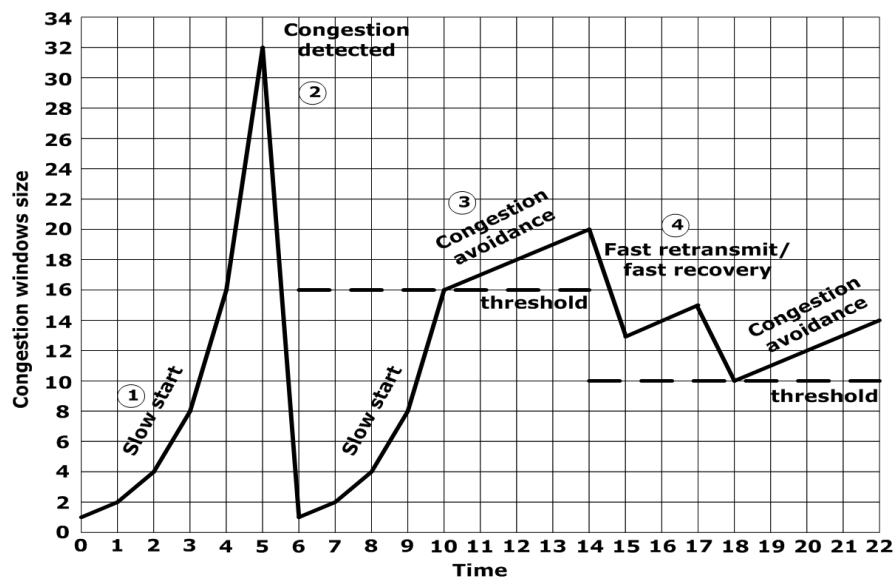


（二）TCP 滑动窗口

- 它允许发送方在等待一个确认信息之前可以发送多个分组。
- 滑动窗口协议在分组序列中放置一个固定长度的窗口，然后将窗口内的所有分组都发送出去；当发送方收到对窗口内第一个分组的确认信息时，它可以向后滑动并发送下一个分组；随着确认的不断到达，窗口也在不断地向后滑动。



(三) TCP 拥塞窗口



这张图是 TCP 拥塞控制的完整过程，核心是“慢开始 + 拥塞避免 + 快重传 / 快恢复”，4 阶段拆解如下：

1. 阶段①：慢开始 (Slow Start)

- 发生时机：TCP 连接刚建立，或刚经历过一次拥塞并恢复后。
- 过程：拥塞窗口 (cwnd) 从 1 开始，指数级增长：每收到一个 ACK，窗口大小翻倍 (1→2→4→8→16...)。
- 目标：快速探测网络可用带宽，避免一开始就发送大量数据引发拥塞。
- 图中可以看到，从时间 0 到 5，窗口大小从 1 快速涨到了 32。

2. 阶段②：拥塞检测 (Congestion Detected)

- 发生时机：窗口涨到峰值后，出现超时重传 (RTO 超时)，这说明网络发生了拥塞。
- 核心动作：
 1. 把门限值 (ssthresh) 设为当前窗口的一半： $ssthresh = cwnd / 2$ ，图中从 32 降到 16。
 2. 把拥塞窗口 (cwnd) 重置为 1。
- 图中表现：时间点 6，窗口大小直接从 32 跌到 1，阈值线从 32 降到 16。

3. 阶段③：慢开始 + 拥塞避免 (Slow Start + Congestion Avoidance)

- 过程:
 1. 先再次进入慢开始: 窗口从 1 指数增长, 直到碰到阈值 (16)。
 2. 超过阈值后, 切换为拥塞避免: 窗口从指数增长变为线性增长, 每个 RTT (往返时间) 只增加 1。
 - 目标: 避免过快增长再次引发拥塞, 以平缓的方式逼近网络最大带宽。
 - 图中表现: 从时间 6 到 10, 窗口从 1 涨到 16; 从 10 到 14, 窗口从 16 线性涨到 20。
-

4. 阶段④: 快重传 / 快恢复 (Fast Retransmit / Fast Recovery)

- 发生时机: 收到 3 个重复 ACK (而不是超时), 说明个别包丢了, 但网络整体还没完全拥塞。
 - 核心动作:

快重传 (Fast Retransmit): 不等超时计时器走完, 收到第 3 个重复 ACK, 就立刻重传那个丢失的包 (比如上面的 2 号包)。

快恢复 (Fast Recovery): 不用把拥塞窗口砍到 1, 而是只砍一半, 降到门限值, 然后直接进入“拥塞避免”阶段, 慢慢涨窗口。

 1. 把门限值 (ssthresh) 设为当前窗口的一半: $ssthresh = cwnd / 2$, 图中从 20 降到 10。
 2. 不把窗口重置为 1, 而是先降到 ssthresh, 然后直接进入拥塞避免阶段, 继续线性增长。
 - 目的: 避免像超时 (一个 ACK 都没收到, 把拥塞窗口直接砍到 1, 重新慢开始) 那样“一刀切”式的窗口暴跌, 减少传输中断, 提升效率。
 - 图中表现: 时间点 14, 窗口从 20 降到 10; 之后又进入线性增长的拥塞避免阶段。
-

一句话总结完整流程

1. 慢开始: 窗口指数级增长, 快速探测带宽;
2. 超时拥塞: 窗口大小重置, 阈值减半, 重走慢开始;
3. 阈值后拥塞避免: 窗口线性增长, 平缓逼近上限;
4. 快重传 / 快恢复: 收到 3 个重复 ACK, 窗口降到阈值后直接进入拥塞避免, 不用重置为 1。

TCP 不适用于 WSN:

- TCP 关注端到端的功能, 中间节点仅负责数据转发。WSN 的中间节点可能需要对数据进行编码和融合等处理。
- TCP 假设网络链路可靠, 数据包丢失由路由器缓存溢出或拥塞引起。WSN 中包丢失可能由于链路传输差错、比特错误、节点故障等, 且有随机性。
- TCP 要求每个节点有唯一的网络地址。WSN 不一定需要。
- TCP 采用基于数据包的可靠传输, 保证源节点发出的每个数据包都成功传输到目的节点。WSN 面向应用, 可能只要传输足够的数据即可完成任务。
- TCP 面向连接, 在发送数据之前必须先建立连接。WSN 的数据分组短, 连接建立过程带来较大开销。
- TCP 为一对一通信模式。WSN 中为多对一通信模式。

传统传输协议的特点:

- 无线 Ad Hoc 网络与 WSN 类似, 但其传输协议也不适用于 WSN
- WSN 规模较大, 节点数可能是无线 Ad Hoc 网络的几十倍甚至几千倍
- 传感器节点的计算能力和能量存储有限, 远小于无线 Ad Hoc 网络节点
- 无线 Ad Hoc 网络的任务是保证移动节点之间的互联, 较多使用对等通信方式。WSN

以数据为中心，获得感知数据为主要任务。

WSN 传输协议的特点：

- 节能优先
- 多对一传输模式
- 以数据为中心
- 应用相关性强

多对一传输模式：

- WSN 面向信息感知，目的是将节点的感知数据传送到汇聚节点。当采用多对一的传输模式时会使大量数据在汇聚节点附近聚集，容易造成拥塞。同时，突发事件导致的流量突发性也会造成局部和全网的拥塞。
- WSN 的传输协议需要考虑网络的上行汇聚传输模式，进行拥塞控制和差错控制，以实现可靠传输。
- WSN 的传输协议需要考虑下行传输中由汇聚节点向网络分发的控制指令或查询消息，确保其能够可靠的发送至所有节点。

传输层协议分类

（一）拥塞控制协议

定义：

防止网络拥塞的产生，或缓解和消除网络中已经发生的拥塞现象，确保网络能够正常运行，避免因拥塞导致的数据丢失和网络性能下降。

拥塞控制设计考察因素：

- （1）能量有效性：减少拥塞控制开销，降低节点能耗，避免加剧拥塞。
- （2）实时性：及时检测拥塞并快速缓解，防止拥塞进一步恶化。
- （3）公平性：保证各节点公平发送数据，避免部分节点因拥塞而无法传输数据。
- （4）面向应用：根据应用需求调整数据传输策略，确保满足应用要求的服务质量。

拥塞控制方法：

- （1）拥塞避免：通过速率分配或传输控制等方法，提前预防拥塞的发生。
- （2）拥塞消除：在网络发生拥塞后，采用速率控制、丢包等方法来缓解和消除拥塞。

（二）可靠传输协议

定义：

保证感知数据能够有序、无丢失和无差错的传输到汇聚节点，向用户提供可靠的数据传输服务，满足不同应用对数据完整性和准确性的要求。

可靠传输机制：

- （1）丢包恢复：通过重传数据包来实现数据的可靠传输，包括丢包检测、丢包反馈和重传恢复等环节。
- （2）冗余传输：发送节点多次发送同一数据包的备份，提高数据传输的成功率。
- （3）速率控制：根据任务需求调节源节点的数据速率，确保任务能够顺利完成。

可靠传输与应用的关系：

WSN 面向应用，传输可靠包括数据可靠和任务可靠。数据可靠要求所有数据成功传输，任务可靠则只需保证任务完成即可，不一定要求所有数据都成功传输。

（三）拥塞控制和可靠传输混合协议

定义：

同时支持拥塞控制和可靠传输两种功能，综合考虑网络的拥塞情况和数据传输的可靠性要求。

优势：

能够更好地适应 WSN 的多对一传输模式和应用相关性特点，提高网络的整体性能和应用效果。

应用场景：

在对数据传输可靠性要求较高且网络容易出现拥塞的应用场景中，如环境监测、工业自动化等领域，混合协议能够发挥重要作用。

传输层设计目标

（1）能量效率

提高有限能量利用率，最大化网络生命周期，因为传感器节点的计算能力和能量存储有限。

（2）拥塞控制

针对 WSN 多对一传输模式导致的汇聚节点附近数据流量过大问题，进行有效的拥塞控制，避免局部或全网阻塞和能量消耗不均。

（3）传输可靠

根据 WSN 面向应用的特点，实现数据可靠和任务可靠，满足不同应用对数据传输可靠性的要求。

（4）可扩展性

协议应适应网络规模变化，为不同规模的网络提供良好的性能，确保网络能够随着节点数量的增加而稳定运行。

（5）自适应性

由于节点能量和信道带宽限制、节点加入或退出以及移动性影响，WSN 的拓扑结构会频繁变化，协议应能快速适应这一特性，保证网络的稳定性和可靠性。

（6）公平性

节点随机部署，距离汇聚节点较远的节点传输数据成功机会少，协议应公平分配带宽，使汇聚节点能够公平地收集数据。

（7）服务质量

根据 WSN 不同应用对服务质量的不同要求，如多媒体应用对带宽要求高，实时监控应用对传输延迟敏感，在性能要求中寻求最佳折中，满足应用需求。

（8）网络寿命

在满足性能要求的同时，延长 WSN 的网络寿命，提高网络的整体效益和应用价值。

传输层设计挑战

（1）节点能力限制

传感器节点的计算能力和能量存储有限，远小于无线 Ad Hoc 网络节点，这给传输协议的设计带来了挑战，需要在有限的资源下实现高效的数据传输。

（2）网络拓扑变化频繁

节点能量和信道带宽限制、节点加入或退出以及移动性影响，导致 WSN 的拓扑结构频繁变化，协议需要具备快速适应能力，以保证网络的稳定性和可靠性。

（3）应用相关性

WSN 具有很强的应用相关性，不同应用对传输的可靠性要求不同，协议需要根据不同应用的特点进行设计，以满足多样化的应用需求。

（4）多对一传输模式

WSN 中为多对一通信模式，大量数据在汇聚节点附近聚集，容易造成拥塞，需要考虑网络的上行汇聚传输模式，进行有效的拥塞控制和差错控制。

传输层基本机制

（一）拥塞控制

拥塞：

一种网络状态。当网络中传输的数据超过网络的负载能力时，网络性能将开始急剧下降。具体表现包括网络节点需要发送过多的数据、多个节点竞争信道接入造成网络排队延迟快速增加、节点缓冲区溢出以及冲突造成的分组丢弃加剧。

拥塞控制:

避免拥塞或及时监测并缓解网络中出现的拥塞现象。

设计考察因素:

- 能量有效性。尽量减少拥塞控制开销, 节点能耗, 避免控制开销加剧拥塞程度。
- 实时性。及时检测网络拥塞出现, 在短时间内缓解拥塞, 避免拥塞进一步加剧。
- 公平性。保证所有需要发送数据的节点都有机会发送数据。
- 面向应用。丢弃过时数据包或调整数据源汇报数据率时不降低满足应用要求的服务质量。

分类:

(1) 拥塞避免

通过速率分配或传输控制等方法来避免在局部或全网范围内出现拥塞。

①速率分配

- 对网络各节点的传输速率进行合理分配和严格限制, 来避免拥塞的产生。
- 要求网络中的节点能够很好的协调与合作。
- 理想情况下, 避免拥塞和丢包, 提高网络吞吐量、传输可靠性和其他服务质量指标。

②传输控制

- 节点根据一些网络参数(如节点缓存状态、网络拓扑信息等)决定是否转发数据或确定转发速率, 以避免拥塞发生。
- 例, 基于缓存状态的传输控制, 避免网络拥塞时的缓存溢出。
- 发送节点仅在接收节点的缓存有足够的剩余接收空间时才向其发送数据, 避免接收节点因缓存溢出造成丢包。
- 多少缓存空间算足够? 门限值设置。
关键问题: 怎么定义“足够的剩余空间”? 要实现上面的逻辑, 就必须提前定一个门限值(阈值), 比如:
 - 当接收节点缓存使用率达到 70% 时, 就认为“不够安全了”, 通知上游节点降速或暂停发送;
 - 当使用率降到 30% 以下时, 再恢复正常发送。这个阈值怎么设, 就是传输控制协议要解决的关键问题之一:
 - 设太高: 节点都快堵死了才通知, 还是会丢包;
 - 设太低: 一有流量波动就降速, 网络利用率低, 传输效率差。

(2) 拥塞消除

在网络发生拥塞后通过采用速率控制、丢包等方法来缓解拥塞, 进一步消除拥塞。

①拥塞检测

采用多种方法检测网络是否发生拥塞, 如基于缓冲区占用率、信道采样、包间隔和包服务时间、丢包率、负载强度、数据逼真度等, 但需多种方法配合使用以提高准确性。

■ 基于缓冲区占用率

节点检测自己缓冲区的占用情况。当数据包接收速率大于发送速率, 缓冲区堆积到一定程度且超过某一门限值时, 预测拥塞即将发生。

实现简单, 不需要任何控制开销。但单独使用并不准确。

■ 基于信道采样

节点周期性的进行信道采样, 以检测信道占用程度, 评估信道负载状况。若采样到信道长时间繁忙, 则认为网络拥塞。

消耗额外能量进行信道检测。

■ 基于包间隔和包服务时间

节点根据从邻居节点收到的数据包到达的时间间隔, 以及数据包从到达缓存区到被发出去的服务时间来判断是否发生拥塞。若到达时间间隔或服务时间过长则认为发生拥塞。

■ 基于丢包率

节点根据丢包的次数或频度判断网络是否发生拥塞。检测功能只在数据包被缓存或丢弃时触发。

并不十分准确，拥塞并非丢包的唯一原因。

■ 基于负载强度

综合考虑节点的流量负载情况、信道竞争情况以及所有邻居节点的本地流量计算出一个综合性度量值。当负载强度超过一定门限时，认为发生了拥塞。

综合考虑集中导致拥塞的因素，比单独考察某一因素更合理，但需要信息较多，时间比较复杂。

■ 基于数据逼真度

通常由汇聚节点执行检测，通过检查收集到的感知数据的准确度来判读是否发生拥塞。一般情况下，若准确度低于某个预定的门限值，则判断网络发生了拥塞。

上述方法各有优缺点，都不能完全准确的反映网络是否拥塞及拥塞的程度，几种检测方法得相互配合着使用。

如何确定拥塞、判断拥塞程度？—— 融合判断逻辑

可以用简单的“投票 + 加权”逻辑来实现：

1. 设置多阈值：给每个指标（缓存占用率、丢包率等）设置“预警值”和“严重值”。
2. 投票机制：如果超过半数的指标触发了预警值，判定为“轻度拥塞”；如果多个指标同时触发严重值，判定为“重度拥塞”。
3. 加权融合：给更可靠的指标（比如负载强度）更高权重，比如缓存占用率占 40%、包服务时间占 30%、丢包率占 30%，计算综合得分，根据得分区间划分拥塞等级。

②拥塞通知

节点检测到拥塞后，通过显式或隐式通知将拥塞状态告知相关节点，包括端到端通知和逐跳通知两种方式。

- 显式通知：直接以控制包的形式通知拥塞信息。
- 隐式通知：用数据包捎带拥塞信息，并由相关节点侦听信道获得。
- 端到端通知：由汇聚节点负责反馈拥塞信息。
- 逐跳通知：由中间节点负责反馈拥塞信息。

③拥塞缓解

通过速率控制、流量调度、数据处理等方法缓解拥塞，如调节源节点或中间节点的转发速率，绕路、分流或重定向数据流，丢弃、压缩或融合数据等。

■ 速率控制

通过调节源节点数据产生速率或中间节点转发速率来缓解拥塞。

源节点速率控制：由目的节点根据接收数据的情况反馈给所有源节点控制信息，来指示这些源节点如何调节发送速率。

转发节点速率控制：节点根据邻居节点的拥塞通知逐跳调整转发速率。

■ 流量调度

通过绕路、分流或者重定向等方式来减少拥塞区域的数据流以缓解拥塞。通常可与多路径路由协议相结合。在网络正常工作时，节点按照主路径路由数据。在拥塞发生后，节点使用备用路径缓解主路径上的拥塞。节点需维护多路由信息，控制开销较大。

■ 数据处理

节点通过对数据进行丢弃、压缩或融合来减少数据量。WSN 中的数据可能存在大量的冗余和相关性，只要传送到汇聚节点的信息满足应用要求，节点可丢弃冗余数据或融合数据包再继续传输。

（二）可靠传输

解决传输过程中因物理环境、网络拥塞、信号干扰、节点故障等造成的数据包丢失问

题，保证目的节点能够获得完整有效的数据信息（基于数据的可靠传输）或能够准确的还原出原始事件状态从而完成感知任务（基于任务的可靠传输）。

（1）丢包恢复

在发生丢包情况下，通过重传数据包来实现数据的可靠传输。

■ 丢包检测和反馈

最常用的就是应答方式。采用端到端或逐跳的应答方式，接收节点根据收包情况返回应答，发送节点根据应答判断是否需要重传，包括 ACK 和 NACK，以及 IACK（Implicit ACK）等机制。

■ 端到端重传和逐跳重传

根据网络情况 and 应用需求选择合适的重传方式，同时考虑最大重传次数，可固定最大重传次数，也可动态调整最大重传次数。

IACK(Implicit ACK): 发送节点发送数据包后缓存该包，监听接收节点的数据传输，若接收节点已将该数据包转发给其下一跳节点，则认为传输成功并清除缓存。不需要控制开销，只能在单跳范围内使用，是一种逐跳检测反馈机制。

一、ACK（确认）

- 全称：Acknowledgement，即确认应答。
 - 工作方式：接收方收到数据包后，主动给发送方回发一个“确认包”，告诉对方：“我收到了，你不用再发了”。
 - 逻辑流程：
 1. 发送方发数据包 → 等待 ACK；
 2. 接收方收到数据包 → 回发 ACK；
 3. 发送方收到 ACK → 确认传输成功，清除缓存的数据包。
 - 特点：
 - 可靠、简单，TCP/IP 里最常用的方式；
 - 缺点：每个数据包都要发一个 ACK，控制开销大，在 WSN 中会额外消耗节点能量和带宽。
-

二、NACK（否定确认）

- 全称：Negative Acknowledgement，即否定应答。
 - 工作方式：接收方只在没收到数据包或收到损坏的包时，才给发送方回发一个“否定包”，告诉对方：“我没收到，麻烦重发一下”。
 - 逻辑流程：
 1. 发送方发数据包 → 等待；
 2. 接收方收到正常包：什么都不做，不回任何包；
 3. 接收方没收到 / 收到坏包：回发 NACK；
 4. 发送方收到 NACK → 重传数据包。
 - 特点：
 - 正常情况下不需要回包，控制开销比 ACK 小；
 - 缺点：如果 NACK 包在路上丢了，发送方会一直以为对方收到了，不会重传，可能导致数据丢失；
 - 常用于对开销敏感、丢包率不高的场景。
-

三、IACK（隐式确认，Implicit ACK）

你笔记里提到的 IACK，是 WSN 中一种更特殊的逐跳确认方式：

- 工作方式：发送方不等待接收方的显式 ACK 包，而是直接监听接收方的行为：

Wireless Sensor Network

1. 节点 A 给节点 B 发数据包;
2. A 不等待 B 的 ACK, 而是监听 B 是否把这个数据包转发给了下一跳节点 C;
3. 如果 A 监听到 B 转发了这个包, 就认为传输成功, 清除缓存。

- 特点:

- 完全不需要额外的 ACK/NACK 包, 控制开销为 0;
- 只能在单跳范围内使用, 属于逐跳确认;
- 依赖无线信道的广播特性, 节点可以直接听到邻居节点的转发行为。

先回顾 IACK 基本流程 (逐跳)

节点 A → 节点 B (单跳传输)

1. A 发数据包, 本地缓存该包;
2. A 监听无线信道, 看 B 有没有把包转发给下一跳;
3. 监听到 B 转发 → 判定成功, 删除缓存;
4. 长时间没监听到转发行为 → 判定传输失败。

两种“没发送成功”的情况 + 对应处理

情况 1: 数据包丢了 / 损坏, B 根本没收到包

B 收不到包, 自然不会去转发。

A 一直监听, 超时后判定传输失败, 主动重传该数据包。

可配合规则: 设置最大重传次数, 多次重传仍失败, 则判定链路故障, 放弃或切换路径。

情况 2: B 收到包, 但自身故障 / 缓存满, 迟迟不转发

B 拿到包却卡在本本地、不往下传, A 同样监听不到转发动作。

超时后 A 依然判定失败, 执行重传。

若反复出现, 说明 B 节点过载 / 异常, 上层路由会尝试绕开该节点。

情况 3: 极端特例: B 转发了, 但 A 没监听到 (无线干扰)

无线环境有干扰, B 确实转发了, 但 A 没收到信道信号。

A 超时后会误判为失败, 触发重传, 造成重复数据包。

WSN 一般靠数据包编号区分新旧包, 接收端丢弃重复包, 不影响业务。

IACK 的短板 (考点)

1. 依靠超时机制判断失败, 没有显式应答, 无法第一时间发现问题;
2. 无线干扰易造成误判, 产生冗余重传;
3. 仅适用于单跳、逐跳场景, 端到端全程没法用;
4. 完全依赖信道监听, 信道环境差时可靠性下降。

简答速记

IACK 依靠超时识别传输失败: 长时间监听不到下一跳转发行为, 发送节点就判定传输失败, 执行数据包重传, 并受最大重传次数约束。无线干扰还可能导致误判、产生重复包, 需靠包序号去重。

(2) 冗余传输

发送节点多次发送同一数据包的备份, 只要接收节点收到至少一个数据包即可。冗余传输可采用多路径方式。多路径传输可至同一个汇聚节点, 也可以至多个不同的汇聚节点。利用多路径空间的不相关性提高数据传送的成功率。要考虑一个数据包多少个冗余复制包 (复制太少, 丢包风险还是高; 复制太多, 资源浪费严重。所以要根据网络丢包率、节点能量来设定最优复制数量), 以及选择什么样的路径 (要尽量选择空间不相关的路径, 比如不共用中间节点的路径, 不然如果几条路径都经过同一个节点, 这个节点一旦拥堵或故障, 所有备份包都可能丢了, 冗余就失去了意义)。消耗网络资源较多。

(3) 速率控制

丢包恢复和冗余传输是为了保证数据包或数据块的端到端传输可靠性, 而速率控制是为

了完成基于任务的可靠传输。在保证任务完成的前提下，调节源节点的数据速率，汇聚节点根据成功接收数据包的数量计算传输可靠度和估测网络拥塞程度，据此通知源节点调整数据发送速率，以实现基于任务的可靠传输。

一般来说，汇聚节点根据一个周期内成功接收数据包的数量计算网络的传输可靠度，同时也估测网络的拥塞程度。如果传输可靠度低于预定要求，则通知源节点提高数据发送速率以提高可靠度。否则，减少源节点数据发送速率，降低网络拥塞。？

第十一讲

RMST 协议

文献：RMST:Reliable Data Transport in Sensor Networks

总体描述文献内容：

本文提出了一种名为 RMST (Reliable Multi-Segment Transport) 的可靠数据传输协议。RMST 通过结合 MAC 层和传输层的可靠性机制，采用选择性 NACK (否定确认) 和路径缓存，解决高误码率、动态拓扑和能量受限环境下的数据传输问题。论文分析了 MAC 层 ARQ (自动重传请求) 与传输层 NACK 的协同作用，并通过仿真实验验证了 RMST 在能耗、可靠性和扩展性上的优势。RMST 与 Directed Diffusion 路由协议集成，支持数据分片重组和逐跳修复，显著提升了多跳传输的可靠性。

文献研究的问题：

- (1) 高误码率：无线链路不稳定导致数据包丢失率高 (2%-30%)。
- (2) 能量效率：传统协议 (如 TCP) 的 ACK 机制和端到端重传能耗过高。
- (3) 动态拓扑适应性：节点失效或移动导致路径频繁变化，传统协议难以维护可靠连接。
- (4) 大数据分片传输：数据实体 (如代码或图像) 需分片传输，但传统协议缺乏高效的分片重组与丢失修复机制。

问题提出的基础：

现有方法的局限性：

- TCP：假设丢包由拥塞引起，依赖端到端 ACK，能耗高且不适应无线链路的高误码率。
- 传统多播协议 (如 PGM、SRM)：控制流量大，依赖全局路径状态，不适合资源受限的传感器网络。
- MAC 层 ARQ (如 802.11)：RTS/CTS 和 ACK 机制在广播和多跳场景下开销大，且无法应对多节点失效。

传感器网络的特殊性 (能量受限、动态性、数据中心路由) 催生了需跨层优化的可靠传输需求。

问题的解决方法：

RMST 的核心机制包括：

(1) 跨层可靠性设计：

MAC 层选择性 ARQ：仅对单播数据使用 ARQ，广播路由发现包不启用 ARQ 以减少开销。

- ARQ：就是自动重传请求，简单说就是“发了包，等对方确认，没确认就重传”。
- 选择性 ARQ：不是所有包都用 ARQ，而是“挑着用”：
 - 对单播数据 (比如发给某个特定节点的数据包) 启用 ARQ，保证关键数据可靠到达；
 - 对广播 / 路由发现包 (比如给所有邻居发的路由广播) 不启用 ARQ，因为这些包丢了影响不大，重传反而浪费能量。
- 作用：既保证了关键数据的可靠性，又避免了给所有包都做确认的高开销，专门适配传感器网络的低功耗需求。

传输层 NACK 机制：接收端检测丢失分片后发送选择性 NACK，沿反向路径（back-channel）逐跳修复。

- NACK（否定确认）：和 ACK 正好相反，只在“丢包了”的时候才发消息，正常收到包啥也不做。
- 选择性 NACK：不是说“所有包都丢了”，而是精准告诉上游节点“我哪几个分片丢了”。
- 逐跳修复：NACK 不是直接发给源节点，而是沿着反向路径一跳一跳往回传，路过的每个节点都看自己有没有缓存的丢包分片：
 1. 接收端发现分片 3 丢了，发 NACK：“我缺分片 3”；
 2. 上一跳节点收到 NACK，如果自己缓存了分片 3，直接发给接收端，修复完成；
 3. 如果上一跳也没有，就继续把 NACK 发给再上一跳，直到找到有缓存的节点或源节点。
- 作用：不用像 TCP 那样等超时，也不用源节点端到端重传，丢包在半路就能修好，既省时间又省能量。

（2）路径缓存：中间节点缓存分片，支持本地修复，减少端到端重传。

每个中间节点都会把转发过的数据包分片缓存一段时间，就像沿途的“临时仓库”。

- 每个中间节点都会把转发过的数据包分片缓存一段时间，就像沿途的“临时仓库”。
- 当接收端的 NACK 传回来时，路过的节点只要缓存里有丢包的分片，就直接拿出来发给接收端，不用再跑到源节点去要。
- 举个例子：源节点 A → 节点 B → 节点 C → 汇聚节点 D。D 发现分片 3 丢了，发 NACK 回传，到 C 发现 C 缓存了分片 3，直接发给 D，就不用让 A 重传整个包了。
- 作用：大幅减少重传的距离和能耗，避免端到端重传带来的“千里送包”，是 RMST 节能的关键。

（3）动态反向路径维护：利用 Directed Diffusion 的路径强化机制，自动构建反向路径（无需额外控制流量）。

- RMST 是和 Directed Diffusion（定向扩散）路由协议深度集成的，这个协议的特点是：数据从源节点往汇聚节点传的时候，会自动“强化”出一条稳定的正向路径。
- RMST 利用这个特性，反向路径会跟着正向路径一起自动建立：正向路径是 A→B→C→D，反向路径就是 D→C→B→A，不需要额外发控制包去建路。
- 作用：不用额外维护反向路径的路由表，也不用发控制包，就能让 NACK 消息沿着反向路径顺利回传，节省了大量控制开销，也适配传感器网络动态变化的拓扑。

（4）分片与重组：数据分片传输，通过唯一标识（RmstNo）和分片序号（FragNo）实现重组。

当要传的数据比较大（比如一张图片、一段代码），传感器节点没法一次发完，RMST 会把它拆成多个分片传输：

- 给每个数据块一个唯一标识 RmstNo（比如文件 1）；
- 每个分片再标上序号 FragNo（比如分片 1、分片 2、分片 3）。

接收端收到分片后，根据 RmstNo 和 FragNo 把它们重新拼成完整的文件；如果某一个分片丢了，只需要单独重传这个分片，不用重传整个文件。

作用：解决了大数据在传感器网络中传输的问题，而且和 NACK 机制配合，实现了“丢哪片补哪片”，效率很高。

解决方法和现有方法的区别与联系：

对比维度	RMST	现有方法（TCP、PGM、SRM）
可靠性机制	选择性 NACK + MAC 层 ARQ	端到端 ACK 或全局多播 NACK
能耗优化	逐跳修复减少重传距离，缓存降低冗余传输	端到端重传能耗高，控制流量大
动态适应性	依赖 Directed Diffusion 自动修复路径	依赖固定路由表，拓扑变化时需全局更新
分片支持	内置分片重组，支持大数据传输	需应用层实现，缺乏统一管理
关联技术	与 Directed Diffusion 深度集成	独立于路由协议，需额外维护路径状态

1. 可靠性机制：选择性 NACK + MAC 层 ARQ

这是 RMST 的“双保险”，分层配合实现可靠传输：

- MAC 层选择性 ARQ：只对关键的单播数据启用链路层确认和重传，广播路由包不启用，既保证了链路传输的基础可靠性，又避免了不必要的能量开销。
- 传输层选择性 NACK：接收方只在检测到分片丢失时，才发送 NACK 报文，精准指出丢失的分片序号，而不是像 TCP 那样用 ACK 确认每个包。这种方式开销极低，且能和路径缓存、逐跳修复配合，实现高效丢包恢复。

2. 能耗优化：逐跳修复 + 路径缓存（核心优势）

这就是你之前看的第二张图的精髓，也是 RMST 最节能的地方：

- 逐跳修复减少重传距离：丢包修复不需要源节点参与，而是在丢包点附近完成。比如分片在节点 2→节点 3 的链路上丢了，节点 3 的 NACK 只需要回传给节点 2，节点 2 直接从缓存里拿出分片重传，重传只发生在节点 2 和 3 之间，不用从源节点重发整条路径。
- 缓存降低冗余传输：每个中间节点都会缓存转发过的分片，NACK 回传时，只要找到有缓存的节点就能直接修复，避免了“端到端重传”带来的长距离、重复传输，大幅降低了节点的发送和接收能耗。

3. 动态适应性：依赖 Directed Diffusion 自动修复路径

RMST 和定向扩散（Directed Diffusion）路由协议是深度集成的，天生适配动态拓扑：

- 定向扩散协议本身会在数据传输过程中，自动强化出一条稳定的正向路径，同时反向路径也会随之自动建立。
- 当节点失效或拓扑变化时，定向扩散会自动更新路径，RMST 的 NACK 报文会沿着新的反向路径回传，不需要额外维护路由表或发送控制包，就能自动适应路径变化，保证丢包修复的连通性。

4. 分片支持：内置分片重组，支持大数据传输

传感器节点的缓存和单次传输能力有限，没法直接传输大文件（比如图像、代码）：

- RMST 会把大数据拆成多个分片传输，每个分片都有唯一标识 RmstNo 和分片序号 FragNo。
- 接收方收到分片后，根据序号重组出完整数据；如果某一个分片丢了，只需要单独重传这个分片，不用重传整个文件。
- 分片和 NACK、路径缓存机制配合，实现了“丢哪片补哪片”，让传感器网络也能高效传输大数据。

5. 关联技术：与 Directed Diffusion 深度集成

这是 RMST 的设计亮点，和传统独立于路由协议的传输层协议完全不同：

- 传统传输协议（如 TCP）需要额外维护路由表和反向路径，控制开销大，拓扑变化时容易失效。
- RMST 直接利用定向扩散的路径强化机制，反向路径随正向数据传输自动建立，NACK 报文可以直接沿反向路径回传，不需要额外的控制流量，既降低了开销，又提升了动态适应性。

解决方法存在的问题：

- （1）高误码率下的性能下降：当链路误码率超过 30% 时，路径维护困难，NACK 成功率降低（需依赖 MAC 层 ARQ）。
- （2）缓存管理复杂性：中间节点缓存需维护分片映射和超时机制，增加内存与计算开销。
- （3）定时器敏感性问题：NACK 触发依赖固定超时，动态环境需自适应调整（论文提到未来研究自适应定时器）。
- （4）反向路径维护开销：动态拓扑变化时反向路径需频繁重建，可能增加控制流量。

改进方法建议：

- （1）自适应定时器机制：根据网络状态（如链路质量、节点密度）动态调整 NACK 触发超时，减少无效重传。
- （2）混合缓存策略：结合 LRU（最近最少使用）和能量感知替换策略，优化缓存利用率。
- （3）多路径传输：在 Directed Diffusion 中引入多路径强化，分散传输负载并提升容错性。
- （4）机器学习优化：利用强化学习预测最优修复路径，减少 NACK 传输次数。
- （5）跨层能量感知：在 MAC 层 ARQ 中引入剩余能量权重，优先选择高能量节点转发修复包。
- （6）地理信息辅助：结合 GEAR 协议的地理路由，优化反向路径构建效率。

RMST 工作模式：

缓存模式下，逐跳节点均需维护缓存，参与丢包检测，确保数据逐步可靠传输，适用于对中间节点可靠性要求高的场景。

非缓存模式仅源节点与 Sink 维护缓存，中间节点不参与，由 Sink 负责丢包检测，减少中间节点负担，适合节点资源受限网络。

RMST 采用跨层优化设计，综合考虑 MAC 层、传输层与应用层，提升整体传输效率。

依赖定向扩散路由机制，节点失效时自动改变路由，保障数据传输可靠性，适应动态网络环境。

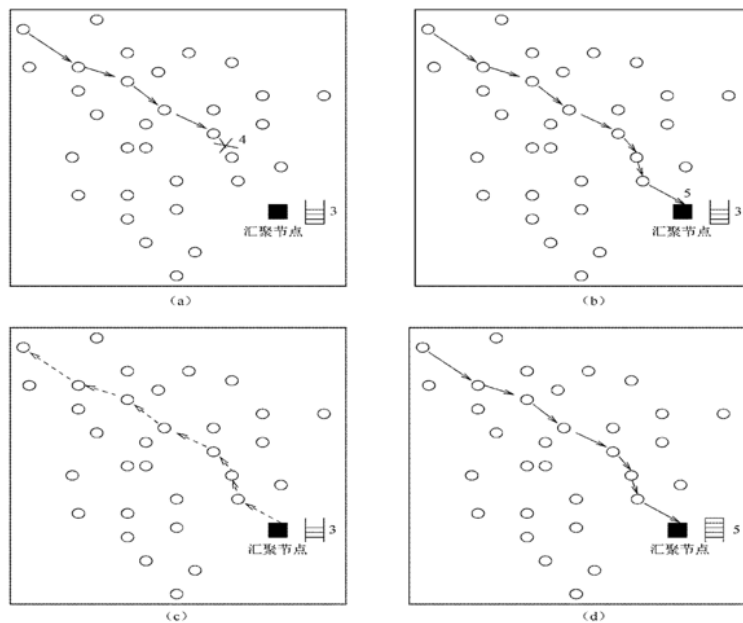
但是，不能保证分组顺序到达，影响对实时性要求高的应用体验。

而且在 MAC 层对单播数据采用 ARQ，广播数据不采用，传输层用 NACK 确认，缓存模式下易出现缓冲区溢出，应限制其在高负载场景的应用。

Wireless Sensor Network

■ 缓存模式——对应于逐跳的可靠传输

源节点到 Sink 的传输路径上的每个节点都需要维护缓存，并参与分组丢失检测。



(a) 数据包丢失 (b) 丢包检测 (c) NACK (d) 重传

过程拆解

(a) 数据包丢失

数据包沿着路径从源节点向汇聚节点传输。

其中序号为 4 的数据包在传输过程中丢失了，所以汇聚节点只收到了前面的包，比如序号 3，但没有收到 4。

(b) 丢包检测

后续的数据包，比如序号 5，又成功到达汇聚节点。

汇聚节点发现自己收到了 3 和 5，中间缺少 4，于是判断：序号为 4 的数据包丢失了。

(c) NACK

汇聚节点向数据源方向发送一个 NACK 请求，意思是：

我没有收到编号为 4 的数据包，请重新发送。

图中的虚线箭头表示这个 NACK 控制消息沿反方向返回。

(d) 重传

收到 NACK 后，源节点或中间缓存了该数据包的节点会重新发送丢失的数据包。

序号为 4 的数据包被再次沿路径传送到汇聚节点，从而完成可靠传输。

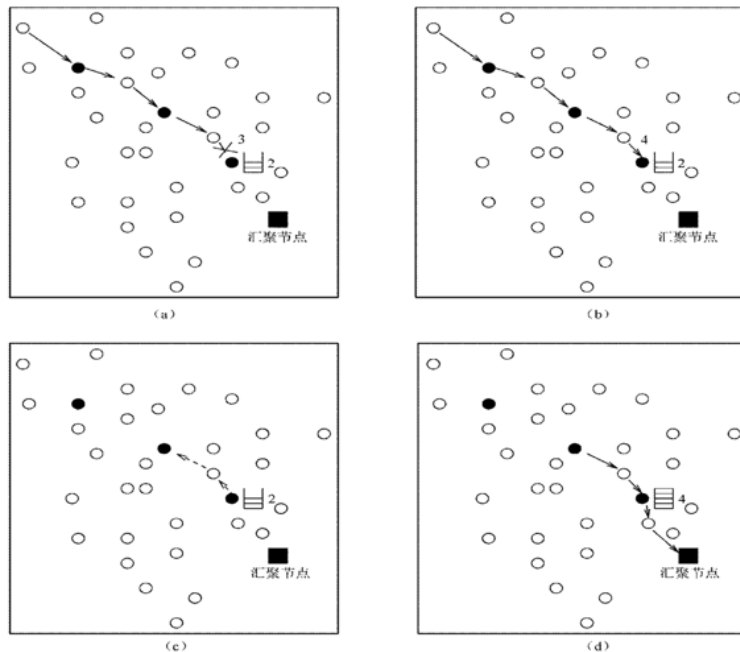
核心问题

重传的距离是整条路径，丢包在半路，却要源节点从头重传，传输距离长、能耗高，这就是传统端到端重传的弊端。这种机制的优点是不用每收到一个包都发送 ACK，而是只有发现丢包时才发送 NACK，因此可以减少控制开销。

Wireless Sensor Network

■ 非缓存模式——对应于端到端的可靠传输

只有源节点与 Sink 需要维护缓存，中间节点不需要维护缓存，分组的丢失检测由 Sink 负责。



(a) 数据包丢失 (b) 丢包检测 (c) 传输NACK到上一跳 (d) 重传

过程拆解

(a) 数据包丢失

数据包沿多跳路径向汇聚节点传输。

其中序号为 3 的数据包在某一跳传输过程中丢失了。

此时后面的节点或汇聚节点只收到了前面的包，例如 2，但没有收到 3。

(b) 丢包检测

后续数据包继续到达，例如节点又收到了序号为 4 的数据包。

接收端发现数据包序号不连续：

收到了 2，又收到了 4，但中间缺少 3。

因此判断：编号为 3 的数据包丢失了。

(c) 传输 NACK 到上一跳

检测到丢包的节点不会马上请求源节点重新发送，而是向上一跳节点发送一个 NACK。

NACK 的意思是：

我没有收到编号为 3 的数据包，请你把它重新发给我。

图中虚线表示 NACK 控制消息，方向是从下游节点返回到上一跳节点。

(d) 重传

上一跳节点如果缓存了丢失的数据包，就直接把编号为 3 的数据包重新发送给下游节点。

这样就完成了局部恢复，不需要让数据包从源节点重新走完整条路径。

核心优势

重传只在丢包点的上一跳和下一跳之间完成，路径缓存让节点变成了“临时仓库”，NACK 只需要传一跳，重传距离极短，大幅节省能耗和时间。

PSFQ 协议

- PSFQ 是一种下行通信的数据块可靠传输协议。可靠性是关注点。

方向	含义	典型场景
上行 (Upstream)	传感器节点→汇聚节点	节点采集数据, 发给基站/网关
下行 (Downstream)	汇聚节点→传感器节点	基站下发命令、代码镜像、配置更新

- 设计目标: 基于 WSN 中传感器节点重编程应用而提出的协议, 保证每个节点都能够收到来自 Sink 的全部分组。
- 基本思想: 将一个消息分组分割成多个短分组; 逐跳进行可靠传输
- 基本过程: 在逐跳传输分组时, 上一跳节点以较慢的速率发送分组, 下一跳节点发现分组丢失后, 立即向上一跳节点请求重传。
- 三种操作: Pump、Fetch、Report。

总体描述文献内容:

PSFQ (Pump Slowly, Fetch Quickly) 是一种可靠传输协议, 其核心思想是通过“泵慢取快”的机制, 解决多跳无线传感器网络中数据传输的高误码率和错误累积问题。协议通过中间节点逐跳检测和恢复丢失数据, 支持本地化快速恢复, 并引入多模态操作 (存储转发与多跳转发的动态切换) 以适应不同信道条件。论文通过仿真和实验验证了 PSFQ 在高误码率下的性能优于传统方法 (如 SRM), 并展示了其在延迟、通信开销和容错性上的优势。

文献研究的问题:

WSN 中可靠数据传输的挑战, 具体包括:

- (1) 高误码率环境下的数据丢失: 传感器网络通常部署在复杂环境中, 无线链路质量差, 导致数据丢失率高。
- (2) 多跳传输的错误累积: 端到端恢复机制在多跳网络中因错误率指数级累积而失效。
- (3) 资源受限: 传感器节点计算能力、存储和能量有限, 传统协议 (如 TCP、SRM) 无法直接适用。
- (4) 动态重编程需求: 如传感器节点需通过无线网络接收完整代码镜像, 单数据包丢失即导致任务失败。

问题提出的基础:

- (1) 应用需求驱动: 传感器网络从单一任务转向动态重编程 (如灾难恢复中更新节点功能), 需可靠传输代码或脚本。

- (2) 现有协议不足:

传统端到端协议 (如 TCP) 依赖高可靠链路, 不适合多跳无线网络。

可靠多播协议 (如 SRM) 需复杂路由 (如 IP 多播) 和频繁信令, 资源消耗大。

- (3) 无线链路特性: 传感器节点使用低功耗射频, 链路误码率高达 5%~10%, 需本地化恢复机制。

问题的解决方法:

PSFQ 通过以下机制实现可靠传输:

- (1) 逐跳错误恢复 (Hop-by-Hop Recovery)

中间节点缓存数据并检测丢失, 触发本地恢复 (NACK 请求), 避免错误累积。

- (2) 泵慢 (Pump Slowly)

控制数据注入速度 (定时器 $T_{\min}$ 和 $T_{\max}$), 减少冗余广播, 为本地恢复预留时间。

- (3) 取快 (Fetch Quickly)

检测到丢失后, 节点主动广播 NACK, 从邻居节点快速重传 (定时器 T_r), 支持多窗口丢失聚合。

- (4) 多模态操作

根据信道质量动态切换存储转发 (高误码率) 与多跳转发 (低误码率) 模式, 平衡延迟

与可靠性。

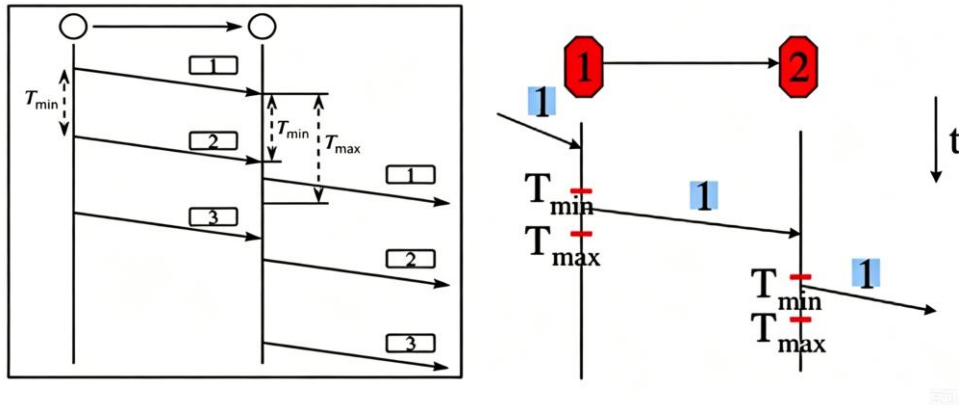
(5) 状态报告 (Report Operation)

通过聚合反馈减少信令开销，仅末端节点触发报告，中间节点捎带状态信息。

工作流程：

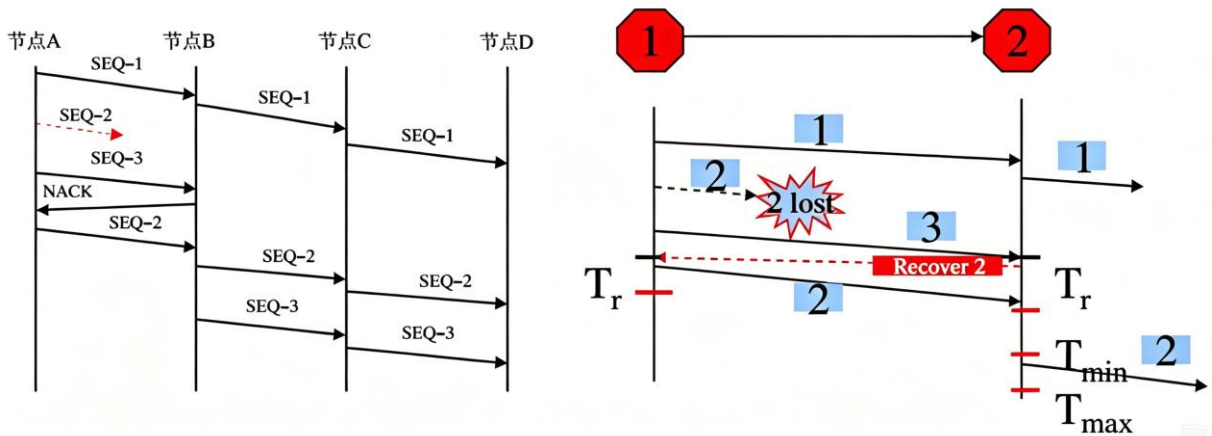
① Pump

- Sink 每 T_{min} 时间广播一个分组，直到所有分组都发送出去。
- 节点每收到一个分组就检查自己的缓冲区，丢弃重复分组；若该分组为新分组，则把分组存储在缓存中，并把 TTL 减 1。若 TTL 非 0 且此分组序列号前的分组都已收到，则延迟随机时间 t 后转发此分组 ($T_{min} < t < T_{max}$)。



② Fetch

- 当接收节点出现序列号空缺时，接收节点向上一跳节点发送 NACK
- 接收节点在未收到重传分组时将不转发任何分组。若一直没有收到重传分组，每 T_r 时间发送一次 NACK, $T_r < T_{max}$ 。
- NACK 只在一跳范围内传播。



③ Report

Sink 需了解节点状态时，发起汇报请求，最后一跳节点立即响应，其他节点状态信息捎带在汇报分组中，无空间则新创建分组，实现状态反馈与监视。

■ 目的

当 Sink 需要了解目标节点的状态信息时，“report”操作可以用来向 Sink 节点反馈数据传输的状态信息，作为反馈/监视机制。

■ 操作

只有最后一跳的节点对 Sink 发送的汇报请求进行立即响应；

其他节点把自己的状态信息捎带在汇报分组中；

若汇报分组中没有空间，节点将新创建一个分组。

Wireless Sensor Network

Sink 发出 REPORT 请求后，不让所有节点同时回报，而是让最外层 / 最后收到请求的节点先回报，其他中间节点在回传过程中顺便加入自己的状态信息。

这样可以避免大量节点同时向 Sink 发报告，造成拥塞。

注意它是下行的。假设网络路径是：

$Sink \rightarrow A \rightarrow B \rightarrow C$

Sink 发出 REPORT 请求，方向是：

$Sink \rightarrow A \rightarrow B \rightarrow C$ ，这里 C 是最后一跳节点，也就是这个请求传播路径上最末端的节点。

所以第一句话：

只有最后一跳的节点对 Sink 发送的汇报请求进行立即响应；

意思是：

C 收到 REPORT 请求后，马上生成一个 REPORT 分组往回发。

也就是：

$C \rightarrow B \rightarrow A \rightarrow Sink$

第二句话：

其他节点把自己的状态信息捎带在汇报分组中；

意思是 B、A 不会一收到请求就马上各自单独发报告，而是等 C 的 REPORT 分组经过自己时，把自己的信息顺便塞进去。

比如 C 先生成：

REPORT = C 的状态

经过 B 时，B 加进去：

REPORT = C 的状态 + B 的状态

经过 A 时，A 再加进去：

REPORT = C 的状态 + B 的状态 + A 的状态

最后 Sink 收到一个分组，就知道了 A、B、C 的状态。

这里的“状态信息”可以是节点是否收到完整数据、缺哪些序号的数据包、缓存情况、恢复情况等。

第三句话：

若汇报分组中没有空间，节点将新创建一个分组。

因为一个 REPORT 分组容量有限，不能无限装所有节点的信息。

比如 C 的 REPORT 到 B 时还能装，B 把自己的状态加进去；

但到 A 时，分组已经满了，A 的状态放不下了。

这时 A 就不能再硬塞，而是新建一个 REPORT 分组：

REPORT 1 = C 的状态 + B 的状态

REPORT 2 = A 的状态

然后两个报告都继续发给 Sink。

所以 PSFQ 中 REPORT 的核心逻辑是：

Sink 发 REPORT 请求

→ 请求向外传播

→ 最末端节点先生成 REPORT

→ REPORT 向 Sink 返回

→ 中间节点把自己的状态捎带进去

→ 包满了才新建 REPORT

它的目的就是：

减少报告分组数量，避免所有节点同时向 Sink 回报，降低拥塞和能耗。

所以这里的“最后一跳节点”不要理解成“离 Sink 最近的节点”，而应该理解成：

REPORT 请求传播方向上的最后一层节点，也就是最外层/叶子节点。

优点：

专为传感器节点重编程应用设计，保证每个节点收到来自 Sink 的全部分组，可靠性高，适用于对节点配置更新要求严格的场景。

缺点：

- (1) 仅靠 NACK 确认机制，无法发现所有分组丢失情况，存在传输漏洞。
- (2) “pump”操作慢，增加传输时延，浪费带宽，降低传输效率。
- (3) 逐跳丢失恢复机制需大缓存空间，对节点资源要求高，限制其在资源受限网络的应用。

适用场景：

适用于对节点重编程可靠性要求高，且网络资源相对充裕，对传输时延要求不苛刻的 WSN 场景，如环境监测网络的节点程序更新。

与现有方法的区别和联系：

对比维度	PSFQ	现有方法（如 SRM、TCP）
恢复机制	逐跳恢复，本地 NACK 请求	端到端恢复，依赖目标节点反馈
多播支持	基于广播，无需 IP 多播基础设施	依赖 IP 多播路由（SRM 需组播树）
资源消耗	轻量级，减少冗余信令（如无周期会话）	信令开销大（SRM 需周期会话消息）
适用场景	高误码率、多跳无线网络	高可靠链路（如有线网络或无线 LAN）
延迟控制	通过泵慢取快平衡延迟与可靠性	延迟随跳数指数增长（端到端恢复）

联系：PSFQ 借鉴了 SRM 的负确认（NACK）机制，但通过本地化恢复和多模态操作优化了其设计。

解决方法存在的问题：

- (1) 参数敏感性：定时器（ T_{min} 、 T_{max} 、 T_r ）需手动配置，实际网络中动态环境可能导致性能波动。
- (2) 缓存管理：节点需缓存完整数据段，对存储资源有限的传感器可能不适用。
- (3) 大规模网络扩展性：实验仅测试小规模网络（13 跳以内），大规模部署时报告机制可能引发信令风暴。
- (4) 主动取回机制限制：若最后一个数据包丢失，需依赖超时触发主动取回，可能增加延迟。

一、先回忆：PSFQ 的正常工作逻辑

PSFQ 的“取快（Fetch Quickly）”机制，是这样处理丢包的：

1. 节点收到数据包，会检查序号是否连续。
2. 如果发现序号跳变（比如收到了 3 号包，却没收到 2 号包），就知道中间的包丢了。
3. 节点会立刻广播 NACK 请求，向邻居节点快速重传丢失的包，不用等源节点端到端重传。

关键点：这个 NACK 触发的前提，是节点收到了后面的包，发现前面的序号断了，才能判断出丢包。

二、问题来了：最后一个包丢了会怎样？

假设源节点（Sink）按顺序发送了 1, 2, 3, 4 四个包，其中最后一个包 4 丢了：

- 下游节点已经收到了 1, 2, 3，但永远不会收到比 4 更大的包了。
- 它没有办法通过“序号不连续”来判断“4 号包丢了”，因为后面没有包来触发这个判断。
- 节点会一直以为：“4 号包还在路上，只是延迟了”，不会主动发起 NACK 请求。

这就是“主动取回机制限制”的核心：PSFQ 的丢包检测，依赖后续包的到达来触发；最后一个包丢了，就没有后续包来触发检测了。

三、为什么会增加延迟？

既然节点没法主动发现丢包，就只能靠超时机制来兜底：

1. 节点会启动一个定时器，等待“最后一个预期包”的到达。
2. 定时器超时了，包还没到，节点才会判定“这个包丢了”，然后发起 NACK 请求。
3. 这个超时等待的时间，就是额外增加的延迟。

举个例子：

- 正常丢包（比如 2 号包丢了）：收到 3 号包就立刻发现，马上发 NACK，几乎没有额外延迟。
 - 最后一个包（4 号包）丢了：节点必须等定时器超时，才会发现丢包，多了一段超时等待的时间，延迟就上去了。
-

一句话总结

PSFQ 的丢包恢复，是“靠后面的包提醒前面的包丢了”；最后一个包丢了，就没有“后面的包”来提醒了，只能靠超时才知道丢包，所以会额外增加延迟。

改进方法：

（1）动态参数调整

根据信道误码率和网络负载自适应调整定时器（如强化学习优化 T_r/T_{max} 比例）。

针对的原问题：

PSFQ 里的定时器 T_{min} （最小发送间隔）、 T_{max} （最大到达间隔）、 T_r （NACK 超时时间）都是手动固定配置的。但实际网络中，误码率、负载是动态变化的，固定参数很容易导致性能波动：

- 信道差、误码率高时， T_r 设短了会误判丢包，设长了又会增加延迟；
- 网络负载高时， T_{max} 设小了会误判丢包，设大了又会让丢包恢复不及时。

改进思路：

根据信道误码率、网络负载，自适应调整定时器参数，比如用强化学习优化 T_r/T_{max} 的比例：

- 误码率高时，适当调大 T_r ，减少不必要的 NACK 重传；
- 负载低、链路质量好时，调小 T_{max} ，让丢包恢复更快。

效果：让协议能适应动态网络环境，不会因为固定参数导致性能忽高忽低。

（2）分层缓存策略

按优先级缓存部分数据段，结合外部存储（如 EEPROM）管理大文件。

针对的原问题：

PSFQ 要求中间节点缓存完整数据段，以备 NACK 触发的本地重传。但传感器节点的 RAM 非常有限，缓存完整大文件很容易溢出，不适合资源受限的节点。

改进思路：

- 按优先级缓存：节点不缓存所有数据段，只缓存关键、高频被请求的数据段，或者缓存文件的前半部分 / 关键分片，降低缓存占用；
- 结合外部存储：把不常用的数据段存在 EEPROM 等外部存储里，需要时再读取，而不是一直占着 RAM，解决大文件存储问题。

效果：在不影响本地恢复能力的前提下，大幅降低节点的缓存压力，让 PSFQ 也能在存储资源有限的节点上运行。

（3）分布式报告优化

引入簇头节点聚合状态报告，减少回传路径上的信令数量。

针对的原问题:

PSFQ 的“Report 操作”依赖末端节点回传状态报告,中间节点捎带信息。但在大规模网络中,大量节点同时回传报告,会导致信令数量激增,形成“信令风暴”,消耗大量能量和带宽。改进思路:

引入簇头节点聚合:把网络分成多个簇,每个簇选一个簇头节点,先收集簇内所有节点的状态信息,再由簇头节点把聚合后的报告一次性回传给 Sink,而不是每个节点单独捎带。效果:大幅减少回传路径上的信令数量,避免大规模网络下的信令风暴,降低控制开销。

(4) 混合恢复机制

结合正向确认(ACK)与 NACK,针对关键数据段(如文件末尾)使用 ACK 确保交付。

针对的原问题:

PSFQ 只靠 NACK 恢复丢包,而最后一个包丢失时,没有后续包触发 NACK,只能靠超时才发现,会增加延迟。

改进思路:

- 对普通数据段,继续用 PSFQ 原有的 NACK 机制,保持轻量、低开销;
- 对关键数据段(比如文件末尾、最后一个分片),改用 ACK(正向确认)机制:接收节点收到后主动回复 ACK, Sink 收到 ACK 才认为传输完成,避免最后一个包丢了却不知道的情况。

效果:既保留了 NACK 的低开销优势,又解决了最后一个包丢失的延迟问题,保证关键数据的可靠交付。

(5) 跨层设计

与 MAC 层协同(如链路层重传),减少传输层恢复压力,提升整体效率。

针对的原问题:

PSFQ 的丢包恢复完全由传输层的 NACK 和本地重传完成,当链路误码率很高时,传输层需要频繁触发恢复,开销大、效率低。

改进思路:

和 MAC 层(链路层)协同工作:

- MAC 层先通过链路层重传(比如 ARQ)解决部分随机丢包;
- 传输层只处理 MAC 层恢复不了的丢包,减少传输层的恢复压力。

效果:把丢包恢复的压力分散到链路层和传输层,提升整体传输效率,降低传输层的信令和重传开销。

假设路径是:源节点 $S \rightarrow A \rightarrow B \rightarrow C \rightarrow \text{Sink}$,数据包从 A 发给 B 时丢了。

MAC 层处理:

A 发给 B 后, B 没收到。A 等不到 B 的 MAC ACK,于是 A 自动重发:

A $\rightarrow$ B 失败

A 没收到 ACK

A 立即重传给 B

链路层 / 局部链路处理:

如果 A 重传多次还是失败,系统可能认为:

A-B 这条链路质量不好

于是可能记录链路质量下降,或者让上层选择其他路径,例如:

A $\rightarrow$ D $\rightarrow$ B

或者在下一跳节点发现序号缺失后,发送局部 NACK,请求上一跳重新发。

总之,MAC 层处理丢包,是处理相邻两个节点之间“这一帧有没有发成功”;链路层处理丢包,是维护相邻节点之间这条链路的可靠性,范围比单纯 MAC ARQ 更宽。

CODA 协议

总体描述文献内容：

本文提出了一种名为 CODA (Congestion Detection and Avoidance) 的拥塞控制方案，识别三种拥塞情形：信源节点附近拥塞、暂时热点、持续热点，将缓存器占用与信道负载联合作为拥塞指标，全面评估网络拥塞状态。通过三种机制实现高效拥塞控制：

- (1) 基于接收端的拥塞检测：结合信道负载和缓冲区占用率，动态检测拥塞。
- (2) 开环逐跳反压 (Open-loop Hop-by-Hop Backpressure)：通过广播抑制消息 (Suppression Message) 快速缓解局部拥塞。
- (3) 闭环多源调节 (Closed-loop Multi-source Regulation)：通过慢速反馈 (如 ACK) 调节多个源的发送速率，解决持续拥塞。

基于接收机的拥塞检测，开环逐跳背压机制向信源节点发送信号，闭环多信源管理机制应对持续大规模拥塞，多机制协同缓解拥塞。

论文通过仿真和实验验证了 CODA 的性能，表明其能显著减少能量消耗 (Energy Tax) 并保持较低的数据保真度损失 (Fidelity Penalty)。

文献研究的问题：

- (1) 事件驱动的高突发流量：传感器网络在事件触发时可能因突发数据流导致拥塞。
- (2) 多跳无线链路的拥塞传播：传统端到端拥塞控制 (如 TCP) 不适用于资源受限的多跳传感器网络。
- (3) 能量效率与数据保真度的权衡：需在减少能量消耗的同时，最小化数据丢失对应用的影响。
- (4) 动态拓扑与异构流量：现有方案 (如 ESRT) 无法区分不同源的优先级或局部拥塞场景。

问题提出的基础：

- (1) 应用需求：如灾难监测中，事件触发时大量节点同时上报数据，导致网络拥塞。
- (2) 现有协议不足：

ESRT [9]：通过全局广播调节所有源节点，能量开销大且无法区分优先级。

传统 TCP：依赖端到端 ACK 和重传，不适合多跳无线网络的高误码率环境。

- (3) 无线链路特性：CSMA-based MAC (如 TinyOS 默认协议) 在负载高时易因隐藏终端和缓冲区溢出导致拥塞。

问题的解决方法：

CODA 通过以下机制实现拥塞控制：

- (1) 拥塞检测：

缓冲区占用率：接近溢出时触发拥塞信号。

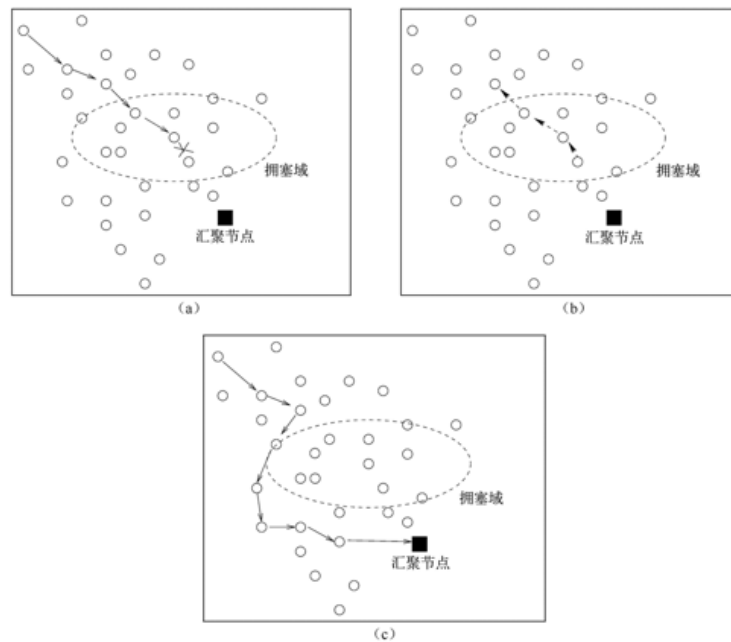
信道负载采样：节点在发送数据时激活信道负载监测 (低能耗)，通过指数加权平均估算负载。

- (2) 开环反压：

拥塞节点广播抑制消息，上游节点根据本地策略 (如丢包或降速) 缓解拥塞。

支持“优先级路径” (Chosen Node)，确保关键数据优先传输。

CODA 开环逐跳背压机制



(a) 拥塞监测 (b) 背压信息传输 (c) 重路由

(a) 拥塞监测：发现问题

- 场景：数据从源节点向汇聚节点（Sink）传输，路径上的节点进入了“拥塞域”。
- 监测方式：节点通过两个指标判断拥塞：
 1. 缓冲区占用率接近溢出；
 2. 信道负载过高（通过采样计算得出）。
- 图中表现：虚线椭圆圈出的区域就是拥塞域，中间带叉的节点是拥塞节点，它的队列快满了、信道也被占满，无法再处理更多数据。

(b) 背压信息传输：通知上游节点

- 核心动作：拥塞节点通过广播发送 抑制消息（Suppression Message），把“我这里堵了”的信号反向传递给上游节点。
- 特点：这是开环、逐跳的，不需要等端到端 ACK，消息一跳一跳往回传，速度极快。
- 图中表现：黑色箭头反向指向源节点，代表背压信号沿着传输路径往上游节点传递，让它们知道下游已经拥塞了。

(c) 重路由：主动绕开拥塞域

- 上游节点的应对：收到背压信号的上游节点，会根据本地策略做出反应：
 1. 降低发送速率；
 2. 直接丢弃非关键数据；
 3. 重路由，选择绕开拥塞域的其他路径向 Sink 传输。
- 图中表现：原来的直线路径被绕开，数据改走了一条不经过拥塞域的新路径，直接缓解了局部拥塞，也避免了拥塞扩散。

一句话总结

这个机制的核心逻辑是：

(a) 本地快速检测到拥塞 → (b) 立刻反向广播背压信号 → (c) 上游节点主动降速或绕路，在拥塞扩散前就把它解决在局部，避免了传统端到端拥塞控制的高延迟和大开销。

另外注意：

一、先搞懂：CODA 里的“广播抑制消息”，为什么能精准反传给上游？

它确实是用广播发出去的，但不是“乱发”，而是有“方向”的：

1. 什么是广播？

无线广播就是：节点发一个信号，所有在它通信范围内的邻居节点，都能收到。

但不是所有收到的节点都会处理这个信号。

2. CODA 里的“广播抑制消息”是怎么定向的？

- 每个节点都知道自己的下一跳节点（往 Sink 方向的邻居）是谁。
- 当一个节点（下游）发“抑制消息”时，它的上游邻居节点（往源方向的邻居）会识别出：“哦，这是我下游节点发来的背压信号，说明我发过去的路堵了”。
- 其他无关的邻居节点，收到这个消息后会直接忽略，不会处理。

所以，它用“广播”的形式，实现了“只让上游节点感知到拥塞”的效果，不需要建立专门的单播连接，开销很低。

二、上下游 vs 上下行，怎么区分？

概念	定义	举个例子
上游 / 下游 (Up/Downstream)	描述数据传输路径上的节点位置关系	源节点 → 节点 A → 节点 B → Sink • 节点 A 是节点 B 的上游节点 （数据从 A 流向 B） • 节点 B 是节点 A 的下游节点
上行 / 下行 (Up/Downlink)	描述数据流向的整体方向	• 上行：传感器节点 → Sink（节点往基站发数据） • 下行：Sink → 传感器节点（基站往节点发命令 / 代码）

三、回到这张图里，具体怎么理解？

- 图里的“上游节点”：就是靠近源节点、离 Sink 更远的节点（比如 (a) 里路径上左边的那些节点）。
- 图里的“下游节点”：就是靠近 Sink、离源节点更远的节点（比如 (a) 里拥塞域里的那个带叉的节点）。

当下游节点（带叉的那个）拥塞时：

1. 它广播“抑制消息”；
2. 所有在它通信范围内的邻居节点都收到了；
3. 其中，它的上游节点（往源方向的邻居）识别出“这是我下游发来的背压信号”，于是：
 - 要么降低发送速率；
 - 要么换一条路走（像 (c) 那样绕开拥塞域）；
 - 其他无关的节点，收到了也不处理。

四、一句话帮你记住

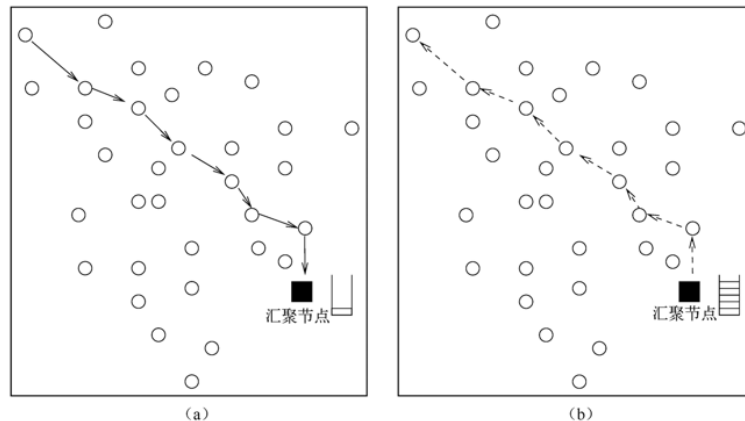
- 广播：形式上是发给所有邻居，但只有上游节点会响应。
- 上下游：是按数据流动方向，把“靠近源”叫上游，“靠近 Sink”叫下游。
- 上下行：是按整体流向，“节点→Sink”叫上行，“Sink→节点”叫下行。

(3) 闭环调节：

源节点通过阈值判断是否需闭环调节，依赖 sink 的 ACK 反馈动态调整速率。

避免全局广播，仅调节与拥塞相关的源节点。

CODA 闭环多信源管理机制



(a) 若传感器节点的速率超过阈值，则进入闭环控制

(b) 每接收到n个数据包，Sink回复一个ACK。

(a) 进入闭环控制：源节点触发条件

- 场景：多个传感器节点（信源）同时向汇聚节点（Sink）发送数据，导致全网出现持续拥塞。
- 触发条件：当源节点的发送速率超过设定阈值（ $r \geq \eta S_{max}$ ）时，就会从“开环”模式切换到“闭环控制”模式。
- 图中表现：节点持续向 Sink 发送数据，Sink 处的缓存开始堆积（图中右侧的缓存示意图），说明数据量已经超过了网络的承载能力，仅靠局部开环背压已经无法解决。

(b) Sink 回复 ACK：速率调节反馈

- 核心动作：Sink 不会对每个包都回复 ACK，而是采用“每收到 n 个数据包回复一个 ACK”的慢速反馈机制。
- ACK 的作用：这个 ACK 会携带网络的拥塞状态信息，反向传递给源节点，让它们根据反馈动态调整发送速率：
 - 如果收到 ACK，说明网络还能承载，可以保持或小幅提升速率；
 - 如果收不到 ACK，说明拥塞严重，需要降低发送速率。
- 图中表现：虚线箭头代表 ACK 沿着传输路径反向传递回源节点，形成了“数据发送 → 拥塞检测 → 反馈调节”的闭环。

一句话总结

当局部开环背压解决不了持续的多源拥塞时，就靠 Sink 的慢速 ACK 反馈，给源节点“踩刹车”，动态调节发送速率，从源头控制流量，避免全网拥塞恶化。

与现有方法的区别和联系：

对比维度	CODA	现有方法（如 ESRT、TCP）
拥塞检测	本地化（缓冲区 + 信道负载）	ESRT 依赖全局拥塞比特，TCP 依赖丢包
控制机制	开环反压 + 闭环调节结合	ESRT 仅全局闭环，TCP 纯端到端
能量效率	动态采样，低信令开销	ESRT 高能耗全局广播，TCP 频繁重传
适用场景	支持异构流量和优先级	ESRT 假设同质流量，TCP 不区分优先级

联系：CODA 借鉴了 ESRT 的拥塞反馈思想，但通过本地化反压和动态调节优化了能耗和响应速度。

解决方法存在的问题：

- （1）参数敏感性：如信道负载阈值 S_{max} 、采样参数等需手动配置，难以适应动态环境。
- （2）闭环调节延迟：依赖 sink 的 ACK 反馈，大规模网络中可能引入较高延迟。

Wireless Sensor Network

(3) 隐藏终端问题: CODA 未完全解决 CSMA 的隐藏终端问题, 可能在高密度网络中失效。

(4) 全局视野局限: 闭环调节依赖 sink 的全局视图, 分布式场景下可能不够灵活。

改进方法:

(1) 自适应参数调整

引入机器学习 (如强化学习) 动态优化采样参数和拥塞阈值。

(2) 跨层优化

与 MAC 层协同 (如 S-MAC 的睡眠调度), 减少冲突和能量浪费。

(3) 混合反馈机制

结合事件触发和周期反馈, 平衡闭环调节的实时性与能耗。

(4) 分布式拥塞协调

在中间节点引入轻量级协作算法 (如博弈论), 替代单一 sink 的全局控制。

(5) 优先级增强

扩展 “Chosen Node” 机制, 支持多级 QoS 区分 (如紧急事件优先)。

优点:

优势在于联合缓存与信道负载评估拥塞, 有效缓解本地与端到端拥塞, 通过避免拥塞提高网络性能。

缺点:

局限是未解决可靠性问题, 闭环多信源管理机制在网络数据流量高时引入额外时延, 影响实时性。

ESRT 协议

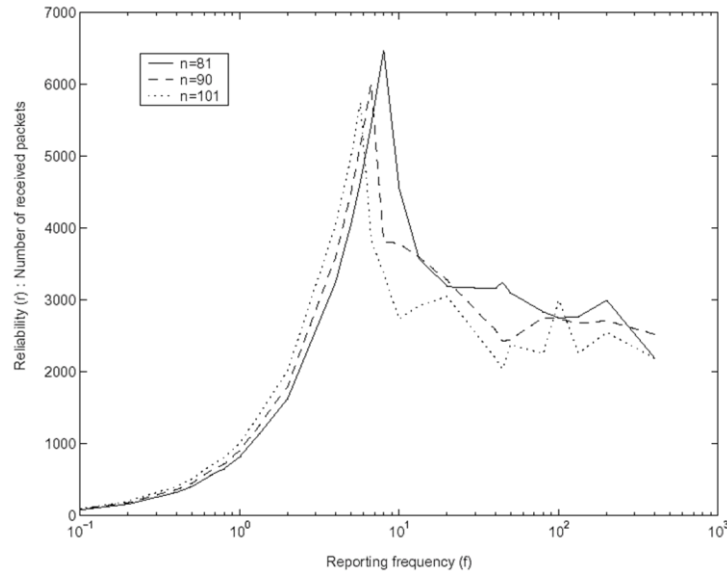
原理与机制:

(1) 事件到 Sink 可靠性概念

基于事件到 Sink 的可靠性概念, 关注综合信息而非单个节点信息, 引入事件 ID 与决策时间间隔 τ , 根据 τ 内收到的分组数判断事件检测可靠性。

(2) 动态调整与拥塞控制

Sink 计算归一化可靠性指标 η_i , 据此动态调整传感器节点报告率 f , 实现可靠性与节能平衡, 同时基于反馈报告进行拥塞决策, 高效控制网络拥塞。



1. 坐标轴含义

- X 轴 (Reporting frequency, f): 源节点每秒发送的数据包数量, 用对数刻度表示 ($10^{-1} \sim 10^3$), 越往右发送频率越高。
- Y 轴 (Reliability, r): Sink 最终收到的数据包总数, 代表事件传输的可靠性。
- 三条曲线: 分别对应不同网络规模的节点数 $n=81$ 、 $n=90$ 、 $n=101$ (节点越多, 网络负载越大)。

2. 曲线的关键阶段解读

整个曲线可以分成三个典型阶段, 对应 ESRT 协议的核心逻辑:

阶段①: 频率过低, 可靠性不足 ($f < \text{峰值点}$)

- 随着发送频率 f 增加, 收到的数据包数 r 快速线性上升。
- 原因: 此时网络没有拥塞, 发送的包几乎都能被 Sink 收到, 频率越高, 收到的包越多, 可靠性越好。
- 这一阶段的问题是: 频率太低, 即使所有包都送达, 总数量也达不到事件检测所需的“集体可靠性”阈值。

阶段②: 峰值点 (最优工作点)

- 曲线到达最高点, 此时收到的数据包数 r 达到最大。
- 这是 ESRT 要找的理想工作点: 发送频率刚好让网络承载饱和, 又没有发生严重拥塞, 传输效率和可靠性都达到最佳。

阶段③: 频率过高, 拥塞导致可靠性下降 ($f > \text{峰值点}$)

- 当频率超过临界值后, 曲线快速下降, 随后进入波动状态。
- 原因: 发送频率太高, 网络出现严重拥塞, 大量数据包在中间节点被丢弃, 导致 Sink 收到的包反而变少了。
- 可以看到, 节点数越多 ($n=101$), 曲线下降得越早、越明显, 说明大规模网络更容易

出现拥塞。

3. 结合 ESRT 协议的核心思想

ESRT 的核心就是动态调整源节点的报告频率，让系统稳定在这个峰值点附近：

- 频率太低 → 提高频率，增加数据包数量，提升可靠性；
- 频率太高 → 降低频率，缓解拥塞，减少数据包丢失；
- 最终目标是在“可靠性”和“拥塞 / 能耗”之间找到平衡点，既保证事件能被可靠检测，又不会浪费能量和带宽。

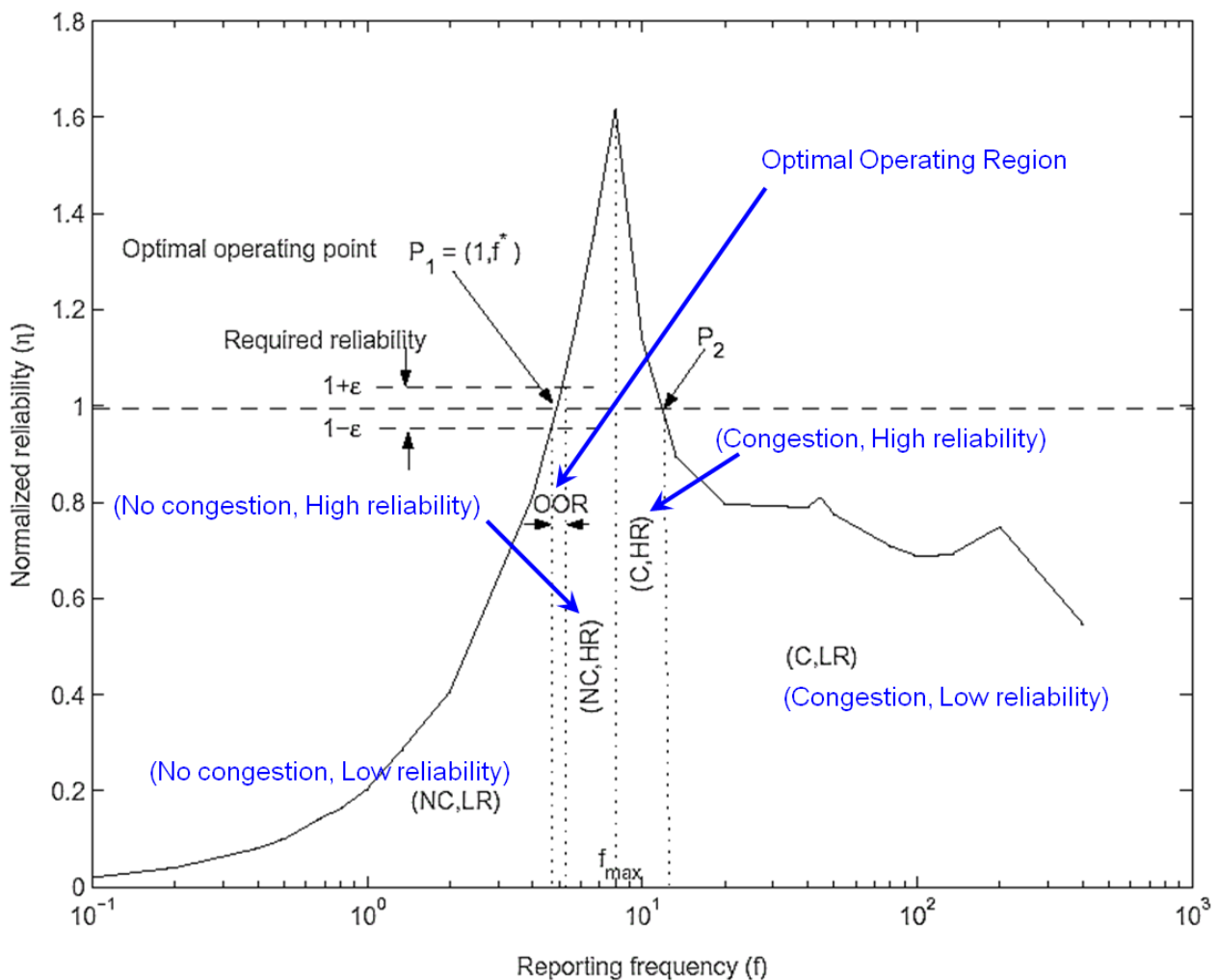
4. 补充：节点数的影响

三条曲线的差异也很关键：

- 节点越多 ($n=101$)，峰值点出现得越早，说明大规模网络的拥塞临界频率更低；
- 节点越多，峰值后的下降也越剧烈，说明高负载网络中，过度提高频率的“惩罚”更严重。

一句话总结

这张图直观地展示了：WSN 中“报告频率”和“可靠性”不是线性关系，过高的频率反而会因为拥塞降低可靠性，而 ESRT 协议的目标，就是把系统动态稳定在曲线的峰值附近。



1. 坐标轴与关键概念

- X 轴 (Reporting frequency, f): 源节点的报告频率，对数刻度，越往右频率越高。
- Y 轴 (Normalized reliability, η): 归一化的可靠性，即 Sink 实际收到的数据包数 /

事件检测所需的最低数据包数。

- $\eta=1$: 刚好满足事件检测的最低可靠性要求。
 - $\eta>1$: 可靠性超过需求。
 - $\eta<1$: 可靠性不足, 无法可靠检测事件。
 - $1\pm\epsilon$ 虚线: 围绕 $\eta=1$ 的误差容忍区间, ESRT 会把系统稳定在这个区间内。
-

2. 曲线与四个状态区域

曲线的形状和你上一张图是对应的, 只是 Y 轴换成了“归一化可靠性”。根据 η 和 f 的不同, ESRT 把系统分成了四个工作状态:

1. (NC, LR) 无拥塞、低可靠性 (No congestion, Low reliability)
 - 位置: 左下角, $\eta<1$, 且曲线处于上升段。
 - 含义: 网络没有拥塞, 但发送频率太低, 收到的包不够, 无法满足事件检测的可靠性要求。
 - 协议动作: 提高报告频率 f , 增加发送的数据包数量, 把 η 拉到 1 以上。
 2. (NC, HR) 无拥塞、高可靠性 (No congestion, High reliability)
 - 位置: $\eta>1$, 且在曲线峰值左侧 (还没到 $f_{\max}$)。
 - 含义: 频率足够高, 收到的包远超需求, 网络也没拥塞。
 - 协议动作: 略微降低报告频率 f , 在保证 $\eta\approx 1$ 的同时, 减少能量消耗。
 3. (C, HR) 拥塞、高可靠性 (Congestion, High reliability)
 - 位置: $\eta>1$, 但已经过了曲线峰值 ($f>f_{\max}$)。
 - 含义: 发送频率过高, 网络出现拥塞, 但收到的包仍然满足可靠性要求。
 - 协议动作: 快速降低报告频率 f , 缓解拥塞, 同时把 η 控制在 1 附近, 避免不必要的能耗。
 4. (C, LR) 拥塞、低可靠性 (Congestion, Low reliability)
 - 位置: 右下角, $\eta<1$, 且在曲线下降段。
 - 含义: 频率太高, 拥塞严重, 大量包丢失, 导致收到的包反而不够了。
 - 协议动作: 大幅降低报告频率 f , 先缓解拥塞, 让网络恢复正常, 再逐步提升频率。
-

3. 最优工作点与最优工作区域

- 最优工作点 (Optimal operating point) $P_1=(1, f^*)$: 理论上的理想点, $\eta=1$, 刚好满足可靠性, 同时 f 最低, 能耗最小。
 - 最优工作区域 (Optimal Operating Region, OOR): P_1 附近的 $1\pm\epsilon$ 区间, 这是 ESRT 实际要稳定的区域。系统只要在这个区间内, 就兼顾了可靠性和能耗效率。
 - $f_{\max}$: 曲线峰值对应的频率, 是网络承载能力的临界点, 超过它就会进入拥塞状态。
-

4. 一句话总结 ESRT 的控制逻辑

这张图就是 ESRT 协议的“状态机”:

根据当前的 (η, f) 状态, 判断系统在哪个区域, 然后按规则调整报告频率 f , 让系统最终稳定在 OOR 区间内, 既保证事件检测的可靠性, 又最小化能量消耗。

文献: Event-to-Sink Reliable Transport in Wireless Sensor Networks

总体描述文献内容:

本文提出了一种新型的 WSN 可靠传输协议——事件到汇聚节点可靠传输 (ESRT) 协议。传统的端到端可靠性模型在 WSN 中不适用, 因为 WSN 是事件驱动的系统, 汇聚节点关注的是事件的集体信息而非单个节点的报告。ESRT 协议通过动态调整源节点的报告频

率，实现了可靠事件检测和能量消耗的最小化，同时具备拥塞控制功能。论文还扩展了 ESRT 协议以支持多事件并发场景，并通过分析和仿真验证了其性能。

文献研究的问题：

如何在 WSN 中实现高效的事件到汇聚节点的可靠传输。具体包括：

- (1) 如何定义和实现事件到汇聚节点的集体可靠性，而非传统的端到端可靠性。
- (2) 如何在保证事件可靠检测的同时，最小化能量消耗。
- (3) 如何处理网络拥塞和多事件并发场景。

问题提出的基础：

问题的提出基于以下观察和背景：

- (1) WSN 是事件驱动的系统，汇聚节点依赖多个传感器节点的集体信息进行事件检测，单个节点的数据丢失不会显著影响整体可靠性。
- (2) 传统端到端可靠性模型（如 TCP）在 WSN 中不适用，因为其资源消耗大且复杂度高。
- (3) 现有 WSN 传输协议（如 PSFQ 和 RMST）未能解决事件到汇聚节点的集体可靠性问题，且未充分考虑拥塞控制和能量效率。

问题的解决方法：

论文提出的解决方法是 ESRT 协议，其核心思想包括：

- (1) 事件到汇聚节点可靠性模型：通过统计汇聚节点接收的数据包数量来衡量事件可靠性，而非单个数据包的端到端可靠性。
- (2) 动态调整报告频率：根据网络状态（拥塞和可靠性水平）动态调整源节点的报告频率，以平衡可靠性和能量消耗。
- (3) 拥塞控制：通过监测节点缓冲区水平检测拥塞，并通知汇聚节点调整报告频率。
- (4) 多事件支持：通过事件 ID 标识和优先级调度机制，处理多事件并发场景。

ESRT 协议的主要操作在汇聚节点完成，传感器节点仅需实现简单的功能（如调整报告频率和拥塞检测），从而降低了资源消耗。

解决方法和现有方法的区别和联系：

- (1) PSFQ: PSFQ 关注传感器到传感器的可靠传输，适用于反向控制和管理数据流，但未解决正向事件数据传输的可靠性和拥塞问题。ESRT 则专注于事件到汇聚节点的可靠传输。
- (2) RMST: RMST 基于定向扩散（Directed Diffusion）提供端到端可靠传输，但依赖于网络内缓存，资源开销大。ESRT 无需中间缓存，直接通过调整报告频率实现可靠性。
- (3) 传统 TCP: 传统 TCP 强调端到端可靠性，不适合 WSN 的集体信息需求和资源限制。ESRT 摒弃了 ACK/NACK 机制，通过集体信息统计和拥塞控制实现高效传输。

解决方法是否还存在问题：

- (1) 动态拓扑适应性：虽然 ESRT 能够适应动态拓扑变化，但在节点频繁失效或移动的场景下，其收敛速度和稳定性可能受到影响。
- (2) 事件 ID 冲突：在多事件场景中，动态随机分配的事件 ID 可能存在冲突风险，尽管论文认为冲突概率低，但在大规模网络中仍需进一步验证。
- (3) 拥塞检测延迟：基于缓冲区水平的拥塞检测可能存在延迟，尤其是在高负载网络中。
- (4) 多事件优先级调度：在多事件并发且共享路由节点的场景中，优先级调度可能导致部分事件的收敛时间延长。

改进的方法：

- (1) 增强动态拓扑适应性：引入机器学习或强化学习算法，动态调整报告频率和拥塞控制策略，以更快适应拓扑变化。
- (2) 优化事件 ID 分配：设计分布式事件 ID 分配协议，避免冲突并支持大规模网络。
- (3) 改进拥塞检测机制：结合流量预测和缓冲区水平监测，提前预测拥塞并调整报告频率。

(4) 多事件负载均衡：在多事件共享路由节点的场景中，引入负载均衡机制，优化资源分配和优先级调度。

(5) 跨层优化：与 MAC 层和路由协议协同设计，进一步提升能量效率和可靠性。

优点：

引入新可靠性概念与传输方案，拥塞控制与可靠性机制将决策转移给能源丰富的 Sink，对资源受限传感器节点要求低，实现可靠性和节能双重目标。

协议性能对比

①可靠性对比

RMST 提供逐跳与端到端可靠传输，但不能保证分组顺序到达；

PSFQ 可靠性高，适用于节点重编程；

CODA 未解决可靠性问题；

ESRT 基于事件到 Sink 可靠性，适用于事件驱动型网络。

②拥塞控制对比

RMST 无专门拥塞控制机制；

PSFQ 无拥塞控制；

CODA 专门针对拥塞检测与避免，效果显著；

ESRT 拥塞控制与可靠性机制结合，高效节能。

③资源消耗对比

RMST 缓存模式下中间节点需较大缓存空间；

PSFQ“pump”操作慢，浪费带宽；

CODA 闭环机制引入额外时延；

ESRT 对资源受限节点要求低，节能效果好。

第十二讲

时间同步

定义：

(1) 时钟准确与精确

时钟准确是指某一时刻，节点时钟和理想时钟相同；时钟精确是指节点时钟变化速率和理想时钟变化速率相同。时钟同步是指两个节点的时钟相同，但时钟同步与时间的准确度和精度没有必然联系。

例如，在一个 WSN 中，一个节点的时钟可能与理想时钟非常接近，但它的时钟变化速率可能与理想时钟不同，因此它可能不是精确的；而另一个节点的时钟可能与理想时钟有一定的偏差，但它的时钟变化速率与理想时钟相同，因此它是精确的。

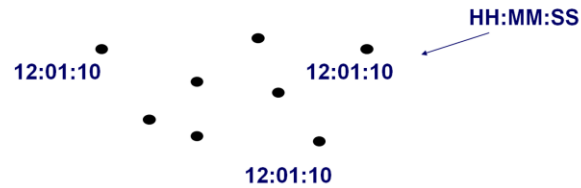
(2) 节点间时钟同步

对于大部分 WSN 应用来说，只需实现节点之间时钟同步，而不需要每个节点的时钟都与理想时钟完全一致。网络中节点之间的时间同步问题是使节点间的时钟同步，以满足应用的需求。

例如，在一个数据融合应用中，只要节点之间的时间同步精度能够满足数据融合的要求，就可以实现准确的数据融合，而不需要每个节点的时钟都与外部标准时间完全一致。

Time Synchronization

What is time synchronization in sensor networks?



Objective: Allow all sensor nodes to maintain the same time frame

目标：让所有传感器节点维持相同的时间基准（时间框架）

原因：

- WSN 通常是一个分布式系统，网络中的每个节点都有自己的本地时钟。由于不同节点的晶体振荡频率总会存在一定偏差，以及温度、湿度和电磁波对晶体振荡器的频率造成干扰，使得节点的本地时钟产生漂移。
例如，在一个用于环境监测的无线传感器网络中，如果节点的本地时钟偏差较大，可能会导致监测数据的时间戳不准确，从而影响对环境变化趋势的分析和判断。
- WSN 中应用需要节点间实现不同程度的时间同步，才能协作完成复杂的环境感知、信息采集或目标监控任务。

目标跟踪系统中的潜在时间同步需求（应用需求驱动、精度要求逐个降低）

- 通过波束阵列确定声源位置进行目标检测，波束阵列需要使用公共基准时间。需要局部节点瞬间时间同步，以确保波束阵列的准确性。
- 通过对目标相邻位置的连续监测，估计目标的运动速率和方向，也需要节点间的时间同步来保证数据的连续性和准确性。
- 使用数据融合减少网络通信量和提高目标跟踪精度。
- 用户与 WSN 进行交互。

特点：

（1）能量效率与同步精度

节点能量有限，因此时间同步机制必须尽量减少网络通信量，降低计算复杂度，以提高能量效率。例如，在一个大规模的无线传感器网络中，如果同步机制需要频繁地进行通信和复杂的计算，将会很快耗尽节点的能量，导致网络的寿命缩短。

同步精度与应用相关，不同的应用对时间同步的精度要求不同，一些应用可能要求同步精确到秒级甚至微秒级，而另一些应用只需要区分消息到达的时间先后即可。

（2）同步寿命与范围

时间同步的寿命可以是瞬时的、短期的或长期的，这取决于应用的需求和网络的特性。例如，在一些临时性的监测任务中，可能只需要瞬时的时间同步；而在长期的环境监测中，则需要长期的时间同步来保证数据的连续性。

同步范围可以是网络内的全体节点，也可以是部分节点，这取决于网络的拓扑结构和应用的需求。例如，在一个分区域的无线传感器网络中，可能只需要在每个区域内实现节点间的时间同步，而不需要在整个网络范围内进行同步。

挑战：

（1）性能参数与网络特性

WSN 时间同步机制的主要性能参数包括最大误差、同步期限、同步范围、可用性、效率、代价和体积等。这些参数之间往往存在矛盾和折衷，例如，提高同步精度可能会增加通信量和能量消耗，而降低同步范围可能会提高效率但会影响应用的性能。

WSN 网络规模大，节点数量多，节点部署密集，这给时间同步带来了很大的挑战。例如，在一个大规模的无线传感器网络中，节点之间的通信延迟和干扰可能会导致时间同步的

Wireless Sensor Network

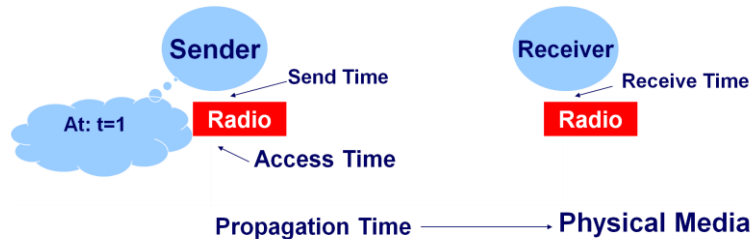
误差增大，而且节点的失效和拓扑结构的变化也会对时间同步产生影响。

(2) 节点状态与同步基准

节点成本低，易失效，这使得时间同步机制需要具有一定的容错能力和自适应能力。例如，当一个节点失效时，时间同步机制应该能够自动调整同步策略，以保证网络的正常运行。

为降低节点能量消耗，WSN 中协议常常使节点在大部分时间处于休眠状态，在这种状态下节点不能持续保持时间同步，唤醒后需要迅速与其它节点取得同步。此外，大规模网络中的同步基准时间的可靠传递也是一个关键问题，需要设计有效的同步基准传递机制来保证时间同步的准确性和可靠性。

影响同步关键因素：



1. 发送方 (Sender) 侧的延迟

(1) At: $t=1$

- 发送方的应用层在时刻 $t=1$ 生成数据，准备发送。

(2) Send Time (发送时间)

- 发送方的操作系统 / 协议栈处理数据的时间：包括数据封装、打包、交给射频模块的过程。
- 这个延迟取决于节点的 CPU 负载和协议栈实现。

(3) Access Time (接入时间)

- MAC 层等待信道空闲、竞争接入的时间（比如 CSMA/CA 的退避等待）。
- 这是无线场景下最不稳定的延迟来源之一，网络负载越高，接入时间波动越大。

(4) Radio (射频模块)

- 把数字信号转换成无线电磁波，开始向外发送的部分。

2. 传输过程中的延迟

Propagation Time (传播时间)

- 信号在物理介质 (Physical Media, 即无线信道) 中，从发送方天线传播到接收方天线的时间。
- 计算公式：传播时间 = 距离 / 光速，这个值本身是固定的，但在多跳网络中会累积。

3. 接收方 (Receiver) 侧的延迟

(1) Radio (射频模块)

- 接收方的天线收到信号，把电磁波转换成数字信号的部分。

(2) Receive Time (接收时间)

- 接收方的协议栈处理数据的时间：包括解包、校验、交给应用层的过程。
- 同样受节点 CPU 负载影响。

4. 核心公式：总延迟

一个数据包从发送方应用层到接收方应用层的总延迟，等于：

$$\text{总延迟} = \text{Send Time} + \text{Access Time} + \text{Propagation Time} + \text{Receive Time}$$

5. 为什么这张图对“时间同步”很重要？

时间同步的本质，就是通过消息交互，把这些延迟从测量结果中剔除，得到两个节点之间真

实的时钟偏差:

- *Send/Receive Time*: 受节点负载影响, 是不确定延迟;
- *Access Time*: 受信道竞争影响, 是最不确定的延迟;
- *Propagation Time*: 在单跳短距离场景下可以近似为常数, 或通过对称路径估算。

不同的时间同步协议 (如 RBS、TPSN), 就是通过不同的消息交互方式, 来尽量消除这些延迟的影响。

基本原理:

时钟模型

- WSN 中节点的本地时钟依靠对自身晶振终端计数实现, 晶振的频率误差和初始计时时刻不同, 使得节点之间的本地时钟不同步。
- 若能估算出本地时钟与物理时钟的关系或者本地时钟之间的关系, 就可以构造对应的逻辑时钟以达成同步。

The time of a computer clock is measured as a function of the hardware oscillator $C(t) = k \int_{t_0}^t \omega(\tau) d\tau + C(t_0)$ where $\omega(\tau)$ is the angular frequency of the oscillator, k is a constant for that oscillator, and t is the time. The change of the value $C(t)$ leads to the events (or interrupts) that can be captured by the sensor.

计算机时钟的时间是由硬件振荡器决定的, 其数学表达式为:

$$C(t) = k \int_{t_0}^t \omega(\tau) d\tau + C(t_0)$$

其中:

$\omega(\tau)$ 是振荡器的角频率,

k 是该振荡器的常数,

t 为真实时间。

时钟值 $C(t)$ 的变化会产生传感器可捕获的事件 (或中断)。

补充说明:

$C(t)$: 节点的本地时钟读数, 不是真实时间, 而是由振荡器计数得到的“虚拟时间”。

积分项 $\int_{t_0}^t \omega(\tau) d\tau$: 表示从 t_0 到 t 这段时间内, 振荡器角频率的累积。角频率决定了时钟的走速。

k : 一个与振荡器硬件相关的常数, 用来把角频率转换成时钟计数单位。

$C(t_0)$: 初始时刻 t_0 的时钟读数, 也就是初始时间偏移。

简单来说, 节点的本地时钟是靠振荡器不停振动来计数的, 而振荡器的频率会受温度、电压等影响产生漂移, 这就是节点之间时钟偏差的根本原因。

- 理想时钟, 时间变化速率应恒定为 1
- 实际由于环境因素变化和影响, 晶体振荡器的频率往往产生随机漂移
- 一般情况下, 漂移幅度并非任意, 局限在一定范围之内
- 时钟漂移, 时钟偏移, 相对漂移, 相对偏移

WSN 中节点的本地时钟依靠对自身晶振终端计数实现, 由于初始计时时刻不同和晶振的频率误差, 使得节点之间的本地时钟不同步。

时钟漂移是指时钟的速率或频率的变化, 而时钟偏移是指相对实际时间的差值。

例如, 一个节点的时钟漂移为每秒 1 微秒, 这意味着该节点的时钟每秒会比实际时间快 1 微秒; 而如果一个节点的时钟偏移为 100 微秒, 那么该节点的时钟比实际时间快 100 微秒。

相对漂移是指两个节点的时钟速率不同, 而相对偏移是指两个节点间相对实际时间之差。相对漂移和相对偏移是时间同步中需要考虑的重要因素, 通过测量和补偿这些参数, 可以实现节点之间的时间同步。

Wireless Sensor Network

例如，在一个 WSN 中，如果两个节点的相对漂移为每秒 2 微秒，那么随着时间的推移，这两个节点的时钟差异会逐渐增大；而如果两个节点的相对偏移为 50 微秒，那么在某一时刻，这两个节点的时钟差异为 50 微秒。

在初始时刻（或漂移还没来得及累积的短时间内），两个节点的时钟差异，就是等于它们的相对偏移（初始差异）。随着时间推移，这个差异会被漂移放大，变成“偏移 + 漂移 × 时间”，不再是单纯的初始值了。

we can approximate its local clock as:

$$C_i(t) = a_i t + b_i,$$

a_i is the *drift*, and $b_i(t)$ is the *offset* of node i 's clock. *Drift* denotes the difference in value from real time t .

we can compare the local clocks of two nodes in a network,

$$C_1(t) = a_{12} \cdot C_2(t) + b_{12}$$

我们可以将节点 i 的本地时钟近似表示为:

$$C_i(t) = a_i t + b_i \text{——让节点时钟与真实时钟对齐}$$

其中:

- a_i 代表节点 i 的时钟漂移 (*drift*)，表示节点时钟相对于真实时间的走速偏差；
- b_i 代表节点 i 的时钟偏移 (*offset*)，表示节点时钟与真实时间的初始差值；
- 漂移描述了节点时钟随时间偏离真实时间 t 的速率。

我们可以用下式描述网络中两个节点的本地时钟关系:

$$C_1(t) = a_{12} \cdot C_2(t) + b_{12} \text{——让两节点时钟对齐}$$

其中:

- a_{12} 是节点 1 与节点 2 之间的相对漂移（节点 1 的走速相对于节点 2 的比例系数）；
- b_{12} 是节点 1 与节点 2 之间的相对偏移（节点 1 与节点 2 的初始时间差）。

补充说明

- 这个线性模型是时间同步协议的基础假设，它认为节点时钟和真实时间、节点时钟之间都可以用“线性关系”来近似描述。
- 时间同步的目标，就是通过消息交互，估算出 a_i 、 b_i 或 a_{12} 、 b_{12} ，从而把不同节点的本地时钟统一到同一个时间基准上。

策略:

(1) 漂移补偿

漂移补偿是通过一定的算法估算出本地时钟与参考时钟之间的相对时钟漂移量，然后对所估算值进行补偿，使得同步时钟不再依赖本地时钟的变化速率，而是以接近参考时钟的速率变化，从而提高同步精度。

例如，可以使用卡尔曼滤波算法来估算时钟漂移量，并对其进行补偿。这种方法精度高，持续时间较长，但算法复杂度较大，需要消耗较多的计算资源和能量。

(2) 偏移补偿

偏移补偿是通过一定的算法估算出某一时刻两个节点之间的相对时钟偏移量，然后通过偏移补偿，实现该时刻的节点时钟同步。这种方法没有考虑两个节点间相对时钟漂移对同步时钟精度的影响，随着同步时间间隔扩大，同步误差会变大。

例如，可以使用简单的平均算法来估算时钟偏移量，并对其进行补偿。这种方法简单易行，但精度较低，适用于对同步精度要求不高的应用。

NTP 协议

NTP（网络时间协议）是一种传统的网络同步时间机制，采用 C/S（客户端/服务器）模式，时间服务器周期性地向客户端发送时间同步消息，同步消息包含服务器的当前时间。客户端产生时间同步请求，服务器回应，通过测量两个消息分组总的往返时间来估测单程的延迟，从而估算时标之间的时间间隔，获得时间同步。

例如，在一个企业网络中，可以设置一台时间服务器，为网络中的所有计算机提供时间同步服务。NTP 协议在 Internet 上广泛使用，精度高、鲁棒性好、易扩展，但假定网络链路失败概率小，网络结构相对稳定，便于为不同位置的节点手工配置时间服务器列表。

NTP 协议采用层次型树形结构，整个体系内多棵树，每棵树的根节点都是一个一级时间基准服务器，一级时间基准服务器与 UTC 时间源相连接。时间信息由一级时间服务器传输到分布式系统的二级时间服务器成员或客户端，依次向下，以此类推。

例如，在一个大型的分布式系统中，可以设置多个一级时间基准服务器，每个一级时间基准服务器连接多个二级时间服务器，每个二级时间服务器又连接多个客户端。成员的层次数越小，越接近一级服务器，时间越准确。

UTC 全称是 *Coordinated Universal Time*，协调世界时。它是全球统一使用的标准时间基准，比如我们平时说的北京时间，其实不是单独的一套时间标准，而是相对于 UTC 加了 8 小时：北京时间 = UTC + 8 所以：UTC 00:00 = 北京时间 08:00

为什么需要 UTC？

因为全世界各地有不同的时区，如果没有统一标准，计算机通信、卫星定位、金融交易、网络同步都会混乱。所以大家统一用 UTC 作为基础时间，再根据地区换算成本地时间。UTC 时间源不是指某一个普通服务器，而是指那些能够提供接近 UTC 标准时间的高精度设备或系统。

一句话总结：

UTC 是全球统一的标准时间，NTP 协议就是让网络中的设备尽量同步到这个标准时间。

基本机制:

TPSN 协议

核心目标:

- (1) 高精度同步: 实现全网节点间的微秒级时间同步。
- (2) 能量高效: 减少通信开销, 适应资源受限的传感器节点。
- (3) 分层结构: 通过层次化拓扑降低同步复杂度。

关键步骤:

TPSN 分为两个阶段: 层次发现阶段 (Level Discovery) 和同步阶段 (Synchronization)。

(1) 层次发现阶段

- 目的: 构建分层的网络拓扑结构 (类似树状结构), 根节点 (通常为基站或参考节点) 位于第 0 层。
- 过程:
 1. 根节点广播包含自身层级 (Level 0) 的报文。
 2. 接收到报文的邻居节点将自身层级设为 1, 并继续广播自己的层级。
 3. 重复此过程, 直到所有节点获得自己的层级 (避免环路)。
- 结果: 形成以根节点为顶点的层次树, 每个节点仅与更高或更低层级的节点同步。

(2) 同步阶段

- 双向消息交换: 基于经典的双向消息对机制 (类似 NTP), 通过计算传播延迟和时钟偏差实现同步。
- 同步过程 (以节点 A (层级 i) 与父节点 B (层级 i-1) 为例):
 1. A → B: 发送同步请求, 携带本地时间戳 T_1 。
 2. B 记录时间戳: 收到时刻 T_2 , 回复时刻 T_3 , 报文包含 T_1, T_2, T_3 。
 3. A 记录时间戳: 收到回复时刻 T_4 。
- 时钟偏差计算:
 - 假设传播延迟 d 对称 $d = \frac{(T_2 - T_1) + (T_4 - T_3)}{2}$ 。
 - 时钟偏差 $\Delta = T_2 - T_1 - d$ 。
 - 节点 A 将本地时钟调整 $+\Delta$ 以与 B 同步。
- 采用层次结构实现整个网络节点的时间同步, 所有节点按照层次结构进行逻辑分级, 分级表示节点到根节点的距离。通过基于发送者-接收者的节点对方式, 每个节点与上一级的一个节点进行同步, 从而使所有节点都与根节点时间同步。
- 采用 NTP 协议部分思想。
- 提供全网节点的时间同步。
- 网络中有一个可以获得外界时间的节点, 该节点为根节点, 可配 GPS。
- 层次性结构, 相邻层次间节点时间同步。
- 基于发送者-接收者同步机制。

(1) 层次发现阶段

- 根节点通过广播级别发现分组 (包含发送节点 ID 和级别), 启动层次发现阶段。
- 接收节点收到分组后, 将自身级别设置为分组中的级别加 1, 然后广播新的级别发现分组。
- 节点一旦建立自己的级别, 就忽略任何其他级别发现分组, 防止网络产生泛洪拥塞。

(2) 同步阶段

- 层次结构建立后, 根节点通过广播时间同步分组, 启动同步阶段。
- 下一级节点收到分组后, 各自分别等待一段随机时间, 通过与上一级节点交换消息进行同步。
- 同步按分层依次进行。

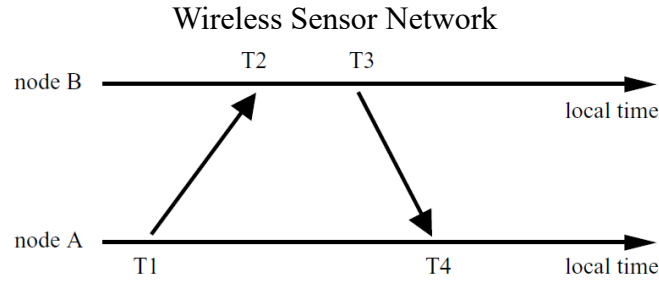


Figure 2: Two way message exchange between a pair of nodes.

A and B as shown in figure 2. Node A initiates the synchronization by sending a *synchronization_pulse* packet at T_1 (according to its local clock). This packet includes A's level number, and the value T_1 . B receives this packet (according to its local clock) at $T_2 = T_1 + \Delta + d$, where Δ is the relative clock drift between the nodes, and d is the propagation delay of the pulse. B responds at time T_3 with an acknowledgement packet, which includes the level number of B and the values T_1 , T_2 , and T_3 . Then, node A can calculate the clock drift and propagation delay as below, and synchronize itself to B.

$$\Delta = \frac{(T_2 - T_1) - (T_4 - T_3)}{2}; \quad d = \frac{(T_2 - T_1) + (T_4 - T_3)}{2}$$

如图所示的节点 A 和节点 B 之间的交互流程:

1. 节点 A 在自身本地时间 T_1 发送一个同步脉冲 (*synchronization_pulse*) 数据包, 发起同步过程。该数据包包含节点 A 的层级编号和时间戳 T_1 。
2. 节点 B 在自身本地时间 T_2 收到该数据包, 满足关系:

$$T_2 = T_1 + \Delta + d$$

其中, Δ 是两个节点间的相对时钟偏移, d 是脉冲信号的传播延迟。

3. 节点 B 在自身本地时间 T_3 回复一个确认 (*acknowledgement*) 数据包, 该包包含节点 B 的层级编号, 以及时间戳 T_1 、 T_2 和 T_3 。
4. 节点 A 收到确认后, 就可以根据下面的公式计算出相对时钟偏移 Δ 和传播延迟 d , 完成与节点 B 的时间同步。

$$\Delta = \frac{(T_2 - T_1) - (T_4 - T_3)}{2}; \quad d = \frac{(T_2 - T_1) + (T_4 - T_3)}{2}$$

- Δ : 两个节点之间的相对时钟偏移 (Offset), 也就是节点 A 和节点 B 的时间差。
- d : 信号在两个节点之间的传播延迟 (Propagation Delay), 也就是信号从 A 到 B 再返回 A 的单程传输时间。

一句话总结:

这是 TPSN 协议中经典的双向时间同步 (Two-way message exchange) 流程, 通过一次“发请求 + 回确认”的消息交互, 节点 A 可以算出它和节点 B 之间的时钟偏移和传播延迟, 从而校准自己的时钟, 实现同步。

- 协议 MAC 层消息开始发送到无线信道时才给同步消息加上时标, 消除访问时间带来的时间同步误差。

问题背景: 之前我们讲过, 一个数据包从发送方应用层到接收方, 总延迟包括: *Send Time* + *Access Time* + *Propagation Time* + *Receive Time*。其中 *Access Time* (MAC 层等待信道的时间) 是最大的不确定误差来源, 因为节点要等信道空闲、退避竞争, 这个时间是随机波动的。

解决办法: 把“打时间戳”的动作, 从应用层 / 网络层, 下沉到 MAC 层, 在射频模块真正开始往信道上发送数据的那一刻, 才打上时标。

效果: 这样就把 *Send Time* 和 *Access Time* 都排除在时间戳之外了, 时间戳直接记录的是信号开始传输的时间, 只留下 *Propagation Time* 和 *Receive Time* 的影响, 大幅消除了不确定误差。

- 利用双向消息交换计算消息的平均延迟, 提高时间同步的精度。

原理: 就是之前的 $A \leftrightarrow B$ 的双向握手流程。

Wireless Sensor Network

它的巧妙之处在于：通过“发送→回复”两次交互，我们可以用公式把传播延迟 d 和时钟偏移 Δ 都算出来，并且两者互相抵消，消除了单向传输延迟的不对称影响。

结果：得到的延迟是“往返延迟的平均值”，精度远高于单次测量。

挑战：

(1) 节点失效

- 传感器节点可能因为没电、故障、环境干扰突然下线。
- 影响：如果同步树的中间节点 / 父节点失效，依赖它的子节点就会失去同步基准，导致整个网络的同步拓扑断裂。
- 应对：协议需要支持重新同步或拓扑重建，让子节点能快速找到新的父节点，恢复同步。

(2) 根节点移动

- 很多 WSN 场景中，作为同步基准的根节点（比如汇聚节点 Sink）可能是移动的。
- 影响：根节点一旦移动，它和子节点之间的传播延迟 d 会发生变化，原来的同步参数失效，甚至导致同步树结构失效。
- 应对：协议需要支持动态更新同步路径，或让节点能快速重新测量与根节点的延迟和偏移。

(3) 新节点加入

- 网络运行过程中，可能会有新的传感器节点加入。
- 影响：新节点没有任何同步信息，需要快速和现有网络建立时间同步。
- 应对：协议需要提供初始化同步流程，让新节点能快速找到父节点，完成首次同步，融入网络。

优点：

- (1) 高精度：通过双向消息交换抵消传播延迟，精度可达微秒级。
- (2) 能量效率：分层结构减少同步跳数，避免全网泛洪。
- (3) 可扩展性：适合多跳网络，层级限制同步范围。

缺点：

- (1) 依赖拓扑稳定性：网络动态变化（如节点移动）需重新构建层级。
- (2) 根节点依赖：根节点故障会导致同步失效（需冗余设计）。
- (3) 时钟漂移未补偿：未考虑长期运行中的时钟漂移问题（需周期性同步）。

与其他协议的对比：

协议	TPSN	RBS（参考广播同步）
同步方式	双向消息交换（层级）	接收者 - 接收者同步（广播）
精度	高（ μs 级）	中高（依赖广播接收间隔）
能耗	中（分层控制）	低（无需层级维护）
适用场景	静态多跳网络	局部密集网络

RBS 协议

在 WSN 中，参考广播同步（Reference Broadcast Synchronization, RBS）是一种经典的时间同步协议，与传统的双向同步协议（如 TPSN）不同，RBS 采用接收者 - 接收者同步机制，利用无线广播的物理特性消除发送方的时间不确定性，显著提高了同步精度。

核心思想：

- 关键创新：

传统协议（如 NTP、TPSN）通过发送方 - 接收方双向同步，但发送方的 MAC 层延迟（如载波监听、退避时间）会引入误差。RBS 则利用无线广播的单跳局部性，让多个接收节点通过同一广播消息彼此同步，从而消除发送方的时间不确定性。

- 核心假设：

同一广播域内的节点接收同一消息的传播延迟差异极小（主要误差来自硬件时钟偏差）。

工作流程：

参考节点广播->接受节点交换时间戳->偏差补偿

一个参考节点（无需时间同步）广播一条空消息（不携带时间戳）广播域内的所有接收节点（如 A、B、C）记录各自的本地接收时间戳 T_A, T_B, T_C 。接收节点通过多跳通信或本地广播交换各自的时间戳。例如，节点 A 和 B 通过比较 T_A, T_B ，计算彼此时钟偏差 $\Delta_{AB} = T_A - T_B$ 。节点根据偏差调整本地时钟（如节点 B 将时钟增加 Δ_{AB} 以对齐节点 A）。

- 基于接收者-接收者的时间同步。
- 一个节点广播发送时间参考分组，广播域内的两个节点分别采用本地时钟记录参考分组的到达时间，然后交换记录时间来确定它们之间的时间偏移量，实现时间同步。
- 参考广播同步。
- 利用无线数据链路层的广播信道特性。
- 一个节点发送广播消息，接收到广播消息的一组节点比较各自接收到消息的本地时刻。
- 通过广播消息使得对所有接收节点而言，消息的发送时间和访问时间相同。

Wireless Sensor Network

1. 先回顾一下无线通信的延迟组成

我们之前说过，一个数据包从发送方到接收方，总延迟分为 4 个部分：

$$\text{总延迟} = \underbrace{\text{Send Time} + \text{Access Time}}_{\text{发送方的不确定延迟}} + \underbrace{\text{Propagation Time} + \text{Receive Time}}_{\text{接收方的延迟}}$$

- **Send Time**：发送方协议栈处理数据的时间（比如封装、交给射频模块）
- **Access Time**：MAC 层等待信道空闲、退避竞争的时间（最不确定的部分）
- **Propagation Time**：信号在空气中传播的时间（光速传播，距离很近时几乎一样）
- **Receive Time**：接收方协议栈处理数据的时间

2. RBS 是怎么做到“发送时间和访问时间相同”的？

关键在于：RBS 的同步不是“发送者→接收者”，而是“接收者→接收者”。

举个具体场景：

节点 S（发送者）给节点 A 和节点 B 发了一条广播消息。

- 节点 S 的 Send Time 和 Access Time，对 A 和 B 来说是完全一样的。
因为消息是同一次发送出来的，发送方的处理和信道竞争过程只发生一次，所以这两个延迟是共享的。
- 当 A 和 B 收到消息时，它们各自记录下自己的本地时间： T_A 和 T_B 。

完整公式（包含接收方处理延迟）：

$$T_A = T_S + \text{Send Time} + \text{Access Time} + d_A + R_A \quad (1)$$

$$T_B = T_S + \text{Send Time} + \text{Access Time} + d_B + R_B \quad (1)$$

其中：

- d_A 和 d_B 是信号从 S 到 A、S 到 B 的传播延迟；
- R_A 和 R_B 是节点 A、B 从 NIC 收到信号到记录时间戳的处理延迟。

把两个式子相减：

$$T_A - T_B = (d_A - d_B) + (R_A - R_B)$$

你看！**Send Time** 和 **Access Time** 直接被抵消掉了！

3. 这句话的真正含义

“通过广播消息使得对所有接收节点而言，消息的发送时间和访问时间相同”，本质是说：

对于同一条广播消息，所有接收节点都共享了完全相同的发送方延迟（Send Time + Access Time），因此
在计算它们之间的时间差时，这部分误差会被自动消除，不会影响同步精度。

这就是 RBS 最巧妙的地方：它直接把发送方的所有不确定延迟都“消掉”了，不用再像 TPSN 那样，通过双向交互来抵消它们。

而剩下的误差，只有两部分：

- 传播延迟差 $d_A - d_B$ （传感器网络中通常只有纳秒级，可忽略）；
- 接收方处理延迟差 $R_A - R_B$ （在 NIC 层打时间戳时已经被压缩到很小，是 RBS 的主要误差源）。

Wireless Sensor Network

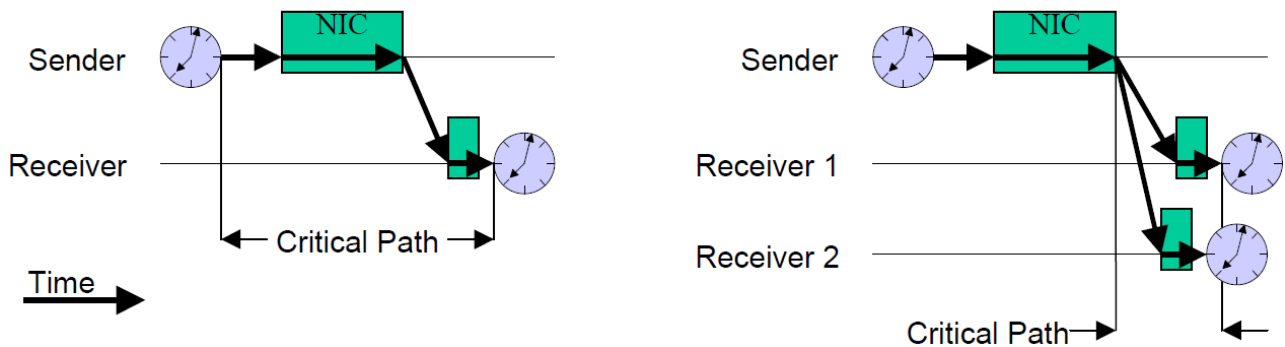


Figure 1: Comparison of traditional synchronization systems to RBS. [5]

1. 图的基本元素说明

- *Sender*: 发送节点
- *Receiver/Receiver 1/2*: 接收节点
- *NIC* (Network Interface Card): 网络接口卡, 代表无线射频模块
- *Clock* (紫色圆形): 节点的本地时钟
- 箭头: 数据 / 时间戳的传递方向
- *Critical Path*: 影响同步误差的“关键路径”, 路径越长、环节越多, 误差越大
- *Time* 轴: 时间向右推进

2. 左侧: 传统时间同步 (如 TPSN、NTP)

传统同步是发送者→接收者的直接同步:

1. *Sender* 节点在本地时钟打时间戳, 把时间戳交给 *NIC* 发送出去。
 2. 数据经过 *NIC* 发送、无线信道传播, 到达 *Receiver* 节点。
 3. *Receiver* 节点收到数据后, 把时间戳交给自己的本地时钟, 完成同步。
- *Critical Path* (关键路径): 从 *Sender* 的时钟, 一直到 *Receiver* 的时钟, 包含了:
 - 发送方处理时间
 - 访问 / 信道竞争时间
 - 传播延迟
 - 接收方处理时间
 - 这些环节的不确定性 (比如访问时间的随机抖动), 都会直接叠加到同步误差里, 所以误差源多、路径长, 精度受限。

3. 右侧: RBS 参考广播同步

RBS 是接收者→接收者的间接同步, 完全避开了发送方的不确定延迟:

1. *Sender* 节点只负责广播一条参考消息, 不在发送方打时间戳。
 2. 广播消息同时到达 *Receiver 1* 和 *Receiver 2*。
 3. 两个接收节点各自在本地 *NIC* 收到消息的瞬间, 记录自己的本地时间戳。
 4. 之后两个接收节点互相交换记录的时间戳, 通过计算差值来同步。
- *Critical Path* (关键路径): 只从 “*NIC* 收到消息的时刻” 开始, 到接收节点的时钟为止。
 - 发送方的所有延迟 ($\text{Send Time} + \text{Access Time}$) 对两个接收节点是完全相同的, 在计算差值时会被抵消。
 - 路径中只剩下: 传播延迟的微小差异 + 接收方处理延迟。
 - 关键路径大大缩短, 误差源被大幅消除, 这就是 RBS 能实现高精度同步的核心原因。
 - 影响协议性能的主要因素包括: 接收节点间的时钟偏差、接收节点非确定性因素和接

收节点的个数

- 为提高时间同步精度，采用统计技术，通过发送节点发送多个消息，获得接收节点之间时间差异的平均值。

单次广播测量的时间差里，混进了很多随机噪声，导致结果不准。

解决办法就是：

- 发送节点广播多次参考消息；
- 接收节点每次都记录下接收时间，算出每次的时间差；
- 最后把这些时间差取平均值。

这样一来，随机的正负噪声会互相抵消，平均值更接近真实的时钟偏移，精度自然就提高了。

- 采用最小平方的线性回归方法进行线性拟合，直线上的点为节点间的时间差异，斜率为两个节点的时钟偏差。

这个方法，是用来解决“时钟走速不一样”的问题的：

- 随着时间推移，两个节点的时间差会越来越大，不是固定值；
- 多次广播后，你会得到一组“时间差 vs 时间”的数据点；
- 用线性回归拟合出一条直线： $\Delta T = a \times t + b$
 - 截距 b ：初始的时钟偏移（静态差值）
 - 斜率 a ：两个节点的时钟偏差 / 漂移（走速差）

这样不仅能算出“当前的时间差”，还能知道“时间差变大的速度”，可以长期补偿漂移，让同步维持更久、精度更高。

多跳网络扩展：

- 层级 RBS：

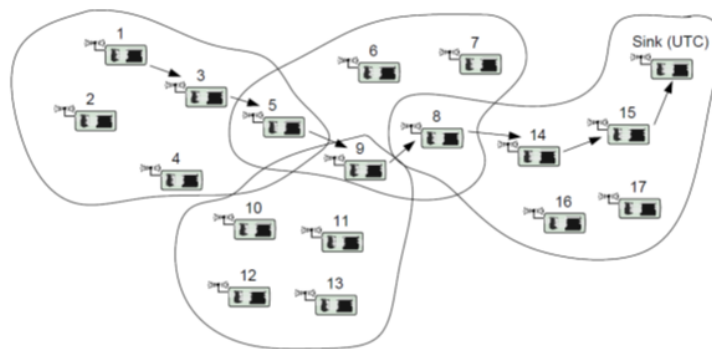
将网络分簇，每簇内使用 RBS 同步，簇头节点负责跨簇同步（类似 TPSN 的层次结构）。

- 全局同步：

通过网关节点传递参考时间，但会累积误差。

RBS 应用于多跳网络

RBS – Network Synchronization



一、图中关键元素说明

- 带数字的节点：无线传感器节点，数字是节点 ID。
- Sink (UTC)：网络的汇聚节点，提供标准时间（如 UTC），是全网的时间基准。
- 不规则椭圆圈：代表一个广播域（同步域），圈内的节点能直接收到同一节点的广播消息。
- 箭头：同步关系，表示“子节点→父节点”的同步方向，最终所有节点都会通过多跳链路同步到 Sink 节点。

二、RBS 在多跳网络中的工作逻辑

RBS 本身是针对单跳广播域设计的（所有节点都能直接收到同一个广播消息），但传感器网络大多是多跳的，所以需要扩展为“多跳 RBS”，流程如下：

1. 第一步：每个广播域内的局部同步

在同一个椭圆圈（广播域）里，选一个节点作为广播参考节点：

- 广播节点发送参考消息，域内所有节点记录接收时间，通过 RBS 的接收者 - 接收者同步，算出彼此的时钟偏移，实现域内同步。

2. 第二步：跨域的“桥接同步”

不同广播域之间，通过公共节点（桥节点）来传递同步信息：

- 比如节点 5，既属于左上方的广播域，也和节点 9 所在的广播域重叠；
- 节点 5 先和左上方域内的节点完成同步，再作为“参考节点”，和节点 9 所在域的节点同步；
- 节点 9 再继续把同步信息传递给节点 8，以此类推，像接力一样把同步从左到右传递。

3. 第三步：最终同步到 Sink 节点

整个网络的同步方向，是沿着箭头从外围节点一步步向 Sink（UTC）汇聚：

- 节点 $1 \rightarrow 3 \rightarrow 5 \rightarrow 9 \rightarrow 8 \rightarrow 14 \rightarrow 15 \rightarrow \text{Sink}$ ；
- 每个节点都和上一跳的节点同步，最终所有节点都间接同步到 Sink 的标准时间。

三、多跳 RBS 的特点与局限

优势

- 保留了 RBS 在单跳内的高精度（消除发送方延迟误差）；
- 无需构建像 TPSN 那样严格的层级树，适配多跳网络拓扑。

局限

- 误差累积：每经过一跳，同步误差都会被放大（比如节点 1 到 Sink 要经过多跳，每跳的误差会叠加）；
- 同步域管理复杂：需要划分广播域、维护桥节点，拓扑变化时容易同步失效。

一句话总结：

这张图展示了多跳 RBS 的“接力同步”逻辑：每个广播域内用 RBS 实现高精度局部同步，再通过公共节点把同步信息从一个域传到下一个域，最终让全网节点都同步到 Sink 的标准时间。

整个过程描述：

图中带箭头的过程表示 RBS 在多跳网络中建立全网时间同步的一条同步链路，其路径大致为 $1 \rightarrow 3 \rightarrow 5 \rightarrow 9 \rightarrow 8 \rightarrow 14 \rightarrow 15 \rightarrow \text{Sink}$ （UTC）。RBS 的核心不是由发送节点给接收节点直接授时，而是由某个节点发送参考广播，处在同一广播范围内的多个接收节点同时接收该广播并记录各自的本地接收时刻，然后交换记录，计算彼此的时钟偏差。因此，箭头并不是普通意义上的数据转发路径，而是表示相邻节点或相邻同步区域之间已经建立了“时间换算关系”。在图中，节点 1 先通过左侧局部区域与节点 3 建立时钟偏差关系，节点 3 再与节点 5 建立关系，节点 5 作为重叠区域中的桥接节点把左侧同步簇连接到中心节点 9；之后节点 9 继续通过节点 8、14、15 把时间关系传递到靠近 Sink 的区域，最后节点 15 与具有 UTC 标准时间的 Sink 同步。这样，远离 Sink 的节点虽然不能直接接收 Sink 的时间信号，也可以通过逐段累加各跳之间的时钟偏差，把自己的本地时间换算成 UTC 时间。图中的灰色闭合区域表示局部 RBS 同步范围，重叠区域中的节点负责连接不同同步簇；没有画在箭头主链上的节点也可以先在本地图同步簇内同步，再通过这些桥接节点接入全网。该过程体现了 RBS 从单跳同步扩展到多跳同步的思想：局部接收者同步、重叠节点桥接、多跳偏差累加，最终

实现全网节点相对于 Sink/UTC 的时间同步。

关键技术：

已解决的主要误差：

- (1) 发送方 MAC 层延迟（被 RBS 完全消除）。
- (2) 接收方中断处理延迟、时钟抖动（通过多次广播取平均降低）。

时间戳位置：

RBS 建议在 MAC 层记录广播消息的实际到达时间（而非应用层），以减少软件栈延迟。

时钟偏差与漂移补偿：

线性回归模型

节点通过多次广播消息的时间戳对 (T_A, T_B) ，拟合时钟偏差和漂移率 $(T_B = \alpha \cdot T_A + \beta)$ ，实现长期同步。

优点：

- (1) 高精度
 - 实验显示单跳同步误差可低至 1~10 μ s（远优于 NTP 的毫秒级）。
 - 消除发送方延迟是关键（占传统协议 90% 以上误差）。
- (2) 能量高效
 - 无需频繁双向通信，广播消息可被多个接收节点复用。
- (3) 鲁棒性
 - 不依赖参考节点的时钟精度（参考节点仅触发同步，无需同步自身）。

缺点：

- (1) 单跳限制
 - 原生 RBS 仅适用于单跳广播域，多跳需结合其他协议（如 TPSN）。
- (2) 通信开销
 - 接收节点需交换时间戳，可能引发局部拥塞。
- (3) 动态网络适应性差：
 - 节点移动或拓扑变化需重新建立同步关系。

1. 先纠正一个关键点：“原生 RBS” 本身只支持单跳

- 标准 RBS 协议的设计初衷，就是解决单跳广播域内多个节点之间的同步问题。它的核心机制（接收者 - 接收者比对），要求所有节点都能直接收到同一个参考节点的广播，否则就无法抵消发送方延迟。
- 所以原文说“原生 RBS 仅适用于单跳广播域”，这句话是完全正确的。

2. 那么多跳网络里的 RBS 是怎么回事？

你之前看到的多跳场景，并不是“原生 RBS 直接跑多跳”，而是 RBS 的扩展方案，常见的有两种：

方案 1：层级 RBS（分簇 + 桥接）

- 把网络分成多个单跳广播簇，每个簇内用原生 RBS 同步；
- 选一个簇头节点，它同时属于两个相邻簇；
- 簇头在簇内同步完成后，再作为参考节点，和下一个簇做 RBS 同步，像桥一样把时间关系传递下去。
- 本质上：每一跳还是单跳 RBS，只是通过簇头接力实现了多跳效果，不是原生协议直接支持的。

方案 2：RBS + TPSN 混合

- 先按 TPSN 的方式构建层级树；
- 树的同一层节点之间，用 RBS 同步；

Wireless Sensor Network

- 再通过 TPSN 的层级结构，把时间关系逐层向上传递。
 - 这也是原文提到的“多跳需结合其他协议（如 TPSN）”的意思。
-

3. 为什么原生 RBS 不能直接跑多跳？

如果强行在多跳里用原生 RBS，会出现两个致命问题：

1. 发送方延迟无法抵消：中间节点转发广播时，会引入新的发送 / 访问延迟，这些延迟对不同接收节点不再是共享的，会直接变成误差；
 2. 误差会跨跳累积：每一跳的同步误差都会被带到下一跳，多跳后误差会越来越大，失去 RBS 的高精度优势。
-

一句话总结

- 原文说的“原生 RBS 仅适用于单跳”是对的；
- 你看到的多跳 RBS，都是扩展 / 混合方案，不是原生协议的直接能力；
- 扩展方案本质上是“单跳 RBS + 接力 / 层级结构”，而不是原生协议本身支持多跳。

TINY/MINI-SYNC 协议

在 WSN 中, Tiny/Mini-Sync 是一种轻量级时间同步协议, 专为资源受限的传感器节点设计。它是经典协议 TPSN 的简化版本, 通过优化同步流程和减少计算开销, 实现了较低能耗下的有效同步。

核心目标:

- 低复杂度: 适应传感器节点的有限计算能力 (如 8 位 MCU)。
- 能量高效: 最小化通信和计算开销。
- 快速收敛: 减少同步所需的时间和数据交换次数。

工作原理:

(一) 基本同步机制

采用双向消息交换 (类似 TPSN), 但通过以下简化提升效率:

(1) 层级发现 (可选):

- 若网络为多跳, 先构建层次结构 (根节点为 0 层, 其余节点逐层递增)。
- 单跳网络可跳过此步骤。

(2) 同步阶段:

- 节点与父节点 (或参考节点) 交换单个双向消息对 (而非多次交换):
 - 请求: 子节点发送同步请求, 记录发送时间 T_1 。
 - 响应: 父节点记录接收时间 T_2 和响应时间 T_3 , 并回复 T_2, T_3 。
 - 完成: 子节点记录响应到达时间 T_4 。
- 时钟偏差计算:
 - 假设传播延迟对称 $d = \frac{(T_2 - T_1) + (T_4 - T_3)}{2}$ 。
 - 时钟偏差 $\Delta = T_2 - T_1 - d$, 子节点调整时钟 $+\Delta$ 。

(二) 轻量化设计

- 单次同步: 不依赖多次平均, 牺牲少量精度以降低能耗。
- 无漂移补偿: 忽略长期时钟漂移, 适合短周期同步。
- 时间戳简化: 仅记录关键时间点 $T_1 \sim T_4$, 减少存储需求。

注意: 一次双向消息交换只能产生一个数据点。

- 简单轻量同步
- 假设节点的时钟漂移遵循线性变化, 那么两个节点之间的时间偏移也是线性的
- 通过交换时间分组来估计两个节点间的最优匹配偏移量

approximated by an oscillator with fixed frequency. As argued in section 2, two clocks, $C_1(t)$ and $C_2(t)$, can be linearly related under this assumption as:

$$C_1(t) = a_{12} \cdot C_2(t) + b_{12}, \quad (3)$$

where a_{12} is the relative drift, and b_{12} is the relative offset between the two clocks.

Both algorithms use the conventional two-way messaging scheme to estimate the relative drift and offset between the clocks of two nodes; node 1 sends a probe message to node 2, timestamped with t_o , the local time just before the message is sent. Node 2 generates a timestamp when it gets the message at t_b , and immediately sends back a reply message. Finally, node 1 generates a timestamp t_r when it gets this reply message. Using the absolute order between these timestamps and equation (3), the following inequalities can be obtained:

$$t_o < a_{12} \cdot t_b + b_{12} \quad (4)$$

$$t_r > a_{12} \cdot t_b + b_{12} \quad (5)$$

The 3-tuple of the timestamps (t_o, t_b, t_r) is called a “data point”. Tiny-sync and mini-sync works with some set of data points, each collected by a two-way message exchange as explained. As the

节点时钟可被视为一个固定频率的振荡器来近似建模。正如第 2 节所述，在此假设下，两个时钟 $C_1(t)$ 和 $C_2(t)$ 之间存在线性关系：

$$C_1(t) = a_{12} \cdot C_2(t) + b_{12} \quad (3)$$

其中， a_{12} 为两时钟之间的相对漂移率， b_{12} 为两时钟之间的相对偏移量。

两种算法均采用传统的双向消息交互机制来估计两节点时钟之间的相对漂移率与偏移量：

1. 节点 1 向节点 2 发送一条探测消息，在发送前以本地时间打上时间戳 t_o ；
2. 节点 2 收到消息时，以本地时间生成时间戳 t_b ，并立即回复一条响应消息；
3. 节点 1 收到回复消息时，生成时间戳 t_r 。

利用这三个时间戳的先后顺序，结合式 (3)，可推得以下不等式：

$$t_o < a_{12} \cdot t_b + b_{12} \quad (4)$$

$$t_r > a_{12} \cdot t_b + b_{12} \quad (5)$$

三元时间戳组 (t_o, t_b, t_r) 被称为一个“数据点”。Tiny-sync 和 mini-sync 算法就是基于一组这样的数据点工作的，每个数据点都通过上述一次双向消息交互采集得到。随着（数据点数量的增加）

number of data points increases, the precision of the algorithms increases. Each data point corresponds to two constraints on the relative drift and relative offset (equations 4, 5). The constraints imposed by data points are depicted in figure 3. Note that the line corresponding to equation (3) must lie between the vertical intervals created by each data point. One of the dashed lines in figure 3 represent the steepest possible such line, satisfying equation (3). This line gives the upper bound for the relative drift (slope of the line, $\overline{a_{12}}$), and the lower bound for the relative offset (y-intercept of the line, $\underline{b_{12}}$) between the two clocks. Similarly, the other dashed line gives the lower bound for relative drift ($\underline{a_{12}}$) and the upper bound for relative offset ($\overline{b_{12}}$). Then the relative drift a_{12} and the relative offset b_{12} can be bounded as:

$$\begin{aligned} \underline{a_{12}} &\leq a_{12} \leq \overline{a_{12}} \\ \underline{b_{12}} &\leq b_{12} \leq \overline{b_{12}}. \end{aligned}$$

... 数据点数量的增加，算法的精度也随之提高。每个数据点都会对相对漂移率和相对偏移量施加两个约束条件（式 4、式 5）。这些数据点施加的约束关系如图 3 所示。

注意，式 (3) 对应的直线（即两时钟的线性关系）必须位于每个数据点所定义的垂直区间内。图 3 中的一条虚线代表了满足式 (3) 的**最陡峭直线**，它给出了相对漂移率的上界（直线斜率 $\overline{a_{12}}$ ），以及相对偏移量的下界（直线的 y 轴截距 $\underline{b_{12}}$ ）。

同理，另一条虚线给出了相对漂移率的下界（ $\underline{a_{12}}$ ）和相对偏移量的上界（ $\overline{b_{12}}$ ）。

因此，相对漂移率 a_{12} 和相对偏移量 b_{12} 可以被界定在以下区间内：

$$\underline{a_{12}} \leq a_{12} \leq \overline{a_{12}}$$

$$\underline{b_{12}} \leq b_{12} \leq \overline{b_{12}}$$

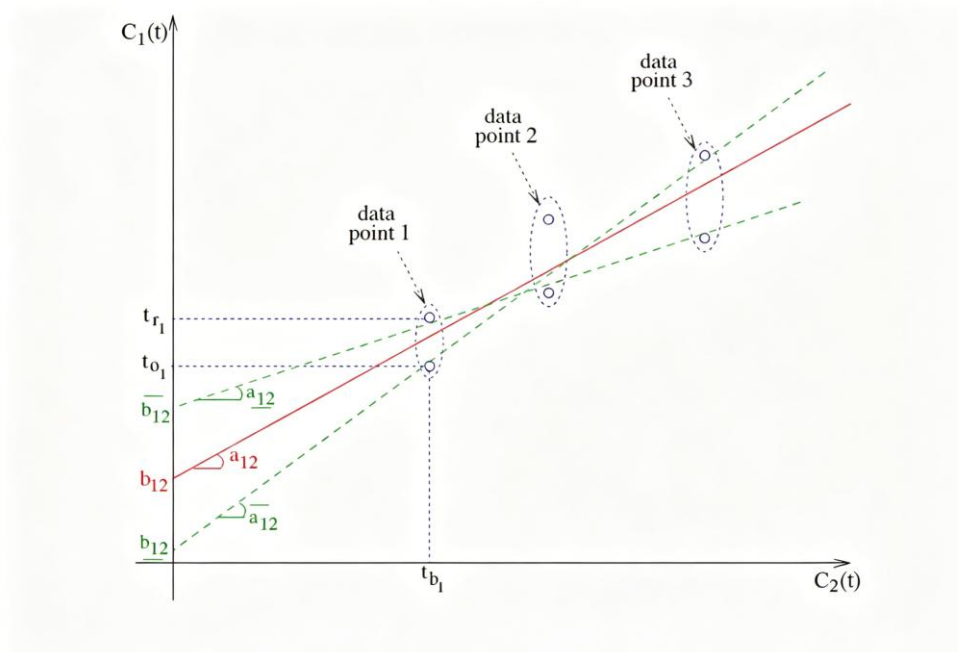


Figure 3: The constraints imposed on a_{12} and b_{12} by data points. [8]

Wireless Sensor Network

这张图展示的是 Tiny-sync / mini-sync 这类时间同步算法中，如何用多个双向报文交换得到的数据点来约束两个节点之间的时钟漂移和时钟偏移。

横轴是节点 2 的本地时钟 $C_2(t)$ ，纵轴是节点 1 的本地时钟 $C_1(t)$ 。假设两个节点的时钟在一段时间内近似线性关系：

$$C_1(t) = a_{12}C_2(t) + b_{12}$$

其中， a_{12} 是两个时钟的相对漂移，也就是图中直线的斜率； b_{12} 是两个时钟的相对偏移，也就是直线在纵轴上的截距。

图中的每一个 data point 来自一次双向消息交换。过程是：节点 1 在本地时间 t_o 发送探测消息给节点 2，节点 2 在自己的本地时间 t_b 收到消息并立即回复，节点 1 在本地时间 t_r 收到回复。由于节点 2 收到消息的真实时刻一定发生在节点 1 发送之后、收到回复之前，所以有：

$$t_o < a_{12}t_b + b_{12} < t_r$$

也就是说，在横坐标 t_b 处，真正的直线值必须落在纵坐标 t_o 到 t_r 之间。

所以图中每个蓝色虚线椭圆/竖直区间表示一个数据点带来的约束：
合法的时钟关系直线必须穿过每个数据点对应的竖直区间。

红色实线表示一种可能的时钟关系：

$$C_1(t) = a_{12}C_2(t) + b_{12}$$

它的斜率是 a_{12} ，截距是 b_{12} 。只要这条线在每个数据点的横坐标位置都落入对应的竖直区间，就说明它满足目前所有观测数据。

绿色虚线表示在所有约束条件下的极限情况：

- 一条绿色虚线是最陡的可行直线，它给出了相对漂移 a_{12} 的上界，同时给出了相对偏移 b_{12} 的下界；
- 另一条绿色虚线是最平缓的可行直线，它给出了相对漂移 a_{12} 的下界，同时给出了相对偏移 b_{12} 的上界。

因此可以得到：

$$\underline{a_{12}} \leq a_{12} \leq \overline{a_{12}}$$

$$\underline{b_{12}} \leq b_{12} \leq \overline{b_{12}}$$

这张图的核心意思是：

一次双向报文交换不能精确确定两个节点的时钟关系，只能给出一个约束区间；随着 data point 增多，满足所有约束的直线范围越来越小，于是相对漂移 a_{12} 和相对偏移 b_{12} 的上下界越来越紧，时间同步精度也就越来越高。

一句话总结：

这张图用几何方式说明：每个数据点都限制了时钟关系直线必须穿过某个竖直区间，Tiny-sync / mini-sync 通过多个数据点不断收紧可行直线范围，从而估计两个节点之间的时钟漂移和时钟偏移。

绿色线是由所有数据点形成的约束共同决定的可行边界线：最陡的可行直线给出漂移上界，最平的可行直线给出漂移下界；两条线之间的范围就是当前数据能够确定的时钟关系范围。

关键优化与特性：

（1）层级动态调整

- 若父节点失效，子节点自动选择邻居中层级最小的节点重新同步。

Wireless Sensor Network

- 适应网络拓扑变化，但需额外信令。

(2) 误差控制

- 主要误差源：
 - 传播延迟不对称（假设对称引入误差）。
 - MAC 层访问延迟（通过低占空比通信降低冲突概率）。
- 精度：实验显示同步误差在 100 微秒~1 毫秒（比 TPSN 略低，但满足多数应用）。

优点：

- (1) 低开销：单次消息交换即可同步，通信量比 TPSN 减少 50% 以上。
- (2) 快速部署：无需复杂初始化，适合动态网络。
- (3) 兼容性：可与 TPSN 共用层次结构。

缺点：

- (1) 精度较低：未补偿时钟漂移，长期运行需频繁同步。
- (2) 依赖拓扑稳定性：父节点故障影响子节点同步。
- (3) 单跳为主：多跳同步误差累积较快。

与其他协议对比：

特性	Tiny/Mini-Sync	TPSN	RBS
同步方式	双向单次交换	双向多次交换	接收者 - 接收者广播
能耗	极低	中	低（但需时间戳交换）
精度	中（100 微秒~1 毫秒）	高（微秒级）	单跳极高（微秒级）
适用场景	资源受限的单跳 / 小规模网络	静态多跳网络	局部密集网络

第十三讲（定位）

必要性：

- 许多应用要求节点知道自身位置，才能向用户提供有用的检测服务。
- 节点位置信息可用于目标定位、目标跟踪和目标轨迹预测，实现对网络覆盖区域内其他目标的定位和追踪。
- 网络节点随机部署时，节点位置不可控。
- 人工测量或设置以获取节点位置不实际。

定义：

- 根据节点位置是否已知，将网络中的节点划分为信标（beacon）节点和未知节点。
- 信标节点可获得自身精确位置，是未知节点定位的参考。
- 未知节点根据信标节点的位置，通过定位技术确定自身或网络中其它节点的绝对位置或相对位置的过程。

特点：

- 节能性：节点能量有限，要求定位算法能够尽量降低计算复杂度和通信量，减少能耗，延长网络寿命。
- 自组织性：节点通常随机部署，难以依靠全局的基础设施协助定位。
- 分布式特性：网络中节点数量较多，不适合由单个节点集中完成定位任务，要求多个节点能够分布式计算自己或其他节点的位置，减少单个节点的计算负载。
- 鲁棒性：节点通信、处理和存储能力有限，测距有一定误差，定位算法应有较强容错性，能够容忍一定的测距误差。
- 可扩展性：网络中节点数量成千上万，定位算法应有较强的可扩展性，以使用不同规模的网络。

节点定位性能指标：

- 定位精度：定位算法提供的位置信息的精确程度。分为绝对精度和相对精度。
- 绝对精度：以长度为单位表示定位误差。

Wireless Sensor Network

相对精度：以定位误差与节点之间距离的比例来表示。

■ 定位范围：一种定位系统或技术能够有效工作的距离的范围。

一般：范围小，精度高；范围大，精度低。

■ 定位速度：定位系统提供位置信息的速度，决定了定位系统提供服务的实时性，影响定位系统刷新位置信息的频率。

■ 节点密度：节点密度越大，定位精度越高，但网络成本提高。

■ 能耗：不同定位算法的复杂度和计算及通信开销不同。

■ 代价：定位算法的配置，所需的基础设施等。

挑战：

■ 由于节点在成本、体积和能量等方面的限制，难以全部配备 GPS 接收器等精确的测距设备，而必须通过网络中节点之间的相互测距和信息交换来进行定位，增加了高效、高精度定位的难度。

■ 由于节点限制，定位算法必须能够尽量降低计算复杂度和由此引起的通信开销，也增加了高效、高精度定位的难度。

计算节点位置的基本方法：

①三边测量法

WSN 中的节点定位是确定网络中未知节点位置的关键技术，三边测量法（Trilateration）是一种基于几何原理的经典定位方法。

基本原理：

三边测量法通过测量未知节点到至少三个已知位置节点（称为锚节点或参考节点）的距离，利用几何原理计算未知节点的坐标。其核心思想是：以每个锚节点为中心、测量距离为半径画圆，多个圆的交点即为未知节点的位置。

关键步骤：

（1）距离测量

首先需要获取未知节点到各锚节点的距离，常用测距技术包括：

- RSSI（接收信号强度指示）：通过信号衰减模型估算距离，但易受环境干扰。
- TOA（到达时间）：基于信号传播时间，要求高精度时钟同步。
- TDOA（到达时间差）：利用不同速度信号（如声波和无线电）的时间差，降低同步要求。
- 超声波 / 超宽带（UWB）：高精度但成本较高。

（2）建立数学模型

三个距离方程构成方程组。

（3）方程组求解

通过线性化方法（如最小二乘法）求解非线性方程组。具体步骤：

1. 将方程组两两相减，消去二次项，转化为线性方程。
2. 用矩阵表示线性方程组： $AX=b$ 。
3. 解为： $X = (A^T A)^{-1} A^T b$ （最小二乘解，适用于超定方程组）。

你这个疑问完全合理：在只有三个参考节点、方程恰好是两个、未知数也是两个的情况下，直接用上面的 $X = A^{-1}b$ 就很好，没必要硬扯最小二乘。

关键是看矩阵 A 的形状。

只有三个参考节点时

三个圆方程相减后得到两个一次方程：

$$AX = b$$

此时：

$$A \in \mathbb{R}^{2 \times 2}, X = \begin{bmatrix} x \\ y \end{bmatrix}$$

只要三个参考节点不共线， A 可逆，直接求：

$$X = A^{-1}b$$

最简单、最直接。

而图中的公式：

$$X = (A^T A)^{-1} A^T b$$

在 A 是可逆方阵时，其实和上面的结果完全一样，因为：

$$(A^T A)^{-1} A^T = A^{-1} (A^T)^{-1} A^T = A^{-1}$$

所以：

$$(A^T A)^{-1} A^T b = A^{-1} b$$

它没有得到一个新的答案，只是把简单问题写成了最小二乘的通用形式。

什么时候才真正需要最小二乘？

当参考节点超过三个时，比如有五个参考节点，相减后可能得到四个方程：

$$A_{4 \times 2} X_{2 \times 1} = b_{4 \times 1}$$

此时 A 不是方阵，根本不存在 A^{-1} 。而且测距有误差，四个方程通常不能同时严格成立，于是只能找一个让总误差最小的坐标：

$$\min_X \|AX - b\|^2$$

- 因为 A 不是方阵且方程组无解，我们只能最小化误差平方和，写出目标函数 $\min \|AX - b\|^2$
- 对这个目标函数求导并令导数为 0，得到正规方程 $A^T A X = A^T b$
- 因为 $A^T A$ 是可逆方阵，两边同时左乘它的逆，就得到了 $X = (A^T A)^{-1} A^T b$

这时才得到：

$$X = (A^T A)^{-1} A^T b$$

所以可以这样记：

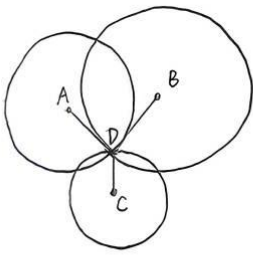
三个参考节点、 A 可逆：直接用 $X = A^{-1}b$ 。参考节点多于三个、方程多于未知数：才使用最小二乘 $X = (A^T A)^{-1} A^T b$ 。

因此你说“用上面的不是好得很吗”是对的。课件写下面那个，主要是为了给出一个能推广到多个参考节点的统一公式，并不是说三节点情况下必须用最小二乘。

（4）误差处理

实际中因测距误差，三个圆可能不交于一点，需采用优化算法（如最小二乘法）拟合最优解。

已知 A, B, C 三个节点的坐标分别是 (x_1, y_1) , (x_2, y_2) , (x_3, y_3) , 以及它们到未知节点 D 的距离 d_1, d_2, d_3
 设节点 D 的坐标是 (x, y) , 则



$$\begin{cases} (x-x_1)^2 + (y-y_1)^2 = r_1^2 & \textcircled{1} \\ (x-x_2)^2 + (y-y_2)^2 = r_2^2 & \textcircled{2} \\ (x-x_3)^2 + (y-y_3)^2 = r_3^2 & \textcircled{3} \end{cases}$$

$$\begin{aligned} \textcircled{1} - \textcircled{3} : & 2(x_1-x_3)x + 2(y_1-y_3)y = x_1^2 - x_3^2 + y_1^2 - y_3^2 - r_1^2 + r_3^2 \\ \textcircled{2} - \textcircled{3} : & 2(x_2-x_3)x + 2(y_2-y_3)y = x_2^2 - x_3^2 + y_2^2 - y_3^2 - r_2^2 + r_3^2 \end{aligned}$$

$$\begin{bmatrix} 2(x_1-x_3) & 2(y_1-y_3) \\ 2(x_2-x_3) & 2(y_2-y_3) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_1^2 - x_3^2 + y_1^2 - y_3^2 - r_1^2 + r_3^2 \\ x_2^2 - x_3^2 + y_2^2 - y_3^2 - r_2^2 + r_3^2 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2(x_1-x_3) & 2(y_1-y_3) \\ 2(x_2-x_3) & 2(y_2-y_3) \end{bmatrix}^{-1} \begin{bmatrix} x_1^2 - x_3^2 + y_1^2 - y_3^2 - r_1^2 + r_3^2 \\ x_2^2 - x_3^2 + y_2^2 - y_3^2 - r_2^2 + r_3^2 \end{bmatrix}$$

优点:

- (1) 原理简单: 易于理解和实现。
- (2) 精度较高: 在测距准确且锚节点布局合理时, 定位精度较好。

缺点:

- (1) 依赖测距精度: RSSI 易受多径效应、遮挡干扰; TOA/TDOA 需硬件支持。
 - (2) 锚节点密度要求: 至少需要三个锚节点 (二维) 或四个 (三维)。
- 未知数个数: n , 要得到唯一解, 至少需要 $(n+1)$ 个锚节点。
- (3) 非视距 (NLOS) 误差: 障碍物导致测距偏差, 需误差补偿算法。

1. 先搞懂什么是 LOS 和 NLOS

- LOS (视距): 未知节点和锚节点之间没有障碍物, 信号是“直线传播”的, 测得的距离 $\approx$ 真实距离
- NLOS (非视距): 两者之间有墙、家具、人等障碍物, 信号无法直线传播, 只能反射、绕射后到达

2. NLOS 会带来什么问题?

信号绕路了, 传播路径变长, 导致测得的距离比真实距离大很多。

比如真实距离是 5 米, 但因为信号绕了一下, 测得的距离变成了 8 米。

这种“系统性的偏大误差”, 会直接导致:

- 三个圆无法正常相交
- 用这些误差很大的距离算出来的位置, 会严重偏离真实节点的位置

3. 为什么需要误差补偿算法?

因为 NLOS 误差是环境导致的、不可避免的, 我们只能通过算法来降低它的影响, 比如:

- 识别出哪些锚节点的测距结果明显偏大, 判断为 NLOS 节点, 降低它们在定位计算中的权重
- 用卡尔曼滤波、加权最小二乘法等, 给 LOS 节点的距离更高权重, 给 NLOS 节点更低权重, 来抵消偏差

②三角测量法

先后逻辑顺序:

先定锚点: 画出 A、B、C, 这是一切的基础。

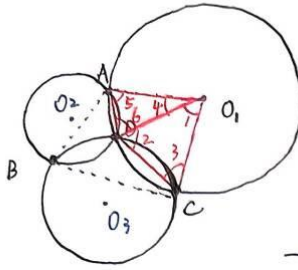
Wireless Sensor Network

画第一个圆 (A、C、 $\angle ADC$): 把 D 限制在一条曲线上 (圆)。

画第二个圆 (A、B、 $\angle ADB$): 把范围缩小到两个点 (两圆交点)。

画第三个圆 (B、C、 $\angle BDC$): 锁定唯一的交点, 也就是 D。

已知 A、B、C 三个节点的坐标分别是 $(x_1, y_1), (x_2, y_2), (x_3, y_3)$, 节点 D 相对于节点 A、B、C 的角度分别是 $\angle ADB, \angle ADC, \angle BDC$, 设节点 D 的坐标是 (x, y)



对于节点 A、C 及 $\angle ADC$, 若 $\widehat{AC}$ 在 $\triangle ABC$ 内, 则可以唯一确定圆, 设圆心 $O_1(x_0, y_0)$, 半径 r

方法一

$$\therefore \angle 2 + \angle 6 = \angle ADC$$

$$\text{又} \because \angle 5 = \angle 6, \angle 2 = \angle 3$$

$$\therefore \angle 5 + \angle 3 = \angle ADC$$

$$\therefore \angle AOC = \angle 1 + \angle 4 = 2\pi - \angle 5 - \angle 6 - \angle 2 - \angle 3 = 2\pi - 2\angle ADC$$

方法二

$$\therefore \text{劣弧 AC 对应圆心角 } \angle AOC$$

$$\therefore \text{优弧 AC 对应圆心角 } 2\pi - \angle AOC$$

$\therefore$ 圆 O_1 是 $\triangle ADC$ 的外接圆, 由圆周角定理:

$$\angle ADC = \frac{1}{2}(2\pi - \angle AOC)$$

$$\therefore \angle AOC = 2\pi - 2\angle ADC$$

对 O_1 :

$$\begin{cases} (x_0 - x_1)^2 + (y_0 - y_1)^2 = r_1^2 \\ (x_0 - x_3)^2 + (y_0 - y_3)^2 = r_1^2 \end{cases}$$

$$(x_1 - x_3)^2 + (y_1 - y_3)^2 = r_1^2 + r_1^2 - 2r_1 \cdot r_1 \cdot \cos \angle AOC$$

先解出 r_1 , 再解出 (x_0, y_0)

$$O_1 \text{ 方程: } (x - x_0)^2 + (y - y_0)^2 = r_1^2 \quad ①$$

$$O_2 \text{ 方程: } (x - x_0)^2 + (y - y_0)^2 = r_2^2 \quad ②$$

$$① - ②:$$

$$2(x_0 - x_0)x + 2(y_0 - y_0)y = r_1^2 - r_2^2 + x_0^2 - x_0^2 + y_0^2 - y_0^2$$

与 ① 或 ② 联立, 解得两圆交点 (可能的 2 个点)

代入

$$O_3 \text{ 方程: } (x - x_0)^2 + (y - y_0)^2 = r_3^2$$

看哪个点满足, 说明该点同时在 3 个圆上。

它就是待求的 $D(x, y)$

对 O_2 :

同样有 $\angle AO_2B = 2\pi - 2\angle ADB$

$$\begin{cases} (x_0 - x_1)^2 + (y_0 - y_1)^2 = r_2^2 \\ (x_0 - x_2)^2 + (y_0 - y_2)^2 = r_2^2 \end{cases}$$

$$(x_1 - x_2)^2 + (y_1 - y_2)^2 = r_2^2 + r_2^2 - 2r_2 \cdot r_2 \cdot \cos \angle AO_2B$$

$$(x_1 - x_2)^2 + (y_1 - y_2)^2 = r_2^2 + r_2^2 - 2r_2 \cdot r_2 \cdot \cos \angle AO_2B$$

先解出 r_2 , 再解出 (x_0, y_0)

对 O_3 :

同样有 $\angle BO_3C = 2\pi - 2\angle BDC$

$$\begin{cases} (x_0 - x_2)^2 + (y_0 - y_2)^2 = r_3^2 \\ (x_0 - x_3)^2 + (y_0 - y_3)^2 = r_3^2 \end{cases}$$

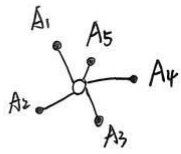
$$(x_2 - x_3)^2 + (y_2 - y_3)^2 = r_3^2 + r_3^2 - 2r_3 \cdot r_3 \cdot \cos \angle BO_3C$$

$$(x_2 - x_3)^2 + (y_2 - y_3)^2 = r_3^2 + r_3^2 - 2r_3 \cdot r_3 \cdot \cos \angle BO_3C$$

先解出 r_3 , 再解出 (x_0, y_0)

③极大似然估计法

已知 $A_1, A_2, A_3, \dots, A_n$ 等 n 个节点的坐标分别是 $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_n, y_n)$, 它们到节点 D 的距离分别是 $r_1, r_2, r_3, \dots, r_n$, 设 $D(x, y)$, 则



$$\begin{cases} (x-x_1)^2 + (y-y_1)^2 = r_1^2 \\ (x-x_2)^2 + (y-y_2)^2 = r_2^2 \\ (x-x_3)^2 + (y-y_3)^2 = r_3^2 \\ \vdots \\ (x-x_n)^2 + (y-y_n)^2 = r_n^2 \end{cases}$$

从第1个方程开始, 分别减去最后1个方程, 得:

$$\begin{cases} 2(x_n-x_1)x + 2(y_n-y_1)y = r_1^2 - r_n^2 - x_1^2 + x_n^2 - y_1^2 + y_n^2 \\ \vdots \\ 2(x_n-x_{n-1})x + 2(y_n-y_{n-1})y = r_{n-1}^2 - r_n^2 - x_{n-1}^2 + x_n^2 - y_{n-1}^2 + y_n^2 \end{cases}$$

线性方程表示为 $AX=b$

$$A = \begin{bmatrix} 2(x_n-x_1) & 2(y_n-y_1) \\ 2(x_n-x_2) & 2(y_n-y_2) \\ \vdots & \vdots \\ 2(x_n-x_{n-1}) & 2(y_n-y_{n-1}) \end{bmatrix}, X = \begin{bmatrix} x \\ y \end{bmatrix}, b = \begin{bmatrix} r_1^2 - r_n^2 - x_1^2 + x_n^2 - y_1^2 + y_n^2 \\ r_2^2 - r_n^2 - x_2^2 + x_n^2 - y_2^2 + y_n^2 \\ \vdots \\ r_{n-1}^2 - r_n^2 - x_{n-1}^2 + x_n^2 - y_{n-1}^2 + y_n^2 \end{bmatrix}$$

利用标准的最小均方误差估计法, 可求得 D 的坐标:

$$X = \begin{bmatrix} x \\ y \end{bmatrix} = (A^T A)^{-1} A^T b$$

2. 求二阶矩阵逆的步骤

假设:

$$A = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$$

第一步, 算行列式:

$$ad - bc = 2 \times 4 - 1 \times 3 = 8 - 3 = 5$$

第二步, 交换主对角线元素:

$$\begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix} \Rightarrow \begin{bmatrix} 4 & 1 \\ 3 & 2 \end{bmatrix}$$

第三步, 副对角线变号:

$$\begin{bmatrix} 4 & -1 \\ -3 & 2 \end{bmatrix}$$

第四步, 除以行列式:

$$A^{-1} = \frac{1}{5} \begin{bmatrix} 4 & -1 \\ -3 & 2 \end{bmatrix}$$

所以:

$$A^{-1} = \begin{bmatrix} \frac{4}{5} & -\frac{1}{5} \\ -\frac{3}{5} & \frac{2}{5} \end{bmatrix}$$

定位算法分类:

(1) 基于测距的定位算法和无需测距的定位算法

- 根据定位过程中是否测量实际节点间的距离进行区分。
- 基于测距的算法需要测量相邻节点间的绝对距离或方位, 并利用节点间的实际距离来计算未知节点位置。

Wireless Sensor Network

- 无需测距的算法利用节点间的估计距离计算节点位置。

(2) 递增式的定位算法和并发式的定位算法

- 根据节点定位的先后次序不同进行划分。
- 递增式的算法通常从信标节点开始，信标节点附近的节点首先开始定位，依次向外延伸，各节点逐次进行定位。定位过程中累积和传播测量误差。
- 并发式的算法中所有节点同时进行位置计算。

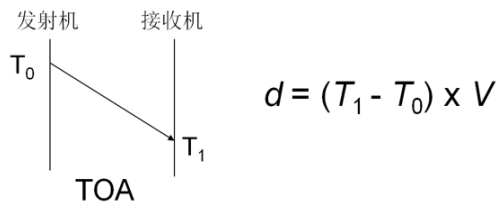
(3) 基于信标节点的定位算法和无线信标节点的定位算法

- 根据定位过程中是否使用信标节点进行划分。
- 基于信标节点的算法在定位过程中，以信标节点作为定位中的参考点，各节点定位后产生绝对坐标系。
- 无线信标节点的算法只关心节点间的相对位置，各节点先以自身为参考点，将邻近节点纳入自己的坐标系中，相邻的坐标系依次转换合并，最后产生整体相对坐标系。

基于测距的定位：

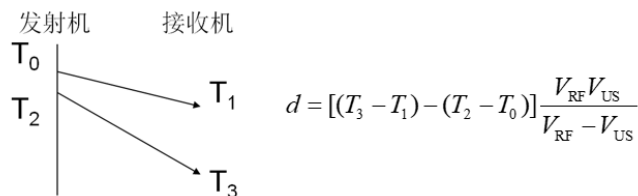
(一) 基于到达时间(TOA)的测距方法

- 使用发射机到接收机间的单程信号传播时间来计算收发机之间的距离。
- 要求收发机严格同步。



(二) 基于到达时间差(TDOA)的测距方法

- 发射节点同时发射两种不同传播速度的无线信号。
- 使用两种信号到达的时间差和不同的传播速度来计算收发机之间的距离。
- 要求节点能感知不同信号，一般使用超声波和射频。



发射机同时发送2种信号 { 射频信号(RF) - 快
超声波信号(US) - 慢

$$RF \text{ 传播时间} = T_1 - T_0 = \frac{d}{V_{RF}}$$

$$US \text{ 传播时间} = T_3 - T_2 = \frac{d}{V_{US}}$$

$$\begin{aligned} (T_3 - T_2) - (T_1 - T_0) &= \frac{d}{V_{US}} - \frac{d}{V_{RF}} = \left(\frac{1}{V_{US}} - \frac{1}{V_{RF}} \right) d \\ d &= \frac{(T_3 - T_2) - (T_1 - T_0)}{\frac{1}{V_{US}} - \frac{1}{V_{RF}}} \\ &= \frac{(T_3 - T_2) - (T_1 - T_0)}{V_{RF} - V_{US}} \cdot V_{US} V_{RF} \end{aligned}$$

AHLos

AHLoS (Ad-Hoc Localization System) 算法是一种经典的分布式节点定位方法，主要用于资源受限的 WSN 环境。该算法通过多跳协作和迭代优化，使未知节点利用锚节点（即信标节点，已知位置）和邻居节点的信息逐步估算自身位置。

核心思想：

通过多阶段协作定位，将定位问题分解为多个局部几何计算步骤，逐步优化未知节点的位置估计。其关键特点包括：

- (1) 分布式计算：无需全局信息，节点仅依赖局部通信。
- (2) 迭代求精：初始粗略估计通过多次迭代逐步优化。
- (3) 混合测距 / 非测距：支持基于距离（如 TOA、RSSI）和基于角度（AOA）的测量。

算法步骤：

AHLoS 的定位过程分为以下几个阶段：

- (1) 初始阶段：锚节点广播
 - 锚节点 (Anchor Nodes) 向周围广播自身的位置信息（如坐标、测距数据）。
 - 未知节点 (Unknown Nodes) 接收来自锚节点或已定位邻居节点的信息。
- (2) 局部几何计算

未知节点根据接收到的信息，选择以下两种方式之一进行定位：

① 基于三边测量 (Trilateration)

- 若未知节点能直接测量到至少 3 个锚节点的距离，则通过三边测量法计算初始位置。

• 数学模型：

$$\begin{cases} \sqrt{(x-x_1)^2+(y-y_1)^2}=d_1 \\ \sqrt{(x-x_2)^2+(y-y_2)^2}=d_2 \\ \sqrt{(x-x_3)^2+(y-y_3)^2}=d_3 \end{cases}$$

用最小二乘法求解超定方程组。

② 基于多跳协作 (Collaborative Multilateration) —— 协作多边算法

- 若未知节点无法直接连接足够锚节点，则利用多跳邻居节点的间接信息。

• 例如，未知节点 U 通过邻居节点 N_1 和 N_2 (已部分定位) 间接获取锚节点 A 的距离：

$$d_{U,A} \approx d_{U,N_1} + d_{N_1,A}$$

• 通过多跳路径的测距数据构建约束方程，迭代求解。

- 通过多跳路径的测距数据构建约束方程，迭代求解。

(3) 迭代优化

- 未知节点将初始估计广播给邻居节点。
- 邻居节点利用新信息修正自身位置，形成 **迭代反馈**：

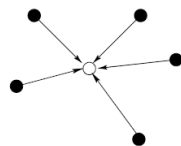
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \sum_{i \in \mathcal{N}} (\hat{d}_i - \|\mathbf{x}_k - \mathbf{x}_i\|) \cdot \frac{\mathbf{x}_k - \mathbf{x}_i}{\|\mathbf{x}_k - \mathbf{x}_i\|}$$

其中：

- $\mathcal{N}$ 为邻居节点集合，
- α 为步长因子，
- $\hat{d}_i$ 为测量距离。

(4) 收敛与终止

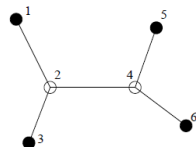
- 当位置更新的变化量小于阈值，或达到最大迭代次数时终止。
- 最终输出节点的估计位置。
- **Ad-Hoc Localization System**
- 迭代的定位算法
- 未知节点首先利用 TDOA 方法测量与其邻居节点的距离。
- 当未知节点的邻居节点中信标节点的数量大于等于 3 时利用极大似然估计方法计算自身位置，随后该节点转变成新的信标节点（转化信标节点），并将自身的位置广播给邻居节点。
- 随着系统内转化信标节点数量增加，原来邻居节点中信标节点数量少于 3 个的位置节点逐渐拥有足够的邻居信标节点，从而能够计算自身位置。
- 过程重复直至所有节点都计算出自身的位置。
- 系统中，位置节点根据周围信标节点的不同分布情况，分别利用相应的子算法计算位置节点的位置。



(a)

■ 原子多边形算法

- 在未知节点的邻居节点中信标节点（非转化信标节点）至少有 3 个时，利用极大似然估计方法计算自身位置。
- 迭代多边形算法
- 未知节点的邻居节点中信标节点少于 3 个，经过一段时间后，其邻居节点中部分未知节点在计算出自身位置后成为转化信标节点，该未知节点基于原始信标节点和转化信标节点，利用极大似然方法计算自身位置。



(b)

■ 协作多边形算法

- 经过多次迭代定位后，部分未知节点的邻居节点中，信标节点的数量仍然少于 3 个，此时需要通过其他节点的协助才能够计算自身位置。

这张图非常典型，讲的就是协作多边形定位为什么有必要。

图中可以这样看：

- 黑色实心点 1、3、5、6：锚节点，位置已知；
- 空心点 2、4：未知节点，位置未知；
- 连线：两个节点之间可以测到距离。

也就是说：

1、3 能和 2 测距

5、6 能和 4 测距

2 和 4 之间也能测距

1. 为什么迭代多边形算法在这里不行？

二维定位中，一个未知节点通常至少需要 3 个已知参考节点的距离 才能独立定位。

看节点 2：

2 连接 1、3、4

其中：

- 1 是锚节点，已知；
- 3 是锚节点，已知；
- 4 是未知节点，未知。

所以节点 2 只有 两个已知参考点，不够独立定位。

再看节点 4：

4 连接 2、5、6

其中：

- 5 是锚节点，已知；
- 6 是锚节点，已知；
- 2 是未知节点，未知。

所以节点 4 也只有 两个已知参考点，也不能独立定位。

因此如果用迭代多边算法，会卡住：

2 需要 4 已知才能定位

4 又需要 2 已知才能定位

谁都不能先定位，迭代过程无法启动。

2. 为什么协作多边算法可以？

协作多边算法不会要求 2 或 4 必须先独立定位，而是把 2 和 4 一起求解。

它利用所有可用距离关系：

$$d_{12}, d_{23}, d_{45}, d_{46}, d_{24}$$

也就是：

- 节点 2 到锚节点 1 的距离；
- 节点 2 到锚节点 3 的距离；
- 节点 4 到锚节点 5 的距离；
- 节点 4 到锚节点 6 的距离；
- 节点 2 和节点 4 之间的距离。

未知量是：

$$(x_2, y_2), (x_4, y_4)$$

也就是两个未知节点的位置。

于是可以联合建立方程组，把节点 2 和节点 4 的位置一起算出来。

3. 这张图的核心意思

这张图说明：

单个未知节点看起来都不满足独立定位条件，但多个未知节点之间如果存在测距关系，就可以通过协作联合定位。

节点 2 单独看不够条件，节点 4 单独看也不够条件；

但是 2 和 4 之间有一条距离约束，所以它们可以互相帮助完成定位。

4. 和两种算法的对应关系

这张图更适合说明协作多边算法，不是普通的迭代多边算法。

因为：

迭代多边算法：要求某个未知节点先能被定位，再帮助别人。

Wireless Sensor Network

协作多边算法：多个未知节点即使都不能单独定位，也可以联合求解。

在这张图里，2 和 4 都不能先单独定位，所以迭代方法会失败；但协作多边算法可以把 2、4 作为一个整体来定位。

考试可以这样说：

图中实心节点 1、3、5、6 为已知锚节点，空心节点 2、4 为未知节点。节点 2 只能直接测得与锚节点 1、3 以及未知节点 4 的距离，节点 4 只能直接测得与锚节点 5、6 以及未知节点 2 的距离。由于二维定位通常需要至少三个已知参考节点，节点 2 和节点 4 都无法通过普通迭代多边算法单独完成定位，因此迭代过程会陷入停滞。但协作多边算法可以利用未知节点 2 和 4 之间的距离约束，将两者的位置作为未知量联合建立方程组，同时结合它们到周围锚节点的距离进行求解。因此，该图体现了协作多边定位的思想，即未知节点之间通过相互测距提供额外约束，从而在单个节点无法独立定位的情况下实现联合定位。

AHLoS 中解坐标统一用极大似然估计，和你之前学的最小二乘数学形式一致：

1. 由 TDOA 测距得到多组距离，建立圆方程组；
2. 两两相减线性化，得到超定方程组 $AX = b$ ；
3. 最优估计解： $X = (A^T A)^{-1} A^T b$ ，最小化测距误差带来的坐标偏差。

五、考点精简总结（答题可用）

1. AHLoS 是自组织迭代式 Range-Based 定位系统，底层依靠 TDOA 实现节点测距；
2. 核心创新：完成定位的未知节点升级为转化信标，迭代扩充全网锚节点密度；
3. 三套子算法分工：
 - 原子多边：原生锚点 ≥ 3 ，独立一轮定位；
 - 迭代多边：原生锚点不足，等待转化锚点补足后定位；
 - 协作多边：迭代后锚点仍不足，多跳节点协作联合求解；
4. 坐标求解统一采用极大似然估计（最小二乘）；
5. 适用场景：大规模、原始锚节点稀疏的无线传感器网络。

特点：

- 对信标节点的密度要求高
- 不适用于大规模的传感器网络
- 迭代过程存在累积误差

优点：

- （1）分布式计算：适应大规模网络，无需中央控制。
- （2）灵活性：支持测距 / 非测距混合模式。
- （3）可扩展性：通过多跳协作覆盖稀疏网络。

缺点：

- （1）误差累积：多跳测距会放大误差。
- （2）收敛速度：迭代次数影响实时性。
- （3）锚节点依赖：初始阶段需足够锚节点。

（三）基于到达角度(AOA)的测距方法

- 接收节点通过天线阵列或多个超声波接收机感知发射节点信号的到达方向。
- 接收节点通过接收信号计算和发射节点之间的相对方位或角度，再通过三角测量法计算出节点的位置。
- 对硬件要求高，维护代价高。

（四）基于接收信号强度(RSS)的测距方法

- 通过信号在传播中的衰减来估计节点之间的距离
- 无线信道的数学模型

$$PL(d) = PL(d_0) - 10n \lg\left(\frac{d}{d_0}\right) - X_\sigma$$

d :发射机和接收机之间的距离

d_0 :参考距离

n : 信道衰减指数, 一般取2-4

X_σ :均值为0, 方差为 σ 的高斯随机噪声变量

$PL(d_0)$:距离发射机 d_0 处的信号强度

$PL(d)$:距离发射机 d 处的信号强度

第十四讲

无需测距的定位:

(一) 质心算法

WSN 中的节点定位是关键技术之一, 而质心算法 (Centroid Algorithm) 是一种简单且广泛使用的基于非测距的平面定位方法。

核心思想:

质心算法利用节点通信范围内的锚节点 (已知位置的参考节点) 的几何质心来估算未知节点的位置。其核心假设是:

未知节点的位置近似于其所有可通信锚节点位置的平均值 (质心)。

算法步骤:

(1) 锚节点广播信息

锚节点周期性地向周围广播自己的位置信息 (如坐标) 和信号覆盖半径 R 。

(2) 未知节点收集信息

未知节点记录在通信范围内 (通常以信号强度或跳数判断) 接收到的所有锚节点的位置信息。

(3) 计算质心位置

设未知节点接收到 N 个锚节点的坐标 $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$, 则其估算坐标 (x_{est}, y_{est}) 为:

$$x_{est} = \frac{1}{N} \sum_{i=1}^N x_i, \quad y_{est} = \frac{1}{N} \sum_{i=1}^N y_i$$

- 多边形的几何中心称为质心
- 多边形顶点坐标的平均值就是质心的坐标
- 质心算法首先确定包含未知节点的区域, 计算这个区域的质心, 并将其作为未知节点的位置。

$$(X_{est}, Y_{est}) = \left(\frac{X_{i_1} + \dots + X_{i_k}}{k}, \frac{Y_{i_1} + \dots + Y_{i_k}}{k} \right)$$

- 信标节点周期性的向邻近节点广播信标分组
- 信标分组包含信标节点的标识号和位置信息
- 当未知节点接收到来自不同信标节点的位置分组数量超过某一个门限 k 或者接收一定时间后, 就确定这些信标节点所组成的多边形的质心为自身位置。
- 完全基于网络连通性, 无需信标节点和未知节点之间的协调 (该类定位算法只依靠节点之间能否收到无线信号 (连通与否) 完成定位, 不需要测距硬件; 锚节点只单向广播坐标, 未知节点仅被动接收, 二者不需要双向交互、时钟同步、迭代反馈等互相配

合的协调操作。)

- 简单，容易实现
- 假设节点都拥有理想球形无线信号传播模型
- 用质心作为位置估计的精确度依赖于信标节点的密度和分布，信标节点密度越大且分布越均匀时，定位精确度越高。

关键特点：

- (1) 无需测距：不依赖距离或角度测量（如 RSSI、TOA 等），仅需锚节点坐标。
- (2) 低计算复杂度：仅需简单算术平均，适合资源受限的 WSN 节点。
- (3) 依赖锚节点密度：定位精度与锚节点数量和分布密切相关。锚节点越多且分布均匀，精度越高。

误差来源与局限性：

- (1) 边缘效应：靠近网络边缘的未知节点，其可通信锚节点可能集中在某一侧，导致质心偏离真实位置。
- (2) 锚节点分布不均：若锚节点分布不均匀（如全部位于一侧），误差会显著增大。
- (3) 通信范围限制：仅适用于锚节点密度较高的场景，稀疏网络中性能下降。

改进方法：

为提高精度，研究者提出了多种改进方案：

- (1) 加权质心算法

根据锚节点与未知节点的信号强度（RSSI）或链路质量赋予不同权重，距离越近的锚节点权重越高。

估算公式：

$$x_{est} = \frac{\sum_{i=1}^N w_i x_i}{\sum_{i=1}^N w_i}, \quad y_{est} = \frac{\sum_{i=1}^N w_i y_i}{\sum_{i=1}^N w_i}$$

(权重 w_i 可基于 RSSI 或跳数)

- (2) 迭代质心算法

部分已定位的未知节点可升级为“虚拟锚节点”，参与后续节点的定位，逐步提高覆盖率。

- (3) 区域划分法

将通信区域划分为网格，结合质心与区域扫描提高精度。

与其他定位算法的对比：

算法类型	质心算法	测距类算法 (如 DV-Hop)	基于几何的算法 (如 APIT)
是否需要测距	否	是	部分需要
计算复杂度	极低	中等	较高
精度	低（依赖锚节点密度）	中等	较高
适用场景	高锚节点密度	中等密度网络	高密度且规则部署

(二) DV-hop

WSN 中的 DV-Hop (Distance Vector-Hop) 算法是一种经典的基于非测距 (Range-free) 的平面节点定位方法, 通过利用网络拓扑中的跳数信息和平均每跳距离来估算节点位置。

核心思想:

DV-Hop 算法的核心是通过距离向量 (Distance Vector) 和跳数 (Hop Count) 信息, 将锚节点 (已知位置的参考节点) 之间的距离转换为平均每跳距离, 进而估算未知节点到锚节点的物理距离, 最终通过多边定位法 (如最小二乘法) 计算未知节点的坐标。

算法步骤:

(1) 计算最小跳数 (Flooding)

- 锚节点广播信标

每个锚节点向全网广播包含自身位置信息 (如坐标) 的数据包, 初始跳数设为 0。

- 跳数累加与转发

其他节点 (包括未知节点和锚节点) 收到信标后:

- 记录到该锚节点的最小跳数 (若收到同一锚节点的多个信标, 保留跳数最小的值)。
- 将跳数 + 1 后转发给邻居节点。

- 结果

最终, 所有节点获得每个锚节点的最小跳数。

(2) 计算平均每跳距离 (Correction Phase)

- 锚节点计算平均每跳距离

每个锚节点 i 根据其他锚节点的位置和跳数信息, 计算自身到其他锚节点的平均每跳距离 $HopSize_i$:

$$HopSize_i = \frac{\sum_{j \neq i} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}}{\sum_{j \neq i} h_{i,j}}$$

(分子为锚节点 i 与 j 的真实欧氏距离, 分母为两者间的跳数)。

- 广播 $HopSize_i$

锚节点将计算的平均每跳距离广播到网络中。未知节点选择接收到的第一个 $HopSize_i$ 或最近的锚节点的 $HopSize_i$ 用于后续计算。

(3) 距离估算与坐标求解 (Positioning)

- 未知节点估算到锚节点的距离

未知节点 k 使用接收到的 $HopSize_i$ 和记录的跳数 $h_{i,k}$, 计算到锚节点 i 的估算距离:

$$d_{i,k} = HopSize_i \times h_{i,k}$$

- 多边定位法求解坐标

当未知节点获得至少3个锚节点的距离估算值后, 通过最小二乘法求解以下方程组:

$$\begin{cases} (x - x_1)^2 + (y - y_1)^2 = d_{1,k}^2 \\ (x - x_2)^2 + (y - y_2)^2 = d_{2,k}^2 \\ \vdots \\ (x - x_n)^2 + (y - y_n)^2 = d_{n,k}^2 \end{cases}$$

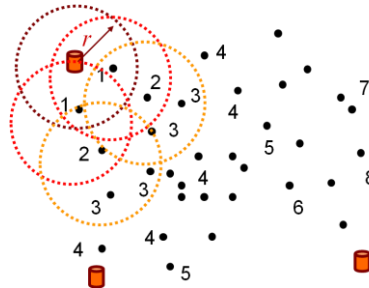
(其中 (x, y) 为未知节点坐标, (x_i, y_i) 为锚节点坐标)。

Wireless Sensor Network

- 距离向量-跳段(distance vector-hop)
- 类似传统网络的距离向量路由机制
- 未知节点首先计算与信标节点的最小跳数，然后估算平均每跳的距离，利用最小跳数乘以平均每跳距离得到未知节点与信标节点之间的估计距离，再利用三边测量法或极大似然法计算未知节点的坐标。
- 定位过程分 3 个阶段：

① 计算未知节点与每个信标节点的最小跳数。

信标节点向邻居节点广播自身位置信息的分组，其中包括跳数字段，初始值为 0。接收节点记录具有到每个信标节点的最小跳数，忽略来自同一个信标节点的较大跳数的分组。然后将跳数值加 1，并转发给邻居节点。最终，网络中所有节点能够记录下到每个信标节点的最小跳数。



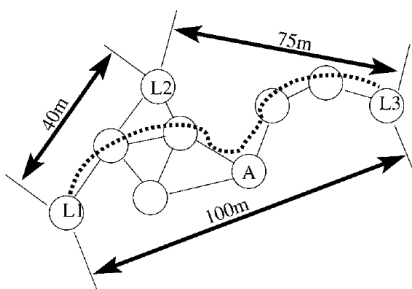
② 计算未知节点与信标节点的实际跳段距离。

每个信标节点根据第一个阶段中记录的其他信标节点的位置信息和相距跳数，估算出平均每跳的实际距离。

$$c_i = \frac{\sum \sqrt{(X_i - X_j)^2 + (Y_i - Y_j)^2}}{\sum h_{ij}}, \quad i \neq j, \text{ 遍历全部锚节点 } j.$$

信标节点将计算的每跳平均距离用带有生存期的分组广播至网络中，未知节点仅记录接收到的第一个每跳平均距离，并转发给邻居节点。未知节点根据每跳平均距离和记录的跳数，计算到每个信标节点的跳段距离。

③ 未知节点利用三边测量法或极大似然估计法计算自身位置。



$$d_{12} = 40 \text{ m} \quad \text{hop}_{12 \min} = 2 \text{ hop}$$

$$d_{23} = 75 \text{ m} \quad \text{hop}_{23 \min} = 5 \text{ hop}$$

$$d_{13} = 100 \text{ m} \quad \text{hop}_{13 \min} = 6 \text{ hop}$$

$$\text{Hopsize} = \frac{d_{12} + d_{23} + d_{13}}{\text{hop}_{12 \min} + \text{hop}_{23 \min} + \text{hop}_{13 \min}} = \frac{40 \text{ m} + 75 \text{ m} + 100 \text{ m}}{2 \text{ hop} + 5 \text{ hop} + 6 \text{ hop}}$$

$$\approx 16.54 \text{ m/hop}$$

例.

Figure 1. "DV-hop" correction example.

$$r_1 = 3 \times \text{Hopsize}, \quad r_2 = 2 \times \text{Hopsize}, \quad r_3 = 3 \times \text{Hopsize} \quad (\text{也是选取最小跳数})$$

再知道 L_1, L_2, L_3 的坐标，就可用三边测量法计算了

关键特点：

(1) 无需测距硬件：不依赖 RSSI、TOA 等测距技术，仅需网络连通性（跳数）。

通俗举例

Wireless Sensor Network

测距类算法 (AHLos、三边测量) 需要传感器硬件测出 A、B 之间有 50 米;

DV-Hop 完全不用测几米, 只看消息从 A 传到未知节点中转了 3 次, 记录跳数 = 3 即可。

实际优势

传感器节点硬件成本、功耗更低, 适合电池供电、资源受限的 WSN 场景。

(2) 分布式计算: 每个节点独立完成跳数记录和距离估算。

通俗举例

集中式: 所有节点把跳数数据发给一台主控电脑, 电脑统一算所有人位置;

分布式: 每个节点自己算自己的跳数、自己算自己在哪, 互不依赖中心。

实际优势

减少网络大量数据传输, 降低通信开销; 单个节点故障不会让整个网络定位瘫痪, 鲁棒性更强。

(3) 适应性: 适用于各向同性网络 (节点分布均匀时精度较高)。

专业解释

1. 当节点均匀铺开 (各向同性): 相邻节点间距相近, HopSize (平均每跳距离) 能真实反映网络内单跳平均长度, 用「跳数 $\times$ 每跳距离」估算的两点直线距离误差很小, 最终定位精度高;
2. 若网络不均匀: 一边节点密集、一边稀疏, 或者存在墙体、障碍物阻挡通信, 相邻节点间距差距很大, “每跳距离相等” 的假设失效, 跳数无法真实反映物理远近, 估算距离偏差巨大, 定位精度暴跌。

通俗举例

均匀网络: 小区楼房整齐排布, 每层住户间距差不多, 跳数能大致代表远近;

不均匀网络: 一片密林 + 一片空地, 密林区 1 跳只有 5 米, 空地 1 跳能传 30 米, 统一用同一个平均跳距计算距离会严重失真。

实际局限

矿山、室内复杂遮挡、节点疏密差距极大的场景, DV-Hop 定位效果很差, 更适合室外均匀部署的无线传感网络。

误差来源与局限性:

(1) 跳数 - 距离线性假设误差

算法假设每跳距离均匀, 但在实际非均匀部署或障碍物环境中, 跳数与真实距离非线性相关。

(2) 锚节点分布影响

锚节点分布不均或数量不足时, 误差显著增大 (如锚节点集中在某一侧)。

(3) 边缘节点问题

网络边缘的未知节点因可接触的锚节点较少, 定位精度下降。

改进方法:

为提高精度, 研究者提出了多种改进方案:

(1) 加权 DV-Hop

根据锚节点间的跳数或拓扑密度赋予不同权重, 优化 HopSize 计算。

原版 DV-Hop 的缺陷

原始公式计算 HopSize 时, 所有锚点对的距离、跳数同等权重, 不管两个锚隔 1 跳还是 10 跳, 全部直接相加。

远距离锚点之间路径绕弯多, 算出来的平均跳距会被严重带偏, 误差变大。

改进思路

给每一组锚点对设置权重:

- 跳数越少、拓扑越密集的锚点对, 权重更高 (参考价值更大);

- 跳数多、路径绕远的锚点对，权重压低，减少它对 HopSize 的干扰。

加权后再计算平均每跳距离，得到的 HopSize 更贴合局部真实单跳长度，距离估算误差缩小。

(2) 局部平均跳距修正

未知节点根据邻居锚节点的 HopSize 动态调整估算值，而非使用单一锚节点的 HopSize。

原版 DV-Hop 的缺陷

原始算法里，未知节点只采用第一个收到的单一锚节点算出的 HopSize，全网共用同一个跳距。

但网络疏密不均时，远处锚算出来的 HopSize 不适合本地节点，全局统一值不匹配局部拓扑。

改进思路

未知节点收集周边多个相邻锚节点各自计算的 HopSize，取局部多个跳距做平均，动态适配自己附近的网络密度，不用远处锚的全局单一数值。

本地拓扑和跳距匹配度更高，估算到锚的距离更精准。

(3) 结合 RSSI 辅助

在部分链路中引入 RSSI 测距，修正跳数到距离的映射关系。

原版 DV-Hop 的缺陷

纯依靠跳数，完全没有物理测距信息，只靠“跳数 × 平均跳距”线性估算，假设每跳距离均等，障碍物、疏密不均时偏差极大。

改进思路

在节点之间短距离链路上，引入 RSSI 信号强度测距：

1. RSSI 可以粗略算出两个直连节点的真实距离；
2. 用真实测距结果修正「跳数→物理距离」的映射关系，打破“每跳距离相等”的刻板假设；

既保留 DV-Hop 免测距的低成本优势，又用少量测距信息大幅降低估算误差。

(4) 迭代定位

将已定位的未知节点升级为“虚拟锚节点”，逐步提高覆盖率。

与其他定位算法对比：

特性	DV-Hop	质心算法	APIT
是否需要测距	否	否	部分需要（接收强度）
计算复杂度	中等（需多边定位求解）	极低（直接平均）	高（区域交集计算）
精度	中等（依赖网络均匀性）	低	较高
适用场景	锚节点分布均匀的网络	高锚节点密度	高密度且规则部署

(三) APIT

WSN 中的 APIT (Approximate Point-In-Triangulation) 算法是一种基于非测距 (Range-free) 的平面节点定位方法, 通过利用锚节点 (已知位置的参考节点) 组成的几何三角形和节点间的相对位置关系来估算未知节点的位置。APIT 算法结合了区域划分和交集逼近的思想, 具有较高的定位精度, 尤其适用于锚节点密度适中的网络。

核心思想:

APIT 算法的核心是通过以下步骤实现定位:

(1) 三角形测试 (PIT Test):

未知节点判断自身是否位于由三个相邻锚节点组成的三角形内部。

(2) 区域交集法:

收集所有包含该未知节点的三角形, 并计算这些三角形的重叠区域, 最终以重叠区域的质心作为估算位置。

关键概念与假设:

完美点 - In - 三角形测试 (Perfect PIT Test)

- 在理想情况下, 若未知节点 M 位于锚节点 A, B, C 组成的三角形内, 则满足:
 M 移动到任意邻域位置时, 至少会靠近或远离所有三个锚节点 (即信号强度变化一致)。
- 反之, 若 M 在三角形外, 存在某些移动方向使其同时远离或接近所有三个锚节点。

近似 PIT 测试 (Approximate PIT Test)

- 实际网络中, 节点无法移动, 因此 APIT 通过邻居节点 (即与 M 直接通信的节点) 模拟移动:
 - 若所有邻居节点均满足 PIT 条件 (即无邻居同时远离 / 接近三个锚节点), 则判定 M 在三角形内。
 - 否则, 判定 M 在三角形外。

算法步骤:

(1) 收集锚节点信息

未知节点监听周围锚节点的信标消息, 记录其坐标和信号强度 (RSSI)。

(2) 执行 APIT 测试

①选择锚节点组合:

从已知锚节点中选取所有可能的三个锚节点组合 (三角形), 例如锚节点 A, B, C 。

②邻居协作测试:

- 未知节点 M 询问邻居节点关于锚节点 A, B, C 的信号强度 (RSSI)。
- 若存在邻居节点 N , 使得 N 相对于 M 同时远离或接近锚节点 A, B, C , 则判定 M 在三角形外; 否则判定在三角形内。

(3) 区域交集与质心计算

①生成包含区域:

收集所有判定包含 M 的三角形, 这些三角形的交集形成一个多边形区域。

②估算位置:

计算该多边形区域的质心, 作为未知节点的最终坐标:

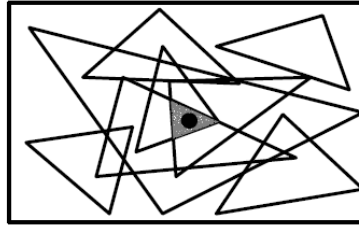
$$(x_{est}, y_{est}) = \left(\frac{\sum x_i}{n}, \frac{\sum y_i}{n} \right)$$

((x_i, y_i) 为多边形的顶点坐标)。

- 近似三角形内点测试法 (approximate point-in-triangulation test)
- 首先确定多个包含未知节点的三角形区域。这些三角形区域的交集是一个多边形, 其确定了更小的包含未知节点的区域。然后计算这个多边形区域的质心, 并将其作为未知节点的位置。

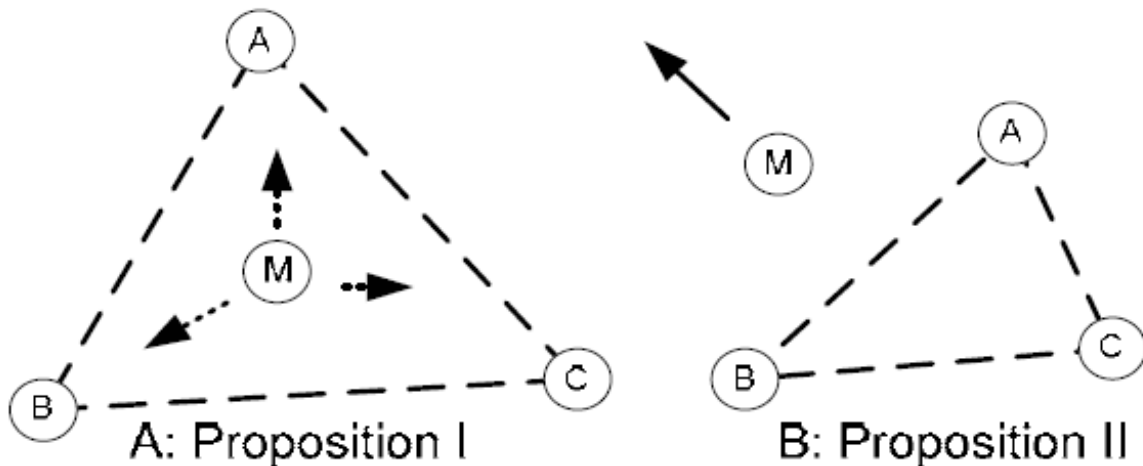
■ 基本思想：

未知节点收集其邻近信标节点的信息，然后从这些信标节点组成的集合中任意选取三个信标节点。假设集合中有 n 个信标节点，则有 C_n^3 种不同的选取方法确定不同的三角形。逐一测试未知节点是否位于每个三角形的内部，直至穷尽所有组合产生的三角形或达到定位所需精度。计算包含目标未知节点所有三角形的重叠区域，将重叠区域的质心作为未知节点的位置。



■ 理论基础：最佳三角形内点测试法 PIT。

- 假如存在一个方向，节点 M 沿着这个方向移动会同时远离或靠近节点顶点 A, B, C ，那么节点 M 位于三角形外；否则，节点 M 位于三角形内。



- 网络中节点静止：需要在静态环境中实现三角形内点测试。
- 近似的三角形内点测试
- 节点 M 的邻居节点中，相对于节点 M ，没有同时远离或靠近三个信标节点，那么节点 M 在三个信标节点组成的三角形内；否则，在三角形之外。
- 利用网络中相对较高的节点密度模拟节点移动。利用无线信号传播特性判断是否远离或靠近信标节点。
- 邻居节点通过交换各自接收到信号的强度，判断距离某一个信标节点的远近，从而模仿 PIT 中的节点移动。

1、先搞懂：原版 PIT 算法本来要干什么（为什么需要“节点移动”）

PIT（点在三角形内测试）原始规则：

让未知节点 M 自己物理移动一小段距离，分两种情况判断：

1. 移动之后， M 到 A, B, C 三个锚的距离同时全部变近 / 同时全部变远

→ M 在 $\triangle ABC$ 外面

2. 移动之后，不可能同时离三个锚更近 / 更远

→ M 在 $\triangle ABC$ 里面

但 WSN 传感器节点是固定死的，不能挪动，硬件没法移动，原版 PIT 直接用不了。这就是 APIT 要解决的核心矛盾。

2、第一句：利用高密度节点模拟节点移动；靠信号强弱判断远近
无线信号有固定规律：

离锚节点越近 → 收到的信号强度 RSSI 越大；

离锚节点越远 → RSSI 越小。

网络节点部署得很密，M 的上下左右全是邻居节点（1、2、3、4）。

这些邻居本身就在 M 四周不同位置，相当于 M 往四周移动后的“虚拟位置”。

不用真的挪动 M，拿旁边邻居的信号数据，就能替代“M 移动后的信号”。

3、第二句：邻居交换 RSSI，模仿 PIT 的移动测试

1. M 和周边所有邻居互相交换数据包，包里带上各自收到 A、B、C 三个锚的 RSSI 信号强度；
2. 拿邻居的 RSSI 和 M 自身的 RSSI 做对比：
 - 邻居 RSSI 更大 = 这个邻居比 M 离锚更近
 - 邻居 RSSI 更小 = 这个邻居比 M 离锚更远
3. 遍历全部邻居，看有没有任意一个邻居，同时对 A、B、C 三个锚的 RSSI 都更大（同时更近），或者同时都更小（同时更远）
 - 有 → 等效于 M 移动后同时远离 / 靠近三个锚，判定 M 在三角形外
 - 没有 → 判定 M 在三角形内

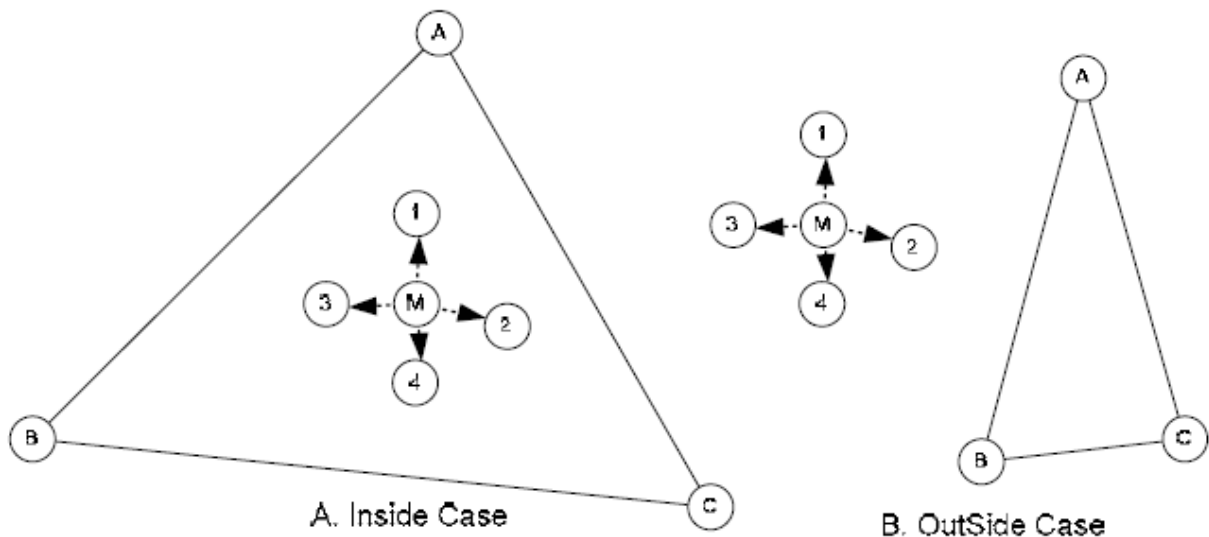


Figure 5: Approximate P.I.T Test

- 在无线信号传播模式不规则和传感器节点随机部署的情况下，算法的定位精度高，性能稳定。
- 对网络的连通性有较高要求。

关键特点：

- (1) 无需测距：仅依赖锚节点坐标和邻居节点的信号强度比较。
- (2) 高精度：通过区域交集法缩小位置范围，精度优于质心算法和 DV-Hop。
- (3) 依赖邻居密度：邻居节点越多，PIT 测试的准确性越高。

误差来源与局限性：

- (1) PIT 测试误判

在网络边缘或邻居节点稀疏时，可能出现误判（如将外部点判为内部）。

- (2) 锚节点分布影响

锚节点分布不均匀时，三角形覆盖不全，导致交集区域过大或无法定位。

(3) 通信开销

需要邻居节点协作，增加了通信负担。

改进方法：

(1) 动态阈值调整：

根据信号强度波动动态调整 PIT 测试的判定阈值，减少误判。

(2) 锚节点优化部署：

规划锚节点的位置，使其形成的三角形尽可能覆盖整个网络。

(3) 混合定位：

结合 RSSI 测距或其他算法（如 DV-Hop）辅助修正交集区域。

与其他算法对比：

特性	APIT	DV-Hop	质心算法
定位原理	三角形区域交集	跳数 - 距离映射	锚节点坐标平均
精度	较高	中等	低
计算复杂度	高（需几何计算）	中等	极低
通信开销	高（需邻居协作）	中等	低
适用场景	锚节点分布均匀且密度适中	大规模均匀网络	高锚节点密度

第十五讲

拓扑控制

拓扑控制（Topology Control, TC）是 WSN 中通过调整节点间的通信关系（如传输功率、邻居选择、休眠调度等），优化网络结构以提高性能（如能效、连通性、覆盖性等）的技术。其核心目标是在保证网络基本功能的前提下，减少能耗、延长网络寿命。

为什么需要拓扑控制？

无线传感器网络通常具有以下挑战：

- 能量受限：节点依赖电池供电，能耗直接影响网络寿命。
- 高密度部署：节点冗余可能导致干扰和资源浪费。
- 动态环境：节点失效、移动或信道变化需自适应调整。
- 多跳通信：需要优化路径以减少延迟和能耗。

拓扑控制通过主动管理网络结构来解决这些问题。

（一）研究问题

①物理拓扑：WSN 中节点在最大通信半径下的网络连接关系。

②逻辑拓扑：在满足网络覆盖率和连通性的前提下，通过信息交互、功率控制等手段，剔除物理拓扑中节点间不必要的物理通信链路，建立逻辑链路后形成的网络连接关系。

WSN 中的拓扑控制是优化网络性能的关键技术，其中物理拓扑和逻辑拓扑是两个核心概念，它们的区别与联系如下：

①物理拓扑（Physical Topology）

- 定义：反映节点之间实际的物理连接关系，由节点的物理位置、通信半径、无线信道特性等决定。
- 特点：
 - 基于硬件：由传感器节点的部署位置和无线信号覆盖范围直接形成。
 - 不可见性：物理拓扑通常是隐式的，需要通过测量信号强度或距离来推断。
 - 动态性：可能因节点移动、能量耗尽或环境干扰（如障碍物）而变化。
- 示例：若节点 A 的无线电波能覆盖节点 B，则两者在物理拓扑中存在直接连接。

②逻辑拓扑（Logical Topology）

- 定义：通过协议或算法人为构建的虚拟连接关系，用于优化数据传输路径或资源分配。

- 特点
 - 基于软件：由路由协议、簇头选举（如 LEACH）、睡眠调度（如 SMAC）等逻辑控制机制形成。
 - 抽象性：可能忽略实际物理连接，仅保留必要的通信路径（如生成树、分簇结构）。
 - 稳定性：可通过算法维持逻辑连接的稳定，即使物理拓扑变化（如通过多跳路由绕过失效节点）。
- 示例：在分簇网络中，簇头节点与成员节点形成逻辑连接，而成员节点之间可能无直接通信。

区别总结：

维度	物理拓扑	逻辑拓扑
构成基础	物理信号覆盖与硬件连接	协议或算法设计的虚拟结构
动态性	随环境或节点状态被动变化	可主动调整以优化性能
可见性	需测量信号强度或距离推断	直接由协议定义（如路由表）
目标	反映真实连接可能性	提高能效、延长网络寿命等

联系与协作：

（1）依赖关系

逻辑拓扑需基于物理拓扑构建（例如，多跳路由需要物理连通性作为基础）。

（2）相互影响

物理拓扑的变动（如节点故障）可能触发逻辑拓扑重构（如重新选举簇头）。

逻辑拓扑的优化（如功率控制）可间接改变物理拓扑（如调整通信半径）。

（3）协同优化

拓扑控制算法（如 TopDisc、HEED）通过结合两者，实现网络覆盖、连通性与能效的平衡。

- 在满足网络覆盖度和连通度的前提下，有效、合理的设置或调整节点的发射功率，并根据一定的原则选择合适的节点处理和传输网络中的数据，优化网络的拓扑结构。

通俗大白话讲解 WSN 物理拓扑 & 逻辑拓扑

先打一个生活化类比，一下子就能分清：

物理拓扑 = 两个人实际能不能喊话听见

逻辑拓扑 = 规定好谁必须跟谁说话、不许乱聊天

一、物理拓扑（实打实、客观存在的连接）

核心本质

节点硬件层面能不能互相收到无线信号，完全由距离、遮挡、发射功率决定，不由程序人为规定，是客观事实。

举例子

1. 节点 A 和节点 B 离得近，A 发信号 B 能收到 → 物理上连通，物理拓扑有连线
2. 节点 A 和节点 C 隔了一堵厚墙，信号被挡住，互相收不到报文 → 物理拓扑没有连线
3. 某个节点电池没电关机了，它和周边所有节点物理连接直接消失，物理拓扑自动改变

关键特点总结

- ✅ 被动形成、天生存在，不是代码写出来的
- ✅ 能通就是能通，不能通就是不能通，人为没法强行修改
- ✅ 环境一变、节点一动、没电失效，物理拓扑就跟着变

二、逻辑拓扑（软件规定、虚拟的连接规则）

核心本质

明明有些节点物理上可以互相通信，但路由 / 分簇算法人为规定：只允许部分节点互相收发数据，屏蔽多余连接，用来省电、减少信号冲突。

逻辑拓扑必须建立在物理拓扑之上：物理不通，逻辑再怎么设置也连不了。

经典例子（LEACH 分簇网络，最好理解）

假设 4 个节点：A（簇头）、B、C、D

1. 物理层面：B、C、D 三个节点距离都很近，两两之间信号全覆盖，物理拓扑里 B-C、B-D、C-D 全部互通
2. 算法做逻辑规定：只允许 B、C、D 把数据发给簇头 A，B、C、D 之间不许互相发消息
3. 最终逻辑拓扑只有三条连线：B-A、C-A、D-A，B、C、D 彼此在逻辑上断开

再举一个路由例子：

节点 1、2、3 两两物理都能通信；路由算法规定数据只能 $1 \rightarrow 2 \rightarrow 3$ 转发，不允许 1 直接发给 3。

- 物理拓扑：1-2、2-3、1-3 三条全通
- 逻辑拓扑：只有 $1 \leftrightarrow 2$ 、 $2 \leftrightarrow 3$ 两条通路

关键特点总结

- ✅ 软件 / 算法人为设计出来的虚拟关系
- ✅ 可以主动修改、裁剪连接，用来节能、抗干扰、优化路由
- ✅ 哪怕物理能互通，我也可以在逻辑上“切断”它们的通信

三、对照表

对比项	物理拓扑	逻辑拓扑
是谁决定的	距离、遮挡、无线信号（硬件客观事实）	路由、分簇、休眠调度算法（软件规则）
能不能人为随便切断	不能，信号能收到就是能收到	可以，物理通也能逻辑禁止通信
变化方式	被动变化（节点移动、没电、遮挡）	主动重构（算法按需调整结构）

四、二者绑定关系（一句话吃透）

1. 逻辑拓扑不能脱离物理拓扑：如果两个节点物理上压根收不到对方信号，你再怎么写协议，逻辑上也不可能建立连接；
2. 物理拓扑变了，逻辑拓扑往往跟着改：某个节点坏掉，周边物理连接消失，算法就会重新选簇头、重算路由，重构逻辑拓扑；
3. 逻辑拓扑调整也能反向改变物理拓扑：比如节点调低发射功率，主动缩小通信范围，原本能连上的邻居物理连接断开，物理拓扑随之改变。

（二）研究意义

■ 影响整个网络的生存时间

传感器节点靠电池供电，发射无线电是最耗电的行为；拓扑控制可以合理控制节点发射功率、让冗余节点休眠，大幅省电，延长整个网络能用的时间。

■ 减少节点间通信干扰

无线信道是共用的，多个节点同时大功率发包会互相撞包、信号打架；拓扑控制精简邻居数量、缩小通信范围，降低信道冲突。

■ 为路由协议提供基础

路由协议要选数据转发路径，前提是先确定“谁和谁能通信”（也就是拓扑结构）；拓扑控制整理好邻居关系、连通结构，路由才能正常算路径。

■ 影响数据融合

WSN 常把多个节点采集的同类数据汇总合并（数据融合，避免重复上传冗余数据）；拓扑控制的分簇结构（LEACH 等）天然适配融合机制。

■ 弥补节点失效的影响

传感器节点容易没电、损坏、信号遮挡离线，造成局部网络断连；拓扑控制具备动态重构能力，节点坏掉后自动调整邻居、重新组网绕开故障点，保证网络整体连通。

（三）设计目标

■ 覆盖

衡量监测区域被传感器感知完整程度，避免监测盲区。

■ 连通

保证全网节点互相可达，网络整体不分裂断网。

■ 网络生命期

网络满足工作要求的总运行时长。

■ 吞吐能力

单位时间网络成功传输的数据总量，代表数据传输上限。

■ 干扰和竞争

多节点抢占无线信道产生信号冲突，不合理拓扑会加剧该问题。

■ 网络延迟

数据包端到端传输耗时，路径不合理、冲突会增大时延。

■ 拓扑性质（节点度有界、可扩展性、负载均衡等）

总述：用来评判拓扑结构好不好、稳不稳定、好不好扩容的一组综合属性

（1）节点度有界（Bounded Node Degree）

- 节点度：一个节点直接相连的邻居节点总数
- 含义：限制每个节点的邻居数量不能无限多，避免某个节点连接太多邻居，转发压力过大、干扰剧增，拓扑更规整可控。

（2）可扩展性（Scalability）

- 含义：当网络规模变大（增加几百上千个新节点），拓扑协议、网络性能不会急剧恶化，依然能正常组网运行。
- 反面例子：小规模网络能用的算法，节点一多就卡顿、瘫痪 = 可扩展性差。

（3）负载均衡（Load Balancing）

- 含义：把数据转发、数据采集、簇头任务均匀分摊给所有节点，避免少数节点长期超负荷工作、耗电过快提前死掉。
- 例子：LEACH 分簇周期性轮换簇头，就是典型负载均衡；如果固定一个节点一直当簇头，该节点电量很快耗尽。

（四）解决方案

■ 部署

含义

在网络投放安装阶段，人为规划传感器节点摆放位置、投放密度，从源头决定初始物理拓扑结构。分为两种模式：

1. 确定性部署：人工精准摆放每个节点，保证监测区域全覆盖、节点间距合理；
2. 随机部署：大面积野外、危险区域飞机抛撒投放，节点位置杂乱，后续靠拓扑算法修正拓扑缺陷。

解决什么问题

Wireless Sensor Network

1. 提前规避监测盲区，保障初始覆盖性能；
2. 控制节点疏密程度，避免局部节点过密产生严重信号干扰；
3. 预先规划连通路经，防止网络天然割裂不通。

举例

大棚监测：人工每隔 5 米放一个温湿度传感器（确定性部署）；山林防火无人机抛撒大量传感节点（随机部署）。

■ 功率控制

含义

动态修改每个传感器节点的无线发射功率，调整自身通信覆盖半径，进而改变邻居关系、重构物理拓扑，是最基础的拓扑控制手段。

- 大功率：通信距离远、邻居多、功耗高、干扰大；
- 小功率：刚好连通必要邻居，省电、冲突更少。

解决什么问题

1. 降低整体能耗，延长网络生命周期；
2. 减少邻居数量，缓解信道干扰与竞争；
3. 约束节点度数，实现节点度有界，优化拓扑规整性；
4. 在保证全网连通前提下，精简冗余链路。

举例

节点原本最大功率能连 8 个邻居，算法下调功率后只保留 2 个必要转发邻居，其余多余连接断开。

■ 活动调度

含义

高密度冗余部署场景下，把节点分成两组或多组，一部分节点开机工作（活跃）完成感知、通信任务，另一组进入低功耗休眠断电，周期性轮换角色。也叫节点休眠调度。

解决什么问题

1. 用最少活跃节点维持区域全覆盖，冗余节点休眠省电；
2. 减少同时在线节点数量，降低无线信道竞争干扰；
3. 全网轮流耗电，实现负载均衡，避免部分节点长期工作提前耗尽电量；
4. 节点故障时唤醒备用休眠节点，弥补节点失效问题。

典型协议：S-MAC、T-MAC 都是基于活动调度的 MAC 层拓扑优化方案。

举例

一片区域有 10 个温度传感器，满足全覆盖只需要 4 个在线，剩下 6 个休眠，每隔半小时轮换工作节点。

■ 层次型拓扑结构控制

含义

把全网节点划分成一个个“簇”，构建分层逻辑拓扑：

1. 每层选出簇头节点：负责收集本簇内所有成员节点感知数据、数据融合、汇总转发给网关；
2. 普通成员节点：只和自己所属簇头通信，成员之间不直接交互；

经典代表算法：LEACH、HEED 都是典型分簇层次拓扑算法。

解决什么问题

1. 天然适配数据融合，簇头聚合冗余数据，大幅减少上传数据量；
2. 簇头周期性选举轮换，实现全网负载均衡，避免固定簇头能耗透支；
3. 精简全网通信链路，降低大范围信号干扰；
4. 网络规模变大时扩展性更好，适配大规模 WSN；
5. 局部节点失效仅影响单个簇，容错性更强。

拓扑对应关系

物理拓扑可以很杂乱，但通过分簇算法构建出规整分层逻辑拓扑，就是层次型拓扑控制的核心。

WSN 中的节点部署问题：

节点部署问题（Node Deployment Problem）是指如何将传感器节点在目标区域内进行空间布置，以满足特定的网络性能需求（如覆盖、连通性、能耗等）。它是 WSN 设计中的关键初始步骤，直接影响网络的可靠性、能效和成本。

核心目标：

节点部署需要优化多个指标，主要包括：

（1）覆盖性（Coverage）

确保目标区域（或目标点）被传感器节点的感知范围有效覆盖。

例如：火灾监测系统需要确保每个区域至少有一个传感器能检测到温度变化。

（2）连通性（Connectivity）

保证所有节点之间能够直接或间接（多跳）通信，数据可传回汇聚节点（Sink）。

例如：在野外监测中，即使某些节点不能直接与基站通信，也要通过中继节点传输数据。

（3）能效性（Energy Efficiency）

优化节点分布以减少通信能耗，延长网络寿命。

例如：密集部署可能导致节点间干扰，增加能耗，需合理分布。

（4）鲁棒性（Robustness）

部署方式需适应节点失效、环境变化（如风力、障碍物）等动态因素。

（5）成本优化（Cost Optimization）

在满足性能需求的前提下，尽量减少节点数量以降低成本。

节点部署的主要分类：

根据部署方式，WSN 节点部署可分为：

（一）确定性部署（Deterministic Deployment）

定义：节点位置由人工或机器精确规划并固定放置。

适用场景：

①环境可控（如工业监测、智能建筑）。

②需要高精度覆盖（如军事侦察、医疗监测）。

优点：

①覆盖和连通性可精确控制。

②能耗优化较好（如通过计算最优布局）。

缺点：

人工成本高，不适合大规模或危险环境（如战场、火山监测）。

典型方法：

①网格部署（Grid-Based）：节点按规则网格（如六边形、方形）排列。

②基于覆盖优化算法：如虚拟力算法（VFA）调整节点位置。

网格部署（Grid-Based Deployment）

一种确定性节点布置策略，其核心思想是将传感器节点按照某种规则的几何网格结构（如方形、六边形或三角形）排列在目标区域，以实现优化的网络覆盖和连通性。

基本原理：

网格部署通过数学计算预先确定节点的最佳位置，确保：

①均匀覆盖（避免覆盖空洞或重叠）

②高效连通（减少多跳通信的跳数）

③可扩展性（易于新增节点）

关键参数：

(1) 节点间距 (d)

由感知半径 R_s 和通信半径 R_c 决定：

- **覆盖优化**：若仅考虑覆盖，最优间距 $d = \sqrt{3}R_s$ (六边形网格)。
- **连通性优化**：若需保证连通，通常要求 $R_c \geq 2R_s$ (避免通信断裂)。

(2) 覆盖与连通性关系

- 完全覆盖：所有目标点至少被一个节点覆盖。
- 完全连通：所有节点可相互通信（直接或间接）。
- k -覆盖 与 k -连通（高可靠性需求）
 - k -覆盖：每个点被至少 k 个节点覆盖（容错）。
 - k -连通：即使使 $k-1$ 个节点失效，网络仍连通。

算法实现：

(1) 方形网格部署

- 节点坐标计算（假设网格边长为 d ）：

$$(x_i, y_j) = (i \cdot d, j \cdot d), \quad i, j \in \mathbb{Z}$$

- 优点：计算简单，适合矩形区域。
- 缺点：覆盖效率低（重叠率约21%）。

(2) 六边形网格部署（最优覆盖）

- 节点坐标计算（蜂窝结构）：

$$(x_i, y_j) = \begin{cases} (i \cdot d, j \cdot \sqrt{3}d) & \text{if } i \text{ even} \\ (i \cdot d, (j + 0.5) \cdot \sqrt{3}d) & \text{if } i \text{ odd} \end{cases}$$

- 优点：覆盖无重叠（理论最优），能耗均衡。
- 缺点：部署复杂度较高。

(3) 三维网格部署（如无人机网络）

- 扩展至立体空间（如立方体、球面网格）。
- 适用于空中监测、水下传感网络。

优点：

- (1) 高覆盖效率（尤其是六边形网格）。
- (2) 连通性有保障（规则结构易于分析）。
- (3) 能耗均衡（节点分布均匀，避免热点问题）。
- (4) 易于扩展（新增节点可按网格规律加入）。

缺点：

- (1) 环境适应性差：不适合复杂地形（如山地、森林）。
- (2) 静态部署：难以应对节点移动或失效。
- (3) 计算复杂度：三维或大规模部署时规划困难。

改进方法：

(1) 混合网格部署

结合确定性网格 + 随机补充节点，填补覆盖漏洞。例如：80% 节点按六边形网格部署，

20% 随机散布。

(2) 动态网格调整

初始按网格部署，后续通过移动节点（如机器人、无人机）微调位置。适用场景：灾害应急网络、战场侦察。

(3) 非均匀网格

在关键区域（如目标密集区）采用更密的网格，边缘区域稀疏部署。例如：城市安防系统中，市中心节点密度高于郊区。

虚拟力算法（Virtual Force Algorithm, VFA）

虚拟力算法（VFA）是一种受物理学启发的动态节点部署优化算法，主要用于 WSN 中调整节点位置，以优化覆盖性、连通性和能耗均衡。它通过模拟物理中的“引力”和“斥力”使节点自主移动，最终达到均匀分布。

基本思想：

VFA 的核心思想是将传感器节点视为带电粒子，节点之间或节点与环境之间产生虚拟力：

①斥力（Repulsive Force）：使节点避免过于密集（减少覆盖重叠）。

②引力（Attractive Force）：使节点避免过于分散（填补覆盖空洞）。

③环境力（Obstacle Force）：避免节点靠近障碍物或危险区域。

通过计算合力，节点调整位置，最终形成均衡分布。

算法流程：

(1) 初始化

节点随机部署在目标区域。

定义感知半径 R_s 和通信半径 R_c 。

(2) 虚拟力计算

对每个节点 i ，计算来自以下方面的力：

① 节点间作用力

- 斥力（当节点距离 $d_{ij} < R_s$ ）：

$$\vec{F}_{rep} = k_{rep} \cdot \frac{1}{d_{ij}^2} \cdot \vec{u}_{ij}$$

- k_{rep} ：斥力系数

- $\vec{u}_{ij}$ ：从节点 i 指向节点 j 的单位向量。

- 引力（当节点距离 $d_{ij} > R_s$ ）：

$$\vec{F}_{att} = k_{att} \cdot (d_{ij} - R_s) \cdot \vec{u}_{ij}$$

- k_{att} ：引力系数。

② 环境作用力

- 障碍物斥力（避免碰撞）：

$$\vec{F}_{obs} = k_{obs} \cdot \frac{1}{d_{io}^2} \cdot \vec{u}_{io}$$

- d_{io} ：节点 i 与障碍物的距离。

③ 目标区域边界约束

防止节点移出监测区域：

$$\vec{F}_{bound} = \begin{cases} k_{bound} \cdot (d_{ib} - d_{safe}) \cdot \vec{u}_{ib} & \text{if } d_{ib} < d_{safe} \\ 0 & \text{otherwise} \end{cases}$$

- d_{ib} : 节点到边界的距离。

(3) 合力计算与位置更新

- 总合力：

$$\vec{F}_{total} = \sum \vec{F}_{rep} + \sum \vec{F}_{att} + \vec{F}_{obs} + \vec{F}_{bound}$$

- 位置更新（离散时间步长 Δt ）：

$$\vec{p}_i(t+1) = \vec{p}_i(t) + \Delta t \cdot \vec{F}_{total}$$

- 限制移动步长，避免震荡。

(4) 终止条件

①节点位置变化小于阈值。

②达到最大迭代次数。

(二) 随机部署 (Random Deployment)

(1) 定义：节点随机散布（如飞机空投、抛洒）。

(2) 适用场景：大规模或难以人工部署的环境（如森林火灾监测、战场侦察）。

(3) 优点：低成本，适用于恶劣环境。

(4) 缺点：可能出现覆盖空洞 (Coverage Holes) 或连通性问题，需后续调整。

(5) 典型方法：

①冗余部署：部署更多节点以弥补随机性缺陷。

②移动节点辅助：使用可移动节点（如无人机）填补覆盖漏洞。

覆盖空洞 (Coverage Hole)

定义：

覆盖空洞 (Coverage Hole) 是指 WSN 中未被任何传感器节点的感知范围覆盖的区域，即目标区域中的某些点无法被有效监测。覆盖空洞会导致数据缺失，影响网络可靠性，是 WSN 部署和优化中需要解决的关键问题。

成因：

成因	说明
节点随机部署	节点随机散布（如空投）可能导致分布不均，某些区域无节点覆盖。
节点失效	节点因能量耗尽、硬件故障或环境破坏停止工作，原覆盖区域空洞。
感知半径受限	节点感知范围 R_s 过小，无法覆盖整个区域。
通信半径不足	节点通信范围 R_c 小，导致部分节点孤立，无法形成有效多跳覆盖。
障碍物阻挡	建筑物、地形等障碍物遮挡信号，导致感知盲区。
动态环境变化	节点移动（如随风/水流漂移）或环境变化（如火灾烧毁节点）。

类型：

(1) 静态覆盖空洞

- 特点：由初始部署不均衡或节点固定失效引起。
- 示例：农田监测中某些土壤区域未被传感器覆盖。

(2) 动态覆盖空洞

Wireless Sensor Network

- 特点：因节点移动、能量耗尽或环境动态变化产生。
- 示例：战场侦察中无人机节点被击落导致监测空白。

(3) k - 覆盖空洞 (k-Coverage Hole)

- 定义：若要求每个点被至少 k 个节点覆盖（冗余监测），则未被满足的区域称为 k -覆盖空洞。
- 应用场景：高可靠性需求（如核辐射监测）。

(三) 混合部署 (Hybrid Deployment)

- (1) 定义：结合确定性和随机部署，部分节点固定，部分可移动。
- (2) 适用场景：需要高覆盖但部分区域难以精确部署（如灾害救援）。
- (3) 优点：平衡成本和性能。
- (4) 典型方法：

静态 + 移动节点协同：固定节点提供基础覆盖，移动节点（如机器人）动态调整位置。

节点部署的关键算法与技术：

(1) 覆盖优化算法

- 虚拟力算法 (VFA, Virtual Force Algorithm)
 - 节点间模拟“引力”和“斥力”，使分布更均匀。
 - 适用于移动传感器网络 (MSN)。
- 基于计算几何的方法
 - 使用 Voronoi 图、Delaunay 三角剖分检测覆盖空洞并优化。

维诺图：

定义

给定平面内一组传感器顶点，将平面划分成若干多边形区域；每个多边形内任意一点，到本多边形对应传感器节点的距离，小于到其他所有传感器节点的距离，该多边形称为该节点的 Voronoi（维诺）多边形，整体集合为 Voronoi 图。

通俗理解

每个传感器圈一块“专属地盘”，地盘里所有位置离这个传感器最近；地盘分界线是相邻两个节点连线的垂直平分线。

在 WSN 怎么实现、怎么优化覆盖

1. 构建步骤

- ① 遍历所有两两传感器节点，作两点连线的垂直平分线；
- ② 多条平分线相交围成闭合多边形，得到每个节点专属维诺多边形；

2. 判断覆盖空洞（核心用途）

取出某个节点的维诺多边形，判断多边形顶点、边中点是否落在该节点感知半径 R_s 内部：

- 若多边形顶点超出节点感知范围 → 该位置存在覆盖空洞；
- 多边形面积过大 → 节点过于稀疏，需要移动节点往里填补；
- 多边形面积过小 → 节点过于密集，存在大量覆盖重叠，节点相互排斥远离。

3. 优化操作

移动传感器节点，调整维诺多边形大小，让每个多边形都被自身节点感知范围完全覆盖，消除盲区、减少重叠，整体覆盖最优。

考试精简答题版

Voronoi 图将监测区域划分为各节点专属多边形，通过判断多边形顶点是否超出节点感知半径，定位覆盖空洞；通过多边形面积判断节点疏密，驱动节点移动，消除覆盖盲区与冗余重叠，优化整体覆盖性能。

德劳内三角：

定义

对平面全部传感器节点连线，生成互不重叠的三角形网格，满足空圆特性：任意一个三角形

的外接圆内部，不包含其他任何顶点，该三角划分即为 Delaunay（德劳内）三角剖分。

两条核心性质（答题加分点）

1. 空圆特性（判定依据）：任一三角形外接圆不含其他节点；
2. 最小最大角特性：剖分后所有三角形最小内角尽可能大，网格最均匀，避免狭长畸形三角形。

WSN 实现步骤 + 覆盖优化原理

1. 剖分构建

- ① 选取全部传感器节点，逐个三点组合构造三角形；
- ② 校验空圆准则，剔除违反规则的边，不断调整，最终得到整张德劳内三角网格；

2. 查找覆盖空洞

遍历每个德劳内三角形，作出三角形外接圆：

- 若外接圆圆心到三个顶点距离 $>$ 节点感知半径 $R_s \rightarrow$ 圆心区域无节点覆盖，此处存在覆盖空洞；
- 三角形过大 $\rightarrow$ 节点分布稀疏，存在盲区；三角形过小 $\rightarrow$ 节点拥挤，覆盖重叠严重。

3. 优化部署

驱动移动传感器向空洞中心移动，缩小超大三角形、打散密集小三角形，最终节点趋近六边形最优蜂窝排布，最大化区域覆盖率。

考试精简答题版

Delaunay 三角剖分将节点连接为满足空圆特性的三角形网格，通过判断三角形外接圆心是否超出节点感知范围定位覆盖空洞；通过调整三角形规整度，引导节点均匀排布，填补盲区、降低覆盖重叠，实现覆盖优化。

二者配套联合使用

1. 由同一组传感器节点，同时生成 Voronoi 图与 Delaunay 三角剖分，二者互为对偶图；
2. 依靠维诺多边形边界判断单个节点覆盖边界，依靠德劳内三角形外接圆精准定位空洞中心位置；
3. 结合虚拟力算法 VFA，给存在空洞方向的节点施加引力、密集区域节点施加斥力，迭代移动节点；
4. 反复更新剖分与维诺图，直到无新增覆盖空洞，节点均匀分布，整体覆盖效率达到最优。

对比虚拟力算法 VFA

1. 虚拟力算法 VFA

依靠节点间虚拟引力、斥力迭代移动节点，实现分布均匀；实现简单，适合移动传感器网络，但只能整体趋于均匀，无法精准定位空洞具体位置。

2. Voronoi+Delaunay 几何方法

依靠几何特征精准定位空洞坐标，针对性优化节点位置，优化精度更高；但几何运算复杂度更高，算法开销更大。

（2）连通性保障方法

- 最小生成树（MST）：确保所有节点连通且总通信代价最小。

把所有传感器节点连成一张整体互通的网络，保证全网连通，同时全部边的总通信开销（距离 / 能耗）总和最小，减少多余通信链路、节省能耗。

原理通俗理解

- 每个节点是图里一个顶点；两个节点能互相通信，就可以连一条边，边的权重一般取节点间距、通信能耗、传输代价。
- 生成树：连接全部 N 个顶点、只用 $(N-1)$ 条边、不存在环路的连通图。
- 最小生成树：在所有可行生成树里，总权重之和最小的那一棵。

Wireless Sensor Network

- **k- 连通部署**: 部署冗余节点, 使得即使部分节点失效, 网络仍连通(如 2- 连通部署)。

k- 连通定义

一个网络是 **k 点连通**: 至少同时删掉 **k 个不同节点**, 整个网络才会分裂断开;

日常最常用: **2- 连通 (双连通)**

2- 连通通俗解释

任意两个节点之间, 至少存在两条完全不共用中间节点的独立路径。

哪怕其中一条路径上某个节点坏掉、断电失效, 还有第二条路径兜底, 网络依然连通。

(3) 移动节点部署

- 基于机器人 / 无人机的部署:
 - 无人机携带节点并动态调整位置 (如用于灾害应急网络)。
- 自组织移动 (Self-Organizing):
 - 节点具备移动能力, 自动调整位置以优化覆盖 (如水下传感器网络)。

用 Voronoi 图检测无线传感器网络中的覆盖空洞:

Voronoi 图是一种基于计算几何的高效覆盖空洞检测方法, 能够精确识别 WSN 中未被节点覆盖的区域。以下是其详细原理、算法步骤及实现方法。

Voronoi 图的基本概念:

(1) 定义

- Voronoi 图将平面划分为若干单元, 每个单元包含一个传感器节点, 且单元内任意点到该节点的距离小于到其他所有节点的距离。

• 数学表达:

对于节点集合 $P = \{p_1, p_2, \dots, p_n\}$, 其 Voronoi 图 $V(P)$ 将空间划分为 n 个单元 $V(p_i)$:

$$V(p_i) = \{x \in \mathbb{R}^2 \mid d(x, p_i) \leq d(x, p_j), \forall j \neq i\}$$

其中 $d(x, p_i)$ 是点 x 到节点 p_i 的欧氏距离。

(2) 关键性质

- 每个 Voronoi 单元是凸多边形。
- 单元边界是节点间的中垂线。
- 单元内的点离所属节点最近。

Voronoi 图检测覆盖空洞的原理:

(1) 覆盖空洞的判定条件

- 若某 Voronoi 单元内存在点 x , 满足:
 - 若某 Voronoi 单元内存在点 x , 满足:

$$d(x, p_i) > R_s$$

(R_s 为节点感知半径), 则该单元存在覆盖空洞。

- 物理意义: 单元内离节点最远的点未被覆盖, 则整个单元未被完全覆盖。

- 物理意义: 单元内离节点最远的点未被覆盖, 则整个单元未被完全覆盖。

(2) 检测步骤

- ①构建 Voronoi 图: 基于所有活跃节点的位置生成 Voronoi 划分。
- ②检查每个 Voronoi 单元: 计算单元内离节点最远的点 (即顶点或边界点) 是否在 R_s 外。
- ③标记空洞单元: 若最远点超出 R_s , 则该单元为覆盖空洞。

用 Delaunay 三角剖分检测无线传感器网络中的覆盖空洞：

Delaunay 三角剖分是另一种基于计算几何的覆盖空洞检测方法，与 Voronoi 图互为对偶图。它通过分析传感器节点之间的拓扑关系来识别网络中的未覆盖区域，具有计算高效、易于分布式实现的优点。

Delaunay 三角剖分的基本概念：

(1) 定义

对于平面上的节点集合 $P = \{p_1, p_2, \dots, p_n\}$ ，Delaunay 三角剖分 (DT) 满足：

- 所有三角形的外接圆内不包含其他节点（空圆性质）。
- 最大化最小内角，避免 "瘦长" 三角形。

为什么要有这个特性 (WSN 用处)

1. 空洞检测更准确

细长畸形三角形外接圆会异常偏大，容易误判出大量虚假覆盖空洞；角度均匀的三角形尺寸合理，外接圆大小贴合实际节点间距，判断盲区更精准。

2. 拓扑结构稳定均匀

三角形规整代表传感器节点排布趋于均匀，契合我们节点部署均匀覆盖的目标；狭长三角形往往对应节点疏密极端，局部过密或过疏。

(2) 关键性质

- 与 Voronoi 图对偶：Delaunay 边对应 Voronoi 图的边（节点连线的垂直平分线）。只要两个节点之间存在一条 Delaunay 连线 (Delaunay 边)，那么这两个节点对应的 Voronoi 多边形的公共分界线，正好就是这条 Delaunay 边的垂直平分线。
- 局部性：三角剖分仅由邻近节点决定，适合分布式计算。
- 覆盖检测基础：若三角形外接圆半径 $R_{\text{circ}} > R_s$ （感知半径），则可能存在覆盖空洞。

这句话基本方向正确，但表述不够严谨。

准确地说：

在一般位置下（尤其是不存在四个节点共圆），两个节点 p_i, p_j 之间存在 Delaunay 边，当且仅当它们对应的 Voronoi 单元共享一条公共边；这条公共边是线段 $p_i p_j$ 的垂直平分线的一部分。

关键是：

Voronoi 公共边位于垂直平分线上，但通常不等于整条垂直平分线

1. 为什么一定在垂直平分线上？

设 Voronoi 公共边上的任意一点为 Q 。

因为 Q 正好处在节点 p_i 和节点 p_j 的区域分界线上，所以：

$$d(Q, p_i) = d(Q, p_j)$$

而平面上所有满足

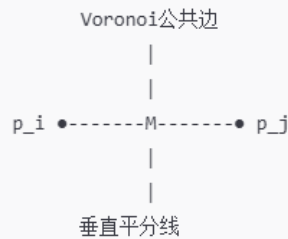
$$d(Q, p_i) = d(Q, p_j)$$

的点，组成的几何图形正是线段 $p_i p_j$ 的垂直平分线。

因此，公共边一定落在垂直平分线上。

text

复制代码



其中：

- M 是 $p_i p_j$ 的中点；
- 公共边与 $p_i p_j$ 垂直；
- 公共边上的点到 p_i, p_j 距离相等。

2. 为什么不是整条垂直平分线？

因为除了 p_i, p_j ，平面上通常还有其他节点。

Voronoi 公共边上的点不仅要满足：

$$d(Q, p_i) = d(Q, p_j)$$

还必须满足：

$$d(Q, p_i) = d(Q, p_j) \leq d(Q, p_k)$$

对所有其他节点 p_k 成立。

也就是说，点 Q 到 p_i, p_j 一样近，并且不能有其他节点比它们更近。

所以其他节点会把整条垂直平分线“截断”，最后留下的可能是：

- 一条有限线段；
- 一条射线；
- 特殊情况下才是整条直线。

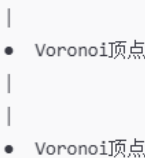
例如：

text [复制代码](#)

整条垂直平分线：



被其他Voronoi区域截断后：



两个 Voronoi 顶点之间的这一段，才是 p_i, p_j 对应单元的公共边。

3. Delaunay 边和 Voronoi 公共边是什么关系?

它们是对偶关系。

假设：

$$p_i p_j$$

是一条 Delaunay 边，那么对应的两个 Voronoi 多边形共享一条边。

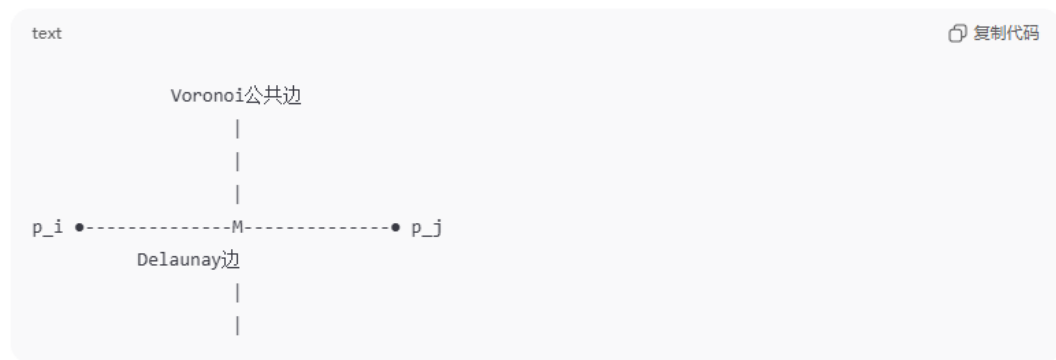
而且：

Delaunay 边与对应的 Voronoi 公共边互相垂直

通常还满足：

Voronoi 公共边经过 Delaunay 边的中点所在垂直平分线

示意如下：



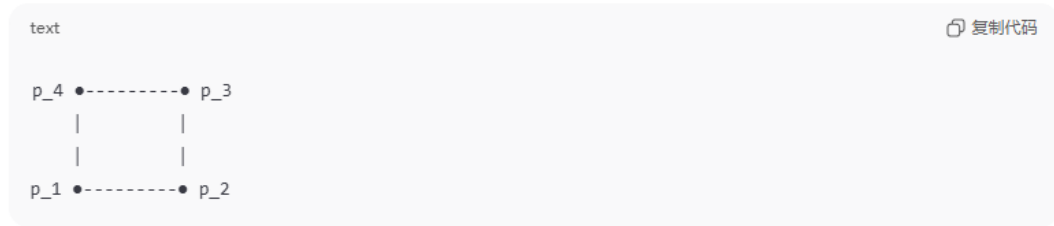
这里：

- 横向的 $p_i p_j$ 是 Delaunay 边；
- 竖向线段是 Voronoi 公共边；
- 二者垂直；
- Voronoi 公共边位于 $p_i p_j$ 的垂直平分线上。

4. 还需要注意一个特殊情况

上述“——对应”在一般位置下最清楚，例如不存在四个节点共圆。

如果出现四个节点共圆，比如正方形四个顶点：



四个 Voronoi 区域可能只在中心点相交。Delaunay 三角剖分则可以任选一条对角线：

$$p_1p_3 \text{ 或 } p_2p_4$$

此时被选中的对角线虽然可以作为 Delaunay 边，但其两个端点的 Voronoi 单元可能只有一个点接触，而不是共享一段公共边。

因此最严谨的说法需要加上：

在无四点共圆等一般位置条件下

推荐你把原句改成

在一般位置条件下，两个节点之间存在 Delaunay 边，当且仅当它们对应的 Voronoi 单元共享一条公共边。该 Voronoi 公共边位于这两个节点连线的垂直平分线上，通常只是垂直平分线的一段，而不是整条垂直平分线。

所以，原句中“正好就是垂直平分线”应改为：

位于垂直平分线上，是垂直平分线的一部分

Delaunay 三角剖分检测覆盖空洞的原理：

(1) 核心理念

- 对节点集合进行 Delaunay 三角剖分。
- 检查每个 Delaunay 三角形：
 - 若三角形外接圆半径 $R_{circ} > R_s$ （感知半径），则其外接圆内可能存在未被覆盖的区域。
 - 进一步验证三角形内是否存在点距离所有顶点 R_s 。

(2) 数学判定条件

对于 Delaunay 三角形 $\triangle p_i p_j p_k$ ：

1. 计算外接圆半径 R_{circ} 和圆心 O 。

2. 若 $R_{circ} > R_s$ ，则：

- 检查圆心 O 是否满足 $d(O, p_i) > R_s \ (\forall i \in \{i, j, k\})$ 。
- 若成立，则 O 为覆盖空洞点。

这个圆心 O 很特殊：

它同时离三个邻近传感器都比较远，是最容易出现覆盖漏洞的候选位置。

注意，准确说法不是“ O 到最远顶点最远”，而是：

O

是一个到最近传感器的距离仍然很大的位置。

节点部署的挑战：

- (1) 动态环境适应
 - 。 环境变化（如风、水流）可能使节点移位，需动态调整。
- (2) 三维空间部署
 - 。 空中、水下等三维场景比二维更复杂（如无人机网络、海洋监测）。
- (3) 安全部署
 - 。 在敌对环境中，节点可能被破坏或干扰，需考虑安全策略。
- (4) 能耗与覆盖的权衡
 - 。 高覆盖通常需要更多节点，但会增加能耗和成本。

第十六讲

功率控制

WSN 基于功率控制的拓扑控制

WSN 中的基于功率控制的拓扑控制是一种关键技术，旨在通过动态调整节点的发射功率来优化网络拓扑结构，从而平衡能量效率、网络连通性、覆盖范围以及干扰等性能指标。

基本概念：

- (1) 目标：在保证网络连通性和覆盖度的前提下，最小化节点能耗（延长网络寿命）并减少通信干扰。
- (2) 核心思想：每个节点根据邻居信息动态调整发射功率，形成高效拓扑（如保持最小生成树、支配集等）。

关键问题与挑战：

- (1) 连通性维护：确保网络无分裂，所有节点至少有一条路径到汇聚节点。
- (2) 能量均衡：避免部分节点因过高功率过早耗尽能量。
- (3) 干扰抑制：高功率会增加信道冲突，需权衡功率与干扰。
- (4) 动态适应性：应对节点移动、失效或环境变化。

(一) LMST

- 基于 MST
- 分布式实现算法
- 信息采集、拓扑创建、发射功率测定
- 与使用最大发射功率相比，总能量消耗降低
- 需要精确的位置信息

LMST (Local Minimum Spanning Tree)

LMST (Local Minimum Spanning Tree) 是一种经典的 WSN 拓扑控制算法，通过分布式方式构建局部最小生成树来优化节点发射功率，从而在保证网络连通性的同时减少能量消耗和通信干扰。

核心思想：

目标：每个节点基于本地邻居信息构建自己的最小生成树 (MST)，仅保留必要的通信链路，降低发射功率。

关键特点：

- (1) 分布式执行：无需全局拓扑信息，适应大规模网络。
- (2) 保持连通性：确保网络无孤立节点。
- (3) 能量高效：减少冗余通信，延长网络寿命。

算法步骤：

- (1) 邻居发现
 - 所有节点以最大功率广播信标 (Beacon)，包含自身 ID 和位置信息 (若可用)。
 - 节点接收邻居的信标，记录邻居的位置和距离，形成邻居表。
- (2) 构建局部最小生成树 (LMST)
 - 每个节点以其邻居为顶点，构建一个局部图，边的权重为节点间的距离。
 - 在该局部图上运行 Prim 算法或 Kruskal 算法，生成局部最小生成树 (LMST)。
 - Prim 算法 (常用):
 1. 选择一个起始节点 (通常是自己)。
 2. 每次选择与当前树距离最短的边加入，直到覆盖所有邻居。
- (3) 确定发射功率
 - 节点仅保留 LMST 中的边对应的邻居作为有效邻居。
 - 调整发射功率至刚好能覆盖最远的有效邻居 (即 LMST 中最长的边)。
 - 删除冗余链路：非 LMST 边对应的邻居不再直接通信，需通过多跳转发。
- (4) 拓扑维护
 - 周期性或事件触发 (如邻居失效) 重新计算 LMST，适应动态网络。

优点：

- (1) 能量高效：最小化发射功率，延长网络寿命。
- (2) 低干扰：减少冗余链路，降低信道冲突。
- (3) 无需全局信息：分布式执行，适应动态网络。

缺点：

- (1) 依赖位置信息：需知道邻居距离 (可通过 RSSI 或 GPS 估计，但可能引入误差)。
- (2) 计算开销：每个节点需运行 MST 算法 (Prim/Kruskal)，对资源受限节点可能负担较大。
- (3) 动态环境适应性：频繁拓扑变化需重新计算 LMST，可能增加能耗。

(二) LMA/LMN

- 节点的度：与节点相距一跳的所有相邻节点的数量。
- 限定节点度的上限与下限。
- 通过动态的调整节点的发射功率，使得节点的度处于给定的上限和下限之间。

算法步骤 (以 LMN 为例)：

- (1) 邻居发现
 - 节点以初始功率广播信标 (Beacon)，交换基本信息 (如 ID、位置、剩余能量)。
 - 维护一个邻居表，记录邻居的动态参数。
- (2) 计算局部均值

- 对目标参数 (如能量、信号强度) 计算邻居的均值：

$$\bar{X}_i = \frac{1}{|N(i)|} \sum_{j \in N(i)} X_j$$

其中 $N(i)$ 是节点 i 的邻居集合， X_j 是邻居 j 的参数值。

- (3) 决策与调整
 - 拓扑控制：若自身参数低于均值 (如能量过低)，则降低活跃度或进入休眠。

- 功率控制：调整发射功率至邻居均值的某个比例（如 $P_{\text{new}} = \alpha \cdot \bar{P}_{\text{neighbors}}$ ）。
 - 数据聚合：用均值替代原始数据上传，减少通信量。
- (4) 周期性更新
- 按固定周期或事件触发重新计算，适应网络变化。
- (三) DRNG/DLMST
- 基于邻近图
 - 把所有节点处于最大发射功率状态下所形成的网络拓扑按照一定的相邻判断条件求出邻近图。
 - 邻近图中每个节点根据与自己相距最远的相邻节点之间的距离来确定其发射功率。

邻近图 (Proximity Graph)

邻近图 (Proximity Graph) 是图论和计算几何中的一类重要图结构，用于描述对象（如点、节点或传感器）之间的空间邻近关系。在无线传感器网络 (WSN)、数据挖掘、计算机视觉等领域有广泛应用。其核心思想是：根据对象之间的距离或其它几何关系，决定是否在图中连接两个对象。

定义：

邻近图是由一组顶点（如传感器节点）和边组成的图，其中边的生成规则基于顶点之间的邻近关系。常见的邻近关系包括：

- 距离阈值：两点距离小于某一阈值则连边。
- 最近邻：每个点连接其最近的 k 个邻居。
- 几何规则：如 Delaunay 三角剖分、Gabriel 图等。

类型：

以下是几种经典的邻近图及其生成规则：

- (1) 单位圆图 (Unit Disk Graph, UDG)
 - 规则：若两节点的欧式距离 $d(u,v) < R$ (R 为通信半径)，则连边。
 - 应用：WSN 中模拟理想无线通信范围（所有节点同构）。
- (2) Gabriel 图 (Gabriel Graph, GG)
 - 规则：连接两点 u 和 v ，当且仅当以 uv 为直径的圆内不包含其他节点。
 - 性质：
 - 是 UDG 的子图，边数更少。
 - 保持连通性且无交叉边。
 - 应用：拓扑控制、地理路由。
- (3) 相对邻居图 (Relative Neighborhood Graph, RNG)
 - 规则：连接两点 u 和 v ，当且仅当不存在其他节点 w 满足 $d(u,w) < d(u,v)$ 且 $d(w,v) < d(u,v)$ 。
 - 性质：
 - 是 GG 的子图，进一步减少边数。
 - 保持连通性但可能增加网络直径。

网络直径：网络中任意两个节点之间最短跳数的最大值。

即：全网相隔最远的一对节点，互相通信最少要经过多少跳中转，这个数值就是网络直径。

- 应用：稀疏化网络拓扑，降低干扰。
- (4) Delaunay 三角剖分 (Delaunay Triangulation, DT)
 - 规则：连接两点 u 和 v ，当且仅当存在一个圆经过 u 和 v ，且圆内不包含其他节点。
 - 性质：
 - 最大化最小内角，避免“狭长”三角形。
 - 是平面图，且对偶图为 Voronoi 图。

- 应用：地理路由、网络生成。
- (5) k - 最近邻图 (k-Nearest Neighbor Graph, k-NNG)
 - 规则：每个节点连接其最近的 k 个邻居。
 - 性质：
 - 可能不连通（尤其当 k 较小时）。
 - 边具有方向性（有向图）。
 - 应用：机器学习、聚类分析。

DRNG/DLMST (Directed Relative Neighborhood Graph / Directed Local Minimum Spanning Tree)

DRNG 是 WSN 中两种基于有向图的分布式拓扑控制算法，旨在通过构建稀疏且能量高效的网络拓扑，优化连通性并延长网络寿命。以下从原理、构造步骤、特性对比及应用场景等方面展开详细介绍。

基本概念与核心思想：

(1) DRNG (Directed RNG)

- 目标：在传统 RNG（无向相对邻居图）基础上引入方向性，进一步减少冗余链路。
- 规则：

对于节点 u 和其邻居 v ，若不存在其他节点 w 满足以下条件，则保留有向边 $u \rightarrow v$ ：

$$d(u, w) < d(u, v) \text{ 且 } d(v, w) < d(u, v)$$

即，若 v 是 u 的“相对邻居”，且无更近的中间节点 w 阻断。

(2) DLMST (Directed LMST)

- 目标：将 LMST（无向局部最小生成树）扩展为有向图，实现更严格的拓扑控制。
- 规则：

每个节点 u 在其邻居集合上构建局部有向最小生成树（基于链路权重，如距离或能量），仅保留树中的有向边。

DRNG (有向相对邻居图) vs 原始无向 RNG: 核心区别通俗详解

一、先回顾原始无向 RNG 规则

无向 RNG: 只要 u 、 v 满足「不存在节点 w ，同时满足 $d(u, w) < d(u, v)$ 、 $d(v, w) < d(u, v)$ 」，直接生成双向无向边 ($u \leftrightarrow v$)

含义： u 能发消息给 v ， v 也一定能发消息给 u ，链路双向对称。

二、DRNG 只保留有向边 $u \rightarrow v$ 的本质改动

DRNG 是每个节点独立、单向判断要不要向外连边：

节点 u 单独判断：我要不要主动发一条有向边 $u \rightarrow v$ ；

判断条件和 RNG 几何规则一致：不存在中间节点 w 满足 $d(u, w) < d(u, v)$ 且 $d(v, w) < d(u, v)$ 。

关键不对称点：

1. u 判定合格，生成 $u \rightarrow v$ ；

2. 但节点 v 单独判断自己的邻居时，未必判定需要连 $v \rightarrow u$ ；

最终只会存在单向边，不再强制双向成对出现。

三、四大具体区别（考试答题可用）

1. 边的对称性不同

- 无向 RNG：边一定双向对称，有 (u, v) 就必然有 (v, u) ，是无向图；
- DRNG：边是单向有向边，允许只有 $u \rightarrow v$ 、没有反向 $v \rightarrow u$ ，属于有向图。

2. 总边数量更少，拓扑更稀疏

无向 RNG 一条判定合格就产生**两条单向链路**；

DRNG 每条边由两个节点各自独立决策，很多反向链路会被各自裁剪，整体冗余链路更少：

- ✅ 信道冲突更少、节点可以进一步压低发射功率、能耗更低。

3. 功率控制逻辑更精细化

- 无向 RNG： u 、 v 必须都开到能互相覆盖的功率，功率受双方约束；
- DRNG：
 u 只要功率够发到 v 即可； v 不需要为了回包专门开大功率匹配 u ，每个节点只根据自己向外需要的最远出边设置发射功率，功率设置更节约、个性化。

4. 构造模式：全局对称构造 vs 纯分布式局部决策

- 无向 RNG：隐含双向一致性，需要隐性协调保证边成对；
- DRNG：完全分布式，每个节点只看自己邻居、自己决定向外连哪些出边，不需要和邻居协商同步，大规模传感器网络部署更简单。

四、优缺点总结

DRNG 优势

1. 链路更稀疏，通信干扰更小，整体能耗更低；
2. 完全分布式运算，不需要节点间协商同步，适配大规模 WSN；
3. 发射功率按需单向设置，功率利用率更高。

DRNG 弊端

1. 拓扑是有向图，路由逻辑更复杂；
2. 虽然原图连通前提下 DRNG 仍保证强连通，但部分路径跳数变长，网络直径略有增大；
3. 双向通信场景下，回传路径需要额外路由规划。

极简考试背诵版

原始 RNG 生成双向对称无向边；DRNG 改为每个节点独立单向判定，仅保留满足 RNG 规则的出边 $u \rightarrow v$ ，不再强制生成反向边。DRNG 拓扑更稀疏、冗余链路更少、节点功率设置更节能，完全分布式易于部署；但拓

算法构造步骤：

（1）DRNG 的分布式构造

1. 邻居发现：
 - 节点以最大功率广播信标（含 ID、位置、剩余能量等）。
 - 接收邻居回复，构建邻居表 $N(u)$ 。
2. 边筛选（对每个邻居 $v \in N(u)$ ）：

Wireless Sensor Network

- 检查是否存在第三方节点 w 使得 $d(u,w) < d(u,v)$ 且 $d(v,w) < d(u,v)$ 。
- 若不存在，则保留有向边 u to v ，否则删除。
- 3. 功率调整：
 - 将发射功率设为覆盖最远保留邻居所需的最小值。
- (2) DLMST 的分布式构造
 - 1. 邻居信息交换：
 - 同 DRNG，获取邻居的链路信息（如距离、能量代价）。
 - 2. 构建局部有向 MST：
 - 每个节点 u 以自己为根，运行 Prim 或 Kruskal 算法，生成以 u 为起点的有向树。
 - 边的权重可定义为 $d(u,v)$ 或能量消耗 $E(u,v)$ 。
 - 3. 保留有向边：
 - 仅保留属于局部有向 MST 的边 u to v 。
 - 4. 动态维护：
 - 周期性更新 MST 以应对拓扑变化。

活动调度

WSN 中基于活动调度的拓扑控制详解

WSN 中的基于活动调度的拓扑控制（Activity Scheduling-based Topology Control）是一种通过动态管理节点的工作状态（活跃/休眠）来优化网络性能的技术。其核心目标是在保证网络覆盖和连通性的前提下，通过周期性或事件驱动节点休眠机制，最大化网络寿命并减少能量消耗。

基本概念与核心思想：

（1）定义：通过协调节点的活跃与休眠时段，使得在任意时刻仅部分节点处于工作状态（负责感知、通信），其余节点休眠以节省能量。

（2）关键目标：

- ①覆盖保持：确保监测区域不被遗漏。
- ②连通保持：活跃节点间可形成连通网络。
- ③能量均衡：避免部分节点因长期活跃而过早死亡。

（2）与传统拓扑控制的区别：

- ①功率控制：调整发射功率（如 LMST、DRNG）。
- ②活动调度：直接控制节点的开关状态，更彻底地节能。

（一）GAF

- 根据节点的地理位置和节点的无线发射半径将网络部署区域划分为若干虚拟单元格，各节点按照位置被划分入相应单元格中。
- 每一个虚拟单元格内，定期选举并保持一个节点作为簇头，代表本单元格向相邻单元格转发数据。只有簇头节点处于工作状态，其它节点进入休眠状态。
- 每个单元格内任意一个节点都可以被选作簇头节点。

GAF（Geographic Adaptive Fidelity）是一种基于地理位置的 WSN 拓扑控制算法，主要用于节能和网络寿命优化。它通过将网络划分为虚拟网格，并在每个网格内动态调度节点的工作状态（活跃/休眠），从而减少冗余通信，同时保持网络连通性和覆盖能力。

核心思想：

- （1）地理分区：将监测区域划分为大小相等的虚拟网格。
- （2）节点角色划分：每个网格内仅保留少量节点处于活跃状态（负责路由和通信），其余节点进入低功耗休眠状态。
- （3）动态轮换：活跃节点定期轮换，以均衡能量消耗。

关键概念：

(1) 虚拟网格划分

- 整个网络区域被划分为多个正方形网格，网格边长 r 满足：

$$r \leq \frac{R}{\sqrt{5}}$$

其中 R 是节点的通信半径（确保相邻网格内的节点可以直接通信）。

- 网格大小计算示例：

◦ 若 $R = 50\text{m}$ ，则 $r \leq \frac{50}{\sqrt{5}} \approx 22.36\text{m}$ ，可取 $r = 20\text{m}$ 。

(2) 节点状态管理

GAF 中的节点有 3 种状态：

状态	描述
Active（活跃）	负责数据转发和感知，保持通信
Sleep（休眠）	关闭无线模块，仅保留定时唤醒功能
Discovery（发现）	过渡状态，用于协商网格内的活跃节点

(3) 活跃节点选举

- 每个网格内通过优先级机制选举活跃节点：
 - 剩余能量高的节点优先。
 - ID 小的节点优先（避免冲突）。
- 活跃节点定期广播信标（Beacon），通知邻居其状态。

执行流程：

(1) 初始化阶段

1. 节点定位：每个节点获取自身位置（GPS 或局部定位算法）。

2. 网格 ID 计算：根据坐标 (x, y) 计算所属网格：

$$\text{Grid}_x = \left\lfloor \frac{x}{r} \right\rfloor, \quad \text{Grid}_y = \left\lfloor \frac{y}{r} \right\rfloor$$

3. 邻居发现：节点广播自己的网格 ID，并记录同网格的邻居。

(2) 状态转换机制

- Discovery → Active：
 - 若当前网格无活跃节点，则自身成为活跃节点。
 - 否则，进入休眠状态。
- Active → Sleep：
 - 活跃节点在超时后若发现其他高优先级节点，则主动休眠。
- Sleep → Discovery：
 - 休眠节点定时唤醒，进入发现状态重新协商。

(3) 数据转发

- 数据包沿网格路径转发（类似棋盘跳跃）：
 - 发送给相邻网格的活跃节点。
 - 若目标网格无活跃节点，唤醒最近节点。

(二) ASCENT

- 均衡网络中骨干节点的数量，并保证数据通路的畅通。
- 节点接收数据时若发现丢包严重，就向数据源方向的邻居节点发出求助消息。
- 节点探测到周围的通信节点丢包率很高或者收到邻居节点发出的帮助请求时，醒来后

主动成为活动节点，帮助邻居节点转发数据包。

ASCENT (Adaptive Self-Configuring Sensor Networks Topologies) 是一种 WSN 中基于邻居协作和自适应唤醒的分布式拓扑控制算法，旨在通过动态调整节点的工作状态（活跃/休眠）来优化网络连通性、覆盖率和能量效率。

核心思想：

ASCENT 的核心目标是：

- (1) 维持网络连通性：确保数据可路由到汇聚节点 (Sink)。
- (2) 动态适应网络密度：在高密度区域减少冗余节点，在稀疏区域激活更多节点。
- (3) 能量高效：通过休眠机制延长网络寿命。

其核心方法是：

- 基于本地邻居信息（如丢包率、邻居数）决定是否激活或休眠。
- 自适应调整：节点根据网络状况动态改变状态，无需全局信息。

关键工作机制：

(1) 邻居探测与评估

- 测试阶段：休眠节点周期性唤醒（进入 Test 状态），广播探测消息：
 - 测量邻居数量和链路质量（如信噪比、丢包率）。
 - 若发现网络性能下降（如丢包率超过阈值），则转为 Active 状态。

(2) 自适应激活规则

- 激活条件（满足任意一条）：

1. 邻居不足：当前 Active 邻居数低于阈值 T_{active} （如 $T_{active} = 3$ ）。

2. 链路质量差：丢包率 $P_{loss} > P_{threshold}$ （如 $P_{threshold} = 20\%$ ）。

- 休眠条件：

◦ 邻居数充足且链路稳定时，返回 Sleep 状态。

(3) 负载均衡

- 主动节点竞争：若多个节点同时满足激活条件，优先级高的节点（如剩余能量高、ID 小）保持 Active，其余进入 Passive 或 Sleep。

状态	工作模式	作用
Active 活跃	射频常开，收发数据包、维护拓扑	组网、数据转发主干节点
Passive 被动	低功耗监听，不主动转发数据	候补备用，随时补位
Sleep 休眠	射频关闭，仅定时短暂唤醒探测	极致省电

层次型拓扑结构控制

层次型拓扑控制 (Hierarchical Topology Control) 是 WSN 中一种重要的网络组织方法，通过将网络划分为多层结构（如分簇、分群或树形结构），优化能量效率、延长网络寿命并提升可扩展性。其核心思想是通过分层管理减少冗余通信，均衡负载，并简化路由。

核心目标：

- (1) 能量高效：减少长距离通信，通过多跳中继降低能耗。
- (2) 负载均衡：避免部分节点（如靠近 Sink 的节点）过早死亡。
- (3) 可扩展性：适应大规模网络部署。
- (4) 简化路由：层次结构天然支持高效数据聚合 (Data Aggregation) 和路由。

(一) HEED

- LEACH 算法中，簇头节点的选举没有考虑节点的地理位置，从而导致簇头分布不均匀
- 根据节点的剩余能量和簇内通信代价选举簇头

HEED (Hybrid Energy-Efficient Distributed Clustering) 是 WSN 中一种分布式、能量高效

Wireless Sensor Network

的分簇算法，通过综合考虑节点的剩余能量和通信代价来选举簇头（Cluster Head, CH），旨在优化网络能耗、延长生命周期并提升数据聚合效率。

核心思想：

HEED 的核心设计原则：

- （1）能量高效：优先选择剩余能量高的节点作为簇头。
- （2）负载均衡：避免簇头过度集中，确保簇大小均匀。

一、什么叫「簇头过度集中」

1. 簇头（CH）：每个簇的管理者，要收集本簇所有节点数据、做数据聚合、转发数据给 Sink，耗电远大于普通节点。
2. 簇头过度集中：

大量簇头扎堆出现在网络某一小块区域，有的地方密密麻麻一堆簇头，有的大片区域一个簇头都没有。

- 坏处 1：扎堆区域簇头互相覆盖、大量冗余，白白浪费能量；
- 坏处 2：空白区域节点太远，没法加入任何簇，网络覆盖断裂；
- 坏处 3：扎堆簇头频繁互相收发报文，信道冲突严重。

举例子：一片方形监测区，左边挤了五六个簇头，右边整片没有簇头，就是典型簇头集中。

二、什么叫「簇大小均匀」

每个簇由一个簇头 + 若干普通成员节点组成，簇大小 = 一个簇内总节点数量

- 如果不加约束：有的簇头附近节点极多，一个簇几十上百个节点；有的簇孤零零只有簇头自己一个节点。
- 节点超多的大簇：簇头要接收、处理海量数据，负载爆炸，耗电飞快，很快电量耗尽死亡；
- 极小簇：簇头开销远大于收发数据收益，整体网络能耗浪费严重。

- （3）分布式执行：无需全局信息，仅依赖本地邻居交互。
- （4）多轮迭代选举：通过多次迭代逼近最优簇头集合。

关键技术：

(1) 簇头选举指标

HEED 使用两个关键参数决定簇头选举：

1. 剩余能量（Residual Energy, E_{res} ）：

- 主参数，直接决定节点成为簇头的初始概率 P_{init} 。
- 能量越高， P_{init} 越大。

2. 通信代价（Communication Cost, C_{cost} ）：

- 次参数，用于在能量相近时优化簇头分布。
- 通常定义为：
 - 节点与邻居的平均距离（减少簇内通信能耗）。
 - 或与 Sink 的距离（减少簇间多跳能耗）。

(2) 自适应簇头概率

- 节点 i 的初始簇头概率 $P_{\text{init}}(i)$ 计算如下：

$$P_{\text{init}}(i) = \min \left(P_{\text{max}}, \alpha \cdot \frac{E_{\text{res}}(i)}{E_{\text{max}}} \right)$$

- P_{max} ：最大簇头概率（如 0.1，避免过多簇头）。
- α ：缩放因子（通常设为 1）。
- E_{max} ：节点初始能量。
- 迭代调整：若节点未当选簇头，逐轮增加概率，直到满足：

$$P_{\text{current}}(i) = \min \left(1, P_{\text{init}}(i) + k \cdot \Delta \right)$$

- k ：当前迭代轮次。
- Δ ：增量步长（如 0.01）。

(3) 簇形成规则

- 强制簇头：若节点在最终轮仍未找到更优簇头，则自立为簇头。
- 最小通信代价选择：普通节点选择通信代价最小的候选簇头加入。

算法执行流程：

阶段 1：初始化

1. 所有节点获取自身剩余能量 E_{res} 和邻居信息。
2. 计算初始簇头概率 P_{init} 。

阶段 2：迭代簇头选举

1. 广播候选消息：
 - 节点以概率 $P_{current}$ 宣布自己为候选簇头 (CH)。
 - 广播消息包含：节点 ID、剩余能量、通信代价。
2. 接收并评估邻居消息：
 - 若收到更优簇头（更高能量或更低 C_{cost} ），则放弃竞选。
3. 概率更新：
 - 未当选的节点更新 $P_{current}$ ，进入下一轮。

阶段 3：簇形成

1. 最终簇头确定：
 - 未加入任何簇的节点成为强制簇头。
2. 普通节点入簇：
 - 选择通信代价最小的簇头加入，发送加入请求 (JOIN-REQ)。
3. 簇内调度：
 - 簇头分配 TDMA 时隙给成员节点。

阶段 4：稳定运行与轮换

1. 数据聚合：
 - 簇头接收成员数据，去冗余后转发至 Sink。
2. 下一轮选举：
 - 固定周期后重新执行簇头选举（避免能量耗尽）。

（二）TopDisc

- 利用颜色区分节点状态，解决骨干网拓扑结构的形成问题。
- 由网络中的一个节点启动发送用于发现邻居节点的查询消息。
- 查询消息携带发送节点的状态信息，算法随消息传播为每个节点着色。
- 按照节点颜色区分出簇头节点，并通过发向寻找查询路由消息的传播路径在簇头节点之间建立通信链路。

TopDisc (Topology Discovery) 是 WSN 中一种基于主动探测的拓扑发现算法，旨在构建高效的网络拓扑结构以支持数据收集和路由优化。其核心思想是通过颜色标记机制和层次化

Wireless Sensor Network

探测，快速识别关键节点（如簇头或网关节点），形成覆盖全网且能量高效的拓扑结构。

核心思想：

TopDisc 通过以下步骤实现拓扑发现：

- ①颜色标记：将节点分为不同颜色类别（如黑、灰、白），表示其在拓扑中的角色。
- ②层次化探测：从汇聚节点（Sink）发起探测，逐跳选择关键节点，形成树状或簇状拓扑。
- ③冗余抑制：通过竞争机制避免过多节点被激活，减少能量消耗。

算法执行流程：

（1）初始化阶段

- 颜色定义：
 - 黑色（Black）：关键节点（如簇头或中继节点），负责数据转发。
 - 灰色（Gray）：辅助节点，部分参与拓扑构建。
 - 白色（White）：普通节点，未被探测到。
- 探测触发：由 Sink 节点发起拓扑发现请求（广播探测消息）。

（2）拓扑发现阶段

①第一轮探测：

- Sink 广播 Topology-Request 消息，标记自己为黑色。
- 收到消息的白色节点：
 - 若距离 Sink 最近，则变为灰色，并继续转发请求。
 - 否则保持白色，等待后续探测。

②第二轮探测：

- 灰色节点广播请求，竞争成为黑色节点：
 - 根据优先级（如剩余能量、信号强度）决定是否升级为黑色。
- 黑色节点继续向外扩散探测，直到全网覆盖。

③拓扑形成：

- 最终黑色节点构成骨干网络（如簇头或路由路径），灰色节点作为辅助，白色节点休眠。

（3）拓扑维护

周期性或事件触发重新探测，适应节点失效或移动。

关键技术：

（1）颜色标记机制

- 黑色节点：关键中继节点，承担数据转发任务。
- 灰色节点：临时角色，用于辅助拓扑发现。
- 白色节点：未被探测的节点，可能进入休眠状态。

（2）优先级竞争

灰色节点通过以下指标竞争成为黑色节点：

- 剩余能量：高能量节点优先。
- 链路质量：信噪比（SNR）高的节点优先。
- 覆盖范围：能覆盖更多白色节点的灰色节点优先。

（3）冗余控制

抑制机制：若多个灰色节点同时竞争，优先级低的节点放弃并退回白色状态。

与其他拓扑控制算法对比：

算法	核心机制	与 TopDisc 的区别
LEACH	随机分簇	TopDisc 主动探测，选择关键节点，更适应动态环境。
HEED	能量 + 通信代价分簇	TopDisc 不依赖分簇，直接构建骨干网络。
GAF	地理网格休眠	TopDisc 基于探测而非固定网格，灵活性更高。